



Reference Manual
Version 2.10
Oct 26 2001

Timo Aila
Ville Miettinen



HYBRID

© Hybrid Holding, Ltd.
Eteläinen Makasiinikatu 4, 6th Floor
00130 Helsinki, Finland

"dynamic PVS" and "dPVS" are trademarks of Criterion Software, Ltd.
The most recent version of this manual is available for download at
<http://www.renderware.com/dpvs>

Contents

I	Programmer's Guide	1
1	Preface	3
2	Acknowledgments	5
3	About This Manual	7
4	Introduction	9
4.1	Why Visibility Optimization Is Vital	9
4.2	dPVS Solution	11
4.3	dPVS Features	12
4.4	Basic Concepts	13
4.5	How Does dPVS Work?	16
4.6	Culling Algorithms Used	19
5	Using the Library	23
5.1	Library Organization	23
5.2	Header Files	23
5.3	Binary and Library Files	23
5.4	Documentation	24
5.5	Debug Build	24
5.6	Memory Checking Tools	24
5.7	dPVS Visualizer	24
5.8	Multi-Threading Issues	25
5.9	Known Bugs and Limitations	25
5.10	Resources	25
5.11	Platform-Specific Notes	25
5.11.1	X86	25
6	dPVS Class Architecture	27
6.1	Conventions	28
6.2	Coordinate Spaces	30
6.3	Vector and Matrix Classes	32
6.4	Reference Counting	33
6.4.1	Using AutoRelease	33
6.4.2	Hard and Soft References	34
6.4.3	Naming Instances	34
6.4.4	User Pointers	34
6.5	The Library Class	35
6.5.1	Querying Statistics	35
6.5.2	Interpreting Statistics	35

6.6	The Commander Class	37
6.6.1	Commander::Viewer	37
6.6.2	Commander::Instance	37
6.6.3	How to Implement Your Commander	37
6.6.4	Commander Architecture	37
6.7	Cells	39
6.7.1	Cell Transformation Matrix	40
6.8	Models	41
6.8.1	Models as Occluders	41
6.8.2	Model Granularity	41
6.8.3	Bounding Volumes of Models	42
6.8.4	Model Simplification	42
6.8.5	Back-Face Culling	43
6.8.6	Good Occluders	43
6.8.7	Level of Detail Models	44
6.8.8	Semi-Transparent Portions of Models	45
6.8.9	Bad Models	45
6.8.10	Mesh Models	46
6.8.11	Sphere Models	46
6.8.12	OBB Models	46
6.9	Objects	47
6.9.1	Derived Classes	47
6.9.2	Placing Objects Into Cells	47
6.9.3	Transformation Matrix	47
6.9.4	Test Models	47
6.9.5	Write Models	49
6.9.6	Visibility Parents	49
6.9.7	Temporarily Disabling Objects	49
6.9.8	Object Costs	49
6.9.9	Bounding Volumes of Objects	50
6.9.10	Animated Object Hierarchies	50
6.9.11	Particle Systems	50
6.9.12	Terrains	51
6.9.13	Unbounded objects	51
6.10	Regions Of Influence	52
6.10.1	Bounding Light Sources	53
6.11	Portals	54
6.11.1	What Are Portals and Where to Place Them	54
6.11.2	The Difference Between Physical and Virtual Portals	55
6.11.3	Defining a Physical Portal	55
6.11.4	Defining a Virtual Portal	55
6.11.5	Warp Matrices	56
6.11.6	Creating a Mirror	56
6.11.7	A Warning About Virtual Portals	57
6.11.8	Importance Decay	57
6.11.9	Limiting Recursion	57
6.11.10	Stencil Models	57
6.11.11	Floating Portals	58
6.12	Cameras	59
6.12.1	Camera Transformation	59
6.12.2	Camera View Frustum	59
6.12.3	Adjustments for Different Pixel Centers	59

6.12.4	Screen Properties	59
6.12.5	Image-Space Sub-sampling	60
6.12.6	Screen-Size Culling	60
6.12.7	Static Coverage Masks and Depth Buffers	60
6.12.8	Visibility Queries	61
6.12.9	Visibility Query Traversal Order	61
6.12.10	Scout Cameras	61
6.12.11	Balancing the Frame Rate	62
6.12.12	Shadow Pre-processing	62
7	Optimizing the Performance of dPVS	65
7.1	On-Demand Loading and Garbage Collection	65
7.2	Integrating Potentially Visible Sets	66
7.3	Image-Based Rendering	67
7.4	Multi-Threading Renderers	68
7.5	Memory Usage	68
7.6	Performance Optimization Checklist	70
II	Application Programming Interface	73
8	API Reference	75
8.1	Camera	76
8.2	Cell	101
8.3	Commander	109
8.3.1	Commander::Instance	122
8.3.2	Commander::Viewer	130
8.4	Frustum	139
8.5	Library	141
8.5.1	Library::Services	163
8.6	MeshModel	168
8.7	Model	170
8.8	OBBModel	177
8.9	Object	179
8.10	ObjectSplitter	199
8.11	PhysicalPortal	201
8.12	ReferenceCount	209
8.13	RegionOfInfluence	220
8.14	SphereModel	222
8.15	Vectors and Matrices	224
8.16	VirtualPortal	234
III	Visibility Theory Guide	241
9	Introduction	243
9.1	Exact Visibility	243
9.2	Conservative Visibility	243
9.3	Scope of This Thesis	244
9.4	Visibility Pipeline	244
9.5	Target Applications of the Library	245
9.6	Plan of Development	245
9.7	New Algorithms and Concepts	245

10 Previous Work	247
10.1 Fundamental Concepts	247
10.2 Design Principles	250
10.3 Exact Visibility Determination	253
10.3.1 Important Algorithmic Properties	253
10.3.2 Grant’s Taxonomy	253
10.3.3 Painter’s Algorithm	254
10.3.4 Z-buffer	255
10.3.5 Ray Casting and Ray Tracing	257
10.3.6 Subdivision	258
10.3.7 Scan-Line Algorithms	260
10.3.8 Beam Tracing	260
10.3.9 Frustum Casting	260
10.3.10 Immediate Mode Ray Casting	260
10.3.11 Conclusion	261
10.4 Conservative Visibility Determination	263
10.4.1 Image-Space Algorithms	263
10.4.2 Hierarchical Z-buffer	264
10.4.3 Hierarchical Polygon Tiling with Coverage Masks	265
10.4.4 Hierarchical Occlusion Maps	267
10.4.5 Object-Space Algorithms	270
10.4.6 Aspect Graph Variants	270
10.4.7 Shadow Volumes	271
10.4.8 Portals	271
10.4.9 Hierarchical View Frustum Culling	275
10.4.10 Hierarchical Back-face Culling	276
10.5 Non-Conservative Visibility Determination	277
10.5.1 Contribution Culling	277
10.5.2 Multi-Resolution Presentation	277
10.5.3 Image-Based Rendering	278
10.6 Data Organization and Traversal	281
11 Contributions	283
11.1 Building a Framework of Inter-Connected Algorithms	283
11.2 Taxonomy of Conservative Visibility Algorithms	286
11.3 Occluder Selection	290
11.3.1 Potentially Good Occluders	290
11.3.2 Incremental Occluder Selection with Feedback	291
11.4 Visible Point Tracking	296
11.5 Object Visibility Types and Actions	298
11.6 Occlusion Buffer and Plan of Development	299
11.7 Occlusion Write Postpone Queue	300
11.8 Silhouettes as Occluder Primitives	301
11.8.1 Temporally Coherent Silhouette Extraction	301
11.8.2 Silhouette Rasterization	305
11.8.3 Silhouettes of Clipping Objects	306
11.8.4 Silhouettes of Alpha Textures	309
11.9 Incremental Occlusion Maps (IOM)	311
11.9.1 HOM and IOM Pipelines	311
11.9.2 Occluder Fusion	311
11.10 Implementing Incremental Occlusion Maps	313
11.10.1 Buffers	313

11.10.2	Tiled Silhouette Rasterization	314
11.10.3	Hierarchical Construction	315
11.10.4	Occlusion Queries	316
11.10.5	Construction-Time Contribution Culling	318
11.10.6	Stencil Buffer	319
11.10.7	Validation Buffer	319
11.10.8	Conclusions on IOM	320
11.11	Hierarchical Depth Estimation Buffer	321
11.12	Object-space Techniques	323
11.13	Virtual Occluders	323
11.14	Portal PVS	324
11.15	Conclusion	326
12	dPVS Architecture	327
12.1	Algorithms Used in dPVS	328
12.2	A Block Diagram of dPVS	329
12.2.1	Concepts	329
12.3	Visibility Pipeline	332
12.4	Accuracy and Robustness Considerations	333
12.4.1	Accuracy	333
12.4.2	Robustness	334
12.5	Data Organization and Traversal	336
12.6	Spatial Database	338
12.6.1	Construction	338
12.6.2	Visibility Query	338
12.6.3	Optimizations	339
12.7	dPVS Visualizer	340
12.7.1	Visual Debugging Information	340
12.7.2	Statistics	344
12.8	Portability	345
13	Results and Conclusions	347
13.1	Test Results	347
13.2	Conclusions	349
14	Future Work	351
14.1	Algorithms	351
14.2	Pre-Processing Tools	351
14.3	Accelerating Shadow Generation	352
14.4	On-Demand Loading	352
14.5	Image-Based Rendering	352
14.6	Interaction Pre-Processor	352
IV	Appendices	355
A	Checklists	357
B	dPVS Visualizer	363
C	Example Implementation of the Commander class	371
D	Troubleshooting	377

E	FAQ	379
F	Glossary	383
G	Technical Information	387
H	Versions: Changes and Porting	391
I	Contact Information	397
V	Index	399
VI	Bibliography	405

List of Figures

4.1	Rendering large scenes	10
4.2	View frustum culling	13
4.3	Occlusion culling	14
4.4	Spatial database	14
4.5	Portal culling	15
4.6	Regions of Influence	16
4.7	Example of temporal bounding volumes	17
4.8	Portals in an architectural environment	21
6.1	Class relationships in the dPVS API	28
6.2	Coordinate spaces	31
6.3	API calls that make hard references	33
6.4	API calls that make soft references	33
6.5	Cells and Portals	39
6.6	Model simplification	42
6.7	Bad occluders	44
6.8	LOD models	44
6.9	Really bad occluders	48
6.10	Visibility parents	49
6.11	Representing terrains in dPVS	51
6.12	Regions of Influence	52
6.13	Portals and Mirrors	54
6.14	Virtual Portals and Reflections	55
6.15	Scout camera	62
7.1	On-demand loading demo	66
7.2	Image-based rendering of reflections	67
10.1	Principle of occluder fusion	249
10.2	Object-space, image-space and temporal coherence	250
10.3	Temporal Bounding Volume	252
10.4	Complete taxonomy of exact visibility algorithms	254
10.5	Classic problem cases of the Painter’s algorithm.	255
10.6	Z-buffer principle	256
10.7	Raycast principle	257
10.8	Warnock subdivision classification cases	259
10.9	Occlusion volume loss	263
10.10	Hierarchical Z-buffer principle	264
10.11	Triage masks	266
10.12	Depth Estimation Buffer principle	267
10.13	An example of Hierarchical Occlusion Maps	267

10.14	Creating partially reflecting surfaces with portals	272
10.15	Overlapping portal bounding rectangles	273
10.16	Using stencil buffer to solve arbitrary portal clipping	273
10.17	Creating simultaneous reflections and refractions using portals	274
10.18	Limiting the recursion depth of portal traversal	274
10.19	Solving the multiple traversal problem of nested portals	275
10.20	Occlusion-Preserving Simplification principle	278
11.1	Taxonomy of conservative visibility determination	287
11.2	Compactness as a tessellation and shape descriptor	290
11.3	Principle of Visible Point Tracking	296
11.4	Example of Visible Point Tracking	297
11.5	Object visibility types and corresponding actions	298
11.6	Re-transforming silhouettes	302
11.7	An example of a re-transformed silhouette	302
11.8	XOR filler	305
11.9	A silhouette clipping to the left plane	307
11.10	Valid backprojection regions of perspective and parallel projection.	307
11.11	Silhouette splatting	308
11.12	Alpha texture to silhouette conversion	309
11.13	HOM and IOM pipelines	311
11.14	Buffer contents of IOM	313
11.15	Byte reorder operation	314
11.16	Vertical and horizontal bounds of IOM tiles	315
11.17	Reflection targets	316
11.18	Contents of Full Blocks Buffer and Coverage Buffer	317
11.19	Portal PVS	324
11.20	Comparison of algorithmic properties of Virtual Occluders, Portals and Occlusion Buffer.	326
12.1	Block diagram of dPVS	329
12.2	Different coverage estimation schemes	333
12.3	Effects caused by a low depth buffer resolution	334
12.4	Different depth estimation schemes	335
12.5	An example scene with occlusion culling.	340
12.6	An example scene without occlusion culling.	340
12.7	Occlusion buffer and voxels for the scene in figure 12.5	341
12.8	A scene with circular mirrors.	341
12.9	Example of the statistics output from dPVS Visualizer	343
B.1	dPVS Visualizer	364
B.2	Statistics menu	366
B.3	Menu System of dPVS Visualizer	366
B.4	Top Level Menus	367
B.5	Menu: Camera Parameters	367
B.6	Menu: Culling	367
B.7	Menu: Debug Output	368
B.8	Menu: Debug Output/Occlusion Buffer	368
B.9	Menu: Global	368
B.10	Menu: Info	368
B.11	Menu: Rendering	368
B.12	Menu: Settings	369

B.13 Keyboard Shortcuts	369
-----------------------------------	-----

Part I

Programmer's Guide

Chapter 1

Preface

Every triangle has its price. That is the first thing you learn when measuring 3D graphics performance.

dPVS is the resulting work of several years at the drawing board, re-thinking the fundamental questions of real-time imaging. Why is 3D rendering such an expensive process? Why is our performance tied to how much goes in, and not how much comes out? dPVS challenges the old rules that have directed and limited the development of 3D applications for years.

For us, dPVS is the first step towards a unified, general solution for dynamic visibility and balanced level of detail. dPVS is the first component in a palette of technologies to raise the bar and take real-time 3D graphics to the next level. dPVS is the most ambitious thing we have ever set out to build.

We are pleased with what we accomplished with dPVS . The technology has reached and exceeded all the goals we set for it. More importantly, though, dPVS has illuminated our way forward.

Every triangle has its price. Have a freebie on us.

Chapter 2

Acknowledgments

This product owes its existence to a number of people both at Hybrid and elsewhere. This manual, as well as the algorithms used in the dPVS library were designed and implemented by Timo Aila (timo@surrender3d.com) and Ville Miettinen (wili@surrender3d.com). Additional programming for the library was done by Jan Achrenius and Otso Mäkinen. Jani Kajala wrote the PS2 assembler version. The dPVS Visualizer system was written by Petri Kero and Janne Hellstén. Models for the examples were created by Mikko Lahti and Santtu Parikka. The figures in the manual are the work of Konsta Hansson. The cover of the manual is the handwork of JuFo Peltomaa. All color images are screenshots from various real-time demos of the SurRender 3D rendering engine and dPVS Visualizer. Harri Holopainen acted as our fearless project leader. The dPVS team would like to thank the rest of the Hybrid crew, whose insight was invaluable during the process.

Special thanks to Janne Kontkanen, who helped designing the first version of the dPVS architecture, and to John Airey, Theodore Jump, Tapio Takala, Matthias Wloka and Hansong Zhang for reviewing the documentation, as well as to Jaakko Lehtinen, Jussi Räsänen and Markus Stein, who helped with many computational geometry problems and other practical issues.

Additional thanks to people at AMD, Criterion Software, Hewlett-Packard, IBM, Intel, Metrowerks, Microsoft and NVidia who provided information and hardware we could not have otherwise obtained.

The research and development of dPVS was partially financed by Tekes, the National Technology Agency of Finland.

Copyright Notice

Dynamic PVS and dPVS are trademarks of Criterion Software Ltd.

Other brand and product names are trademarks or registered trademarks of their respective holders.

Copyright (c) 1994-2001 by Hybrid Holding, Ltd.

Information in this document is subject to change without notice. No part of this document can be duplicated without prior permission from Hybrid Holding, except under conditions set forth by your license agreement.

Chapter 3

About This Manual

The documentation of dPVS is divided into three distinct parts. The first part is the Programmer's Guide, containing mainly technical information on using the library. The second part of the documentation is the API Reference, which contains more specific information about all the classes and member functions of the Application Programming Interface of dPVS. The third part¹ contains a generic overview of existing visibility algorithms, the new algorithms created for dPVS, and how all of them co-operate in the dPVS framework. However, this last part is not needed for using the library and is intended only for people who are *really* interested in how visibility algorithms work in practice.

We make certain assumptions on the existing computer graphics knowledge of our readers. We assume that you have been using some high-level rendering engine and are familiar with concepts such as the scene graph, geometry pipeline, and rasterization. Knowledge of elementary vector and matrix mathematics is also required.

Even though wrappers of dPVS for different languages such as C, Java and Python exist, the C++ forms of function names and classes are used throughout this documentation. All of the code examples use the OpenGL API, as OpenGL has become the *lingua franca* of the 3D visualization community; even if you have not ever written a single application using it, reading and understanding code with OpenGL calls should be relatively straightforward.

The Programmer's Guide has been divided into several chapters. Chapter 4 gives a very broad overview of the dPVS system and visibility problems. Chapter 5 contains information about installing the library, including header files and compiling the sample code. Chapter 6 describes all classes used in the library. Chapter 7 is intended for advanced users who want to maximize the performance of their application.

The appendices A, C, D and E can be useful when learning to use the library. Appendix H contains changes made to the dPVS API - please refer to this when you are porting an application to a newer version of the library.

The most recent electronic version of this manual can be found from the dPVS web page at <http://renderware.com/dpvs>.

¹The original version of the third part was submitted as Timo Aila's Master's thesis.

Chapter 4

Introduction

4.1 Why Visibility Optimization Is Vital

Visibility optimization is currently the most effective technique in speeding up rendering performance in large 3D scenes. The primary reason for this is that during each frame, the rendering subsystem needs to determine the visibility of each piece of geometry individually. Even though most of the geometry does not contribute to the final image, time is spent in finding out which parts are. In other words, 3D performance in larger scenes becomes input-dependent.

In real life, most objects are not in view because something, such as a wall, a ceiling, a cover or another object limits the eyesight. The larger the world, the smaller the percentage that is visible at a given view. Implementing any 3D scene that is bigger than a single room therefore requires a solution for visibility optimization, i.e. a method for detecting what is visible. In the ideal system, the performance depends on the complexity of geometry that is actually visible on the screen. Therefore 3D application performance should be output-sensitive, not input-sensitive.

Visibility optimization in large scenes speeds up rendering more than any other optimization. Current 3D applications solve the problem by pre-processing or using application-specific solutions. The best-studied case, indoor environments, can be made output-sensitive by implementing a system that switches off rooms and their content based on the position of the camera.

Outdoor environments are trickier, and occlusion optimization with arbitrary inter-object occlusion is not usually implemented due to complexity of the problem. Static visibility algorithms such as Potentially Visible Sets [4] require extensive pre-processing and often break down when objects are moved, added or deleted from the scene.

Writing custom pre-processing tools generates a significant amount of work for both the programmer and the artist. The former needs to design and implement the tool and the latter need to learn to use them and know their limitations. To avoid artifacts, these solutions usually impose artificial limitations to the content. Implementing more advanced visibility solutions is complicated and takes substantial amounts of time to develop and debug. Owing to the cost-benefit ratio, it is hardly cost-effective to write a general purpose visibility system for only a single application.



Figure 4.1 When visualizing extremely large scenes, such as the city scene above, visibility determination is the single most important optimization method. The city in the pictures above consists of more than 16,000 buildings and 4,000 moving vehicles.

4.2 dPVS Solution

Known algorithms and currently available implementations rely heavily on pre-processing of scenes, have problems with moving objects and offer limited solutions such as static indoor environments only. dPVS is the first general-purpose solution that supports dynamic environments and does not require pre-processing.

dPVS is implemented as a portable C++ class library, and designed to be easily integrated into any 3D game engine or rendering engine.

The library is a result of several years of research and programming work. All details of used algorithms are hidden from the user. Thereby using dPVS does not require knowledge of advanced visibility technology.

dPVS functionality is very strictly defined for solving the visibility problem only. It is not dependent on any external hardware, 3D APIs or tool libraries. The library simply lists the visible objects seen by a given camera in the specified 3D world. Because of this, dPVS can be used in solving other related problems such as minimizing network traffic for massive multiplayer games. It can also be used to speed up rendering for traditional 3D modeling packages, illumination pre-processing tools and level editors.

4.3 dPVS Features

Advantages

- speeds up rendering by view frustum, occlusion, portal and contribution culling
- reduces geometry and triangle processing, texture memory consumption and hardware pixel fill rate
- speeds up illumination computations
- helps memory and resource management by providing lazy loading of large scenes
- cuts development costs by dramatically reducing scene pre-processing and writing of custom tools
- can be used with any game engine or low-level rendering API
- portable cross-platform solution

Features

- output-sensitive visibility framework for dynamic scenes with zero pre-processing
- hierarchical view frustum culling
- occlusion culling with occluder fusion and dynamic occluders
- advanced portal culling with support for arbitrary-shaped portals and effects such as reflections and refractions
- screen-size and contribution culling
- on-demand loading of scene data
- Region Of Influence overlap detection, allowing extensive use of light sources
- other techniques such as LODs, multiresolution geometry and image-based rendering can be used with dPVS for further enhancing performance and content scalability

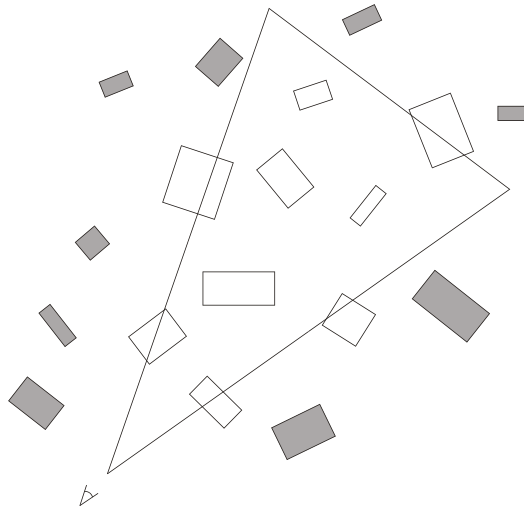


Figure 4.2 The picture shows how portions of the scene database can be view frustum culled. The objects marked in gray are completely outside the view frustum and can thus be rejected.

4.4 Basic Concepts

As mentioned before, the sole purpose of dPVS is to determine the set of visible objects in a 3D scene for a specific camera view description. When performing rendering in an interactive visualization application, such as a game, this information has to be calculated several times each second. Visualization applications typically have frame rates in the range of 20-70 frames per second. Using dPVS to perform the visibility determination should result in an increase of frame rate in all but the simplest of cases.¹ dPVS could therefore be defined as a *visibility pre-processor*.

The process of querying the list of visible objects from dPVS is called a *visibility query*. During the visibility query, dPVS determines which objects are hidden - the rejection of hidden objects is called *culling*. There are several different kinds of culling. The rejection of objects falling completely outside the camera's *view frustum* is called *view frustum culling* (see figure 4.2).

The culling of objects that are completely hidden by other objects is called *occlusion culling*. The related term *occluder* is used to describe an object or a set of objects that can hide another object, the *occludee*. The term *occluder fusion* is used to describe the ability of the culling algorithm to combine the occlusion power of multiple occluders, thus hiding objects that could not be hidden by any of the occluders individually. Figure 4.3 shows a scene where the occluded objects have been faded.

The term *contribution culling* is used for algorithms that cull away objects that have little significance to the rendered image. Such algorithms are not *conservative*, i.e. using them will produce a different output image than what would otherwise be produced. dPVS does not enable any contribution culling algorithms by default - they have to be specifically turned on by the user.

dPVS organizes objects in the world into a *spatial database*; sometimes the term *spatial index* is used instead. Such a database divides the objects into different groups based on their spatial extents (see figure 4.4). Using a spatial database is absolutely necessary for fast visibility queries in large

¹If all of the objects in the scene are visible and inside the view frustum, dPVS cannot accelerate the rendering.

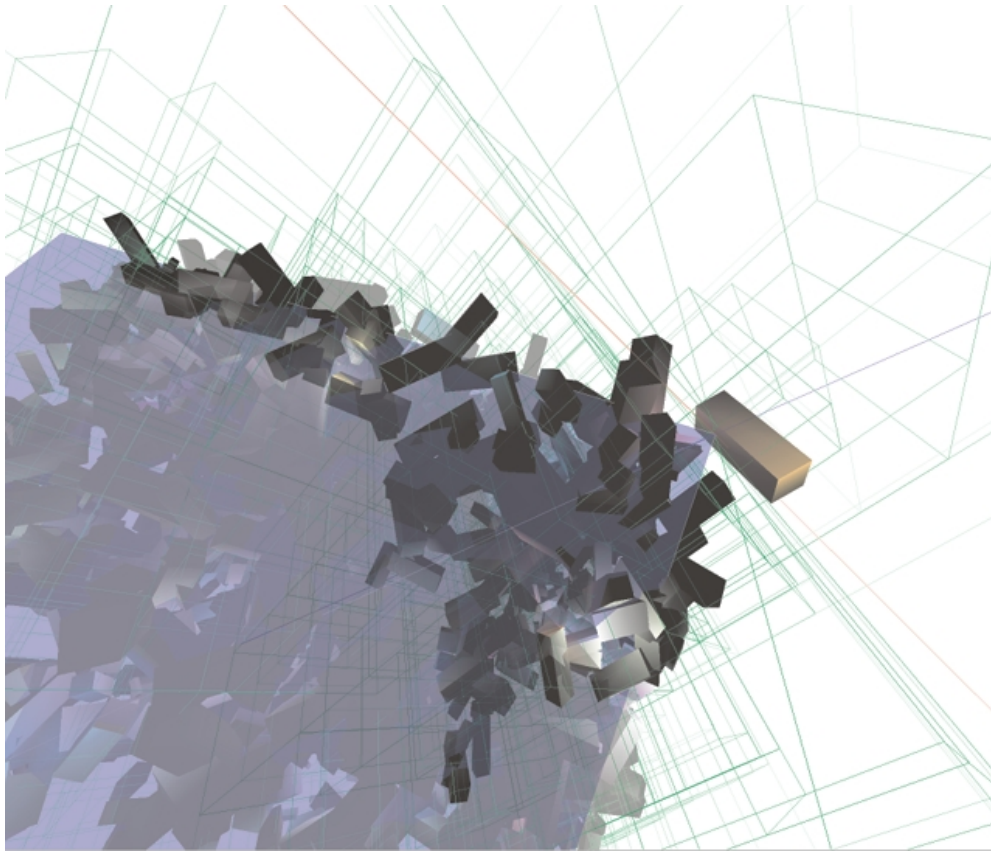


Figure 4.3 Occlusion culling is used to select only the objects visible to the viewer. In the picture the blue area shows the view frustum. The objects drawn as solid are used as occluders and the semi-transparent objects are culled away. The green lines show the spatial subdivision used.

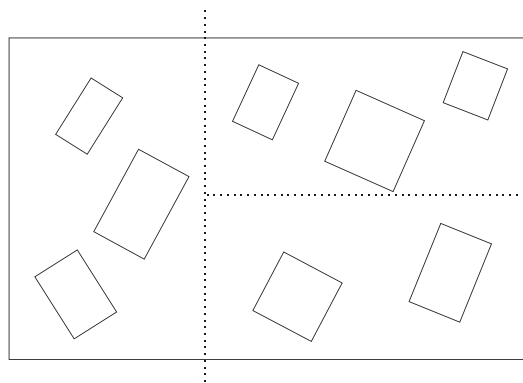


Figure 4.4 By organizing objects into a spatial database, operations can be performed simultaneously for a group of objects. The picture shows a simple axis-aligned binary space partitioning (BSP).

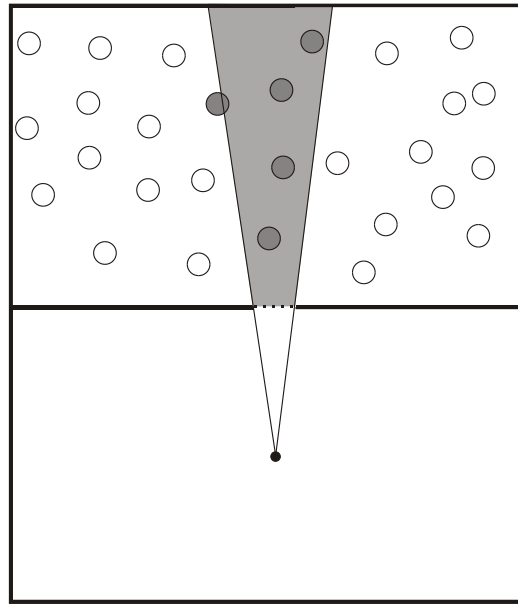


Figure 4.5 In portal culling, a view frustum is formed by the viewer and the portal (doorway). Only objects intersecting this frustum (marked in gray) are visible.

environments. The spatial database constantly adapts to the motion of objects placed into it.

The user can additionally partition the world into separate entities, or *cells*. The connections between the cells are called *portals*. A portal can either be a physical link between cells, such as a doorway between two rooms, or a virtual link affecting the visualization, such as a mirror. See figure 4.5 for an example of *portal culling*. In the picture objects visible through the portal are drawn in gray.

In order to define which parts of the world should be visualized, one or more *cameras* are defined by the user. All visualization systems have this concept, so the cameras used in dPVS have a counterpart on the application side.

Additionally, dPVS supports the concept of *Regions of Influence* (ROI). These are objects that affect somehow other objects (figure 4.6). For example, the light sources of a rendering system can be described using ROIs. During a visibility query dPVS also determines which ROIs affect which objects - this information can be used by the user to dramatically speed up the illumination computations of the rendering engine.

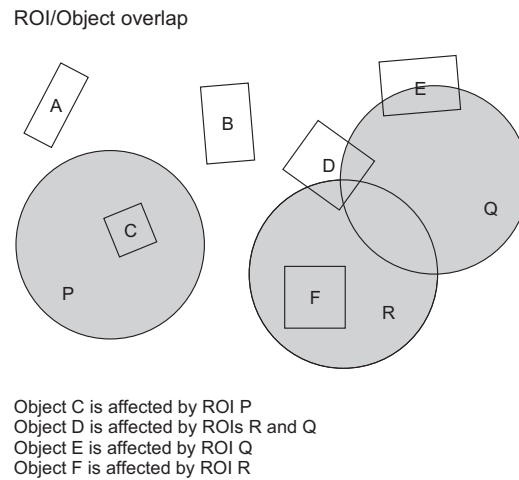


Figure 4.6 Regions of Influence can be used for detecting overlaps between objects and light sources. The gray circles show the regions of influence of three light sources. Illumination computations have to be performed only for the objects that intersect these regions.

4.5 How Does dPVS Work?

dPVS uses a combination of several new and many existing algorithms to perform the visibility determination as quickly and reliably as possible. The sole objective of the library is to produce the tightest possible set of visible objects in the smallest amount of time. dPVS finds a conservative visible set; this means that it will produce all visible and a small number of hidden objects as the output.

The library uses a number of techniques and data structure organizations that make the visibility determination process *output-sensitive*. Output-sensitivity means that the time used for solving a problem, i.e. visibility determination, is dependent on the size of its output - the number of visible objects - rather than its input - the number of objects in the scene.

All of the complicated data structures are internal to the library. The application programmer does not need to know anything about the space partitioning, silhouette simplification or the temporal bounding volumes used internally. Users can use their own scene graph formats, object data formats etc. All the heuristics are internal and self-adjusting - there is no need to submit any tweak parameters. The culling algorithms are all conservative and artifact-free by default. Memory consumption is linear compared to the input data set.²

Lazy Evaluation

Lazy evaluation is used in most of the algorithms in the library. This means that an operation is not performed until the results are *truly* needed. In many cases a substantial amount of work can be avoided by using this technique. For example, moving hidden objects does not really update the space partitioning structures until the objects become visible, buffers are cleared only when they are written into, and writes into data structures are performed only when the corresponding data is read.

²This means that if a scene has N objects, the memory consumption of dPVS is N times some constant.

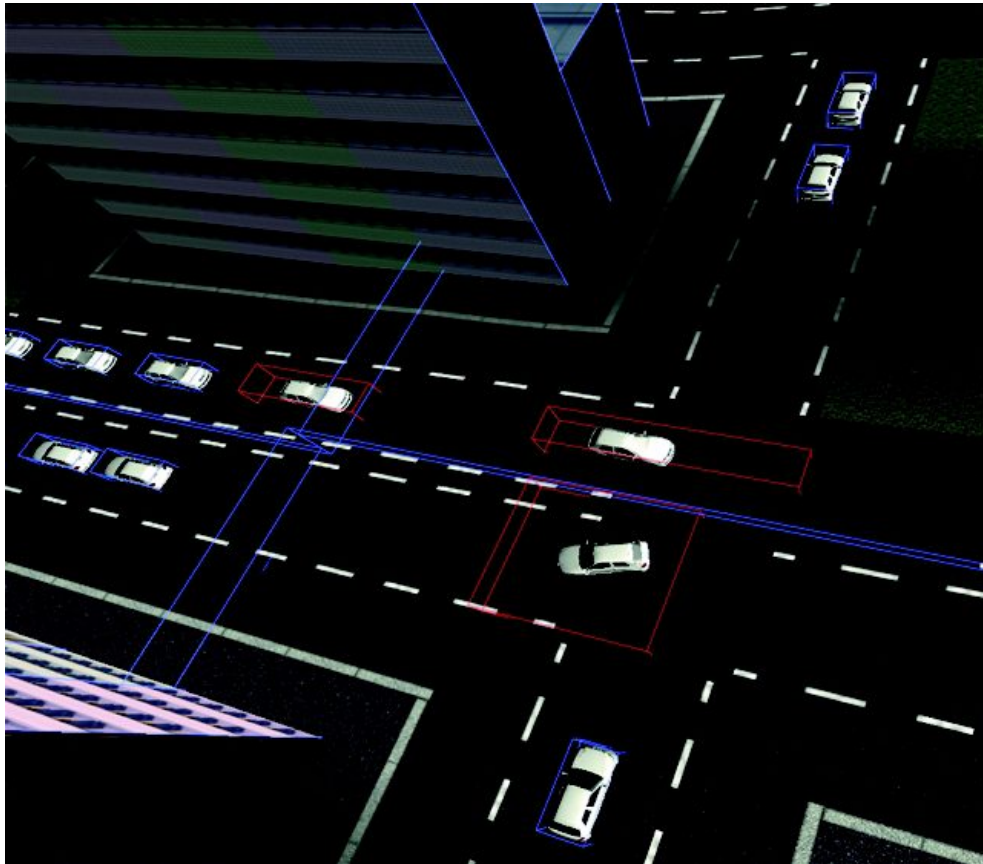


Figure 4.7 When objects are moving in the scene, dPVS internally computes *temporal bounding volumes* for objects. The internal spatial database needs to be updated only when objects violate these bounds. The image above shows the temporal bounding volumes (marked in red) of cars in a traffic simulation. The cars with blue boxes are waiting for the traffic lights to change and have not moved for a couple of seconds.

Hierarchical Data Structures

Almost all data structures inside the library are hierarchical. This allows early exit for many queries. For example, if the collective bounding volume of a group of objects is found to be hidden, then all of the contained objects are hidden and not processed at all. All of the hierarchical data structures are generated inside the library at run-time, and are not exposed in the API. The data structures are dynamic and adapt quickly to even radical changes in the scene. Updating the hidden portions of the scene incurs only a minimal cost on the accompanying data structures.

Algorithm Co-Operation

dPVS uses several different occlusion algorithms, each of which has its own strengths and weaknesses. The algorithms have been designed to be *co-operative*, so that they can share results and even solve difficult sub-problems for each other - problems that would be extremely hard to solve with one algorithm alone. Often multiple algorithms are used together to prove that an object is hidden, and different algorithms are chosen based on the expected results. ³

Temporal Coherence

Temporal coherence or *frame coherence* means that the positions of the objects and cameras in the scene tend to change slowly, i.e. that two subsequent visibility queries have similar - although not exactly the same - input or output data. All of the key algorithms exploit temporal coherence and degrade gracefully when temporal coherence is lost - that is when objects jump in a random fashion around the scene. The temporal coherence is exploited by using different caches, error estimators, adaptive data structures and area visibility determination. Also, many expensive operations can be amortized over a number of frames by using temporal bounding volumes for objects and other similar methods.

Area Visibility

Most occlusion culling algorithms are capable of only determining point visibility: their results are valid only for a single point in space and for a certain view frustum. If a nearby point is used, the solution has to be calculated from the beginning. dPVS uses several dynamic area visibility algorithms to determine if an object is hidden from an entire area or volume in space. As long as the camera stays inside this volume, the object is known to be hidden without any further computation. Such areas are then organized hierarchically, so that if the camera moves outside the current volume, some information is still available. Also, if multiple cameras are used simultaneously, the work done by one camera can be used by the others.

Occluder Fusion and Dynamic Occluders

dPVS utilizes *occluder fusion*. This means that two or more objects can together hide an object that would not be hidden by any of the objects alone. This feature is very useful in scenes where

³dPVS tracks history data for many objects. As there are often multiple ways for proving something, the history data can be used for selecting the fastest algorithm for the specific case.

4.6 Culling Algorithms Used

the occluding objects are small, spatially distinct or moving. The occlusion algorithms can also use *dynamic*, i.e. moving or rotating, objects as occluders.

Simplified Representations

dPVS conservatively approximates the shapes of complicated objects to reduce the costs of the occlusion detection algorithms. This allows using complicated objects consisting of thousands of triangles as both occludees and occluders. The simplification process is purely internal and performed at run-time.

Cost-Benefit Analysis

Almost all of the algorithms used are subject to a *cost/benefit analysis*. This means that the cost of executing an algorithm is compared against the expected benefit: the algorithm is only executed if the benefits outweigh the costs involved. Heuristics and fuzzy logic are used to guide the cost and benefit determination. For example, such an analysis is used for solving the typically very tricky occluder selection problem. Also, some objects are reported visible, even though they are actually hidden. The reason for this is that proving that they are hidden is more expensive than rendering them.

Prediction and Work Amortization

Some of the algorithms use predictors to predict the future transformations of objects and cameras. The prediction information is then used to balance the work more evenly over different queries; this produces more stable per-frame computation costs. Also, some expensive operations, such as database partitioning, are amortized over multiple frames by using *tabu counters* and similar constructs.

Regions of Influence

The library can also determine overlaps between objects and *Regions of Influence*. Imagine a city with thousands of objects and thousands of lights with relatively short attenuation ranges. The cost of calculating the illumination for even the visible objects would be unacceptably high. dPVS can solve the light source range vs. object overlaps in the scene traversal process. Therefore the exact illumination calculations have to be done only for the lights that can actually affect the object.

4.6 Culling Algorithms Used

dPVS uses the following culling algorithms in the visibility determination process:

Hierarchical View Frustum Culling

Objects, object groups and volumes of space that do not intersect the view frustum of the camera are culled in a hierarchical fashion. The algorithm uses highly optimized OBB, AABB and convex hull intersection routines, as well as *plane mask propagation*, i.e. no view frustum tests are performed for objects that are known to be inside the view frustum.

Occlusion Buffer Culling

The key occlusion algorithm that supports occluder fusion and highly dynamic occluders is the occlusion buffer algorithm, which is an optimized combination of Hansong Zhang's Hierarchical Occlusion Map [125] and Ned Greene's Triage Coverage Mask [52] algorithms. This image-space occlusion algorithm is pixel-exact. The occluder writes are optimized by using simplified occluders, massively parallel pixel-processing and hierarchical, Warnock-style subdivision [114]. The occlusion buffer culling is hierarchical in both object- and image-space.

Hierarchical DHS

Definitely Hidden Set (DHS) culling is the complement of the popular Potentially Visible Set (PVS) algorithm. Hierarchical DHS can detect objects and volumes of space that are definitely hidden from a certain part of the scene - such objects are skipped completely and not processed by the other culling algorithms. DHS differs from PVS by being completely dynamic.

Portal Culling

Portal culling is supported and used mostly for generating special effects such as mirrors, refractive surfaces and other optical tricks. Portal culling requires extensive pre-processing or specialized modeling tools.

Screen-Size and Contribution Culling

Screen-size culling can be used to cull objects having screen projections smaller than a user-specified threshold. Contribution culling is used for removing objects that have little influence to the output image. These forms of culling are not enabled by default as they are not conservative. However, they can produce important savings in rendering time.

ROI vs. Object Overlap Testing

The ROI vs. object overlap testing is done only for visible objects and visible ROIs. The overlap detection is a multi-dimensional "sweep and prune" algorithm, where the scanning is performed simultaneously in all three dimensions.

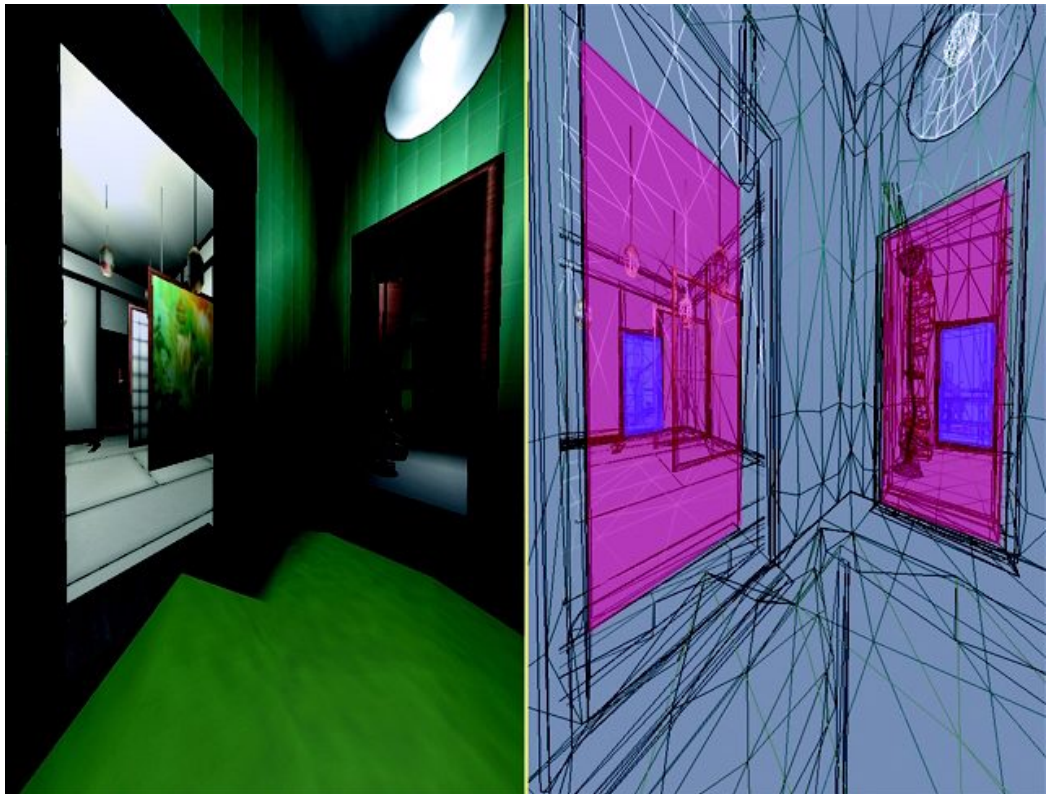


Figure 4.8 This wireframe picture shows how objects can be clipped against a portal sequence to reduce overdraw and fillrate requirements.

Chapter 5

Using the Library

5.1 Library Organization

The library is divided into four separate categories that can be installed separately:

- binary, library and header files
- binary and library files for debug build
- documentation
- sample source code

Only the files in the first category are needed to compile and link an application using dPVS . When distributing the final application, only the dynamic link library file of dPVS needs to be included.

The debug build library and binary files are intended to be used solely during application development. They should not be distributed along with the final application.

5.2 Header Files

All public API header files can be found from the directory **/dpvs/interface**. The header files are all named **dpvsXXX.hpp** where **XXX** corresponds with the name of the class. For example, the Library interface is in the header file called **dpvsLibrary.hpp**. In order to include all dPVS header files, one can use the header file **dpvs.hpp**. Note that all classes in the Model family are in the header **dpvsModel.hpp** and all Object classes in the header **dpvsObject.hpp**.

5.3 Binary and Library Files

All library and binary files are stored in the directories **/dpvs/lib/XXX** and **/dpvs/bin/XXX** where **XXX** corresponds with the name of the platform/compiler. Debug builds of the library have files

named with the 'd' postfix. For example, **dpvs.lib** is the release build library and **dpvsd.lib** is the corresponding debug build library.

The library files of dPVS are backwards binary-compatible, unless separately noted. This means that one can install a newer version of dPVS by just downloading the new library file.

5.4 Documentation

The directory **/dpvs/doc** has several text files containing information about the current release of dPVS. The file **dpvsReleaseNotes.txt** has the most up-to-date information about the current release. Please have a look at this file as it may contain corrections to this manual. This directory also contains an electronic version of this documentation.

5.5 Debug Build

The debug build of the library has been compiled with default optimizations, but with debug information and assertions turned on. The debug build is intended only for debugging purposes during the development of an application, and should not be used in the final application. The debug build can be quite slow as it performs various internal consistency checks of the state of dPVS.

The debug build contains two types of assertions: API asserts and internal asserts. The former are used to trap user errors, such as submitting a NULL pointer when one is not allowed. The latter are purely internal checks; should you encounter an internal assertion, please send a bug report to the Hybrid staff.

The debug and release builds of dPVS can be freely mixed. This means that one can use the debug build of dPVS with the release build of the application and vice versa.

5.6 Memory Checking Tools

If tools such as BoundsChecker or Purify report any dPVS-related memory leaks - and you are sure you have released everything correctly - please inform the Hybrid staff. dPVS should not leak under any circumstances. The debug build performs automatic leak checking and asserts when `Library::exit()` is called if any dPVS-related resources have not been released.

5.7 dPVS Visualizer

dPVS Visualizer is the basic framework used by all of the sample applications. It contains a simple rendering engine that uses OpenGL, a windowing and menu system using GLUT and simple scene database importing facilities. No platform-specific external libraries are used.

dPVS Visualizer can be used either for educational purposes or for debugging and profiling the library itself. dPVS Visualizer provides many interesting statistics about the performance of the library. Further information about dPVS Visualizer can be found in [Appendix B](#).

5.8 Multi-Threading Issues

dPVS can be used by a multi-threaded application. However, only one thread at a time can access any dPVS components. There are no internal mutexes or critical sections protecting the components. On platforms that support multiple threads, dPVS has been linked to use the run-time C and C++ libraries that are thread-safe. dPVS does not use any non-thread-safe library functions, such as `rand()` in the C library.

5.9 Known Bugs and Limitations

Should you find a bug in the library or ever get an internal assertion, i.e. not an API assertion, please report the problem by e-mail to rw-support@csf.com.

Hybrid maintains a zero bug tolerance policy. If bugs are found, they are patched in the next release. A list of reported and recently fixed bugs, as well as the most recent binaries of the library, can be found from the main dPVS web page at <http://renderware.com/dpvs>. The same web page also has a list of upcoming modifications; please take a look at this list before submitting a wish list of features.

5.10 Resources

The bibliography at the end of this manual contains a list of research papers on visibility algorithms, spatial databases and image-based rendering. Additionally, the dPVS web site at <http://renderware.com/dpvs> has links to other web pages containing on-line publications about these and related subjects.

5.11 Platform-Specific Notes

The following notes apply to different builds of the dPVS library. Although the library interface itself is exactly the same for all platforms, there are certain subtle differences between the builds regarding internal optimizations, memory usage and raw performance.

5.11.1 X86

dPVS detects the CPU used during library start-up. Portions of the library are specifically optimized for certain instruction sets. For example, image-space occlusion culling algorithms are optimized for processors with MMX instructions. The application user can determine which optimizations are enabled by calling the function `Library::getInfoString(Library::CPU_OPTIMIZATIONS)`

In debug build dPVS unmask floating-point exceptions during the `Camera::resolveVisibility()` query. This means that if invalid input data in matrices or model data has been specified, a hardware floating-point exception will occur. The exception masks are returned to normal before the Commander callback functions are performed so this should not affect the rendering code of the user.

Chapter 6

dPVS Class Architecture

The dPVS public API has been designed to be as minimal as possible. This means that features that can be easily and efficiently implemented outside the library are not provided in the API.

This chapter contains an overview of the classes in the dPVS API. However, it does not contain any function-level documentation, as this is provided in chapter 8.

Functions that have global effects are provided in the `Library` class (section 6.5). The world is divided into `Cells` (section 6.7) that are connected using `PhysicalPortals` and `VirtualPortals` (section 6.11). The cells are populated with `Objects` (section 6.9) and light sources are described using the `RegionOfInfluence` (section 6.10) class. The shapes of the objects are described using the `Model` and derived classes (section 6.8).

Visibility queries are made using `Cameras` (section 6.12). Whenever dPVS needs to inform the user about something during a visibility query, the `Commander` class is used (section 6.6). Almost all classes in the library are derived from the `ReferenceCount` class (section 6.4).

See figure 6.1 for a diagram of the relationships between the classes in the public interface of dPVS.

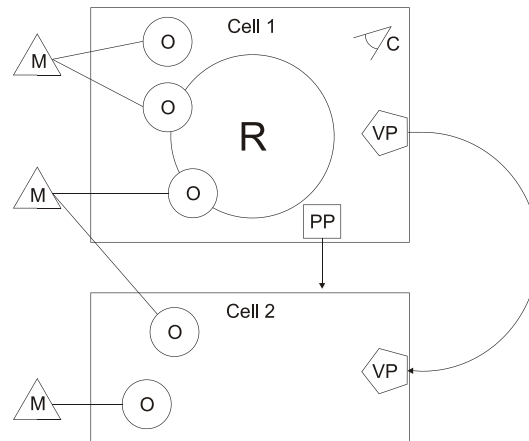


Figure 6.1 Models have been marked using the letter *M*, objects with *O*, Regions Of Influence with *R*, cameras with *C*, physical portals with *PP* and virtual portals with *VP*.

6.1 Conventions

The following conventions have been used throughout the library:

- all classes are wrapped inside the namespace `DPVS`; however, for the sake of clarity, this manual often omits the namespace prefix, i.e. `Object` is used instead of `DPVS::Object`
- no global functions, enumerations or variables are used
- classes do not have public data members
- the Bridge Pattern is used in all public classes to fully hide the implementation details
- all class names start with a capital letter. In case of compound names, the subsequent words also start with capital letters, i.e. `DPVS::MeshModel`
- the first word in a member function name does not start with a capital letter, however subsequent ones do, i.e. `DPVS::Cell::getWorldToCellMatrix()`
- enumerations are written in all capitals and words are delimited by underscores, i.e. `DPVS::Camera::OCCLUSION_CULLING`
- class interfaces can be found from a header file with name **`dpvsXXX.hpp`** where **`XXX`** corresponds with the name of the class
- function parameters use signed integers and single-precision floats¹. Unsigned integers are used only when a bit mask should be submitted
- member functions that query parameters are named `getXXX()` and functions that set parameters are named `setXXX()`. For example: `Camera::getWidth()` and `Object::setTestModel()`.
- member functions that do not modify the object are marked as `const`

¹Certain platforms execute double-precision floats using an emulator.

6.1 Conventions

- header files include only the header files that are absolutely needed
- neither standard C++ library nor STL headers are used in the public API
- platform-specific headers, such as `<windows.h>` are not used by the public API
- neither exception handling nor RTTI is used internally or externally
- member functions dealing with transformation matrices are named so that the direction of the transformation is clear: `Camera::getCameraToWorldMatrix()` returns the matrix needed to transform vectors from the camera-space into the world-space
- all member functions that take references to matrices as arguments support both single- and double-precision formats

6.2 Coordinate Spaces

A 3D pipeline involves using many different coordinate systems. However, people tend to use different names for the coordinate systems. Throughout this documentation we have used the following conventions:

- as the world is constructed from connected cells, the term *cell-space* is used to describe the local coordinate system of a cell. Object and camera transformations are expressed in the cell-space.
- the concept of a *world-space* may or may not exist in your world. If all of your cells are connected by physical portals and no discontinuities in space exist, a world-space coordinate system is defined.
- the term *object-space* is used to indicate the local coordinate system inside an object. The vertices of a model are defined in object-space²
- *eye-space* is a coordinate system where the camera is located in the origin, looking along the negative (or positive) Z-axis. Vertex data is transformed by the object-to-camera matrix³ to get vertices into the eye-space
- points are transformed from eye-space into *clip-space* by the *projection matrix*. The vectors are now 4D homogenous coordinates, where points inside the view frustum satisfy the equation

$$-W \leq X \leq +W \quad \wedge \quad -W \leq Y \leq +W \quad \wedge \quad -W \leq Z \leq +W$$

- by dividing the first three components of the homogenous coordinate by the *W* component, *screen-space* coordinates are obtained. Points inside the view frustum now have *X*, *Y* and *Z* values in the range [-1,+1]
- to obtain *raster-space* coordinates, the values in the range [-1,+1] should be mapped to the dimensions of the current screen. This is done by scaling the *X* and *Y* coordinates by (*width of the screen*/2, *-height of the screen*/2) and adding the coordinates of the center of the screen. The raster-space *Z* coordinate can be obtained by: $Z' = Z/2 + 0.5$. This effectively scales the resulting *Z*-range to [0,1]. The reason why the *Y* value has to be negated is that raster-space is expressed as having its origin in the upper-left corner of the screen, whereas the other spaces treat the positive *Y*-axis as *up*.

²Note that the term *object-space* is also used for culling algorithms that operate in a coordinate system where the camera projection has not been performed. Correspondingly, *image-space* algorithms operate with projected data.

³sometimes called the *modelview* matrix

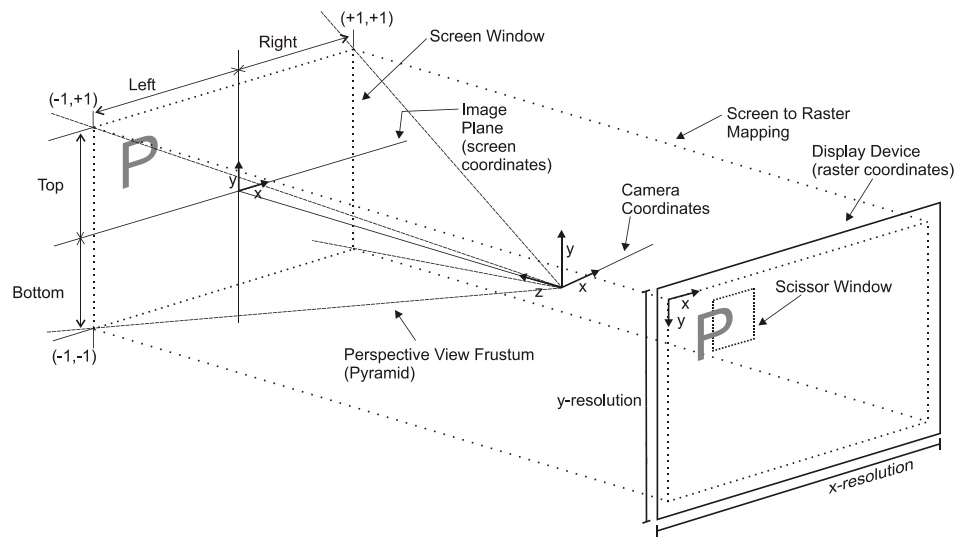


Figure 6.2 Visualization of coordinate spaces and related concepts. All concepts are identical to the RenderMan standard. If the camera coordinate system of an application is different from the one defined by dPVS, special care needs to be taken. For instance, if the x-axis points to the opposite direction, the concepts of left and right get swapped in the image-space, and the origin of the raster-space coordinates is in the top-right corner. Flips like this can be compensated by submitting a frustum having left and right values flipped. See documentation for `Camera::setFrustum()` for further details.

6.3 Vector and Matrix Classes

The dPVS API defines, but does not implement, several vector and matrix classes. It is likely that users want to use their own implementation for these classes. This can be achieved by first defining the macro `DPVS_OVERRIDE_VECTOR_TYPES` before including any dPVS headers and then using typedefs. The data layout for the vector and matrix classes can be found from the header file **dpvsDefs.hpp**. The matrix data organization, i.e. whether it is column or row major, can be selected at run-time when `Library::init()` is called. Functions dealing with projection matrices take additional parameters for indicating the matrix handedness.

The vector classes are named as follows:

- single-precision floating-point vectors are named `VectorX` where `X` is a number denoting how many components the vector has; for example `Vector3`
- double-precision floating-point vectors have the same naming convention, except that the postfix `'d'` is appended to the name. For example, `Vector4d` is a double-precision vector with four components.
- integer vectors use the same convention, except that the postfix used is `'i'`; for example `Vector3i`
- matrix classes contain the matrix dimensions in their name: `Matrix4x4`. The postfix `'d'` is used to indicate that double-precision floats are used.
- whenever matrices are passed to functions of the dPVS API, 4x4 matrices are used. This is done even if the function does not accept matrices containing projections.

6.4 Reference Counting

Object::create()
Object::setTestModel()
Object::setWriteModel()
PhysicalPortal::create() (<i>test model</i>)
PhysicalPortal::setStencilModel()
RegionOfInfluence::create()

Figure 6.3 API calls that make hard references

Camera::setCell()
Object::setCell()
Object::setVisibilityParent()
PhysicalPortal::create() (<i>cell</i>)
PhysicalPortal::setTargetCell()
VirtualPortal::setTargetPortal()

Figure 6.4 API calls that make soft references

6.4 Reference Counting

In order to properly release allocated resources, and not to release them too early, dPVS uses a mechanism called *reference counting*. This means that any object derived from the `ReferenceCount` class - almost all dPVS classes are derived from it - has a *reference count* value indicating how many other objects are referring to it. Every time a pointer to an object is held by another object - for example, a `Model` is assigned to an `Object` - the model's reference count is increased by one. When the object no longer needs the model, for example when the object is destroyed, the model's reference count is decreased by one. When the model's reference count reaches zero, its destructor is called.

When any reference counted object is constructed, its reference count is initialized to 1. This means that from the viewpoint of the programmer, a call to a constructor must be matched by a single call to the release function. The destructors of reference counted objects cannot be called directly - instead a call to `ReferenceCount::release()` should be made.

When using the debug build of the library, assertions are made if any dPVS resources exist when `Library::exit()` is called.

The `ReferenceCount` class interface can be found from the header file **`dpvsReferenceCount.hpp`**.

6.4.1 Using AutoRelease

The programmer may want to let dPVS manage the destruction of certain objects, such as models. This can be achieved by calling `ReferenceCount::autoRelease()`. The function will decrease the reference count of the model by one, but not destroy it. When all referring objects have been destroyed, the reference count of the model reaches zero and it is finally released.

6.4.2 Hard and Soft References

Some API calls of dPVS automatically increase and decrease the reference counts of the parameters. For example, if a test model is assigned to an object with `Object::setTestModel()`, its reference count is automatically increased. Functions that modify the reference counts of their parameters are said to take *hard references*.

Some other functions, such as the assignment of a visibility parent, take *soft references*. This means that if the *reference target* is destroyed, the *reference maker* will be internally notified and the link will be automatically broken.

The following piece of code shows how hard and soft references work in practice.

```
objectA->setVisibilityParent(objectB); // make a soft reference
objectA->setWriteModel(modelA); // make a hard reference
objectB->release(); // break the soft visibility parent relation
modelA->release(); // modelA is not destroyed
assert (!objectA->getVisibilityParent());
objectA->setWriteModel(modelB); // modelA is destroyed
modelB->release(); // modelB is not destroyed
objectA->release(); // objectA and modelB destroyed
```

6.4.3 Naming Instances

To help debugging, a name string can be assigned for each instance. By default, instances do not have names. dPVS does not use the names for anything internally. The names can be queried and set by using the member functions `ReferenceCount::getName()` and `ReferenceCount::setName()`.

6.4.4 User Pointers

Typically the application code has a corresponding class for each dPVS class. In order to facilitate the mapping between the dPVS objects and the user's objects, a *user pointer* can be assigned for each dPVS object. dPVS does not use this value internally and never modifies it. See documentation for `ReferenceCount::setUserPointer()` for further information.

6.5 The Library Class

The `Library` class provides all global functions of the library, i.e. functions that are not directly related to any other class. This includes library initialization and shutdown, as well as querying of statistics and version information.

dPVS is initialized with a single call to `Library::init()`. It is a programming error to perform any dPVS -related actions, such as construction of any objects, prior to this call. The library is shut down with a single call to `Library::exit()`. All dPVS -related resources should be released before making this call.

The `Library` class also provides version information. The user can query the version number of the library, a copyright string, the build time and certain functionality-related strings, such as the build type or the name of the customer.

The last important function of the `Library` class is to provide internal statistics information. A large portion of the statistics is interesting only for performance tuning and debugging purposes.

The `Library` class interface can be found from the header file **`dpvsLibrary.hpp`**.

6.5.1 Querying Statistics

dPVS provides some 120+ different statistics counters about its internal performance. A statistic can be queried using the `Library::getStatistic()` function. All statistics can be simultaneously reset using the `Library::resetStatistics()` function. The statistics can be useful when debugging the application; for example, if the `STAT_WRITEQUEUEWRITESPERFORMED` statistic is zero, it is a good indication that the user has forgotten to assign write models for the objects.

The internal statistics counters are 32-bit integers. Therefore it is recommended that all statistics be occasionally reset to prevent overflow. Such resetting will also give more current information.

6.5.2 Interpreting Statistics

Below is a list of some statistics values that can act as indicators for potential problems:

- if `STAT_WRITEQUEUEWRITESPERFORMED` is zero, no occlusion writes have been performed. Perhaps no write models have been assigned for the objects?
- the ratio between the statistics `STAT_DATABASETRAVERSALS` and `STAT_CAMERARESOLVEVISIBILITYCALLS` indicates the number of times a visibility query has passed through a portal. If this ratio is very high, there may be a large number of reflective portals.
- if the ratio between `STAT_DATABASEOBSSTATUSCHANGES` and `STAT_DATABASEOBSVISIBLE` is very high, there is very little temporal coherence available in the scene
- the statistic `STAT_DATABASEOBSVISIBLE` indicates how many objects were determined visible during the traversal
- the statistic `STAT_ROISTATECHANGES` indicates how many times the status of a ROI has been changed from active to inactive

- the statistic `STAT_ROIOBJECTOVERLAPS` tells how many object/ROI overlaps have been detected. If the ratio between this and the visible object count is high, it is typically an indicator of very large ROI volumes

6.6 The Commander Class

All dPVS -to-user communication happens using the Commander class. The user creates a Commander object by deriving a new class from Commander and overriding the `Commander::command()` function. A reference to the Commander object is passed when calling functions that may require user callbacks, such as `Camera::resolveVisibility()`. The Commander class interface can be found from the header file **dpvsCommander.hpp**.

dPVS first performs the entire traversal process and gives the commands to the user *after* the traversal. Thus changes made to the scene or objects will become visible during the *next* traversal. If this is not acceptable, a re-traversal should be made before rendering the objects.

During certain commands, the user might want to have further information, such as a transformation matrix. This information is provided through the *smart objects* `Commander::Viewer` and `Commander::Instance`. See documentation for `Commander::command()` about when the Viewer and Instance objects are available.

6.6.1 Commander::Viewer

`Commander::Viewer` is an internally created class that passes camera-related information to the user during callbacks. The available information includes the view frustum description, the scissor area, and the current projection matrix. The projection matrix returned is either left- or right-handed based on the user's preference. Additionally, camera-space $\rightarrow$ world-space transformation is available, but this information is required only for creating secondary views.

6.6.2 Commander::Instance

`Commander::Instance` is an internally created class that passes object-related information to the user. The same object can be reported visible multiple times with different matrices if mirrors or other virtual portals are used. The `Commander::Instance` class contains a pointer to the object and current object-space $\rightarrow$ camera-space matrix (the same as the modelview matrix in OpenGL). Additionally some assistance to level-of-detail selection is available. The object's accumulated importance value, which is based on the importance decay terms of the portal sequence, can be queried from Instance. The object's screen projection rectangle and its depth range are also available. The clipping status of the object can be queried.

6.6.3 How to Implement Your Commander

The basic idea is very simple. The user derives a new commander class from `DPVS::Commander` class and overrides the `Commander::command()` function. All commands are enumerations. See Appendix C for source code of an example implementation.

6.6.4 Commander Architecture

There are two primary reasons why we prefer the Commander architecture over the traditional approach of using virtual functions. The first reason is clarity. Since all user callbacks are collected

to the same place, it is easy for the user to make sure every callback has been implemented. Additionally, when virtual function tables are modified, new DLL versions would no longer be binary-compatible. As Commander's commands are enumerations, adding a new command does not break the binary compatibility.



Figure 6.5 Architectural environments can be subdivided into cells (rooms) and portals (doors and windows connecting the rooms). In the picture, there are two portals connecting the room to the outside environment.

6.7 Cells

Cells are used to divide the world into separate entities. Each object in dPVS must belong to one cell⁴; an dPVS object cannot exist simultaneously in multiple cells. However, it is very easy to support such multi-cell objects on the application side.

Cells are connected by using *portals*. A portal is a single-directional link object between two cells. A portal can have an arbitrary shape and can contain an arbitrary transformation; this allows implementation of mirrors and refracting surfaces, such as glass walls and water.

If the application does not need portals, only a single cell needs to be created and all objects can be placed there.

Object transformations are always represented in the corresponding cell's local coordinate system. This means that a cell can be moved by modifying its transformation matrix, and the objects contained will move along with it.

dPVS creates a separate spatial database for each cell. This means that an object moving in one cell cannot affect the internal layout of the database in another cell. A cell takes soft references to all objects and cameras that are assigned into it.

Nested cells are supported. In such a case, the inner cell must be bounded with a portal that triggers the traversal of the cell.

The `Cell` class interface can be found from the header file `dpvsCell.hpp`.

⁴Objects can also belong to a `NULL` cell, indicating that the object does not belong to any cell.

6.7.1 Cell Transformation Matrix

Cell transformations are always represented in world-space. The cells are object containers and can be regarded as world-space objects in the sense that they behave like objects behave in the cell-space.

A scene consisting of multiple cells should define the transformation matrices of the cells according to their physical positions inside the scene.

Cell transformations are freely modifiable. Thus moving cells can be created and large entities such as space stations can be modeled as separate cells.

6.8 Models

Models are used for representing transformation-independent shapes and bounding volumes of objects. A model itself does not have any physical location or orientation; the physical existence of a model is represented by the `Object` class. Any number of objects can share a model. This means that once a model - say a chair - has been created, it can be instantiated arbitrarily many times. All modifications made to the model will be reflected to all the instances. The concept of instancing is an important memory-saver in large scenes where there are many duplicate objects. Due to cache-coherence and internal data structure issues, instancing improves the performance of dPVS and should be used whenever possible. For example, an entire forest can be represented using a single tree model that is instantiated thousands of times.

The `Model` class is a pure virtual base class for the derived model classes. Even if `Models` cannot be directly constructed, there are several common functions for manipulating them. The following derived classes exist:

- `SphereModel` for representing a simple bounding sphere
- `MeshModel` for representing models consisting of vertices and triangles
- `OBBModel` for representing an oriented bounding box

The interfaces for all of the `Model`-derived classes can be found from the header file **`dpvsModel.hpp`**.

6.8.1 Models as Occluders

Models are used as occluders if they are assigned to objects as write models and the type of the model permits occlusion writes. Currently only the `MeshModel` class supports occlusion writes.

6.8.2 Model Granularity

Since dPVS reports object visibility at object granularity⁵, selecting the proper composition is important. For example, the model of a large building with tens of thousands of polygons should be split into several different models and objects. The optimal granularity is highly dependent on the topology of the model, the application type and the rendering engine and hardware used. In typical applications, each model should contain somewhere between 100-1000 vertices. When splitting a model into sub-models, the rule of thumb is to attempt to retain spatial locality in each sub-model. Therefore it is much better to divide a house so that each room is a separate model, rather than to place the walls in a single model, and the furniture in another.

Also, keep in mind that objects cannot directly occlude themselves.⁶ This means that if a model contains very large and good occluder polygons, such as the walls of a room, it might be beneficial to subdivide the room into several separate models so that the outer wall can occlude the rest of the

⁵This means that the visibility of portions of the objects, such as individual polygons, is not reported. Most of the time it is faster to just process such hidden polygons, and to let the renderer's exact-visibility algorithm (usually a Z-buffer) handle the exact culling.

⁶They can indirectly occlude their reflections from mirrors.

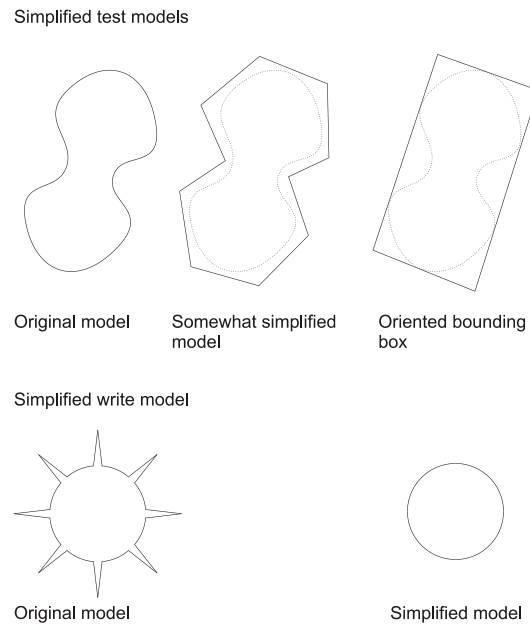


Figure 6.6 dPVS does not need to use the same models that are used for rendering. Test and write models can be simplified as long as they satisfy certain rules.

room. Note that sometimes the subdivision will produce planar or near-planar objects; these objects can then be completely back-face culled from certain viewpoints.

When subdividing a model into multiple parts, a dummy *visibility parent*, for example the bounding box of the original model, can be assigned to the objects. This will speed up culling, if the entire original model is occluded as its children will not have to be individually processed. The usage of visibility parents is not absolutely necessary, since the spatial hierarchy created internally by dPVS achieves this behavior to some degree.

For an automatic splitting of unstructured data, one can use the `DPVS::ObjectSplitter` utility class.

6.8.3 Bounding Volumes of Models

dPVS calculates and maintains several bounding volumes internally. Both axis-aligned and oriented bounding boxes as well as bounding spheres can be queried from models by calling `Model::getAABB()`, `Model::getOBB()` and `Model::getSphere()` respectively. These functions return the bounding volumes in object-space.

6.8.4 Model Simplification

The user does not *need* to use the same model data for rendering and for visibility determination. Instead, simplified models *can* be submitted to dPVS. The basic rules for using simplified models are as follows:

6.8 Models

1. If the model is used for visibility testing, it must be conservatively *larger* or the same size as the rendered model. This means that the test model has to fully contain the model that is rendered. For example, the bounding box, the bounding sphere or the convex hull of any model satisfies this condition.
2. If the model is used for occlusion writes, i.e. it is a *write model*, it must be conservatively *smaller* or the same size as the rendered model. This means that when viewed from any direction, the write model should not occlude something that the rendered model would not. Note that by just removing polygons from the original model, one can always create a valid write model. For example, it might be a good idea to remove all insignificant details - such as the keys in a computer keyboard - when using the model as a write model.
3. Unnecessary tessellation can and should always be removed. The user may have had to tessellate a flat surface for rendering purposes - perhaps to improve the per-vertex illumination quality. It is much better to submit the untessellated version of the model to dPVS, as less geometry operations need to be made during the visibility query, thus improving run-time performance.

See figure 6.6 for examples of simplified test and write models.

6.8.5 Back-Face Culling

Sometimes test models are planar or near-planar, such as individual walls or portals. Such models can be completely hidden from some view directions, as none of their front-facing polygons are visible. The user can indicate this to dPVS by enabling back-face culling of the model by calling `Model::set(Model::BACKFACE_CULLABLE, true)`. Back-face culling is used internally by dPVS to fully reject the object - no polygon-level back-face culling is done, as this is something that should be handled by the renderer. Also some non-planar convex objects, such as spheres or boxes, could be back-face culled when the camera is completely inside or outside them.

Note that if a model is intended to be used as a portal, back-face culling *must* be enabled - otherwise endless recursion cases may occur⁷. Such cases are trapped and handled internally, but may cause undesirable rendering artifacts and loss in performance.

It is not possible for dPVS to detect internally when back-face culling can or cannot be used⁸. The reason for this is that dPVS cannot know whether a test model represents the actual geometry or a simplified bounding volume. Therefore the responsibility for indicating this is left to the user.

6.8.6 Good Occluders

A good occluder is an object that is able to hide a lot of geometry either by itself, or in conjunction with other objects. The task of selecting the good occluders is an internal issue and no user assistance is required. However, some objects are expensive to use as occluders and are hardly ever able to hide geometry. The performance can be improved by disabling occlusion writes of such objects. The concepts of a good occluder and run-time occluder selection are reviewed in detail in section 11.3. See figure 6.7 for examples of some bad occluders.

As a general rule, occlusion writes should be disabled for the following models:

⁷See section 5.7 for details.

⁸The only exception is the case of absolutely planar models. dPVS enables back-face culling for them automatically.

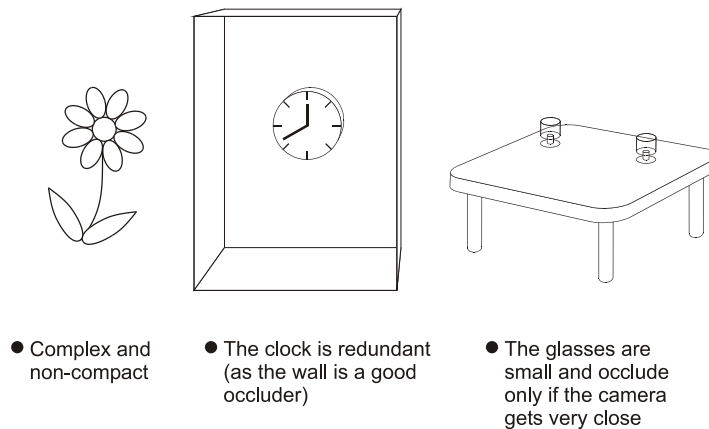


Figure 6.7 The picture shows several occluders that are considered 'bad' under most circumstances.

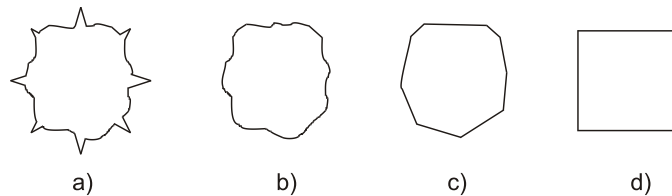


Figure 6.8 Complex models can be represented as several simplified models. The actual model used for rendering and culling can be selected based on the screen size of the object. The model a) is the original model, b) is a simplified LOD level that is a valid write model, c) and d) are simplified LOD levels that are not valid write models. Only a) is a valid test model.

- complex trees, bushes and plants
- complex models that are physically small
- models with many holes, such as a fence
- models that are somehow known not to hide other objects

An occluder can be made better by removing unnecessary detail and tessellation while preserving most of its occlusion properties.

6.8.7 Level of Detail Models

If *level-of-detail* (LOD) models are used in the renderer and one wants to use such objects as occluders, there are two options. The first option is to define only a single reasonably simple write model that represents conservatively all of the detail levels. The other option is to use the `Object::REPORT_IMMEDIATELY` flag for reporting that an object will be visible. The object's

6.8 Models

write model can be changed during the `INSTANCE_IMMEDIATE_REPORT` command. The first option should be used if plausible. See figure 6.8 for further information about creating valid LOD models.

6.8.8 Semi-Transparent Portions of Models

The occlusion culling algorithms of dPVS do not support texturing. Therefore, if the model to be rendered contains polygons with chroma key or alpha channel textures, the offending polygons should be removed from the models before submitting them to dPVS. Sometimes these polygons would contribute significantly to the occlusion. An example could be the case where an artist has modeled the walls of a house with large polygons and drawn the windows using transparent pixels. In such a case, it is better to model the opaque geometry of the walls, and to submit that model to dPVS, regardless of the model used for actual visualization.

Other typical cases where texture transparencies are used include fences and trees. Fences usually contribute very little to the occlusion, thus they should not be specified as write models. Trees are somewhat more problematic because scenes such as forests often contain many trees that can together occlude large portions of the scene. The recommended approach is to at least model the completely solid parts, such as the trunk and the main branches, and use that as a write model. If slight artifacts are allowed, a group of leaves can be modeled by using some simple shape. In scenes such as rain forests, the vegetation density will eventually become virtually solid. The user can model this by placing a simple flat model at the desired height.

The reasons why dPVS cannot handle texture transparencies are simple: first of all, modifying object-space visibility algorithms to support texturing would be next to impossible. Although the image-space algorithms could in theory handle texturing, the rasterization fill rate could be as much as 500 times smaller than what the currently used parallel bit-processing provides. Future versions of dPVS with hardware occlusion culling capabilities may support limited usage of alpha channels.

6.8.9 Bad Models

Because dPVS performs its visibility determination mostly in image-space, i.e. by means of high speed rasterization, it suffers from the same problems as the rendering engine used to draw the final image. If the models contain artifacts, such as T-junctions, gaps can appear between triangles of the model. This in turn may cause many more objects to be determined as visible, since they are seen through the tiny gaps. It is relatively easy to find out if your artwork contains T-junctions: set the background color of your screen to a high-contrast value, such as bright red, and look for red pixels. If the modeling work has been done well, no such gaps should exist.

If there are gaps in your models, there are three options. First of all, you can ignore the problem. This is a valid approach if the gaps are not in important occluders, such as walls. The second option is to manually fix the models. This process requires splitting each triangle that has a T-junction into two triangles and ensuring that the vertices are shared. This process can also be done algorithmically. The last option is to allow dPVS deal with the problem by letting it perform non-conservative culling. See documentation for `Camera::setObjectMinimumCoverage()` for further details.

Similar problems may appear also if connected objects are modeled separately and expressed in different local coordinate systems. This problem can be easily fixed by using a shared coordinate system for such objects.

6.8.10 Mesh Models

`MeshModel` is the most common model type. The topology of the model is represented by vertex positions and triangle connectivity information. All vertex and triangle data is submitted as constructor parameters. The mesh model data cannot be modified at run-time. Mesh models take a copy of the input data provided by the user. The data is optimized, analyzed and organized in a more efficient format by the constructor. Because this operation takes some time, new `MeshModels` should not be constructed too often.

6.8.11 Sphere Models

Sphere models are the simplest type of models. A `SphereModel` is a spherical shape defined by a translation vector in the object's local coordinate system and a positive radius value. Although many shapes are usually bounded poorly by spheres, this class is provided so that certain complicated systems - such as particle systems or point light sources - can be represented easily. An additional advantage is that sphere models consume very little memory. Note that `SphereModels` cannot be used as write models.

6.8.12 OBB Models

An `OBBModel` is a box shape defined by a center and a local coordinate system indicating the scales and the orientation. Oriented bounding boxes are simple model types with a very low memory consumption, yet various objects can be rather tightly bounded using OBBs. An alternative constructor accepts axis-aligned bounding boxes (AABB) as input. Note that `OBBModels` cannot be used as write models.

6.9 Objects

Objects are used to place `Models` into the world. For each object, a transformation matrix is defined. Each object also has a *test model* used for visibility testing and an optional *write model* used for occluding other objects.

6.9.1 Derived Classes

There are several classes in the `Object` family. The `Object` class acts both as a base class and a common implementation class for normal objects. The `RegionOfInfluence` class is used for describing objects, such as light sources, that need to be tested against other objects. The `PhysicalPortal` class is used to describe portals between cells. The `VirtualPortal` class is used to describe connections between portals. Such connections include mirrors, teleports and other special effects.

The interfaces for all `Object`-derived classes can be found from the header file `dpvsObject.hpp`.

6.9.2 Placing Objects Into Cells

In order to be processed in a visibility query, an object must belong to a cell. Objects cannot belong to multiple cells. This cell is defined by the member function `Object::setCell()`. If the application requires that objects can span multiple cells - for example, when an object is crossing through a portal - two separate dPVS objects must be created. The reason why dPVS does not do this automatically is, that unsolvable ambiguities would rise in certain portal cases. The application programmer has more knowledge to solve such problems herself.

6.9.3 Transformation Matrix

The transformation matrix of an object is used to place the object physically into the cell. The matrix should be a 4x3 transformation matrix, i.e. no projection term is allowed.⁹ See documentation for the functions `Object::setObjectToCellMatrix()` and `Object::getObjectToCellMatrix()`.

The transformation matrix can contain arbitrary rotation and scaling terms and non-uniform scaling is supported. Negative or zero scaling terms can cause surprises - as they may affect image-space back-face culling - and should thus be avoided.

6.9.4 Test Models

Each dPVS object must have a test model. This model describes the shape that is used to determine whether the object is visible or not. The test model is initially defined in the object's constructor but can be changed later by calling `Object::setTestModel()`.



Figure 6.9 Example of an extremely bad occluder. The object is very complex and does not occlude much.

6.9 Objects

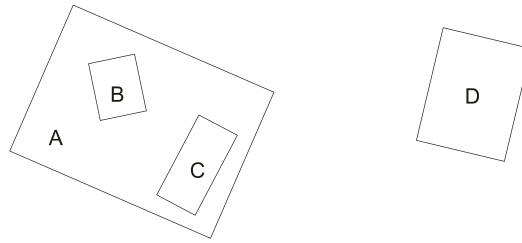


Figure 6.10 Visibility relationships between objects can be expressed using *visibility parents*. Objects *B* and *C* can use *A* as their visibility parent, whereas object *D* cannot.

6.9.5 Write Models

If the object is intended to be used as an occluder, a write model must be assigned to it. This can be done by using the member function `Object::setWriteModel()`. The write model can be the same as the test model - however it may be wise to use conservatively simplified test models and optimized write models. See figure 6.6 for examples of simplified models.

6.9.6 Visibility Parents

Sometimes the user may have *a priori* knowledge about certain visibility relationships between objects. For example, the user may know that a chest is closed and thus objects inside the chest cannot be visible if the chest is not visible. Such visibility relationships can be denoted by using the concept of *visibility parents*. Visibility parents are a generalized method for expressing bounding volume hierarchies. See documentation for the function `Object::setVisibilityParent()` and the figure 6.10 for further details. Note that a visibility child cannot occlude its own parent, since the parent is processed first during the traversal.

Disabling a visibility parent does *not* disable its children.

6.9.7 Temporarily Disabling Objects

Objects can be temporarily disabled by two separate methods: either the object can be moved to a dummy (NULL) cell or the ENABLED flag of the object can be turned off.

Both of the methods achieve the same result: the object will be skipped with zero overhead in the visibility query. The latter approach is recommended because the object will stay in its cell.

This same method can be used for temporarily disabling portals or Regions of Influence.

6.9.8 Object Costs

During its visibility determination process, dPVS performs some cost-benefit analysis to determine which objects should be used as occluders. In order to get a better idea about the actual rendering

⁹The debug build of the library will assert this.

costs of objects, the user can give a hint to dPVS by telling the actual vertex and triangle counts of an object by using the function `Object::setCost()`. dPVS estimates initially the rendering cost of an object from the complexity of the first test model assigned to it. However, this can be an arbitrarily bad estimate - such as in the case of bounding a complicated particle system with a simple box.

The vertex count specified in `Object::setCost()` is used to calculate the expected geometry computation cost. The triangle count is used to compute the expected rasterization cost. The function also takes a *complexity* parameter. How to use this factor is completely up to the user; however it is recommended that value 1.0 be used when rendering normal, single-pass objects. The value 2.0 could then be used to express the cost of objects rendered in two passes.

Usually the user should not assign costs to portals or ROIs based on their model complexities. dPVS internally assigns extremely large cost values for them in order to indicate that a portal or a ROI should always be culled, i.e. that it is worthwhile to attempt to use several occluders to cover a portal or to hide a ROI.

6.9.9 Bounding Volumes of Objects

dPVS calculates and maintains several bounding volumes internally. AABB, OBB and Sphere can be queried from objects (and models) by calling `getAABB()`, `getOBB()` and `getSphere()` respectively. Potential uses include pre-processing for collision detection.

6.9.10 Animated Object Hierarchies

dPVS does not allow specifying transformation hierarchies between objects. This means that if a scene graph has complicated, animated transformation hierarchies, the inherited transformation updates need to be specified manually to dPVS. This is not a problem if a hierarchy is updated infrequently. However, certain hierarchical systems are animated continuously, and updating the transformation matrices of hidden objects would not be output-sensitive. This would slow down the system in very large scenes, where large portions of the environment is hidden to the camera.

The suggested approach for such highly dynamic hierarchical transformations is not to insert the child objects into dPVS at all. Instead, find a common bounding volume for the object group and use that as the test model. The bounding volume can be updated every frame, or it can be a temporal bounding volume that is valid for a certain period of time or until invalidated by object motion. Do not use such objects as occluders. If one part of the hierarchy is a very good occluder, for example the body of a car in a hierarchy consisting of the body and four wheels, use only that part as the write model and ignore the detail objects.

If the object hierarchy has only one parent that is being transformed, another option is to represent the entire hierarchy as a `Cell`. For example, a huge space station or a driving bus full of people could be expressed as a cell. Note that one or more portals need to be set in the enclosing cell so that a traversal can enter the inner one.

6.9.11 Particle Systems

When visualizing complex particle systems, such as explosions or splashing water, it is best to represent the entire system as a single object rather than to insert hundreds of individual objects into

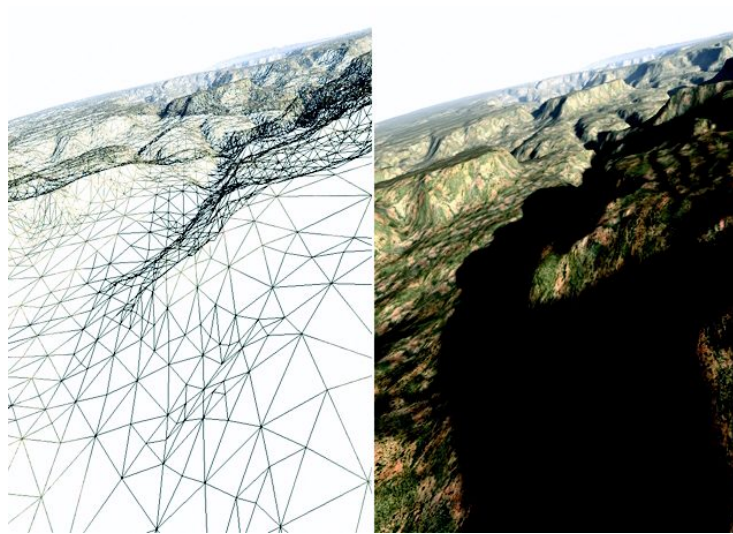


Figure 6.11 The easiest way to represent terrains in dPVS is to subdivide the heightfield into regular-sized blocks and to assign an dPVS Object for each block. In the image above, a 4 million triangle heightfield of the Grand Canyon has been subdivided into tiles of 8x8 quads, i.e. each object has 128 triangles.

dPVS . If the particle system is not expected to significantly occlude other objects, no write model should be defined. See notes above on animated object hierarchies and temporal bounding volumes.

6.9.12 Terrains

The recommended method for representing terrains in dPVS is to split the terrain into a large number of objects. Most modern terrain rendering engines use multi-resolution mesh representations. If a multi-resolution representation is available, conservatively lower-resolution write models can be used as occluders. Unlike many visibility algorithms written specifically for terrains, dPVS can use the 'non-heightfield' portions of the terrain, such as bridges, as occluders.

When splitting a heightfield into multiple objects, it is best to use a common coordinate system for all of the objects, i.e. expressing the vertex coordinates of the models in world-space rather than in local object-space. If this approach is not used, gaps may appear between the objects, producing rendering artifacts and also potentially growing the visible set of objects.

6.9.13 Unbounded objects

If an object has infinite size - for example, it is a sky box or a directional light, this can be indicated to dPVS by specifying the object as **UNBOUNDED**. Such objects are always visible, never used as occluders and have faster ROI vs. object overlap processing. The advantage of using the **UNBOUNDED** flag is that these objects are now placed into the internal spatial database and thus cannot negatively influence the visibility determination process. Also, much fewer intersection tests need to be performed during the ROI-object overlap detection.

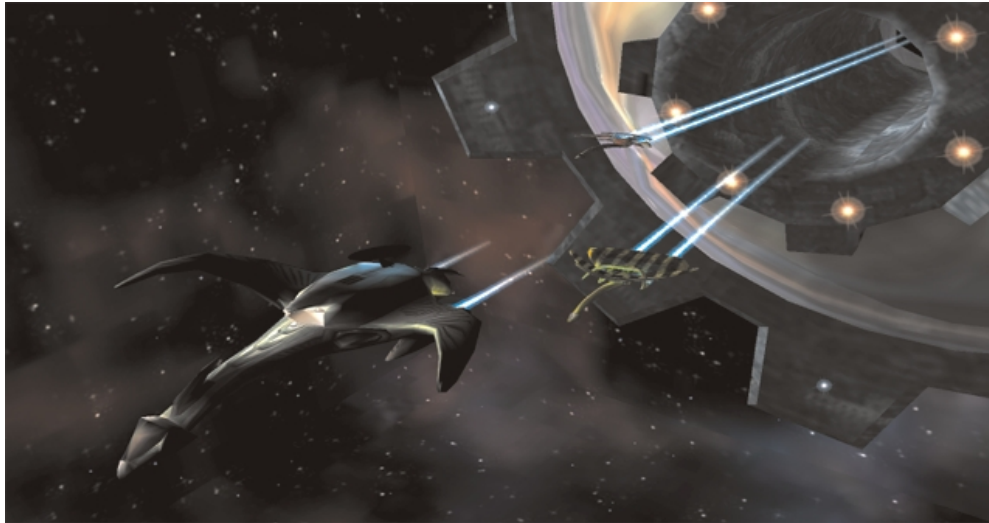


Figure 6.12 Regions of Influence can be used to support massive amounts of light sources. Each bright dot in the image is a separate light source with a small region of influence.

6.10 Regions Of Influence

dPVS supports the concept of *Regions of Influence*, or ROIs for short. Such objects do not participate in the visibility determination process - although they are culled in the same fashion as other objects. Instead the visibility query determines overlaps between the ROIs and other objects. ROIs are typically used for representing light sources. In many real-life scenes light sources have relatively limited attenuation ranges; thus an average object in the world is affected only by a small number of lights, even though the world can contain millions of objects and millions of lights.

ROIs are manipulated exactly like other objects. A transformation matrix and a test model are defined. The test model is used for determining the region of influence - so, a spotlight should be modeled using a model that approximates the spotlight cone. Directional lights, i.e. lights having an infinite volume, should be specified as **UNBOUNDED** - for further information see section 6.9.13.

ROIs are culled exactly the same way as other objects. This means that if a ROI falls completely outside the view frustum, or is occluded by other objects, it is not activated. If one uses ROIs for representing light sources and the rendering engine computes illumination only at vertex granularity - i.e. it has no per-pixel lighting - one should extend all the Regions Of Influence by the length of the longest object edge in the scene ¹⁰ - otherwise slight artifacts can occur. On the other hand, if vertex-only illumination is used, the edges of the objects should not be very long in the first place - the lighting calculations would otherwise be too imprecise, and cause annoying flickering.

ROIs can only affect objects in the same cell. This means that in a scene where there are two cells, *A* and *B*, connected by a portal, a light source in *A* will not affect objects in *B*. If the user would like the ROI to affect objects in cell *B*, a separate copy of the ROI should be placed into cell *B* as well.

All ROIs will be in a deactivated state at the end of a visibility query.

¹⁰This extra radius is called the *safe distance* of the ROI

6.10.1 Bounding Light Sources

The method used for defining the attenuation ranges differs between rendering engines. Many production-quality modeling and rendering packages use the approach where the absolute limits of the light's falloff are expressed by user-defined *near attenuation* and *far attenuation* ranges. This approach has the advantage that the artist can explicitly bound the region of influence of the light source. Typically the falloff curve between these ranges can be modified in order to obtain attenuation that is visually more pleasing, although physically incorrect.

Many real-time rendering libraries have less sophisticated controls. In OpenGL, for example, only the shape of the attenuation curve can be explicitly controlled. The shape is defined using an inverse second-order polynomial. This has the unfortunate property of always producing positive illumination values - a light source can affect objects infinitely far away.

The only method for bounding such infinite lights is to select a threshold illumination value that is considered to be *insignificant*, and calculating the distance where the attenuation falls below this value - objects farther than this distance are then considered unaffected by the light source. Although this approach will produce artifacts, it is really the only option. The larger the threshold value is, the tighter the bounds are. Smaller threshold values produce larger bounds, but cause less artifacts. Depending on the color resolution of the rendering engine and the number of light sources in the scene, good values for the threshold are in the range $1/32$ - $1/256$, assuming that 0.0 corresponds with no illumination and 1.0 with maximum illumination.

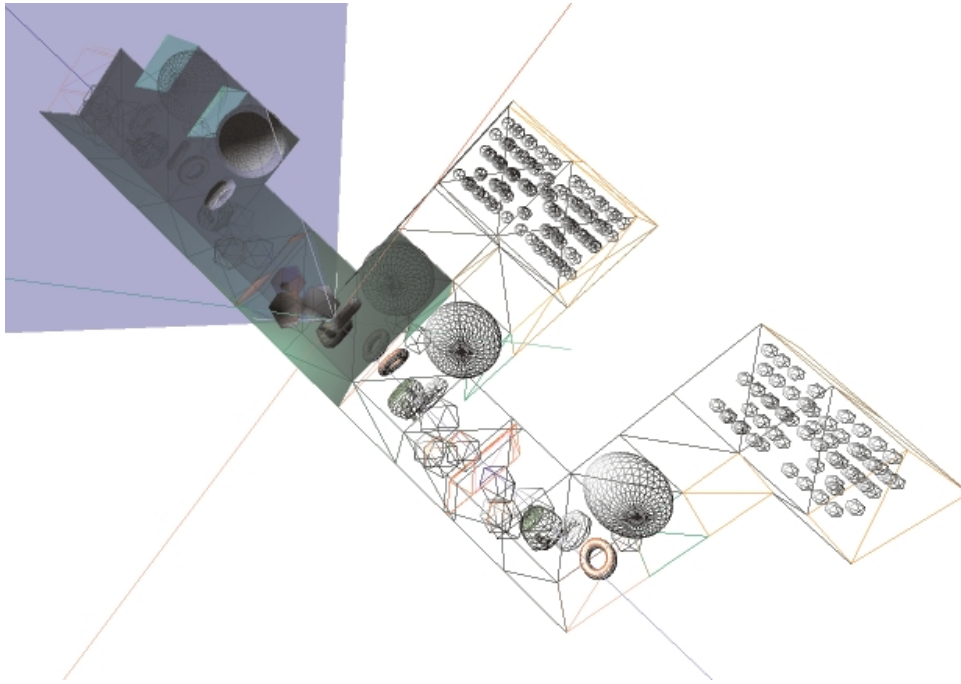


Figure 6.13 The image shows how dPVS renders mirrored objects. The objects drawn in wireframe are culled away either by occlusion culling or view frustum culling.

6.11 Portals

Portals and problems related to them are reviewed in detail in section 10.4.8. Both physical and virtual portal classes are derived from the `Object` class, and can be manipulated with the `Object` member functions. Thus, a portal is placed into a cell by using `Object::setCell()`, it respects the same property flags etc.

Portals are culled using the same culling algorithms that are used for other objects. See figure 6.13 for an example of combined portal and occlusion culling. Real and reflected objects that are visible are drawn normally. Hidden objects and reflections are drawn using wireframe rendering.

6.11.1 What Are Portals and Where to Place Them

Portals are connections between two cells. Whenever the database traversal encounters a visible portal, it creates a new view frustum and continues the traversal to the destination cell.

In an architectural scene cells correspond with rooms and portals with the doors and windows connecting the rooms. If a scene is not naturally divisible into rooms and holes between the rooms, it is likely that portals should not be used.

6.11 Portals



Figure 6.14 Virtual portals can be used for generating planar reflections. The Reflecting Table Demo of dPVS Visualizer shows how recursive reflections can be rendered correctly.

6.11.2 The Difference Between Physical and Virtual Portals

Physical portals are physical, i.e. real-world, connections between cells. Virtual portals are mainly useful for creating special effects, such as mirrors and teleports. Virtual portals are connections between two portals, not cells. The main difference is that the virtual portals can warp the light rays passing through the portal, whereas physical portals mainly act as separators between scene entities. See figure 6.13 for an example of a scene that can be created using virtual portals.

6.11.3 Defining a Physical Portal

To define a physical portal, one must assign the portal to a cell, set the portal's transformation inside the cell, and to assign a test model and a target cell. When a physical portal is determined visible, dPVS first transforms the camera position into world-space and from there into the coordinate system of the target cell.

Note: portals should always be back-face cullable. Be sure to set up the geometry of the portal so that the visible side of the portal can be seen from the source cell.

6.11.4 Defining a Virtual Portal

Creating a virtual portal is more involved than creating a physical portal. First of all, a virtual portal is a connection from portal A to portal B. If a mirror is being created, portal B must also be created. The transformation inside the cell must be set, a test model provided and the target portal defined. Additionally, a *warp matrix* must be set.

6.11.5 Warp Matrices

A warp matrix expresses the coordinate system of the portal surface and is defined in the space of the portal. Since the warp matrices are defined in the portal's space, they automatically rotate along with the portal object. It might initially seem that warp matrices could be automatically extracted from the transformation matrix and the model data of an object. This is true only in limited cases, such as planar portals, and would lead to unsolvable ambiguities in the more generic case. Therefore the user must define the warp matrices manually.

Advanced text follows. Readers without proper understanding of matrix mathematics should skip to [6.11.7](#).

When a virtual portal is determined visible, dPVS executes a rather complex viewpoint transformation:

$$\begin{aligned} \text{CameraToTargetCell} = & \text{CameraToCell1} * \text{Cell1ToPortal1} * \text{Portal1ToWarp1} \\ & * \text{Warp2ToPortal2} * \text{Portal2ToTargetCell} \end{aligned} \quad (6.1)$$

As can be seen from the equation [6.1](#), source and destination warp spaces are set equal and thus no concept of world-space is involved. Some portal engines only use a single transformation matrix to describe the whole process. Even if such an approach is simpler and faster to implement, it is nearly impossible to use! We have selected as generic an approach as possible to allow the widest use of virtual portals.

Creating correct warp matrices is one of the most difficult issues when using dPVS. The warp matrix is an identity transformation, if the portal is modeled into (x,y)-plane and its normal points towards the positive z-axis. If this is not the case, perhaps the easiest way to create a warp matrix (W) would be:

- construct a plane equation of the portal surface.
- select a point T which lies on the portal plane. T should preferably be in the middle of the portal
- assign T to W 's translation
- assign normal of the portal plane to W 's direction-of-flight (DOF) vector
- construct the two remaining axes so that the base is orthogonal. The easiest way is to use cross products
- normalize the base vectors
- optionally apply scaling to the base vectors

6.11.6 Creating a Mirror

- create two virtual portals A and B and set them to a cell
- set the same transformation and test model for both A and B
- set B to A 's target portal and vice versa

6.11 Portals

- disable B
- create warp matrix W of A 's surface
- set W to A 's warp matrix
- negate W 's DOF vector
- set W to B 's warp matrix

6.11.7 A Warning About Virtual Portals

Virtual portals should be mainly used for creating special effects. Moving through a virtual portal should not be encouraged, since several unsolvable issues follow. How should a character walking through a magnifying portal be visualized? Physics systems may lose the concept of *up vector*. Infinite loops might be constructed inside the scene. See section 10.4.8 for a thorough discussion of the problem associated with virtual portals.

6.11.8 Importance Decay

An *importance decay* term can be assigned to a portal. It is internally used for terminating recursion, if the accumulated importance of a portal sequence falls below a user-specified threshold. More importantly, the current accumulated importance value is provided to the user when an object instance is reported visible. For example, if the reported importance is 0.4, there is a good reason for rendering the object using a lower detail level. Usually physical portals should set the importance decay to 1.0, i.e. no decay. Virtual portals, such as mirrors, which often do not require full resolution models, can use a much smaller decay term.

6.11.9 Limiting Recursion

The visibility query function `Camera::resolveVisibility()` takes the maximum recursion depth as a parameter. Mirrors seeing each other in a circular fashion create infinite loops. dPVS solves this issue by calculating the recursion depths and terminating traversal when this limit is exceeded. All portal traversal can be disabled for debugging purposes by setting the recursion depth to 0.

6.11.10 Stencil Models

Optionally, a stencil model can be set to portals. Stencil models require that the rendering engine used supports stencil buffers. Stencil models are required to define arbitrary portals that do not visualize correctly without a stencil buffer. Consult section 10.4.8 for a detailed explanation of the problem. Stencil models are set to portals by calling the function `PhysicalPortal::setStencilModel()`.

6.11.11 Floating Portals

dPVS processes all objects in a cell before traversing through the portals in the cell. Thus if a portable mirror floats in the middle of a cell, special treatment is required. In such a case the depth buffer contains wrong information, and the offending portion must be cleared. This operation causes additional overhead and is such a rare case that it must be explicitly mentioned to dPVS. A portal can be defined to be a *floating portal* by calling `PhysicalPortal::set(Object::FLOATING_PORTAL, true)`. Only portals that define a stencil model should be declared as floating portals. When dPVS reports that it passes through a portal that is declared as floating portal, the depth buffer should be cleared using stencil test.

6.12 Cameras

The Camera class in dPVS is used for defining a virtual viewer in the world. All visibility queries are made through the Camera interface. Multiple cameras can exist in the world simultaneously.

6.12.1 Camera Transformation

The camera must be physically placed into the world by defining the cell where the visibility queries of the camera start from; this is very important because the cells can be connected by virtual portals and therefore no global coordinate system may exist. Additionally, a transformation matrix must be specified. This matrix is used for defining only the position and orientation of the camera, i.e. it is a 4x3 matrix. The view frustum of the camera is defined separately. The camera transformation matrix can be set by using the member function `Camera::setCameraToCellMatrix()` - this will define the transformation of the camera in relation to the current cell. The camera is placed into a cell by `Camera::setCell()`.

6.12.2 Camera View Frustum

The view frustum of the camera is used for determining the portion of the world that the camera can see. The frustum is defined by six coordinates, two of which define the *near* and *far* (hither and yon) plane distances, the latter four defining the horizontal and vertical fields of view. The frustum is assigned to the camera using the member function `Camera::setFrustum()`. Note that the frustum definition is identical to the one used in OpenGL's `glFrustum()` and `glOrtho()` functions, except that the Z-axis is flipped.

dPVS cameras support two kinds of projection: *perspective* and *orthographic*. The type of projection used is selected by setting the member variable `type` in the `Frustum` structure to either `Frustum::PERSPECTIVE` or `Frustum::ORTHOGRAPHIC`. Orthographic views can be useful for visualizing floor plans or CAD models.

6.12.3 Adjustments for Different Pixel Centers

Some rendering engines and hardware do not agree with the convention of OpenGL for defining the *pixel center*. If dPVS performs its internal image-space culling with a different pixel center convention than the user's rendering engine, slight artifacts can occur. The pixel center used by dPVS can be defined by using the member function `Camera::setPixelCenter()`.

6.12.4 Screen Properties

Some of the occlusion culling algorithms of dPVS operate in image-space. For this reason the final output resolution of the image must be specified to the camera. Also, some information returned by `Commander` callbacks are expressed in raster-space coordinates. The screen resolution used can be defined by the member function `Camera::setParameters()`.

Also, the culling methods used by dPVS can be selected with the `Camera::setParameters()` call. By default, all culling methods are enabled. However, for experimental or debugging purposes,

some of them can be turned off.

6.12.5 Image-Space Sub-sampling

By default, the image-space culling is pixel-perfect and conservative. This means that the image rendered with occlusion culling turned on and the one without occlusion culling are pixel-exact. Sometimes the user may want to accelerate the occlusion culling process, and to speed up the rendering by allowing slight artifacts. This can be done, for example, when rendering extremely high-resolution images. Also many games programmers value the speedup gained over the slight artifacts produces. This is achieved by defining image-space scaling values in the `Camera::setParameters()` calls that are smaller than 1.0. If the screen resolution is defined to be 640x480 and the image-space scaling values are (.5,.5) dPVS will internally perform the occlusion culling using buffers of resolution 320x240.¹¹ Such sub-sampling has the additional side effect that objects that are just *barely visible*, i.e. only a few pixels of the object would be drawn, *may* be culled out.

6.12.6 Screen-Size Culling

dPVS supports a non-conservative culling method called *screen-size culling*. This means that objects - as well as ROIs and Portals - having an axis-aligned screen space bounding rectangle smaller than a user-specified threshold, can be culled away in the visibility query. This threshold can be specified with the member function `Camera::setObjectMinimumCoverage()`. The threshold specified is only a hint and some objects smaller than the threshold size may be reported to the user as being visible.¹²

Because screen-size culling causes rendering artifacts, the default minimum coverage is set to (0,0) in the constructor of the `Camera` indicating that screen-size culling is disabled.

6.12.7 Static Coverage Masks and Depth Buffers

Some applications have a fixed viewpoint where objects move in a pre-rendered scene. In some other cases, a fixed part of the display is always covered by opaque menus or head-up displays. Such information can be passed to dPVS by using the member functions `Camera::setStaticCoverageMask()` and `Camera::setStaticZBuffer()`. dPVS will combine this pre-calculated information into its dynamic occlusion buffers. The coverage mask and the depth buffer can be provided in any pixel resolution, dPVS will internally convert it to match the current image resolution. The conversion takes some time, so these buffers should not be updated for every frame.

The camera does not take a copy of the user-provided static buffers. If the output resolution is changed by a call to `Camera::setParameters()`, the static buffers need to be redefined by the user.

¹¹The same effect could be achieved by creating the buffer in 320x240 resolution in the first place. This approach would however suffer from the drawback that some `Commander::command()` callbacks return coordinates in raster-space. The user would then have to manually map these lower-resolution raster coordinates back to the full-resolution values.

¹²Not all of the culling routines calculate the bounding rectangle.

6.12.8 Visibility Queries

dPVS supports two kinds of visibility queries: traversal queries and immediate queries. The former is the recommended method for performing visibility queries. In such a query, dPVS will start a spatial database traversal from the camera, perform visibility tests internally, traverse through visible portals and eventually report all visible objects to the user. Such a query can be made by calling the member function `Camera::resolveVisibility()`. This kind of a query supports virtual portals and resolves object-ROI overlaps.

The immediate queries are available only after a traversal query has been made. These queries can report if a point or a rectangle defined in raster-space is occluded by the occluders used in the last traversal query. The usefulness of these queries is quite limited. They should be used only for certain special effects, or for testing the visibility of objects that cannot be physically placed into the scene.

In order to make a traversal query, a `Commander` object must be created. dPVS reports the visible objects, their transformation matrices, ROI activation/deactivation information etc. through the `Commander` object. As long as the user responds properly to all commands, the correct image is rendered. Note that dPVS handles all calculations needed by the reflective and refractive mirrors, tells when to apply stencil masks, and when to clear portions of the depth buffer.

6.12.9 Visibility Query Traversal Order

dPVS performs the visibility query traversal internally as follows. The traversal starts from the cell where the camera is located. dPVS collects all visible objects, ROIs and portals when sweeping through the cell. After that, it proceeds through the visible portals in depth-first order. When the traversal finally ends, dPVS starts reporting events through the `Commander` object specified by the user. The intra-cell ROI vs. object overlap testing is done during this reporting.¹³

This means that changes to the database in `Commander` callbacks will not affect the results of the visibility query in any way. Making any modifications - such as creating new objects or deleting existing objects - during the callbacks is an extremely bad idea, and should not be done. Such modifications should be performed after the `Camera::resolveVisibility()` function returns.

6.12.10 Scout Cameras

One can improve the occluder selection process and smoothen on-demand loading by using *scout cameras* (see figure 6.15). A scout camera is either the normal camera used for visibility queries or a separate camera dedicated for scouting. The scout camera is used for occasionally making a visibility query at a predicted future camera position and orientation. The optimal frequency for scout queries varies, but something like one query every 1-2 seconds would probably be enough. The location and orientation of the future camera position can be extrapolated by using the current physics engine parameters, such as the velocity and the angular velocity of the camera. The prediction moment should be approximately the same as the scout query frequency, i.e. if queries are made once every second, the prediction moment should be one second ahead of current time. Another option is to monitor how well the predictions work in practice, and to make scout queries whenever the camera does not behave as predicted.

If a separate camera is used for scouting, a smaller screen resolution can be used as the scout does not

¹³Remember that ROIs can affect only the objects located in the same cell.

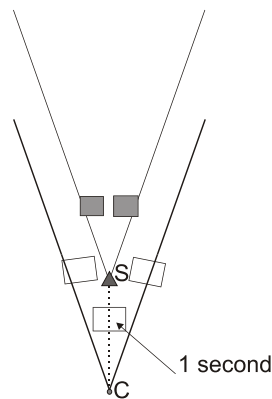


Figure 6.15 Scout cameras can be used for prefetching portions of the object database from the disk just before the objects become visible. Objects drawn in gray are hidden to the camera *C*, but will become visible when the camera moves forward. The scout camera at point *S* will see the objects and load them from the disk before the camera reaches the point *S*.

need to produce conservative results. Also, a dummy Commander object that does not do anything should be used for the scout query to avoid processing the callbacks. The SCOUT flag of the camera can be turned on to indicate that the camera is used for scouting.

6.12.11 Balancing the Frame Rate

When occlusion culling is used, the rendering frame rate is highly dependent on the number of visible objects, which can vary from frame to frame. The library user can attempt to balance the frame rate by performing additional non-rendering work during frames that are fast to render. The OPTIMIZE flag of the camera is provided for exactly this purpose. During frames when the flag is enabled, visibility queries perform additional work that can speed up future more expensive frames. Such work includes enhanced occluder benefit analysis, virtual occluder determination, spatial database reorganization and memory optimization.

6.12.12 Shadow Pre-processing

The visibility query mechanism of dPVS can be used as a pre-processor for dynamic shadow computation systems. All of the dominating point light shadow algorithms, the *shadow depth buffer*, the *shadow frustum* algorithm and the *projection shadow* algorithm, require finding the objects visible from the viewpoint of the light source. By placing an dPVS camera to the position of the light source, and making a visibility query, a tight list of the objects that can contribute to the shadow volume can be obtained. Note that several cameras might be needed to properly model point light sources.¹⁴ Most spot lights can be modeled with a single camera.

Future versions of dPVS will support additional performance optimizations for static cameras and situations where most of the objects are static. These optimizations will be purely internal and

¹⁴The camera model in dPVS supports field-of-view angles up to 180 degrees. However, very wide angles cause distortions in the image-space algorithms. Therefore it is recommended that a point light source is modeled using six separate 90-degree views.

6.12 Cameras

directly enhance the setup speed for dynamic shadow computations.

Chapter 7

Optimizing the Performance of dPVS

This chapter deals with several topics that are related to improving the run-time performance of dPVS and conserving memory resources. These topics are quite advanced and not necessarily interesting for somebody who has just started using the library.

7.1 On-Demand Loading and Garbage Collection

On-demand loading means that objects are inserted into the database only when they become visible or near-visible - they do not even have to be loaded into the system memory before that. A simple method is to create a spherical volume around the camera and to load objects when their bounding volumes first overlap this sphere. The radius of the sphere should be slightly larger than the longest distance the camera can see.¹ The larger the sphere, the less often the overlap checks have to be made.

If the number of objects falling into such a view sphere is still too high, as in the case of a 100-story building, dPVS can be used for finding the objects that become visible. A volume in space containing many objects can be inserted into the dPVS database. Until this *container volume* becomes visible, the contained objects do not need to be loaded. When dPVS reports that the volume has become visible for the first time, the objects can be loaded from the disk, inserted into the database of dPVS, the container volume destroyed or disabled, and a new visibility query made.

The different actions that can be performed when a container volume first becomes visible are:

- deciding that the objects are not visible during the first frame. This way the objects can be loaded as a background job and inserted into the database only once the loading has finished. This approach causes slight artifacts - however the artifacts can be lessened by somewhat enlarging the original container volume.
- inserting the objects into the database and performing the visibility query again. Because neither the camera nor most of the scene objects have moved between the queries, this second query has much coherence and proceeds faster than the original query.²

¹This is not the same as the distance to the far clip plane. It is the distance to the farthest vertex in the view frustum.

²Do not insert the objects into the scene during the `Commander` callback. See section 6.12.9 for details.

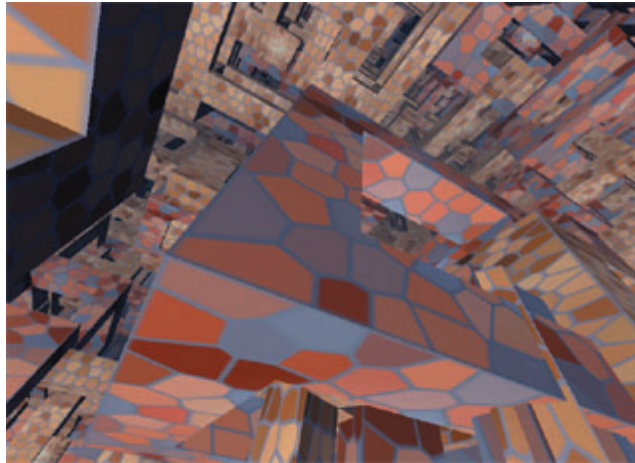


Figure 7.1 The database in the picture above contains two million objects. All models have 108 triangles and 258 vertices, thus pumping the total triangle count to 216 million. The database file requires 130Mb of disk space. The memory used by the visible portion of the database (and other dPVS allocations) is in the order of a few megabytes. The database is not static: 120,000 of the objects are moving around every frame. Objects in the database are on-demand loaded just before they become visible. The demonstration is part of dPVS Visualizer demo package.

- inserting the objects into the database and deciding that all of them are visible for the duration of the first frame
- inserting the objects into the database and performing immediate mode visibility queries to determine whether they are visible during the first frame

Note that *scout cameras* can be used to do the opening of container volumes in advance, so that the file I/O can be performed as a background process. This will increase the chances that the objects have already been loaded when the container volume really becomes visible, and a stall can be avoided.

Garbage Collection

When the camera has continued its traversal through the scene for a longer period of time, certain objects will have become and stayed hidden for a while. Such objects can be removed from the dPVS database to conserve memory resources, and can be collapsed back into a container volume. The easiest way to determine objects that have not been visible for a certain period of time is to use the `Library::suggestGarbageCollect()` function. This function performs quite a bit of work, so it should not be called too often; several times per minute or even less frequently should be enough.

7.2 Integrating Potentially Visible Sets

In some cases, one may have pre-calculated *Potentially Visible Sets* (PVS) [4] of objects for different portions of the scene. Such sets may have been calculated by geometric means, or by sampling the

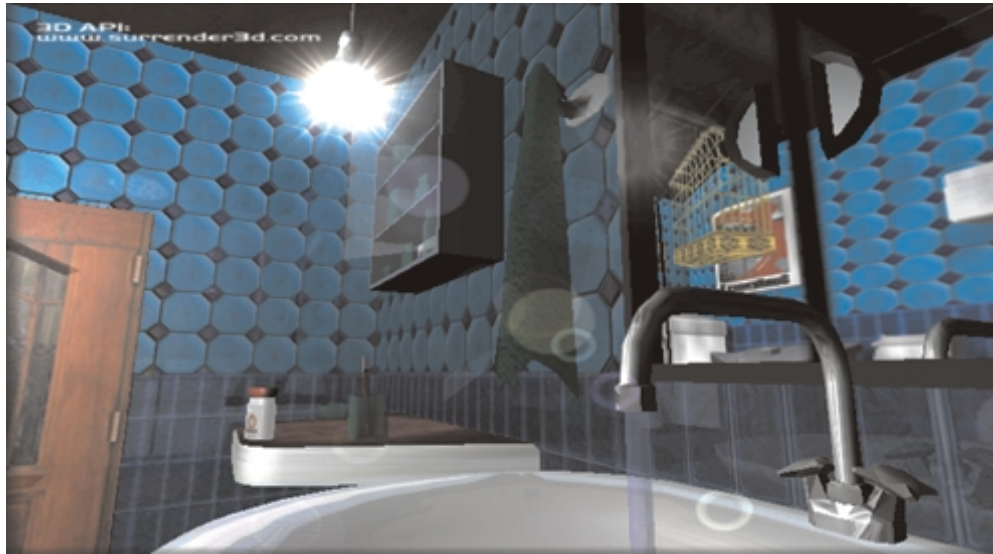


Figure 7.2 The picture shows how reflections can be rendered using image-space methods. The reflections shown on the walls and the mirror are actually the inside of a texture-mapped box. The glossiness effect is implemented by texture filtering.

scene with either a renderer or dPVS . The easiest way to provide the PVS information to dPVS is to temporarily disable the objects *not* contained in the current PVS. The easiest and most efficient way of doing this is clearing the `ENABLED` flag of the hidden objects. Objects that do not have the `ENABLED` flag set are skipped during the visibility query. This skipping is done at zero cost - it is therefore equally fast to process a scene with a 100,000 disabled objects than it would be if the disabled objects were not inserted into the scene. Entire cells can also be disabled by clearing `ENABLED` flag from them.

7.3 Image-Based Rendering

Image-based rendering (IBR) is a rendering technique that converts portions of the scene geometry into images, i.e. *impostors*, and uses these images instead of the real geometry during rendering. See section 10.5.3 for an overview of impostor techniques. Using impostors can greatly improve the rendering speed in scenes where the number of visible objects is high. Impostors can reduce the work that needs to be done after dPVS has been run. On the other hand, dPVS can reduce the work needed for creating the impostors. Visibility determination and image-based rendering are both needed in order to render arbitrarily complex scenes.³

A few things that are good to keep in mind when using impostors in conjunction with dPVS :

- the visibility of impostors can be tested exactly like the visibility of the real geometry
- if the impostor image is completely opaque, i.e. it does not contain any transparent pixels, it can be used as an extremely good occluder. Use a write model consisting of a single quad

³The upcoming product dPVS Impostor will provide automatic impostor support for rendering engines.

- impostors can be easily combined with portals, i.e. the geometry seen through a portal is collapsed into an image. The portal object is then disabled and a planar model is displayed in its place [7]
- if virtual portals, such as mirrors, are represented using impostors, the image data does not necessarily need to be updated for every frame. See figure 7.2 for an example of such portals
- interesting special effects can be done by post-warping impostor images. Reflections from curved surfaces can be simulated by using six portals located around an object combined with cubic environment mapping techniques. Non-linear lens distortions can be simulated by texture coordinate manipulation.
- if 2D manipulation of the image data is possible, effects such as glossy reflections can be performed

7.4 Multi-Threading Renderers

Some high-end rendering engines support multi-threaded scene traversal. This can significantly improve performance when using multi-processor machines. Performance improvements can be achieved even on single-processor machines, if the rasterization is performed using a separate rendering hardware. Such an engine is typically organized in the form APP-CULL-DRAW⁴, where the rendering pipeline has been divided into separate stages; multiple frames are rendered simultaneously by processing concurrently different pipeline parts for different frames. The workload of dPVS falls mainly into the CULL category. Some of the hidden-surface removal is usually performed in the DRAW stage, i.e. primitive-level back-face culling and pixel-level visibility determination.

Because the code of dPVS is not protected by any mutexes or similar mechanisms, no parts of dPVS should be accessed simultaneously by multiple threads.⁵ The responsibility for synchronizing this is left to the application programmer. There are two simple ways to achieve this: either let only a single thread access dPVS or create a single mutex and synchronize all accesses to dPVS using it.

Note that functions such as `Camera::resolveVisibility()` can perform major modifications to the internal databases. This means that it is not even safe to call the constructor of an `Object` while another thread performs a visibility query.

On platforms that support multiple threads, dPVS is linked to use thread-safe versions of the shared C/C++ run-time libraries. This means that dPVS does not internally perform operations that could conflict with the user's code.

7.5 Memory Usage

Although the actual memory usage of dPVS may vary quite a bit, and is subject to changes in future versions, below are some basic guidelines and information about how much dPVS uses memory:

- each `Camera` will allocate ~3 bits for each screen pixel specified in the `Camera::setParameters()` call

⁴Application-specific (i.e. database updates, physics), culling and rendering phases

⁵This design decision was mainly made for performance reasons. dPVS uses internally many lazy evaluation techniques and synchronizing them internally would be quite expensive. It is likely that the user can get a better performance by using more coarse-grained synchronization methods.

7.5 Memory Usage

- if static coverage buffers or Z-buffers are used, an internal resampled copy is kept. The memory usage is approximately 1 bit per output resolution pixel.
- `MeshModels` take a copy of the data submitted to them when they are constructed. Each vertex will consume 12 bytes and each triangle 12 bytes.
- each `Object` consumes approximately ~200 bytes of memory. This figure will be optimized in future releases of the library.
- the spatial database memory allocations are sublinear to the object memory usage. In most cases the spatial database memory usage is less than 10% of the memory consumption of the objects.
- the silhouette cache constantly attempts to conserve memory. Its absolute peak usage has been set to 256 kB
- the debug build of the library uses slightly more memory than the release build

The following methods can be used to bring the memory usage down:

- using simpler models
- sharing models between objects
- using on-demand loading and garbage collection
- destructing cameras when they are not used for a while
- calling `Library::minimizeMemoryUsage()`

Future versions of dPVS will support further lazy allocations of object data. This means that portions of the internal object structure will be allocated only for visible and near-visible objects, and memory is released when an object has been hidden for a short while. In a similar fashion, internal model-specific resources will be temporarily released when a model has not been accessed for a while.

The amount of memory used by dPVS can be queried with the `Library::STAT_MEMORYUSED` statistic.

Releasing Temporary Allocations

dPVS uses various internal caches and temporary *recyclers* in order to avoid unnecessary memory allocations and to speed up computations. When memory resources are scarce, the user can reclaim the memory used by such caches by calling `Library::minimizeMemoryUsage()`.

7.6 Performance Optimization Checklist

Because dPVS is a highly heuristic and self-optimizing system, it is impossible to give exact rules for performance optimization. Below is a listed set of very broad guidelines on how certain events can affect the performance and the quality of the library.

General Guidelines

- do not pass any non-modified parameters to dPVS . For example, if an object does not move, do not re-specify its transformation matrix. In a similar fashion, do not set the camera's parameters for every frame. dPVS checks some of such changes internally, but it is still better to avoid making the unnecessary calls in the first place
- use on-demand loading and garbage collection to keep the memory footprint as small as possible
- all temporary memory allocations can be released by calling `Library::minimizeMemoryUsage()`
- dPVS aggressively exploits all kinds of coherence - thus the more coherent your camera motion, object motion, scene layout etc. is, the faster the visibility queries are
- although all functions that accept matrices have both single- and double-precision variants, try to use the single-precision versions whenever possible
- the statistics provided by the `Library` class can be used for analyzing the reasons for performance problems

Cameras

- the view volume can be limited by bringing the back-clipping (far) plane closer. Fog effects can be used to make this transition smoother. Backdrop imagery can be used to impose as the far-away geometry
- sub-sampling can be used for reducing image-space occlusion culling work and, as a side-effect, also for culling out some barely visible objects. See section 6.12.5 for further details
- contribution and screen-size culling (section 6.12.6) can be used for discarding objects that contribute very little to the final image
- if a portion of the screen is covered by menus or some other opaque surface, a static coverage mask can be defined (section 6.12.7)
- if the camera has a fixed transformation matrix, a static depth buffer can be defined (section 6.12.7)
- both camera velocity and angular velocity are inversely related to temporal coherence - the slower the camera moves, the faster the visibility queries are
- avoid using the immediate mode visibility queries if possible
- some culling systems, such as portal scissor rectangles, are raster-space axis-aligned. Rolling the camera will make these rectangles larger and thus cause more objects to be reported visible

7.6 Performance Optimization Checklist

- if the rendering frame rate is uneven, i.e. jerky motion occurs, the `OPTIMIZE` flag can be used to balance the frame rate
- if the camera is used for scouting, enable its `SCOUT` flag

Models

- try to keep the test models relatively simple, but tight-fitting
- remember that most of the time dPVS operates only with the convex hulls of the test models; thus specifying inconcave test models can be a waste of memory resources
- remove unnecessary detail from write models. For example, a computer keyboard can be represented as a solid box unless you are looking from the viewpoint of an ant crawling on the keyboard (section 6.8.4)
- even small holes in objects can turn a good occluder into a bad one. Make sure your models are properly welded, i.e. no cracks exist between triangles due to numerical inaccuracies or poor modeling
- if your model is near-planar, i.e. there are view directions from which the model is completely back-facing, enable back-face culling for the model (section 6.8.5)
- if the model is definitely a bad occluder, do not assign it as a write model. This will ensure that the model is never used as an occluder
- if simple portions of a model can hide more complex portions of the model, split the model into several parts. An example of this could be a computer monitor where the outside parts are relatively simple, but internal parts are quite complicated. Keep in mind that dPVS reports visibility only at object/model granularity (section 6.8.2)
- if your model contains a high number of primitives, try splitting it into multiple parts (section 6.8.2)

Objects

- if your objects are huge and almost always visible, avoid placing them into the dPVS database. For example, the sky in an outdoors environment could be such an object. Alternatively, if the object is very large and only portions of it are usually visible, it can be split into multiple objects. Such splitting can help if the rendering is bound by the fill rate
- share models between objects as much as possible. Remember that you can assign different transformation matrices for each individual object. Different objects can share a test model as long as the test model is conservative for all of them
- if you know that an object will be an extremely bad occluder, do not assign a write model for it
- if you have *a priori* knowledge that an object is going to be hidden, tell dPVS about it by disabling the object temporarily by clearing its `ENABLED` flag (section 6.9.7)
- if you have knowledge about the visibility relationships of the objects - i.e. if A is hidden, then B is hidden as well, pass this knowledge to dPVS by using the *visibility parent* mechanism (section 6.9.6)

- give dPVS better knowledge about rendering costs by using the `Object::setCost()` method (section 6.9.8)
- moving an object in the database has associated costs. If the transformation matrix of an object is updated several times between two visibility queries - perhaps a physics engine runs multiple iterations per each rendered frame - perform the update to dPVS only once. If the object does not move, do not make the update at all
- some of the internal data organizations prefer axis-aligned data
- try to model complex dynamic systems such as particle systems only with a single or with very few objects (section 6.9.11)
- far-away dynamic objects can be represented using *temporal bounding volumes* (TBVs), i.e. a bounding volume that is valid for a longer period of time. This way the number of object transformation updates to dPVS can be reduced. Note that dPVS calculates TBVs internally as well. However, its motion prediction capabilities are more limited than those of the user, as the user can have additional information from the underlying physics engine
- note that all these guidelines apply to portals and ROIs as well
- if your rendering engine benefits from disabling primitive-level clipping when an object does not clip the view frustum, query the current clip mask by calling `Commander::Instance::getClipMask()`
- avoid unnecessary use of `REPORT_IMMEDIATELY` and subsequent write model changes.

Portals

- if two cells are connected by multiple portals, and the portals are very close together - say five windows next to each other - the portals should be combined into a larger portal instead
- when using virtual portals, the recursion depth parameter of `Camera::resolveVisibility()` can be used for controlling how many times the traversal can proceed through each portal (section 6.11.9)
- if portals act as filters - for example, when looking through a colored window - the importance decay values (section 6.11.8) can be used for allowing early termination. The importance decay values can also be used with normal portals for limiting the traversal depth
- try to keep models used for portals as simple as possible. For example, a round window between two cells should not be modeled with a 100-vertex circle - instead a conservative pentagon or a hexagon should be used
- floating portals introduce an additional clipping plane. This can slow down the rendering engine in some cases. The clip mask of the instance may be queried from `Commander` to find out whether the clipping plane is needed for an object.

Regions Of Influence

- try to make your Regions Of Influence as small as possible
- if you have global Regions Of Influence, i.e. ROIs that will certainly affect almost all of the objects in the world, do not insert them into dPVS at all - they will only unnecessarily slow down the ROI vs. object overlap evaluation

Part II

Application Programming Interface

Chapter 8

API Reference

This is the Reference Guide for the Application Programming Interface of dPVS . It contains short descriptions for each class, enumeration and member function of dPVS .

The classes are represented in an alphabetical order. Member functions and enumerations inside each class are organized as follows:

- enumerations in alphabetical order
- constructors
- other member functions in alphabetical order

All classes are wrapped inside the namespace DPVS; however, for the sake of clarity, the API reference omits the namespace prefix, i.e. `Object::setTestModel()` actually means `DPVS::Object::setTestModel()`.

Classes that are reference-counted do not have publicly accessible destructors, copy constructors or assignment operators. These functions are not documented in the API reference guide.

8.1 Camera

C++ SPECIFICATION

```
class Camera
```

DESCRIPTION

Class for representing a virtual viewer

BASE CLASSES

ReferenceCount

HEADER FILE

```
dpvsCamera.hpp
```

NOTES

The Camera class is used for determining visible objects in a scene. The camera defines the position and orientation of the viewer and the shape of the view frustum. The camera interface is also used to select culling methods used by the visibility query.

An overview of the Camera class can be found from section [6.12](#).

Because the class is derived from ReferenceCount, it does not have a public destructor. Use the function `ReferenceCount::release()` to release instances of the class.

PUBLIC INTERFACE

```
enum Property  
create()  
getCameraToCellMatrix()  
getCameraToWorldMatrix()  
getCell()  
getFrustum()  
getHeight()  
getObjectMinimumCoverage()  
getPixelCenter()  
getProperties()  
getScissor()  
getWidth()  
isPointVisible()  
isRectangleVisible()  
resolveVisibility()  
setCameraToCellMatrix()  
setCell()  
setFrustum()  
setObjectMinimumCoverage()  
setParameters()  
setPixelCenter()  
setScissor()  
setStaticCoverageMask()  
setStaticZBuffer()
```


C++ SPECIFICATION

```
enum Camera::Property
```

DESCRIPTION

Enumeration of culling methods used by visibility queries

VALUES

VIEWFRUSTUM_CULLING	Enables view frustum culling
OCCLUSION_CULLING	Enables occlusion culling
DISABLE_VIRTUALPORTALS	Disables traversal through any virtual portals
SCOUT	Indicates that the camera is being used as a <i>scout</i>
PREPARE_RESEND	Prepares for RESEND.
RESEND	Resend the cached results of the previous query (requires PREPARE_RESEND)
OPTIMIZE	Allow visibility query to spend additional time for optimizing future queries

NOTES

These enumerations are used when constructing the property mask for the API calls `Camera::setParameters()` and `Camera::getProperties()`.

If the camera is used purely for scouting, the SCOUT flag should be set. This flag affects the internal occluder selection algorithms and some other sub-systems.

The RESEND flag can be used when the results of a visibility query are needed multiple times. Once the RESEND flag is turned on, subsequent calls to `Camera::resolveVisibility()` will perform no internal visibility queries. Instead the results of the last query defining PREPARE_RESEND are sent to the user. Notice that RESEND must always be prepared using PREPARE_RESEND.

The VIEWFRUSTUM_CULLING and OCCLUSION_CULLING flags are coupled so that if occlusion culling is enabled, view frustum culling is automatically enabled as well.

The DISABLE_VIRTUALPORTALS flag is mainly used for debugging. If the flag is enabled, the visibility query will never traverse through a virtual portal.

The OPTIMIZE flag is used for combatting uneven frame rates. When occlusion culling is used, the rendering frame rate is highly dependent on the number of visible objects, which can vary from frame to frame. The library user can attempt to balance the frame rate by performing additional non-rendering work during frames that are fast to render. The OPTIMIZE flag is provided for exactly this purpose. During frames when the flag is enabled, visibility queries perform additional work that can speed up more expensive frames in the future. Such work includes enhanced occluder benefit analysis, virtual occluder determination, spatial database reorganization and memory optimization.

SEE ALSO

`Camera::getProperties()`, page 86

`Camera::setParameters()`, page 96

C++ SPECIFICATION

```
static Camera* Camera::create()
```

DESCRIPTION

Constructs a new camera

RETURNS

Pointer to the camera or NULL on failure

NOTES

When a camera is constructed, its transformation matrix is initialized to identity, the view frustum is non-existent, the camera is not placed into any cell and the screen parameters have not been defined. Therefore the following steps should be taken after the constructor has been called:

1. Assign the camera into a cell by calling `Camera::setCell()`
2. Call `Camera::setCameraToCellMatrix()` to assign a transformation matrix to the camera
3. Define a view frustum by calling `Camera::setFrustum()`
4. Call `Camera::setParameters()` to define the screen resolution and culling methods
5. Optionally call `Camera::setStaticZBuffer()` to define a pre-computed depth buffer
6. Optionally call `Camera::setStaticCoverageMask()` to define a pre-computed coverage mask
7. Optionally call `Camera::setObjectMinimumCoverage()` to enable screen-size culling

The only reason this function can fail is that there is not enough memory in order to allocate the camera.

C++ SPECIFICATION

```
void Camera::getCameraToCellMatrix(Matrix4x4& mtx) const  
void Camera::getCameraToCellMatrix(Matrix4x4d& mtx) const
```

DESCRIPTION

Returns the camera-to-cell transformation matrix

PARAMETERS

mtx Reference to matrix structure where the result is stored

NOTES

The matrix format used has been defined in the call to `Library::init()`. The matrix has been initialized to identity in the constructor.

SEE ALSO

`Camera::setCameraToCellMatrix()`, page 92

C++ SPECIFICATION

```
void Camera::getCameraToWorldMatrix(Matrix4x4& mtx) const  
void Camera::getCameraToWorldMatrix(Matrix4x4d& mtx) const
```

DESCRIPTION

Returns the camera-to-world transformation matrix

PARAMETERS

mtx Reference to matrix structure where the result is stored

NOTES

This function calculates the camera-to-world transformation matrix. The matrix is calculated by multiplying the camera-to-cell matrix by the cell-to-world matrix. If no cell has been defined for the camera, the cell-to-world matrix is assumed to be an identity matrix.

The matrix format used has been defined in the call to `Library::init()`.

SEE ALSO

`Camera::getCameraToCellMatrix()`, page 79

`Camera::setCell()`, page 93

`Cell::setCellToWorldMatrix()`, page 107

C++ SPECIFICATION

Cell* Camera::getCell() const

DESCRIPTION

Returns pointer to cell where the camera resides

RETURNS

Pointer to camera's cell

NOTES

If the camera has not been assigned to any cell, NULL is returned. Note that such cameras cannot be used for performing visibility queries.

SEE ALSO

Camera::setCell(), page 93

C++ SPECIFICATION

```
void Camera::getFrustum(Frustum& frustum) const
```

DESCRIPTION

Returns the view frustum of the camera

PARAMETERS

frustum Reference to Frustum structure where the result is stored

NOTES

The function returns the view frustum defined by the last `Camera::setFrustum()` call. Note that it does not return the *current* view frustum (the one created by a portal sequence) during a traversal. To obtain that frustum, use the function `Commander::Viewer::getFrustum()`.

The `type` field of the frustum returned indicates the projection type (either perspective or orthographic).

SEE ALSO

`Camera::setFrustum()`, page 94

`Commander::Viewer::getFrustum()`, page 133

C++ SPECIFICATION

```
int Camera::getHeight() const
```

DESCRIPTION

Returns height of the screen

RETURNS

Height of the screen in pixels

NOTES

The screen width and height have been determined by the previous call to `Camera::setParameters()`. If no screen has yet been defined, the return value is zero.

SEE ALSO

`Camera::getWidth()`, page 88

`Camera::setParameters()`, page 96

C++ SPECIFICATION

```
void Camera::getObjectMinimumCoverage(float& width, float& height,  
float& opacity) const
```

DESCRIPTION

Returns the minimum object coverage values assigned to the camera

PARAMETERS

<i>width</i>	Reference to float where to store the horizontal minimum coverage value
<i>height</i>	Reference to float where to store the vertical minimum coverage value
<i>opacity</i>	Reference to float where to store the opacity threshold

NOTES

This function returns the minimum coverage values defined by the previous call to `Camera::setObjectMinimumCoverage()`. These values define an area for *screen-size culling*. Objects having raster-space bounding boxes smaller than the area specified may be culled by dPVS during a visibility query.

For further information about *screen-size culling*, see section [6.12.6](#).

The minimum coverage values are initialized to (0,0) in the camera constructor, i.e. no screen-size culling is performed by default.

SEE ALSO

`Camera::setObjectMinimumCoverage()`, page 95

C++ SPECIFICATION

```
void Camera::getPixelCenter(float& xoffset, float& yoffset) const
```

DESCRIPTION

Returns pixel center used by the rasterizer

PARAMETERS

<i>xoffset</i>	Reference to float where to store the horizontal offset
<i>yoffset</i>	Reference to float where to store the vertical offset

NOTES

This function returns the pixel center used by the internal rasterizer. The default value is (0.5,0.5) and can be changed by calling `Camera::setPixelCenter()`.

This function was introduced in version 2.3.

SEE ALSO

`Camera::setPixelCenter()`, page 97

C++ SPECIFICATION

```
unsigned int Camera::getProperties() const
```

DESCRIPTION

Returns property mask of the camera

RETURNS

A bitmask containing culling properties of the camera

NOTES

The property mask has been defined by the previous call to `Camera::setParameters()`. The bitmask contains an ORed mask of `Camera::Property` enumerations.

The following code can be used to find out if occlusion culling has been enabled:

```
printf ("occlusion culling: %s", (camera->getProperties() &  
Camera::OCCLUSION_CULLING) ? "enabled" : "disabled");
```

SEE ALSO

`Camera::Property`, page 77

`Camera::setParameters()`, page 96

C++ SPECIFICATION

```
void Camera::getScissor(int& left, int& top, int& right, int& bottom)
const
```

DESCRIPTION

Returns the scissor rectangle

PARAMETERS

<i>left</i>	Reference to integer where to store the left scissor coordinate (inclusive)
<i>top</i>	Reference to integer where to store the top scissor coordinate (inclusive)
<i>right</i>	Reference to integer where to store the right scissor coordinate (exclusive)
<i>bottom</i>	Reference to integer where to store the bottom scissor coordinate (exclusive)

NOTES

This function returns the scissor rectangle defined by the previous call to `Camera::setScissor()`. If no scissor rectangle has been defined, the dimensions of the entire viewport are returned.

Note that the interface differs from OpenGL's `glScissor()` call. The convention used here defines the screen origin to be the upper-left corner, whereas OpenGL uses the lower-left corner. Also, the scissor width and height are not returned. They can be computed trivially as follows: *width* = *right-left*, *height* = *bottom-top*.

See section [6.2](#) for a detailed description about the coordinate system used.

SEE ALSO

`Camera::setParameters()`, page 96
`Camera::setScissor()`, page 98

C++ SPECIFICATION

```
int Camera::getWidth() const
```

DESCRIPTION

Returns width of the screen

RETURNS

Width of the screen in pixels

NOTES

The screen width and height have been determined by the previous call to `Camera::setParameters()`. If no screen has yet been defined, the return value is zero.

SEE ALSO

`Camera::getHeight()`, page 83

`Camera::setParameters()`, page 96

C++ SPECIFICATION

```
bool Camera::isPointVisible(const Vector3& xyz)
```

DESCRIPTION

Tests whether specified point is visible

PARAMETERS

xyz Point in raster-space

NOTES

The query is performed against the internal state of the previous `Camera::resolveVisibility()` call. If the database has been modified in any way after the call (by moving any objects or the camera), the results of the query can be wrong.

The point's coordinates are in raster-space, i.e. the *X* and *Y* coordinates should be in the range `[0,screen width[` and `[0,screen height[`. The *Z* coordinate should be in the range `[0,1]`, where 0.0 corresponds to the near clip plane and 1.0 to the far clip plane. The screen width and height have been defined by the previous call to `Camera::setParameters()`.

See section [6.2](#) for information about how the raster-space coordinates of a 3D point can be calculated.

See section [6.12.8](#) for further information about immediate mode visibility queries.

PERFORMANCE

Immediate mode visibility queries are less efficient than the retained mode query provided by `Camera::resolveVisibility()`.

SEE ALSO

`Camera::isRectangleVisible()`, page 90

`Camera::resolveVisibility()`, page 91

C++ SPECIFICATION

```
bool Camera::isRectangleVisible(float xmin, float ymin, float xmax,  
float ymax, float zmax)
```

DESCRIPTION

Tests whether specified rectangle is visible

PARAMETERS

<i>xmin</i>	Rectangle left corner in raster-space
<i>ymin</i>	Rectangle top corner in raster-space
<i>xmax</i>	Rectangle right corner in raster-space (exclusive)
<i>ymax</i>	Rectangle bottom corner in raster-space (exclusive)
<i>zmax</i>	Rectangle depth value in raster-space

NOTES

The query is performed against the internal state of the previous `Camera::resolveVisibility()` call. If database has been modified in any way after the call (by moving any objects or the camera), the results of the query can be wrong.

The coordinates of the rectangle are in raster-space, i.e. the *X* and *Y* coordinates should be in range `[0,screen width[` and `[0,screen height[`. The *Z* coordinate should be in range `[0,1]`, where 0.0 corresponds to the near clip plane and 1.0 to the far clip plane. The screen width and height have been defined by the previous call to `Camera::setParameters()`.

See section [6.2](#) for information about how the raster-space coordinates of a 3D point can be calculated.

See section [6.12.8](#) for further information about immediate mode visibility queries.

PERFORMANCE

Immediate mode visibility queries are less efficient than the retained mode query provided by `Camera::resolveVisibility()`.

SEE ALSO

`Camera::isPointVisible()`, page 89

`Camera::resolveVisibility()`, page 91

`Commander::command()`, page 113

C++ SPECIFICATION

```
void Camera::resolveVisibility(Commander* c, int maxRecursion, float
minImportance)
```

DESCRIPTION

Performs a visibility query using the current viewing and culling parameters of the camera

PARAMETERS

<i>c</i>	Pointer to Commander object (non-NULL)
<i>maxRecursion</i>	Maximum portal recursion depth (0 or greater)
<i>minImportance</i>	Allow culling of portals with smaller cumulative importance than the threshold (set to 0.0 to disable such culling)

NOTES

Visible objects, regions of influence etc. are informed using the Commander functions. If no recursive portals, such as mirrors are used, use value 1 for the recursion depth. If value 0 is used, the visibility query never traverses through any portals. In debug build assertions are made that the Commander object is non-NULL and recursion depth is one or greater. The camera must belong to some cell in order for this function to do anything (traversal is started from that cell). The properties of the camera determine which culling methods are used during the traversal.

For further information about performing a visibility query, see section 6.12.8. For information about setting up the Commander, see section 6.6.

See documentation for `Camera::create()` for a checklist of things to perform before making this call.

The function returns without reporting any of the objects as visible in the following cases:

1. the screen width or height is zero
2. the camera has not been assigned into any cell
3. the cell where the camera is located is not enabled
4. the frustum is degenerate

PlayStation2: The contents of the scratchpad are trashed during `resolveVisibility()`. Therefore the `INSTANCE_IMMEDIATE_REPORT` commands must not trash the scratchpad. When the reporting phase starts (`INSTANCE_VISIBLE`), dPVS no longer needs the scratchpad.

PERFORMANCE

All updates to bounding volumes of objects, internal spatial databases and models are executed during the visibility query. Therefore the execution time is dependent on the coherence between subsequent queries. If a number of objects have been moved and two queries are made in a row, the second query executes much faster than the first one.

SEE ALSO

`Camera::create()`, page 78

`Camera::setParameters()`, page 96

C++ SPECIFICATION

```
void Camera::setCameraToCellMatrix(const Matrix4x4& mtx)
void Camera::setCameraToCellMatrix(const Matrix4x4d& mtx)
```

DESCRIPTION

Defines camera-to-cell transformation matrix

PARAMETERS

mtx Reference to the camera-to-cell transformation matrix

NOTES

The camera-to-cell matrix is used to place the camera physically into a cell. It defines the position and transformation of the camera in the local coordinate system of the cell.

The matrix may not contain a projection term, i.e. the last row or column (depending on matrix format) must be (0,0,0,1). The matrix must be invertible, thus zero scaling terms are not allowed. These conditions are asserted in the debug build. The matrix format used has been defined in the call to `Library::init()`.

The matrix has been initialized to identity in the constructor.

SEE ALSO

`Camera::getCameraToCellMatrix()`, page 79

C++ SPECIFICATION

```
void Camera::setCell(Cell* cell)
```

DESCRIPTION

Places camera into specified cell

PARAMETERS

cell Pointer to the cell

NOTES

The camera must be set into some (non-NULL) cell before it can be used for visibility queries. The visibility queries are started from the cell where the camera belongs. A camera cannot belong simultaneously to multiple cells.

If the *cell* parameter is NULL, the camera is removed from all cells and cannot be used for performing visibility queries until placed again into a cell.

SEE ALSO

Camera::getCell(), page 81

Camera::setCameraToCellMatrix(), page 92

C++ SPECIFICATION

```
void Camera::setFrustum(const Frustum& frustum)
```

DESCRIPTION

Defines the view frustum used by subsequent visibility queries

PARAMETERS

frustum Reference to Frustum structure

NOTES

For information about setting up the view frustum, see section [6.12.2](#) or OpenGL documentation for the functions `glFrustum()` and `glOrtho()`.

This function can define either a perspective or an orthographic projection. The projection type is defined by the *type* member of the Frustum structure.

The *near* and *far* frustum values must be greater than zero. Also, the *far* value must be greater than the *near* value. These conditions are asserted in the debug build.

Note that if *left* is greater than *right* or *bottom* is greater than *top*, the screen-space coordinate axes are flipped. This will affect the back-face culling of polygons in the rendering engine. See `Commander::Viewer::isMirrored()` for further information about handling flipped view frustums.

SEE ALSO

`Camera::getFrustum()`, page 82

`Commander::Viewer::isMirrored()`, page 138

C++ SPECIFICATION

```
void Camera::setObjectMinimumCoverage(float pixelWidth, float  
pixelHeight, float opacity)
```

DESCRIPTION

Defines minimum object coverage values

PARAMETERS

<i>pixelWidth</i>	Minimum width in pixels (zero or greater)
<i>pixelHeight</i>	Minimum height in pixels (zero or greater)
<i>opacity</i>	Opacity threshold value [0,1] (1.0 indicates fully opaque)

NOTES

For information about *screen-size* culling, see section [6.12.6](#).

dPVS reserves the right to cull objects that have raster-space axis-aligned bounding rectangles smaller than the area defined by this function, but does not promise to do so. In practice this feature is effective when appropriate intermediate results are available. This policy was chosen not to decrease performance in any case. The initial minimum coverage values are both zero indicating that no contribution culling is performed by default.

The *opacity* value can be used to control contribution culling. A value of 0.8 means that if more than 80% of the pixels of an object are not visible, dPVS can consider it hidden. The value defaults to 1.0, i.e. no contribution culling is performed. The main use of this feature is to correct errors, such as T-junctions, in the original modeling artwork.

The width and height values must be zero or greater. Negative values are asserted in the debug build and clamped to zero in the release build.

SEE ALSO

Camera::getObjectMinimumCoverage(), page 84
Object::Property, page 180
Object::set(), page 191

C++ SPECIFICATION

```
void Camera::setParameters(int screenWidth, int screenHeight, unsigned  
int propertyMask, float imageScaleX, float imageScaleY)
```

DESCRIPTION

Defines resolution of the screen, culling methods used and some associated parameters

PARAMETERS

<i>screenWidth</i>	Width of screen in pixels (0-16384)
<i>screenHeight</i>	Height of screen in pixels (0-16384)
<i>properties</i>	Bitmask of property enumerations (see <code>Camera::Property</code>)
<i>imageScaleX</i>	Image-space scaling horizontal factor (defaults to 1.0)
<i>imageScaleY</i>	Image-space scaling vertical factor (defaults to 1.0)

NOTES

This function is used for defining the screen parameters, so that dPVS can perform image-space occlusion culling. See section 6.12.4 for further information about the subject. For information about using the image-space scaling factors, see section 6.12.5.

The property enumeration mask is used during subsequent calls to `Camera::resolveVisibility()` to determine which culling methods are used. The screen size and image-space scaling factors determine the resolution at which image-space occlusion culling is performed and the ranges for screen-space coordinates used in query callbacks.

If the `OCCLUSION_CULLING` flag is set in the property mask, the `VIEWFRUSTUM_CULLING` flag will be automatically set as well.

Note that if either a static coverage mask or a static z-buffer is being used, it must be redefined to the camera after this call. The existing scissor rectangle is lost. If scissors are being used, a call to `Camera::setScissor()` must be performed after this call.

The maximum screen dimensions after image-space scaling supported without artifacts by the `Camera` class are 16384x16384 pixels. The *imageScaleX* and *imageScaleY* values must be greater than zero. This is asserted in the debug build.

PERFORMANCE

This function may perform expensive memory reallocations. Therefore do not call the function if the parameters are not really changing. In other words do not put the call into the main per-frame rendering loop.

Note that if only the states of the `PREPARE_RESEND`, `RESEND`, `SCOUT` and `OPTIMIZE` flags change, and no other changes are made to the parameters, no memory reallocations are performed.

The speed of a visibility query is partially dependent on the screen size. Image-space scaling can be used to improve the performance of visibility queries by allowing slight artifacts.

SEE ALSO

`Camera::getHeight()`, page 83
`Camera::getWidth()`, page 88
`Camera::setObjectMinimumCoverage()`, page 95
`Camera::setScissor()`, page 98
`Camera::setStaticCoverageMask()`, page 99
`Camera::setStaticZBuffer()`, page 100

C++ SPECIFICATION

```
void Camera::setPixelCenter(float xoffset, float yoffset)
```

DESCRIPTION

Defines pixel center used by the rasterizer

PARAMETERS

<i>xoffset</i>	Pixel center horizontal offset in pixels [-1,+1]
<i>yoffset</i>	Pixel center vertical offset in pixels [-1,+1]

NOTES

In order to synchronize the internal rasterization of dPVS with the external rasterizer used to draw the image, the pixel center convention must match. If the pixel center is not the same for the two rasterizers, minor artifacts will occur, and small objects may be incorrectly culled.

dPVS uses (0.5,0.5) as its default pixel center. This corresponds with the convention used by most rendering APIs, including OpenGL. If your rasterizer uses a different convention, this function can be used to define an alternative pixel center.

This function was introduced in version 2.3.

SEE ALSO

Camera::getPixelCenter(), page 85

C++ SPECIFICATION

```
void Camera::setScissor(int left, int top, int right, int bottom)
```

DESCRIPTION

Defines a scissor rectangle for culling objects

PARAMETERS

<i>left</i>	Scissor left value in pixels (inclusive)
<i>top</i>	Scissor top value in pixels (inclusive)
<i>right</i>	Scissor right value in pixels (exclusive)
<i>bottom</i>	Scissor bottom value in pixels (exclusive)

NOTES

The scissor rectangle can be used for culling objects outside the specified region on the screen.

The default scissor rectangle has the same size as the camera viewport (defined in the previous call to `Camera::setParameters()`).

The scissor rectangle can be partially or fully outside the viewport. However, if the *right* value is smaller than *left* or the *bottom* value is smaller than *top*, the function asserts in the debug build.

The scissor rectangle must be re-specified every time the viewport is resized by `Camera::setParameters()`.

Note that the interface differs from OpenGL's `glScissor()` call. The convention used here defines the screen origin to be the upper-left corner, whereas OpenGL uses the lower-left corner. Also, the scissor width and height are not specified explicitly. Instead, they are expressed using the *right* and *bottom* coordinates that can be initialized as: *right* = *left*+*width*, *bottom* = *top*+*height*.

SEE ALSO

`Camera::getScissor()`, page 87

`Camera::setParameters()`, page 96

C++ SPECIFICATION

```
void Camera::setStaticCoverageMask(const unsigned char* buffer, int
width, int height, int pitch)
```

DESCRIPTION

Defines a static coverage mask used to mask out portions of the screen.

PARAMETERS

<i>buffer</i>	Pointer to source byte array (may be NULL), where a non-zero value indicates that a pixel is covered
<i>width</i>	Width of the mask buffer in pixels (positive value)
<i>height</i>	Height of the mask buffer in pixels (positive value)
<i>pitch</i>	Pitch of the buffer in bytes (may be negative)

NOTES

If parts of the screen are constantly being covered by some opaque blockers, it is a good idea to hint the occlusion sub-system to take this into account. The convention is similar to using stencil write masks. Bytes with values other than zero are considered to be covered.

A copy of input buffer is taken and resampled to match the current internal resolution. This function must be called after the viewport is resized, i.e. whenever `Camera::setParameters()` reallocates the screen. The function asserts in debug build if either *width* or *height* are negative.

If NULL is submitted as the *buffer* pointer, the existing mask is destroyed. In such a case, all other input parameters are ignored.

For further information about static coverage masks, see section [6.12.7](#).

PERFORMANCE

When this function is called, a lot of internal processing is performed. Therefore the function should not be called every frame.

SEE ALSO

`Camera::setParameters()`, page 96

`Camera::setStaticZBuffer()`, page 100

C++ SPECIFICATION

```
void Camera::setStaticZBuffer(const float* buffer, int width, int height, int pitch, float farValue)
```

DESCRIPTION

Defines a static depth buffer used to mask out portions of the screen.

PARAMETERS

<i>buffer</i>	Pointer to source depth buffer (may be NULL)
<i>width</i>	Width of the buffer in pixels (positive value)
<i>height</i>	Height of the buffer in pixels (positive value)
<i>pitch</i>	Pitch of the buffer in bytes (may be negative)
<i>farValue</i>	Depth value of the back plane [0,1]

NOTES

Applications with a static background can mix dynamic occlusions by submitting the pre-rendered depth buffer to the system.

A copy of input buffer is taken and conservatively resampled to match current internal resolution. This function must be re-called whenever the viewport is resized, i.e. whenever `Camera::setParameters()` reallocates the screen. The function asserts in debug build if either *width* or *height* are negative.

If NULL is submitted as the *buffer* pointer, the existing static depth buffer is destroyed.

See section 6.2 for information on how the depth values of 3D points can be calculated. The Z-buffer submitted in the function should have depth values in the range [0,1], where 0.0 corresponds to the near clipping plane and 1.0 to the far clipping plane.

For further information about static depth buffers, see section 6.12.7.

PERFORMANCE

When this function is called, a lot of internal processing is performed. Therefore the function should not be called every frame.

SEE ALSO

`Camera::setParameters()`, page 96

`Camera::setStaticCoverageMask()`, page 99

8.2 Cell

C++ SPECIFICATION

```
class Cell
```

DESCRIPTION

A cell defines a region in space with an arbitrary topology

BASE CLASSES

ReferenceCount

HEADER FILE

dpvsCell.hpp

NOTES

Every object and camera in the world must belong to some cell in order to be used in a visibility query. The effects of ROIs are limited to the cell where they belong.

Multiple cells can be connected by using portals.

Because the class is derived from ReferenceCount, it does not have a public destructor. Use the function `ReferenceCount::release()` to release instances of the class.

For an overview of the Cell class, see section [6.7](#).

PUBLIC INTERFACE

```
enum Property  
create()  
getCellToWorldMatrix()  
getWorldToCellMatrix()  
set()  
setCellToWorldMatrix()  
test()
```

C++ SPECIFICATION

```
enum Cell::Property
```

DESCRIPTION

Enumeration of cell properties

VALUES

ENABLED	Enables or disables the entire cell
---------	-------------------------------------

NOTES

If the ENABLED flag of the cell is not set, a visibility query can never enter the cell. This flag is an easy method for disabling large portions of the world without having to disable individual objects. Note that if a visibility query starts from a disabled cell, i.e. the camera's cell has been disabled, no objects are reported as visible.

The cell properties are initialized as follows in the constructor:

ENABLED true

SEE ALSO

`Cell::set()`, page 106

`Cell::test()`, page 108

C++ SPECIFICATION

```
static Cell* Cell::create()
```

DESCRIPTION

Constructs a new cell

RETURNS

Pointer to the cell or NULL on failure

NOTES

The cell-to-world matrix is initialized into identity. See `Cell::Property` for default values of the cell's properties.

The only reason this function can fail is that there is not enough memory in order to allocate the cell.

SEE ALSO

`Cell::Property`, page 102

C++ SPECIFICATION

```
void Cell::getCellToWorldMatrix(Matrix4x4& mtx) const  
void Cell::getCellToWorldMatrix(Matrix4x4d& mtx) const
```

DESCRIPTION

Returns the cell-to-world transformation matrix of the cell

PARAMETERS

mtx Reference to matrix structure where the result is stored

NOTES

The matrix format used has been defined in the call to `Library::init()`. The matrix has been initialized to identity in the constructor.

SEE ALSO

`Cell::setCellToWorldMatrix()`, page 107

C++ SPECIFICATION

```
void Cell::getWorldToCellMatrix(Matrix4x4& mtx) const  
void Cell::getWorldToCellMatrix(Matrix4x4d& mtx) const
```

DESCRIPTION

Returns the world-to-cell transformation matrix of the cell

PARAMETERS

mtx Reference to matrix structure where the result is stored

NOTES

This matrix is the inverse of the cell-to-world matrix and can be used for transforming objects from the world-space into the local cell-space.

The matrix format used has been defined in the call to `Library::init()`.

SEE ALSO

`Cell::getCellToWorldMatrix()`, page 104

C++ SPECIFICATION

```
void Cell::set(Property p, bool b)
```

DESCRIPTION

Enables or disables specified property

PARAMETERS

<i>p</i>	Property enumeration
<i>b</i>	Boolean value indicating new state of the property (true = enabled)

NOTES

The default values of the properties are described in the documentation for `Cell::create()`.

SEE ALSO

`Cell::Property`, page 102
`Cell::create()`, page 103
`Cell::test()`, page 108

C++ SPECIFICATION

```
void Cell::setCellToWorldMatrix(const Matrix4x4& mtx)
void Cell::setCellToWorldMatrix(const Matrix4x4d& mtx)
```

DESCRIPTION

Defines the cell-to-world transformation matrix of the cell

PARAMETERS

mtx Reference to cell-to-world matrix where the result is stored

NOTES

The cell-to-world matrix can be used to place a cell into a global coordinate system, i.e. *world-space*. Not all applications use global coordinates.

The matrix format used has been defined in the call to `Library::init()`. The matrix has been initialized to identity in the constructor.

PERFORMANCE

The layout of the internal spatial database is *not* modified by this call.

SEE ALSO

`Cell::getCellToWorldMatrix()`, page 104

C++ SPECIFICATION

```
bool Cell::test(Property p) const
```

DESCRIPTION

Returns state of specified property

PARAMETERS

p Property enumeration

RETURNS

True if property is set, false otherwise

SEE ALSO

Cell::Property, page 102

Cell::set(), page 106

8.3 Commander

C++ SPECIFICATION

```
class Commander
```

DESCRIPTION

Class used for dPVS -to-user message passing

HEADER FILE

```
dpvsCommander.hpp
```

NOTES

The Commander interface handles dPVS -to-user communication. This approach was taken both to avoid extending the API in future releases, and to keep future libraries binary-compatible.

The application user should create a new class derived from Commander and override the `Commander::command()` function to provide a valid implementation. See documentation for the function for further information.

For an overview of the Commander class, see section [6.6](#).

PUBLIC INTERFACE

```
enum Command  
class Instance  
class Viewer  
command()  
~Commander()
```

PROTECTED INTERFACE

```
Commander()  
getBuffer()  
getInstance()  
getLine2D()  
getLine3D()  
getStencilValues()  
getTextMessage()  
getViewer()
```

C++ SPECIFICATION

```
enum Commander::Command
```

DESCRIPTION

Enumeration of different commands that dPVS may send to the `Commander::command()` function

VALUES

QUERY_BEGIN	Start of <code>Camera::resolveVisibility()</code>
QUERY_END	End of <code>Camera::resolveVisibility()</code>
PORTAL_ENTER	Traversal passed through a portal
PORTAL_EXIT	Traversal came back through a portal
VIEW_PARAMETERS_CHANGED	New viewing parameters
INSTANCE_VISIBLE	Object should be rendered
INSTANCE_IMMEDIATE_REPORT	Change write model if necessary
REMOVAL_SUGGESTED	Object should be garbage collected
REGION_OF_INFLUENCE_ACTIVE	Region of influence turned on
REGION_OF_INFLUENCE_INACTIVE	Region of influence turned off
STENCIL_VALUES_CHANGED	Stencil buffer test and write values have changed
STENCIL_MASK	New object should be written into the stencil buffer
TEXT_MESSAGE	<i>Debug function:</i> Text message from the library
DRAW_LINE_2D	<i>Debug function:</i> Draw two-dimensional line
DRAW_LINE_3D	<i>Debug function:</i> Draw three-dimensional line
DRAW_BUFFER	<i>Debug function:</i> Draw monochromatic buffer

SEE ALSO

`Commander::command()`, page 113

C++ SPECIFICATION

Commander::Commander()

DESCRIPTION

Constructs a new Commander

NOTES

Note that the constructor is protected, i.e. Commanders cannot be created directly. One must *always* create a derived class that implements the Commander::command() function.

C++ SPECIFICATION

```
virtual Commander::~~Commander()
```

DESCRIPTION

Destructor

NOTES

The function is virtual because the `Commander` class has virtual functions that are expected to be overridden by the user.

C++ SPECIFICATION

```
virtual void Commander::command(Command cmd) = 0
```

DESCRIPTION

Performs an action requested by dPVS

PARAMETERS

cmd Command enumeration

NOTES

This function is called by dPVS whenever the library user should be informed about something (such as an object being visible or a ROI becoming active or inactive). It is also used to request actions, such as clearing a stencil mask. No parameters other than the command enumeration are passed. Instead, depending on the command, the user should query further information using the `Commander::getViewer()` and `Commander::getInstance()` calls.

This function is defined as 'pure virtual'. This means that the user should create a new class that inherits the `Commander` class and then override the `Commander::command()` function. For further information, see 6.6.3. For future compatibility, a default label should be used in order to correctly handle new commands.

Notes for the different commands:

QUERY_BEGIN

Marks the beginning of the visibility query. No action needs to be taken. This is always the first command.

QUERY_END

Marks the end of the visibility query. No action needs to be taken. This is always the last command.

PORTAL_ENTER

Passing through a portal. No action needs to be taken.

PORTAL_EXIT

Coming back through a portal. No action needs to be taken. Special effects, such as a blending portal material on top of the reflected scene should be done here.

VIEW_PARAMETERS_CHANGED

Set frustum, projection matrix and scissor to your rendering engine. The data can be accessed through the `Commander::Viewer` interface.

INSTANCE_VISIBLE

Render object instantly with specified matrix and current view parameters. Note that the same object can be reported visible several times with different matrices, if there are mirrors or other virtual portals present in the scene. The instance data can be accessed through the `Commander::Instance` interface.

INSTANCE_IMMEDIATE_REPORT

Do *not* render the object, rendering command will be given later. Write model can be changed if desired. Usually this relates to level-of-detail mechanisms. The changes made during this command are made effective immediately, before using the write model in any way. The instance data can be accessed through the `Commander::Instance` interface.

REGION_OF_INFLUENCE_ACTIVE**REGION_OF_INFLUENCE_INACTIVE**

Toggle on/off ROI from active light list, reflect changes to the renderer. The instance data can be accessed through the `Commander::Instance` interface.

STENCIL_VALUES_CHANGED

Stencil test and write values have changed. A write value of -1 is invalid and indicates that

the stencil buffer should be disabled. A test value of -1 is invalid and indicates the subsequent stencil writes are unconditional (no test). The values can currently exceed 255 (hardware limit) in *extreme* cases. Such an extreme case is seeing through 256+ consecutive portals, where each one uses a stencil mask. That is a very unlikely scenario.

STENCIL_MASK

Draw object using previously specified stencil test and write mask values. If your rendering system does not support stencils (or destination alpha test), ignore this event. Accurate portal rendering is then impossible, and incorrect results are evident with floating portals, such as mirror cubes or portable mirrors. The only known way to solve this issue without stencils is to create a complex clipper that clips objects to *any* shape. This approach only has theoretical value and is not really an option. The instance data can be accessed through the `Commander::Instance` interface.

REMOVAL_SUGGESTED

Swap out specified object, if considered appropriate. This is only a hint and a consequence of calling `Library::suggestGarbageCollect()`. User does not *need* to take any action. The instance data can be accessed through the `Commander::Instance` interface.

TEXT_MESSAGE

Use `Commander::getTextMessage()` to access the output string. This command is used by dPVS to pass vital debug information.

The following messages provide debug information. The debug messages are sent only if the corresponding flags have been enabled by `Library::setFlags()`.

DRAW_LINE_2D

Use `Commander::getLine2D()` to access the line data.

DRAW_LINE_3D

Use `Commander::getLine3D()` to access the line data.

DRAW_BUFFER

Use `Commander::getBuffer()` to access the surface data.

SEE ALSO

`Commander::getInstance()`, page 116

`Commander::getViewer()`, page 121

`Library::suggestGarbageCollect()`, page 161

C++ SPECIFICATION

```
Library::BufferType Commander::getBuffer(const unsigned char*& buffer,  
int& width, int& height) const
```

DESCRIPTION

Returns a buffer from the library

PARAMETERS

<i>buffer</i>	Reference to data pointer
<i>width</i>	Reference to integer where the surface width is stored
<i>height</i>	Reference to integer where the surface height is stored

RETURNS

Type of the buffer

NOTES

This is a debug function. This function can only be called during a DRAW_BUFFER callback. The buffer data is valid only for the duration of the callback. Therefore, a copy of the buffer must be taken, if the data is not used instantly. The DRAW_BUFFER command should be ignored in the final product.

Buffers belong to several categories defined in Library class. Call Library::clearFlags() and Library::setFlags() to select the desired debug buffers.

SEE ALSO

Commander::command(), page 113
Library::BufferType, page 142
Library::setFlags(), page 160

C++ SPECIFICATION

Commander::Instance* Commander::getInstance() const

DESCRIPTION

Returns a pointer to instance object

RETURNS

Pointer to current instance object or `NULL` on failure

NOTES

Access to the instance object is granted only during certain `Commander::command()` callbacks. See documentation of the function for further details. Access to the instance is available only for the duration of the current `Commander::command()` callback.

The function returns `NULL` if an instance is not available for the current callback.

SEE ALSO

`Commander::command()`, page 113

C++ SPECIFICATION

`Library::LineType Commander::getLine2D(Vector2& a, Vector2& b, Vector4& color) const`

DESCRIPTION

Returns a 2D line from the library

PARAMETERS

<i>a</i>	Line start point in raster-space
<i>b</i>	Line end point in raster-space
<i>color</i>	Line color in RGBA format (components in [0,1] range)

RETURNS

Enumeration indicating what the line represents

NOTES

This is a debug function. This function can only be called during a `DRAW_2D_LINE` callback. The `DRAW_LINE_2D` command should be ignored in the final product.

Lines belong to several categories defined in `Library` class. Call `Library::clearFlags()` and `Library::setFlags()` to select the desired debug lines.

SEE ALSO

`Commander::command()`, page 113
`Library::LineType`, page 145
`Library::setFlags()`, page 160

C++ SPECIFICATION

Library::LineType Commander::getLine3D(Vector3& a, Vector3& b, Vector4& color) const

DESCRIPTION

Returns a 3D line from the library

PARAMETERS

<i>a</i>	Line start point in world-space
<i>b</i>	Line end point in world-space
<i>color</i>	Line color in RGBA format (components in [0,1] range)

RETURNS

Enumeration indicating what the line represents

NOTES

This is a debug function. This function can only be called during a DRAW_3D_LINE callback. The DRAW_LINE_3D command should be ignored in the final product.

Lines belong to several categories defined in Library class. Call Library::clearFlags() and Library::setFlags() to select the desired debug lines.

SEE ALSO

Commander::command(), page 113

Library::LineType, page 145

Library::setFlags(), page 160

C++ SPECIFICATION

```
void Commander::getStencilValues(int& test, int& write) const
```

DESCRIPTION

Returns current test and write stencil values

PARAMETERS

<i>test</i>	Reference to int where to store the test stencil value
<i>write</i>	Reference to int where to store the write stencil value

NOTES

The stencil values are provided for correct rendering of certain portal overlap cases. See [6.11.10](#) for further information on the subject.

The stencil values are available only during the `Commander::STENCIL_VALUES_CHANGED` callback. At other times garbage values can be returned by the function.

SEE ALSO

`Commander::command()`, page 113

C++ SPECIFICATION

```
const char* Commander::getTextMessage() const
```

DESCRIPTION

Returns a text message from the library

RETURNS

String containing a text message

NOTES

This is a debug function. The function should be called only during a TEXT_MESSAGE callback. The TEXT_MESSAGE command should be ignored in the final product. The string returned by this function exists only for the duration of the TEXT_MESSAGE callback.

SEE ALSO

Commander::command(), page 113

C++ SPECIFICATION

`Commander::Viewer* Commander::getViewer() const`

DESCRIPTION

Returns a pointer to current viewer object

RETURNS

Pointer to current viewer object or NULL on failure

NOTES

Access to the viewer object is granted only during certain `Commander::command()` callbacks. See documentation of the function for further details. Access to the viewer is available only for the duration of the current `Commander::command()` callback.

The function returns NULL if a viewer is not available.

SEE ALSO

`Commander::command()`, page 113

8.3.1 Commander::Instance

C++ SPECIFICATION

```
class Commander::Instance
```

DESCRIPTION

Class used for passing object-related information to the user during certain Commander callbacks

HEADER FILE

```
dpvsCommander.hpp
```

NOTES

This class is used solely by the Commander for providing additional object-specific information during Commander::command() callbacks.

The application user cannot construct or destruct Instances. The only way to get an access to an Instance is through the Commander::getInstance() method.

PUBLIC INTERFACE

```
enum Clip  
struct Projection  
getClipMask()  
getImportance()  
getObject()  
getObjectToCameraMatrix()  
getProjectionSize()
```

C++ SPECIFICATION

```
enum Commander::Instance::Clip
```

DESCRIPTION

Enumeration of first six clip plane masks

VALUES

FRONT	Front clip plane of the view frustum
BACK	Back clip plane of the view frustum
LEFT	Left clip plane of the view frustum
RIGHT	Right clip plane of the view frustum
TOP	Top clip plane of the view frustum
BOTTOM	Bottom clip plane of the view frustum

NOTES

These enumerations can be used for interpreting the clip mask returned by `Commander::Instance::getClipMask()`.

SEE ALSO

`Commander::Instance::getClipMask()`, page 125

`Commander::Viewer::getFrustumPlane()`, page 134

`Commander::Viewer::getFrustumPlaneCount()`, page 135

C++ SPECIFICATION

```
struct Commander::Instance::Projection
```

DESCRIPTION

Structure for passing object projection information

VALUES

int left	Left screen coordinate (inclusive)
int right	Right screen coordinate (exclusive)
int top	Top screen coordinate (inclusive)
int bottom	Bottom screen coordinate (exclusive)
float zNear	Near depth value
float zFar	Far depth value

NOTES

This structure is used for passing data to the member function `Commander::Instance::getProjectionSize()`. See section [6.2](#) for further information about raster-space coordinates and their ranges.

SEE ALSO

`Commander::Instance::getProjectionSize()`, page 129

C++ SPECIFICATION

```
UINT32 Commander::Instance::getClipMask() const
```

DESCRIPTION

Returns clip mask of the object

RETURNS

A 32-bit mask indicating which clipping planes are crossed by the object

NOTES

The clip mask of an object can be used for optimizing the performance of the rendering engine. Some rendering engines allow disabling primitive-level clipping if the user can guarantee that the object is fully inside the view frustum. Current HWTL rendering cards support only the first six clip planes at zero cost - subsequent planes can reduce the performance considerably. Therefore it may be beneficial to disable the user-defined clipping planes whenever they are not needed.

The first six planes of the clip mask always correspond to the six planes (front, back, left, right, top, bottom) that form the view frustum expressed by the current projection matrix. An additional plane can be introduced by *floating portals*. The clip planes can be queried from `Commander::Viewer` during certain `Commander` callbacks.

Note that if the traversal has passed through portals, the view frustum may not be the same as the original view frustum specified by the `Camera`.

The clip mask is conservative and it is formed by testing the bounding volume of the *test model* of the object against the view frustum. The clip mask is always formed during the visibility query - therefore this function does not have to perform any additional work.

If the clip mask is zero, it is guaranteed that the object does not cross any clipping planes and thus primitive-level clipping can be skipped.

SEE ALSO

`Commander::Instance::Clip`, page 123

`Commander::Viewer::getFrustumPlane()`, page 134

`Commander::Viewer::getFrustumPlaneCount()`, page 135

C++ SPECIFICATION

```
float Commander::Instance::getImportance() const
```

DESCRIPTION

Estimates the importance of an object

RETURNS

A value in range [0, 1], where 1.0 indicates full importance.

NOTES

The importance is estimated by calculating the cumulative importance decay terms of the portal sequence through which the object is visible. If the object is in the same cell as the camera (and not seen through a mirror), the importance is 1.0. Note that the importance term is computed only from the portal importance values - it cannot be used alone to make estimates for object LOD level calculations.

See documentation for the function `Commander::Instance::getProjectionSize()` for information on selecting proper LOD levels.

SEE ALSO

`PhysicalPortal::setImportanceDecay()`, page 206

C++ SPECIFICATION

Object* Commander::Instance::getObject() const

DESCRIPTION

Returns pointer to corresponding Object

RETURNS

Pointer to Object

NOTES

It is an invalid operation to modify the cell, transformation matrix or the test model of an Object during the visibility query. One can use all of the 'const' query functions of the Object or change its write model.

C++ SPECIFICATION

```
void Commander::Instance::getObjectToCameraMatrix(Matrix4x4& mtx) const  
void Commander::Instance::getObjectToCameraMatrix(Matrix4x4d&) const
```

DESCRIPTION

Returns object-to-camera transformation matrix of the instance

PARAMETERS

mtx Reference to matrix structure where the result is stored

NOTES

The matrix format used has been defined in the call to `Library::init()`. This matrix corresponds with the OpenGL concept of a *modelview* matrix.

C++ SPECIFICATION

```
bool Commander::Instance::getProjectionSize(Projection& p) const
```

DESCRIPTION

Estimates the projection size of the instance

PARAMETERS

p Reference to a Projection structure where the results are stored

RETURNS

true if *p* is valid (object is at least partially inside view frustum), false otherwise

NOTES

The projection size of an object can be used for user-side detail culling or level-of-detail model selection.

The left/right/top/bottom values are clamped to the current viewport rectangle.

If the object is completely outside the view frustum, the function returns `false`. In such a case, the values in the Projection structure are not valid.

SEE ALSO

Commander::Instance::Projection, page 124

8.3.2 Commander::Viewer

C++ SPECIFICATION

```
class Commander::Viewer
```

DESCRIPTION

A smart object for passing view-related information to the user during certain Commander callbacks

HEADER FILE

```
dpvsCommander.hpp
```

NOTES

This class is used by Commander to provide additional view-specific information during certain Commander::command() callbacks.

The application user cannot construct or destruct Viewers. The only way to gain access to a Viewer is through the method Commander::getViewer().

PUBLIC INTERFACE

```
enum Handedness  
getCameraToWorldMatrix()  
getFrustum()  
getFrustumPlane()  
getFrustumPlaneCount()  
getProjectionMatrix()  
getScissor()  
isMirrored()
```

C++ SPECIFICATION

```
enum Commander::Viewer::Handedness
```

DESCRIPTION

Enumeration specifying handedness of the matrix returned by a function

VALUES

LEFT_HANDED	The matrix is left-handed
RIGHT_HANDED	The matrix is right-handed

NOTES

This enumeration is used as a parameter to some `Commander::Viewer` member functions to specify the handedness of the matrix returned.

SEE ALSO

`Commander::Viewer::getProjectionMatrix()`, page 136

C++ SPECIFICATION

```
void Commander::Viewer::getCameraToWorldMatrix(Matrix4x4& mtx) const  
void Commander::Viewer::getCameraToWorldMatrix(Matrix4x4d& mtx) const
```

DESCRIPTION

Returns the current camera-to-world transformation matrix

PARAMETERS

mtx Reference to matrix structure where the result is stored

NOTES

This matrix is not needed for the actual rendering. It is supplied for debug visualization purposes, as the user may want to transform objects into a global coordinate system. The matrix format used has been defined by the user in the call to `Library::init()`.

C++ SPECIFICATION

```
void Commander::Viewer::getFrustum(Frustum& frustum) const
```

DESCRIPTION

Returns current view frustum

PARAMETERS

frustum Reference to the view frustum structure where the result is stored

NOTES

If portals are used, the frustum can be different than the one originally specified by the user in the previous call to `Camera::setFrustum()`, as dPVS creates a new frustum whenever its visibility traversal passes through a portal. The existence of a new frustum is reported to the user by the Commander callback `Commander::VIEW_PARAMETERS_CHANGED`.

SEE ALSO

`Camera::getFrustum()`, page 82

`Camera::setFrustum()`, page 94

C++ SPECIFICATION

```
void Commander::Viewer::getFrustumPlane(int n, Vector4& plane) const
```

DESCRIPTION

Returns Nth plane of the current view frustum

PARAMETERS

<i>n</i>	Number of the plane [0,numPlanes[
<i>plane</i>	Reference to vector where to store the camera-space plane equation (normals point outside)

NOTES

The frustum can be different than the one originally specified by the user in the previous call to `Camera::setFrustum()`, if portals are used, as dPVS creates a new frustum whenever its visibility traversal passes through a portal. The existence of a new frustum is reported to the user by the Commander callback `Commander::VIEW_PARAMETERS_CHANGED`. The frustum planes are always in camera-space (eye-space).

This function should only be called if primitive-level view frustum culling is desired. Generally this is only worth doing if the underlying hardware supports *free clipping planes*.¹

SEE ALSO

`Commander::Instance::getClipMask()`, page 125

`Commander::Viewer::getFrustumPlaneCount()`, page 135

¹Clipping planes beyond the initial six planes that form the view frustum. Sometimes the term *user clipping planes* is used.

C++ SPECIFICATION

```
int Commander::Viewer::getFrustumPlaneCount(void) const
```

DESCRIPTION

Returns number of planes in the current view frustum

NOTES

If portals are used, the frustum can be different than the one originally specified by the user in the previous call to `Camera::setFrustum()`, as dPVS creates a new frustum whenever its visibility traversal passes through a portal. The existence of a new frustum is reported to the user by the Commander callback `Commander::VIEW_PARAMETERS_CHANGED`.

This function should only be called if primitive-level view frustum culling is desired. Generally this is only worth doing if the underlying hardware supports free clipping planes.

SEE ALSO

`Commander::Instance::getClipMask()`, page 125

`Commander::Viewer::getFrustumPlane()`, page 134

C++ SPECIFICATION

```
void Commander::Viewer::getProjectionMatrix(Matrix4x4& mtx, Handedness  
h) const  
void Commander::Viewer::getProjectionMatrix(Matrix4x4d& mtx, Handedness  
h) const
```

DESCRIPTION

Returns current projection matrix

PARAMETERS

<i>mtx</i>	Reference to the matrix structure where the result is stored
<i>h</i>	Handedness of the matrix returned

NOTES

The matrix format used has been defined in the call to `Library::init()`.

SEE ALSO

`Commander::Instance::getObjectToCameraMatrix()`, page 128

C++ SPECIFICATION

```
void Commander::Viewer::getScissor(int& left, int& top, int& right,  
int& bottom) const
```

DESCRIPTION

Returns current scissor rectangle

PARAMETERS

<i>left</i>	Smallest x-coordinate of an axis-aligned rectangle (inclusive)
<i>top</i>	Smallest y-coordinate of an axis-aligned rectangle (inclusive)
<i>right</i>	Largest x-coordinate of an axis-aligned rectangle (exclusive)
<i>bottom</i>	Largest y-coordinate of an axis-aligned rectangle (exclusive)

NOTES

The coordinates returned are in raster-space. See section 6.2 for further details about coordinate systems. The coordinates of the top left corner of the window are (0,0).

The scissor rectangle has at the beginning of a visibility query the size defined by the previous call to `Camera::setScissor()`. If a scissor has not been defined, a scissor box having the size of the full screen is used.

When the traversal passes through a portal, the axis-aligned screen projection of the portal sequence is calculated and reported to the user. This "scissor rectangle" can be used for fast polygon-level rejection of primitives. Also, many hardware rasterizers are able to conserve fill rate somewhat by rejecting the parts of objects that are not visible through the portal scissor sequence.

SEE ALSO

`Camera::setParameters()`, page 96
`Camera::setScissor()`, page 98

C++ SPECIFICATION

```
bool Commander::Viewer::isMirrored (void) const
```

DESCRIPTION

Returns boolean value indicating whether subsequent objects are mirrored

RETURNS

Boolean value

NOTES

When virtual portals are used, two-dimensional back-face culling routines must know if the view is a reflected one, as they need to change their winding. Most rendering engines perform back-face culling with screen-space coordinates, so the mirroring status should be honored - otherwise incorrect back-face culling will occur.

Also, the use of a mirrored view frustum will affect the winding of polygons. See documentation for `Camera::setFrustum()` for further information.

If the application uses 2D back-face culling, it should query the mirror status whenever the commander receives the `VIEW_PARAMETERS_CHANGED` message. If this function returns *true*, the application should change the current polygon winding.

Note that a 3D back-face culling routine that operates fully in object-space² does not need to flip the winding.

SEE ALSO

`Camera::setFrustum()`, page 94

²I.e. a routine that transforms the camera position into object-space and performs back-face culling by testing the camera position against the object-space plane equations of the primitives.

8.4 Frustum

C++ SPECIFICATION

```
struct Frustum
```

DESCRIPTION

Structure for passing view frustum information

HEADER FILE

```
dpvsDefs.hpp
```

VALUES

float left	Frustum left coordinate
float right	Frustum right coordinate
float top	Frustum top coordinate
float bottom	Frustum bottom coordinate
float zNear	Frustum near plane distance
float zFar	Frustum far plane distance
Type type	Type of the frustum (default = PERSPECTIVE)

NOTES

All of the member variables are initialized to zero in the constructor. The type field defaults to PERSPECTIVE.

When used to pass information to projection routines, the near and far clipping values must be greater than zero. Also, the far clipping value must be greater than the near clipping value.

PUBLIC INTERFACE

```
enum Type  
Frustum()
```

SEE ALSO

Camera::getFrustum(), page 82
Camera::setFrustum(), page 94
Commander::Viewer::getFrustum(), page 133
Frustum::Type, page 140

C++ SPECIFICATION

```
enum Frustum::Type
```

DESCRIPTION

Enumeration of projection types

VALUES

PERSPECTIVE	Perspective projection (default)
ORTHOGRAPHIC	Orthographic projection

NOTES

These enumerations are used in the `type` field of the `Frustum` structure to indicate the projection type of a frustum.

This enumeration was introduced in version 2.3.

SEE ALSO

`Camera::getFrustum()`, page 82

`Camera::setFrustum()`, page 94

`Commander::Viewer::getFrustum()`, page 133

8.5 Library

C++ SPECIFICATION

`class Library`

DESCRIPTION

dPVS global functions

HEADER FILE

`dpvsLibrary.hpp`

NOTES

This class contains functions for initializing and shutting down the library. It also has functions for querying performance statistics and other debug information.

All of the functions in the `Library` class are static. This means that `Library` objects cannot be constructed.

For an overview of the `Library` class, see section [6.5](#).

PUBLIC INTERFACE

```
enum BufferType
enum FlagType
enum InfoString
enum LineType
enum MatrixFormat
enum Statistic
class Services
clearFlags()
exit()
getFlags()
getInfoString()
getStatistic()
init()
minimizeMemoryUsage()
resetStatistics()
setFlags()
suggestGarbageCollect()
textCommand()
```

C++ SPECIFICATION

```
enum Library::BufferType
```

DESCRIPTION

Enumeration of buffer type categories

VALUES

BUFFER_COVERAGE	Coverage buffer
BUFFER_DEPTH	Depth buffer (8x8)
BUFFER_FULLBLOCKS	Full 8x8 blocks version of coverage buffer
BUFFER_STENCIL	Final stencil buffer

NOTES

These enumerations are only required while using visual debug information. Buffer contents and representations are subjects to change, but provide valuable information about the behavior of the library.

SEE ALSO

Library::FlagType, page 143
Library::LineType, page 145
Library::clearFlags(), page 152
Library::getFlags(), page 154
Library::setFlags(), page 160

C++ SPECIFICATION

enum Library::FlagType

DESCRIPTION

Enumeration of flag type categories

VALUES

LINEDRAW	Select line draw flags
BUFFER	Select buffer draw flags

SEE ALSO

Library::BufferType, page 142
Library::LineType, page 145
Library::clearFlags(), page 152
Library::getFlags(), page 154
Library::setFlags(), page 160

C++ SPECIFICATION

```
enum Library::InfoString
```

DESCRIPTION

Enumeration of input values for the function `Library::getInfoString()`

VALUES

VERSION	Version string of the library (in format: "1.9.1")
COPYRIGHT	Library copyright info string
BUILD_TIME	Library build date and time
FUNCTIONALITY	"Registered", "trial/non-functional (tampered/no authorization)", "trial/fully functional"
CUSTOMER	Name of the customer to whom the library is licensed
CPU_OPTIMIZATIONS	String containing CPU-specific optimizations enabled for current processor

SEE ALSO

`Library::getInfoString()`, page 155

C++ SPECIFICATION

```
enum Library::LineType
```

DESCRIPTION

Enumeration of line type categories

VALUES

LINE_MISC	Lines that do not belong to any other category
LINE_OBJECT_BOUNDS	Object axis-aligned bounding volumes
LINE_VOXELS	Voxel bounds
LINE_RECTANGLES	Axis-aligned bounding rectangles of objects
LINE_SILHOUETTES	Silhouettes of occluders
LINE_VIRTUAL_CAMERA_AXII	Reflected etc. camera coordinate systems
LINE_PORTAL_RECTANGLES	Axis-aligned bounding rectangles of portals
LINE_VPT	Visible Point Tracking
LINE_TEST_SILHOUETTES	Silhouettes of occludees
LINE_REGIONS_OF_INFLUENCE	Regions of influence

NOTES

These enumerations are only required while using visual debug information.

SEE ALSO

Library::FlagType, page 143
Library::LineType, page 145
Library::clearFlags(), page 152
Library::getFlags(), page 154
Library::setFlags(), page 160

C++ SPECIFICATION

```
enum Library::MatrixFormat
```

DESCRIPTION

Enumeration of matrix formats

VALUES

COLUMN_MAJOR	Matrix format of OpenGL, Direct3D, Matlab, Renderman
ROW_MAJOR	Matrix format of SurRender3D

NOTES

The matrix format enumeration is used in `Library::init()` for specifying the format used by all dPVS API functions that deal with matrices. The matrix format cannot be changed after the library has been initialized.

SEE ALSO

`Library::init()`, page 157

C++ SPECIFICATION

```
enum Library::Statistic
```

DESCRIPTION

Enumeration of different statistics

VALUES

```

STAT_SILHOUETTECACHEINSERTIONS
    Number of silhouettes created and inserted into the cache
STAT_SILHOUETTECACHEREMOVALS
    Number of silhouettes removed from the cache
STAT_SILHOUETTECACHEQUERIES
    Number of silhouette queries
STAT_SILHOUETTECACHEQUERYITERS
    Number of silhouette query inner iterations
STAT_SILHOUETTECACHEHITS
    Number of silhouette cache hits (misses = m_queries-m_hits)
STAT_SILHOUETTECACHEMEMORYUSED
    Number of bytes used by the silhouette cache currently
STAT_OBJECTMATRIXUPDATES
    Number of times object → cell matrices have been updated
STAT_OBJECTCAMERAMATRIXUPDATES
    Number of times object*camera matrices have been updated
STAT_OBJECTVPTFAILED
    Number of times object VPT has failed
STAT_SPHEREBOUNDSQUERIED
    Number of times sphere bounding rectangles queried internally
STAT_CAMERARESOLVEVISIBILITYCALLS
    Number of times DPVS::Camera::resolveVisibility() has been called
STAT_CAMERAVISIBILITYCALLBACKS
    Number of times camera has performed visibility callbacks
STAT_HZBUFFERLEVELUPDATES
    Number of times a HDEB level has been updated
STAT_WRITEQUEUEPOINTQUERIES
    Write queue point visibility queries
STAT_WRITEQUEUESILHOUETTEQUERIES
    Write queue silhouette visibility queries
STAT_WRITEQUEUEOBJECTQUERIES
    Write queue object visibility queries
STAT_WRITEQUEUEOCCLUDEESDEPTHREJECTED
    Write queue occludees rejected due to "closing depth"
STAT_WRITEQUEUEWRITESREQUESTED
    Writes submitted to write queue
STAT_WRITEQUEUEWRITESPERFORMED
    Writes submitted to occlusion buffer
STAT_WRITEQUEUEWRITESPOSTPONED
    Writes postponed by the write queue (bad Z)
STAT_WRITEQUEUEWRITESDISCARDED
    Writes discarded (invalid silhouette, beyond far plane)
STAT_WRITEQUEUEOVERFLOW
    Insufficient allocation space in write queue (expensive)
STAT_WRITEQUEUEOVERFLOWEVERYTHING
    Insufficient allocation space in write queue (even more expensive)

```

STAT_WRITEQUEUEDEPTHWRITES
Depth buffer writes performed by write queue

STAT_WRITEQUEUEDEPTHCLEARS
Depth buffer clears performed

STAT_WRITEQUEUEFLUSHES
Number of write queue flushes performed

STAT_WRITEQUEUEBUCKETFLUSHWORK
Number of buckets traversed in flush work

STAT_WRITEQUEUEHIDDENOCCLUDERS
Number of hidden occluders used

STAT_WRITEQUEUEFRONTCLIPPINGOCCLUDERSTESTED
Number of front clipping occluder tests

STAT_WRITEQUEUEFRONTCLIPPINGOCCLUDERSUSED
Number of front clipping occluders used

STAT_OCCLUSIONSILHOUETTEQUERIES
Number of occlusion buffer silhouette queries

STAT_OCCLUSIONPOINTQUERIES
Number of occlusion buffer point queries

STAT_OCCLUSIONRECTANGLEQUERIES
Number of occlusion buffer rectangle queries

STAT_OCCLUSIONACCURATEPOINTQUERIES
Number of points where full precision query was performed

STAT_OCCLUSIONACCURATEBLOCKQUERIES
Number of blocks where full precision query was performed

STAT_OCCLUSIONACCURATEPOINTUSEFUL
Number of points where full precision query was useful

STAT_OCCLUSIONACCURATEBLOCKUSEFUL
Number of blocks where full precision query was useful

STAT_OCCLUSIONBUFFERBUCKETSCLEARED
Number of times occlusion buffer buckets were cleared

STAT_OCCLUSIONBUFFERBUCKETSPROCESSED
Number of times occlusion buffer buckets were processed

STAT_OCCLUSIONBUFFEREDGESRASTERIZED
Number of edges rasterized (write+test)

STAT_OCCLUSIONBUFFEREDGESTESTED
Number of edges tested (test only)

STAT_OCCLUSIONBUFFEREDGESCLIPPING
Number of edges clipping (both rasterized+tested)

STAT_OCCLUSIONBUFFEREDGESINGLEBUCKET
Number of single-bucket edges

STAT_OCCLUSIONBUFFEREDGEPIXELS
Number of edge pixels rasterized

STAT_OCCLUSIONBUFFERTESTEDGEPIXELS
Number of test edge pixels rasterized

STAT_OCCLUSIONBUFFEREXACTZTESTS
Number of pixels exact Z-tested

STAT_DATABASETRAVERSALS
Number of top level traverse calls

STAT_DATABASENODESINSERTED
Number of nodes created

STAT_DATABASENODESREMOVED
Number of nodes removed

STAT_DATABASENODEDIRTYUPDATES	Number of nodes dirty updated
STAT_DATABASENODESTRAVERSED	Number of nodes traversed
STAT_DATABASELEAFNODESTRAVERSED	Number of leaf nodes traversed
STAT_DATABASENODESSKIPPED	Number of nodes node-skipped during traversal
STAT_DATABASENODESVFTESTED	Number of nodes view frustum tested
STAT_DATABASENODESVFCULLED	Number of nodes view frustum culled by primary test
STAT_DATABASENODESVFCULLED2	Number of nodes view frustum culled by secondary test
STAT_DATABASENODESOCCLUSIONTESTED	Number of nodes occlusion tested
STAT_DATABASENODESOCCLUSIONCULLED	Number of nodes occlusion culled
STAT_DATABASENODEVPTFAILED	Number of node VPT guesses gone wrong
STAT_DATABASENODEVPTSUCCEEDED	Number of node VPT guesses gone right
STAT_DATABASENODEVPTUPDATED	Number of node VPT updates (by feedback)
STAT_DATABASENODESVISIBLE	Number of nodes visible after VF/occlusion culling
STAT_DATABASENODESPLITS	Number of nodes split
STAT_DATABASENODENEWROOTS	Number of times a new root node has been created (i.e. upward collapses)
STAT_DATABASENODESFRONTCLIPPING	Number of front clipping nodes traversed
STAT_DATABASEOBSTATUSCHANGES	Number of times object's visibility status has changed from visible to hidden or vice versa
STAT_DATABASEOBSINSERTED	Number of objects inserted into the database
STAT_DATABASEOBSREMOVED	Number of objects removed from the database
STAT_DATABASEOBSUPDATED	Number of object update calls
STAT_DATABASEOBSUPDATEPROCESSED	Number of times object update calls really caused work
STAT_DATABASEOBINSTANCETRAVERSED	Number of times an ob instance has been touched
STAT_DATABASEOBSTRAVERSED	Number of objects traversed
STAT_DATABASEOBSVFTESTED	Number of objects view frustum tested
STAT_DATABASEOBSVFCULLED	Number of objects view frustum culled
STAT_DATABASEOBSVFEXACTTESTED	Number of objects view frustum tested by exact tests

STAT_DATABASEOBSVFEXACTCULLED	Number of objects view frustum culled by exact tests
STAT_DATABASEOBSVISIBLE	Number of objects determined visible
STAT_DATABASEOBSVISIBILITYPARENTCULLED	Number of objects culled by relation to visibility parents
STAT_DATABASEOBSOCCLUSIONSKIPPED	Number of objects whose occlusion test was skipped
STAT_DATABASEOBSOCCLUSIONTESTED	Number of objects occlusion tested
STAT_DATABASEOBSOCCLUSIONCULLED	Number of objects occlusion culled
STAT_DATABASEOBNEWVISIBLEPOINTS	Number of times a new "visible point" has been computed
STAT_DATABASEINSTANCESINSERTED	Number of instances inserted
STAT_DATABASEINSTANCESREMOVED	Number of instances removed
STAT_DATABASEINSTANCESMOVED	Number of instances moved (i.e. skipped a removal/insertion pair)
STAT_ROIACTIVE	Number of regions of influence found visible
STAT_ROISTATECHANGES	Number of region of influence state changes (active ↔ inactive)
STAT_ROIOBJECTOVERLAPTESTS	Number of ROI vs. object overlap tests performed
STAT_ROIOBJECTOVERLAPS	Number of ROI vs. object overlaps
STAT_HOAX0 - STAT_HOAX7	Hoax counter 0-7, reserved for internal debugging
STAT_MODELRECTANGLESQUERIED	Number of times a bounding rectangle has been calculated for a model1
STAT_MODELEXACTRECTANGLESQUERIED	Number of times an exact bounding rectangle has been calculated for a model
STAT_MODELTESTSILHOUETTESQUERIED	Number of times a test silhouette has been queried
STAT_MODELTESTSILHOUETTESCLIPPED	Number of times a test silhouette has been clipped (front-clipping)
STAT_MODELTESTSILHOUETTESREJECTED	Number of times a test silhouette has been rejected (i.e. determined automatically as visible)
STAT_MODELWRITESILHOUETTESQUERIED	Number of times a write silhouette has been queried
STAT_MODELTOPOLOGYCOMPUTED	Number of times model derived topology has been computed
STAT_TIME	Time in seconds since last reset (timer accuracy is platform-dependent)
STAT_MEMORYUSED	Number of bytes of memory currently used by dPVS
STAT_MEMORYCURRENTALLOCATIONS	Number of memory allocations dPVS currently has

STAT_MEMORYOPERATIONS
 Number of memory allocation/release operations since last reset

STAT_LIVECAMERAS
 Number of cameras in the world

STAT_LIVECELLS
 Number of cells in the world

STAT_LIVEMODELS
 Number of models in the world

STAT_LIVEOBJECTS
 Number of objects in the world (includes ROIs and portals)

STAT_LIVEPHYSICALPORTALS
 Number of physical portals in the world

STAT_LIVEREGIONSOFINFLUENCE
 Number of ROIs

STAT_LIVEVIRTUALPORTALS
 Number of virtual portals in the world

STAT_LIVENODES
 Number of spatial database nodes in the world

STAT_LIVEINSTANCES
 Number of object instances in nodes in the world

NOTES

Many of the statistics are not interesting for everyday development. See section [6.5.1](#) for information about some important statistics.

SEE ALSO

Library::getStatistic(), page 156
Library::resetStatistics(), page 159

C++ SPECIFICATION

```
static void Library::clearFlags (FlagType type, unsigned int flags)
```

DESCRIPTION

Clears flags of desired type

PARAMETERS

<i>type</i>	Flag type
<i>flags</i>	Mask of flags to deactivate

SEE ALSO

Library::FlagType, page 143
Library::getFlags(), page 154
Library::setFlags(), page 160

C++ SPECIFICATION

```
static void Library::exit()
```

DESCRIPTION

Shuts down dPVS

NOTES

Call this function once at the end of your application, after releasing *all* dPVS resources.

In the debug build, dPVS asserts that all dPVS -related resources have been released. If this call succeeds, Purify, BoundsChecker and other similar memory-leak detection tools should report zero leaks caused by dPVS .

SEE ALSO

Library::init(), page 157

C++ SPECIFICATION

```
static unsigned int Library::getFlags (FlagType type)
```

DESCRIPTION

Returns flags of desired type

PARAMETERS

type Flag type

RETURNS

Flag mask of specified type

SEE ALSO

Library::FlagType, page 143

Library::clearFlags(), page 152

Library::setFlags(), page 160

C++ SPECIFICATION

```
static const char* Library::getInfoString(InfoString t)
```

DESCRIPTION

Returns information about the library

PARAMETERS

t Enumerated value indicating which information string should be returned

RETURNS

Corresponding information string

NOTES

The information strings can be used to find out which version of dPVS is being used and what internal features have been enabled.

SEE ALSO

Library::InfoString, page 144

C++ SPECIFICATION

```
static float Library::getStatistic(Statistic s)
```

DESCRIPTION

Returns specified statistic value

PARAMETERS

s Statistic enumeration

RETURNS

Corresponding statistic value

NOTES

For further discussion about statistics, see section [6.5.1](#).

SEE ALSO

Library::Statistic, page 147

Library::resetStatistics(), page 159

C++ SPECIFICATION

```
static void Library::init(MatrixFormat format, Services* services)
```

DESCRIPTION

Initializes dPVS

PARAMETERS

<i>format</i>	Matrix format enumeration
<i>services</i>	Pointer to Services object (may be NULL)

NOTES

Call this function once at the beginning of your application, before constructing *any* dPVS - related objects.

The *format* parameter is used to specify the matrix format used by all of the library functions. The matrix format cannot be changed after this call.

The *services* parameter is used to assign a Services object. The use of this parameter is optional.

SEE ALSO

Library::MatrixFormat, page 146

Library::exit(), page 153

Library::Services, page 163

C++ SPECIFICATION

```
static void Library::minimizeMemoryUsage()
```

DESCRIPTION

Minimizes memory usage of all dPVS sub-systems

NOTES

When this function is called, dPVS will inform all of its sub-systems to minimize their memory usage. This will cause freeing of all temporary allocations, invalidation of all caches etc.

PERFORMANCE

This function should not be called often, as it has a negative impact on the speed of the following visibility queries as the caches are rebuilt and new temporary memory allocations are performed.

SEE ALSO

Library::suggestGarbageCollect(), page 161

C++ SPECIFICATION

```
static void Library::resetStatistics()
```

DESCRIPTION

Resets all statistics counters

SEE ALSO

[Library::Statistic](#), page 147

[Library::getStatistic\(\)](#), page 156

C++ SPECIFICATION

```
static void Library::setFlags (FlagType type,unsigned int flags)
```

DESCRIPTION

Activates flags of desired type

PARAMETERS

<i>type</i>	Flag type enumeration
<i>flags</i>	Mask of flags to activate

NOTES

Performs a logical OR with existing flags of the specified type.

SEE ALSO

Library::FlagType, page 143

Library::clearFlags(), page 152

Library::getFlags(), page 154

C++ SPECIFICATION

```
static void Library::suggestGarbageCollect(Commander* c, float t)
```

DESCRIPTION

Suggest garbage collection of objects that have not been visible in specified time period

PARAMETERS

<i>c</i>	Pointer to Commander to catch the suggestions (non-NULL)
<i>t</i>	Time in seconds (must be zero or positive)

NOTES

The function will report through the Commander : :REMOVAL_SUGGESTED command all objects that have not been visible (in any camera) during the last *t* seconds. The object pointer can be queried in the callback by using Commander : :getInstance().

If the commander pointer is NULL or the time value is negative, the function will not do anything in the release build. In debug build such cases are asserted.

PERFORMANCE

This function performs a lot of internal work and should not be called very frequently.

SEE ALSO

Library::minimizeMemoryUsage(), page 158

C++ SPECIFICATION

```
static int Library::textCommand(Commander* commander, const char* cmd)
```

DESCRIPTION

Submits a command to the library in text format

PARAMETERS

<i>commander</i>	Pointer to commander object
<i>cmd</i>	String describing the command

RETURNS

Command-specific information

NOTES

This function is used only for debugging/profiling dPVS and not needed in everyday development.

8.5.1 Library::Services

C++ SPECIFICATION

```
class Library::Services
```

DESCRIPTION

Class for routing certain operating system and application dependent calls to the user

HEADER FILE

```
dpvsLibrary.hpp
```

NOTES

This class is used for routing certain calls to the user. These calls include error-handling, memory management and timer services.

The services are passed to dPVS when `Library::init()` is called.

PUBLIC INTERFACE

```
allocateMemory()  
error()  
getTime()  
releaseMemory()
```

C++ SPECIFICATION

```
virtual void* Library::Services::allocateMemory (size_t bytes)
```

DESCRIPTION

Function for allocating memory

PARAMETERS

bytes Number of bytes of memory to allocate

RETURNS

Pointer to a block of memory

NOTES

Whenever dPVS needs to allocate memory, it calls this function. The default implementation of the function calls the C library function `malloc()`.

The memory allocated using this function is always released with a call to `Library::Services::releaseMemory()`. If this function is overridden, the release function should be overridden as well.

The *bytes* parameter is always greater than zero.

dPVS assumes that the alignment of the memory block is at least eight.

SEE ALSO

`Library::Services::releaseMemory()`, page 167

C++ SPECIFICATION

```
virtual void Library::Services::error(const char* message)
```

DESCRIPTION

Error-management function

PARAMETERS

message String containing information about the error

NOTES

This function is called when dPVS encounters a fatal error in the debug build. The function must be overridden by the application user. The typical implementation would be to display the text message. dPVS expects that this function does not return, i.e. that the user somehow aborts the execution of the program.

C++ SPECIFICATION

```
virtual float Library::Services::getTime()
```

DESCRIPTION

Function for querying current time

RETURNS

Time since application startup in seconds

NOTES

This function is called whenever dPVS needs to know the current time. The function should return the time since application startup in seconds. dPVS assumes the timer has at least 20Hz accuracy.

The default implementation of this function calls the C library function `clock()`. This function is not implemented on Sony PlayStation2, so the user must always implement this function on that platform.

C++ SPECIFICATION

```
virtual void Library::Services::releaseMemory (void* p)
```

DESCRIPTION

Function for releasing a memory block

PARAMETERS

p Pointer to memory block to be released

NOTES

dPVS calls this function to release a memory block earlier allocated by a call to `Library::Services::allocateMemory()`.

The default implementation of this function calls the C library function `free()`.

SEE ALSO

`Library::Services::allocateMemory()`, page 164

8.6 MeshModel

C++ SPECIFICATION

```
class MeshModel
```

DESCRIPTION

A model type where the topology of the model is described using triangles and vertices

BASE CLASSES

ReferenceCount, Model

HEADER FILE

dpvsModel.hpp

NOTES

This is the most common model type used. Note that when using a MeshModel as a test model, any simplified model that is (at least) larger than the true model can be used. When using them as write models, the shape must be conservatively smaller - otherwise artifacts will occur.

See section [6.8](#) for further information about constructing and using models.

Because the class is derived from ReferenceCount, it does not have a public destructor. Use the function ReferenceCount::release() to release instances of the class.

PUBLIC INTERFACE

create()

C++ SPECIFICATION

```
static MeshModel* MeshModel::create(const Vector3* pos, const Vector3i*  
ind, int vn, int tn, bool clockwise)
```

DESCRIPTION

Constructs a new MeshModel

PARAMETERS

<i>pos</i>	Vertex positions in object-space (cannot be NULL)
<i>ind</i>	Triangle vertex indices (cannot be NULL)
<i>vn</i>	Number of vertices in mesh (must be greater than zero)
<i>tn</i>	Number of triangles in mesh (must be greater than zero)
<i>clockwise</i>	Boolean value indicating whether vertex indices are in clockwise or counter-clockwise order (clockwise = true)

RETURNS

Pointer to the model or NULL on failure

NOTES

A copy of the input vertex and triangle data is taken internally. The application user is free to release the original mesh data after this call.

Each triangle should have three vertex indices in the range $[0, vn[$. The *clockwise* boolean value is used to indicate the winding of the vertex indices. If the value is `true`, indices are specified in clockwise order.

In the debug build, the library asserts that the vertex coordinates are valid floating-point numbers and that the vertex indices are in the valid range.

The function can fail if there is not enough memory to store the model data or if the input data is invalid.

PERFORMANCE

dPVS performs some mesh analysis/optimizations inside the constructor - this takes a while. For example, duplicate vertices and degenerate triangles are removed.

8.7 Model

C++ SPECIFICATION

```
class Model
```

DESCRIPTION

Base class for all models in dPVS

BASE CLASSES

ReferenceCount

DERIVED CLASSES

MeshModel, SphereModel

HEADER FILE

dppsModel.hpp

NOTES

Models are used to describe shapes of objects. They do not "exist" by themselves - in order to physically place a model into a world, an `Object` must be created to provide the location and orientation information. Models can be shared by multiple objects.

The `Model` class is a pure virtual base class. One cannot construct `Models` directly. Instead, one of the derived classes must be used. See section 6.8 for further information about constructing and using models.

When a `Model` is constructed, its properties are initialized as follows:

```
BACKFACE_CULLABLE false  
SOLID false
```

Models that are going to be used by portals *must* have the `BACKFACE_CULLABLE` flag set. Models that are going to be used by objects other than portals should generally disable the `BACKFACE_CULLABLE` flag.

Because the class is derived from `ReferenceCount`, it does not have a public destructor. Use the function `ReferenceCount::release()` to release instances of the class.

PUBLIC INTERFACE

```
enum Property  
getAABB()  
getOBB()  
getSphere()  
set()  
test()
```

C++ SPECIFICATION

```
enum Model::Property
```

DESCRIPTION

Model property enumeration

VALUES

BACKFACE_CULLABLE	Entire model can be back-face culled
SOLID	Model is completely solid

NOTES

These enumerations are used by the `Model::set()` and `Model::test()` functions.

When a `Model` is constructed, its properties are initialized as follows:

```
BACKFACE_CULLABLE false  
SOLID false
```

The `BACKFACE_CULLABLE` flag indicates that the entire model can be back-face culled from certain viewpoints. For example, certain single-sided models and all convex models have this property. See section [6.8.5](#) for further details.

`SOLID` flag indicates that the model is completely solid and the user guarantees that the camera may never go inside the object. This information may be used to optimize certain scenarios.

SEE ALSO

`Model::set()`, page 175

`Model::test()`, page 176

Reference to float where to store the opacity threshold

C++ SPECIFICATION

```
void Model::getAABB(Vector3& min, Vector3& max) const
```

DESCRIPTION

Returns an axis-aligned bounding box (AABB) of the model in object-space

PARAMETERS

<i>min</i>	Reference to vector where to store the minimum coordinates of the AABB
<i>max</i>	Reference to vector where to store the maximum coordinates of the AABB

NOTES

The AABB returned may be calculated from other bounding volumes, i.e. OBB and Sphere.

This function was introduced in version 2.3.

SEE ALSO

[Object::getAABB\(\)](#), page 183

[Model::getOBB\(\)](#), page 173

[Model::getSphere\(\)](#), page 174

C++ SPECIFICATION

```
void Model::getOBB(Matrix4x4& m) const
```

DESCRIPTION

Returns oriented bounding box of the model

PARAMETERS

m Reference to matrix where to store the OBB transformation matrix

NOTES

The transformation matrix is the matrix used to transform a unit cube, i.e. a cube with corners at (-1,-1,-1) and (+1,+1,+1). The matrix contains rotation, scale and translation terms, but not a projection term.

This function was introduced in version 2.3.

SEE ALSO

Model::getAABB(), page 172

Model::getSphere(), page 174

Object::getOBB(), page 185

C++ SPECIFICATION

```
void Model::getSphere(Vector3& center, float& radius) const
```

DESCRIPTION

Returns bounding sphere of the model

PARAMETERS

<i>center</i>	Reference to vector where to store the origin of the sphere
<i>radius</i>	Reference to float where to store the radius of the sphere

NOTES

The center of the sphere is expressed in object-space, i.e. the local coordinate system of the model.

This function was introduced in version 2.3.

SEE ALSO

`Model::getAABB()`, page 172

`Model::getOBB()`, page 173

`Object::getSphere()`, page 187

C++ SPECIFICATION

```
void Model::set(Property p, bool b)
```

DESCRIPTION

Enables or disables specified property

PARAMETERS

<i>p</i>	Property enumeration
<i>b</i>	Boolean value indicating whether property will be enabled (true) or disabled (false)

SEE ALSO

Model::Property, page 171

Model::test(), page 176

C++ SPECIFICATION

```
bool Model::test(Property p) const
```

DESCRIPTION

Returns state of specified property

PARAMETERS

p Property enumeration

RETURNS

Boolean value indicating if property is set (true=enabled, false=disabled)

SEE ALSO

Model::Property, page 171

Model::set(), page 175

8.8 OBBModel

C++ SPECIFICATION

```
class OBBModel
```

DESCRIPTION

A model type describing an oriented bounding box using a matrix

BASE CLASSES

ReferenceCount, Model

HEADER FILE

dpvsModel.hpp

NOTES

OBBModels are boxes that are oriented, translated and scaled to fit an object. Many shapes are usually bounded rather well by OBBs. An additional advantage is that OBB models consume very little memory. Note that OBBModels cannot be used as write models. Because the class is derived from ReferenceCount, it does not have a public destructor. Use the function `ReferenceCount::release()` to release instances of the class.

PUBLIC INTERFACE

```
create()
```

C++ SPECIFICATION

```
static OBBModel* OBBModel::create(const Matrix4x4& obb)
static OBBModel* OBBModel::create(const Vector3* vertices, int
numVertices)
static OBBModel* OBBModel::create(const Vector3& min, const Vector3&
max)
```

DESCRIPTION

Constructs a new OBB model

PARAMETERS

<i>obb</i>	Reference to matrix describing an OBB
<i>vertices</i>	Array of object-space vertex locations
<i>numVertices</i>	Number of vertices
<i>min</i>	Reference to vector containing minimum coordinates of an AABB
<i>max</i>	Reference to vector containing maximum coordinates of an AABB

RETURNS

Pointer to the model or NULL on failure

NOTES

If the OBB is constructed from a transformation matrix, the matrix may not contain projection terms or zero scale values. These conditions are asserted in the debug build.

If a vertex array is used to construct the model, dPVS calculates an oriented bounding box that bounds the vertex set. The vertex data is not needed later, so it can be discarded by the user.

If an AABB is used to construct the model, the input AABB will be copied and converted into an OBB transformation matrix.

The function may fail if there is not enough memory in order to allocate the model or the input data is invalid. In such a case, NULL is returned.

8.9 Object

C++ SPECIFICATION

```
class Object
```

DESCRIPTION

A physical instance of a `Model` in the scene database

BASE CLASSES

`ReferenceCount`

DERIVED CLASSES

`PhysicalPortal`, `RegionOfInfluence`, `VirtualPortal`

HEADER FILE

`dpvsObject.hpp`

NOTES

Objects are used to place `Models` into the world. For each object, a transformation matrix is defined. Each object also has a *test model* used for visibility testing and an optional *write model* used for occluding other objects.

See section 6.9 for further information about the `Object` class.

Because the class is derived from `ReferenceCount`, it does not have a public destructor. Use the function `ReferenceCount::release()` to release instances of the class.

PUBLIC INTERFACE

```
enum Property
create()
getAABB()
getCell()
getOBB()
getObjectToCellMatrix()
getSphere()
getTestModel()
getVisibilityParent()
getWriteModel()
set()
setCell()
setCost()
setObjectToCellMatrix()
setTestModel()
setVisibilityParent()
setWriteModel()
test()
```

C++ SPECIFICATION

```
enum Object::Property
```

DESCRIPTION

Object property enumeration

VALUES

ENABLED	Is the object active?
INFORM_VISIBLE	Should the user be informed when the object is found visible?
CONTRIBUTION_CULLING	Can object be contribution or screen-size culled?
INFORM_PORTAL_ENTER	Inform when traversing through the portal (portal objects only)
INFORM_PORTAL_EXIT	Inform when traversing backwards through the portal (portal objects only)
FLOATING_PORTAL	Object is a floating portal (portal objects only)
REPORT_IMMEDIATELY	Should the user be informed <i>immediately</i> when the object is found visible? (new in version 2.3)
UNBOUNDED	Does the object have an infinite size

NOTES

These enumerations are used by the `Object::set()` and `Object::test()` functions to define some object-specific properties. The object properties are initialized as follows:

```
ENABLED true
INFORM_VISIBLE true
CONTRIBUTION_CULLING true
INFORM_PORTAL_ENTER false
INFORM_PORTAL_EXIT false
FLOATING_PORTAL false
REPORT_IMMEDIATELY false
UNBOUNDED false
```

When an object is not **ENABLED**, it will be interpreted as the object did not exist at all during visibility queries. Non-enabled objects have zero processing overhead during visibility traversals. The user can thus disable large portions of a scene and be sure that the disabled objects will not slow down the visibility processing.

If **INFORM_VISIBLE** is disabled, the visibility of the object will still be determined and the object can be used as an occluder. However, a visibility callback is not made, i.e. no message is sent to the Commander. This flag applies only to **Objects** and **Portals** as **Regions Of Influence** ignore this flag as they never create a visibility callback. In order to disable a ROI, clear its **ENABLED** flag.

If **REPORT_IMMEDIATELY** is enabled, a message is sent to **Commander** immediately. The purpose is to make it possible to change write models, for instance LOD levels, during the visibility query.

If contribution or screen-size culling is enabled with a call to `Camera::setObjectMinimumCoverage()`, an object can be subjected to contribution culling unless the **CONTRIBUTION_CULLING** flag is disabled.

If the object has been tagged as **UNBOUNDED**, the object is treated as having an infinite size, regardless of its test or write models. An unbounded object is always visible if its cell is visible - no view frustum or occlusion culling is ever performed. The unbounded object is never used as an occluder. When ROI vs. object overlaps are detected, unbounded ROIs affect all objects in

the cell. An unbounded object is affected by all ROIs in a cell. The main purpose of this flag is to easily support extremely large objects, such as directional lights or sky boxes.

SEE ALSO

`Camera::setObjectMinimumCoverage()`, page 95

`Object::set()`, page 191

`Object::test()`, page 198

C++ SPECIFICATION

```
static Object* Object::create(Model* testModel)
```

DESCRIPTION

Constructs a new Object

PARAMETERS

testModel Pointer to test model (non-NULL)

RETURNS

Pointer to object or NULL on failure

NOTES

See `Object::Property` for default values of the object's properties.

The object's transformation matrix is initialized to identity. The object's cost is initialized based on the test model cost. The object is not located in any cell during initialization, thus a separate call to `Object::setCell()` is needed to place the object into some cell.

The *testModel* pointer *must* be non-NULL. This is asserted in the debug build.

Checklist for creating and configuring an object:

1. assign the object to some cell by calling `Object::setCell()`
2. assign the object $\rightarrow$ cell transformation matrix with `Object::setObjectToCellMatrix()`
3. to give dPVS a better idea about the rendering cost of the object, call `Object::setCost()` (section 6.9.8)
4. if you intend to use the object as an occluder, assign a write model to it by using `Object::setWriteModel()`
5. if you know that the object will only be visible if some other (enclosing) object is visible, assign that object as the current one's *visibility parent* by calling `Object::setVisibilityParent()` (section 6.9.6)
6. if you want to debug objects with names, assign a name for the object by `ReferenceCount::setName()` (section 6.4.3)
7. if you want the object to be automatically destroyed when its cell is destroyed, call `ReferenceCount::autoRelease()`

The function fails if there is not enough memory in order to allocate the object or the *testModel* pointer is NULL.

SEE ALSO

`Object::setTestModel()`, page 195

`Object::setWriteModel()`, page 197

C++ SPECIFICATION

```
void Object::getAABB(Vector3& min, Vector3& max) const
```

DESCRIPTION

Returns cell-space axis-aligned bounding box (AABB) of the object

PARAMETERS

<i>min</i>	Reference to vector where to store the minimum coordinates of the AABB
<i>max</i>	Reference to vector where to store the maximum coordinates of the AABB

NOTES

The axis-aligned bounding box is expressed in cell-space coordinates. If the transformation matrix of the object is changed, the AABB will change as well.

This function was introduced in version 2.3.

SEE ALSO

Model::getAABB(), page 172

Object::getOBB(), page 185

Object::getSphere(), page 187

C++ SPECIFICATION

```
Cell* Object::getCell() const
```

DESCRIPTION

Returns pointer to cell where the object belongs to

RETURNS

Pointer to cell where the object belongs or NULL

NOTES

If the object has not been placed into any cell, NULL is returned. Note that in such a case, the object will never be reported as visible by a visibility query.

SEE ALSO

Object::setCell(), page 192

C++ SPECIFICATION

```
void Object::getOBB(Matrix4x4& m) const
```

DESCRIPTION

Returns cell-space oriented bounding box of the object

PARAMETERS

m Reference to matrix where to store the OBB into

NOTES

The oriented bounding box is expressed in cell-space coordinates. If the transformation matrix of the object is changed, the OBB will change as well.

The matrix returned does not contain a projection term.

This function was introduced in version 2.3.

SEE ALSO

Model::getOBB(), page 173

Object::getAABB(), page 183

Object::getSphere(), page 187

C++ SPECIFICATION

```
void Object::getObjectToCellMatrix(Matrix4x4& mtx) const  
void Object::getObjectToCellMatrix(Matrix4x4d& mtx) const
```

DESCRIPTION

Returns the object-to-cell transformation matrix

PARAMETERS

mtx Reference to the matrix structure where the result is stored

NOTES

The matrix format used has been defined in the call to `Library::init()`.

The matrix is initialized to identity in the object constructor.

SEE ALSO

`Object::setObjectToCellMatrix()`, page 194

C++ SPECIFICATION

```
void Object::getSphere(Vector3& center, float& radius) const
```

DESCRIPTION

Returns cell-space bounding sphere of the object

PARAMETERS

<i>center</i>	Reference to vector where to store the origin of the sphere
<i>radius</i>	Reference to float where to store the radius of the sphere

NOTES

The bounding sphere is expressed in cell-space coordinates. If the transformation matrix of the object is changed, the bounding sphere will change as well.

This function was introduced in version 2.3.

SEE ALSO

`Model::getSphere()`, page 174

`Object::getAABB()`, page 183

`Object::getOBB()`, page 185

C++ SPECIFICATION

```
Model* Object::getTestModel() const
```

DESCRIPTION

Returns pointer to the test model of the object

RETURNS

Pointer to test model

SEE ALSO

Object::setTestModel(), page 195

NOTES

The function never returns a NULL pointer, as a test model *has* to be defined for an object.

C++ SPECIFICATION

`Object* Object::getVisibilityParent() const`

DESCRIPTION

Returns pointer to visibility parent of the object

RETURNS

Pointer to visibility parent of the object

NOTES

If a visibility parent has not been defined, the function returns `NULL`.

Note that if the visibility parent of an object has been destroyed, the return value of this function is `NULL`, i.e. the function never returns dangling pointers.

SEE ALSO

`Object::setVisibilityParent()`, page 196

C++ SPECIFICATION

```
Model* Object::getWriteModel() const
```

DESCRIPTION

Returns pointer to write model of the object

RETURNS

Pointer to write model

NOTES

If a write model has not been defined, the function returns NULL.

SEE ALSO

[Object::setWriteModel\(\)](#), page 197

C++ SPECIFICATION

```
void Object::set(Property p, bool b)
```

DESCRIPTION

Enables or disables specified property

PARAMETERS

p Property enumeration

b Boolean value indicating new state of the property (true = enabled)

NOTES

The default values of the properties are described in the documentation for `Object::create()`.

SEE ALSO

`Object::create()`, page 182

`Object::test()`, page 198

C++ SPECIFICATION

```
void Object::setCell(Cell* cell)
```

DESCRIPTION

Assigns object to specified cell

PARAMETERS

cell Pointer to cell (may be NULL)

NOTES

The object is removed from its previous cell. By default, objects do not belong to any cell. In order to get an object reported as visible during a visibility query, it *must* be placed into some cell.

The object can be temporarily disabled by assigning it into a NULL cell. However, the `ENABLED` flag method for disabling objects should be preferred instead (section 6.9.7).

SEE ALSO

`Object::getCell()`, page 184

C++ SPECIFICATION

```
void Object::setCost(int vnum, int tnum, float complexity)
```

DESCRIPTION

Defines rendering cost of the object

PARAMETERS

<i>vnum</i>	Number of vertices (0 or more)
<i>tnum</i>	Number of triangles (0 or more)
<i>complexity</i>	Relative complexity value (must be ≥ 0.0 , default = 1.0)

NOTES

The rendering cost of an object can be defined to allow dPVS to make better decisions in its occluder selection process. The complexity term can be used to indicate the relative rendering complexity of the triangles (number of passes during multi-passing or relative cost of shader).

The rendering cost is assigned on a per-object basis because many objects can share simplified bounds (test models) - such models would not give any idea about the actual rendering cost.

The rendering cost is initialized using the test model provided in the `Object` constructor.

See section [6.9.8](#) for more information about assigning object costs.

SEE ALSO

`Object::create()`, page 182

C++ SPECIFICATION

```
void Object::setObjectToCellMatrix(const Matrix4x4& mtx)
void Object::setObjectToCellMatrix(const Matrix4x4d& mtx)
```

DESCRIPTION

Defines the object-to-cell transformation matrix

PARAMETERS

mtx Reference to object-to-cell matrix

NOTES

This function is used for physically placing the objects inside a cell. The matrix defines the object's transformation inside the cell's local coordinate system.

The matrix can contain anisotropic scaling components. However, using negative or zero scales is highly discouraged as they may cause surprising problems in the user's rendering engine.

The matrix may not contain any projection (i.e. the fourth column/row must be 0, 0, 0, 1) The matrix format used has been defined in the call to `Library::init()`.

The matrix is initialized to identity in the object constructor.

PERFORMANCE

Updating an object's transformation matrix is not a cheap operation. Call this function only when an object really moves, and at most once per visibility query. If the physics system updates the object transformations at a rate higher than the visualization frame rate, specify the changed transformation to dPVS only once per frame.

SEE ALSO

`Object::getObjectToCellMatrix()`, page 186

C++ SPECIFICATION

```
void Object::setTestModel(Model* testModel)
```

DESCRIPTION

Defines a new test model for the object

PARAMETERS

testModel Pointer to test model (cannot be NULL)

NOTES

This function defines the test model of the object. The test model is used for determining the visibility of the object during a visibility query. See section 6.8 for a discussion of test models.

Back-face culling *must* be enabled for portal models. If portal models cannot be back-face culled, infinite recursion cases are evident. Even though dPVS employs the concept of recursion depth, the results would be wrong.

The object takes a reference to the model. The reference is released when a new test model is assigned or when the object is destroyed. Objects must always have test models. Thus a NULL pointer cannot be passed to the function (asserted in debug build and rejected in release build).

PERFORMANCE

The effect of changing the object's test model is evaluated during the next visibility query. Therefore updating an object's test model also slows down the next call to `Camera::resolveVisibility()`.

SEE ALSO

`Object::getTestModel()`, page 188

`Object::setWriteModel()`, page 197

C++ SPECIFICATION

```
void Object::setVisibilityParent(Object* parent)
```

DESCRIPTION

Defines a visibility parent for the object

PARAMETERS

parent Pointer to visibility parent (can be NULL)

NOTES

Visibility parents are used for expressing inter-object visibility relationships. See section [6.9.6](#) for further information about visibility parents.

The visibility parent must exist in the same cell as the object. Visibility parents can be used in some cases to create visibility relationships between objects. An object may be found *hidden* in a visibility query if its visibility parent is hidden. An object should be fully contained inside its visibility parent's test bounds - otherwise artifacts will occur.

If the visibility parent of an object is destroyed, the visibility parent link is automatically cleared. This means that after the destruction of the parent, a call to the child's `Object::setVisibilityParent()` function will return NULL, not a dangling pointer.

The visibility parent relationship means that dPVS has the *right* to cull children of hidden visibility parents. It does *not* guarantee that the children will be culled.

An object may have at most one visibility parent. If the function is called multiple times, the last call determines the visibility parent.

By default, objects do not have visibility parents.

SEE ALSO

`Object::setVisibilityParent()`, page 189

C++ SPECIFICATION

```
void Object::setWriteModel(Model* model)
```

DESCRIPTION

Defines a write model for the object

PARAMETERS

model Pointer to model (may be NULL)

NOTES

If the object is intended to be used as an occluder, a write model must be assigned to it. See sections 6.8.1 and 6.9.5 for further information about write models.

The object takes a reference to the model. The reference is released when a new write model is assigned or when the object is destroyed. The write model can be disabled by assigning a NULL write model. By default, objects do not have write models.

If no write model is assigned, the object is never used as an occluder.

PERFORMANCE

Defining a new write model is much less expensive than defining a new test model, as the write model does not affect the internal spatial databases in any way.

SEE ALSO

Object::getWriteModel(), page 190

Object::setTestModel(), page 195

C++ SPECIFICATION

```
bool Object::test(Property p) const
```

DESCRIPTION

Returns state of specified property

PARAMETERS

p Property enumeration

RETURNS

True if property is set, false otherwise

SEE ALSO

`Object::set()`, page 191

8.10 ObjectSplitter

C++ SPECIFICATION

```
class ObjectSplitter
```

DESCRIPTION

Utility class for partitioning large unstructured scenes

HEADER FILE

```
dpvsUtility.hpp
```

NOTES

The `ObjectSplitter` class is used for partitioning large, unstructured scenes into objects. The class has only a single static function.

PUBLIC INTERFACE

```
split()
```

C++ SPECIFICATION

```
static int ObjectSplitter::split(UINT32 objectIDs[], const Vector3*  
vertices, const Vector3i* triangles, const UINT32* triMaterials, int  
numVertices, int numTriangles)
```

DESCRIPTION**PARAMETERS**

<i>objectIDs</i>	Output object ID array (non-NULL)
<i>vertices</i>	Input vertex positions (non-NULL)
<i>triangles</i>	Input triangle vertex indices (non-NULL)
<i>triMaterials</i>	Input triangle materials (may be NULL)
<i>numVertices</i>	Number of vertices (must be positive)
<i>numTriangles</i>	Number of triangles (must be positive)

RETURNS

number of objects the input object was split into

NOTES

The output array will contain object IDs for each triangle. The object IDs are in range $[0, numObjects[$, where *numObjects* is the return value of the function.

The *triMaterials* array is optional, i.e. NULL may be submitted. If a triangle material array is provided, arbitrary UINT32 'material tags' can be used. The materials are used to improve the partitioning process.

The function does not take any input parameters regarding the "size" of the scene. It assumes the entire static portions of the scene are passed as input. This means that one should not send *portions* of the scene separately to the splitter, instead everything should be passed in using a single call.

8.11 PhysicalPortal

C++ SPECIFICATION

```
class PhysicalPortal
```

DESCRIPTION

Class describing a physical link between two cells

BASE CLASSES

ReferenceCount, Object

DERIVED CLASSES

VirtualPortal

HEADER FILE

dpvsObject.hpp

NOTES

Physical portals are physical links between cells. Such links cannot cause discontinuities in space and are only used for visibility determination.

See section 6.11 for an overview of physical and virtual portals.

Because the class is derived from ReferenceCount, it does not have a public destructor. Use the function ReferenceCount::release() to release instances of the class.

PUBLIC INTERFACE

```
create()  
getImportanceDecay()  
getStencilModel()  
getTargetCell()  
setImportanceDecay()  
setStencilMode()  
setTargetCell()
```

C++ SPECIFICATION

```
static PhysicalPortal* PhysicalPortal::create(Model* testModel, Cell*  
targetCell)
```

DESCRIPTION

Constructs a new physical portal

PARAMETERS

<i>testModel</i>	Pointer to test model (non-NULL)
<i>targetCell</i>	Pointer to target cell(can be NULL)

RETURNS

Pointer to the portal or NULL on failure

NOTES

The function will fail if there is not enough memory in order to allocate the portal. It will also fail if *testModel* is NULL. This case is asserted in the debug build.

C++ SPECIFICATION

```
float PhysicalPortal::getImportanceDecay() const
```

DESCRIPTION

Returns importance decay factor of the portal

RETURNS

Importance decay factor of the portal

SEE ALSO

PhysicalPortal::setImportanceDecay(), page 206

C++ SPECIFICATION

```
Cell* PhysicalPortal::getTargetCell() const
```

DESCRIPTION

Returns target cell the portal leads to

RETURNS

Pointer to target cell

NOTES

The function returns NULL if no target cell has been defined.

SEE ALSO

`PhysicalPortal::setTargetCell()`, page 208

C++ SPECIFICATION

```
Model* PhysicalPortal::getStencilModel() const
```

DESCRIPTION

Returns stencil mask model of a portal.

RETURNS

Pointer to stencil mask model of the portal (or NULL if none defined)

SEE ALSO

PhysicalPortal::setStencilModel(), page 207

C++ SPECIFICATION

```
void PhysicalPortal::setImportanceDecay(float decay)
```

DESCRIPTION

Sets an importance decay factor to the portal.

PARAMETERS

decay Decay factor in range [0, 1]

NOTES

Defaults to 1.0 and physical portals that are solely used for visibility determination should always use the factor 1.0. Virtual portals used to create special effects such as mirrors should set this to (1.0 - average transparency value). This information is visible for the user when the LOD level of an object is being estimated.

For further information about the importance decay term, see section [6.11.8](#).

The input parameter is silently clamped into [0, 1] range.

C++ SPECIFICATION

```
void PhysicalPortal::setStencilModel(Model* model)
```

DESCRIPTION

Defines stencil mask model of a portal.

PARAMETERS

model Pointer to stencil model (may be NULL)

NOTES

Stencil modes are used for correctly visualizing certain difficult portal cases. See section [6.11.10](#) for further information.

The portal takes a reference to the stencil model specified.

SEE ALSO

`PhysicalPortal::getStencilModel()`, page 205

C++ SPECIFICATION

```
void PhysicalPortal::setTargetCell(Cell* cell)
```

DESCRIPTION

Defines the target cell the portal leads to

PARAMETERS

<i>cell</i>	Pointer to target cell
-------------	------------------------

NOTES

This function is used for defining the target cell of the portal. The source cell of the portal is defined by placing the portal into a cell by `Object::setCell()`.

SEE ALSO

`Object::setCell()`, page 192

`PhysicalPortal::getTargetCell()`, page 204

8.12 ReferenceCount

C++ SPECIFICATION

```
class ReferenceCount
```

DESCRIPTION

Reference counting base class

DERIVED CLASSES

Camera, Cell, MeshModel, Model, Object, PhysicalPortal,
RegionOfInfluence, SphereModel, VirtualPortal

HEADER FILE

dpvsReferenceCount.hpp

NOTES

Classes in the library that use reference-counting are all derived from this class. Instances of reference counted classes cannot be destructed by calling `delete`, instead the member function `ReferenceCount::release()` should be used.

See section [6.4](#) for an overview of the ReferenceCount class.

PUBLIC INTERFACE

```
addReference()  
autoRelease()  
getName()  
getReferenceCount()  
getUserPointer()  
release()  
setName()  
setUserPointer()
```

PROTECTED INTERFACE

```
ReferenceCount()  
~ReferenceCount()
```

C++ SPECIFICATION

ReferenceCount::ReferenceCount()

DESCRIPTION

Constructor

NOTES

Initializes the object's reference count to one. Sets the user pointer to NULL.

Some implementations of reference-counting use the convention of initializing the counter to zero. We have found it to be more natural to initialize it to one.

SEE ALSO

ReferenceCount::autoRelease(), page 213

ReferenceCount::release(), page 217

ReferenceCount::setUserPointer(), page 219

C++ SPECIFICATION

```
virtual ReferenceCount::~~ReferenceCount()
```

DESCRIPTION

Destructs a reference counted object

NOTES

The destructor of all reference-counted objects is protected. This means that one cannot directly destruct any reference-counted objects. Instead the function `ReferenceCount::release()` should be used.

The destructor is virtual because the class is intended to be inherited by other classes.

C++ SPECIFICATION

```
void ReferenceCount::addReference()
```

DESCRIPTION

Increases reference count of the object by one

NOTES

This function is used internally whenever any dPVS resource takes a reference to a reference-counted resource. It can also be used by the library user for implementation of *auto pointers* and similar constructs.

SEE ALSO

ReferenceCount::autoRelease(), page 213

ReferenceCount::release(), page 217

C++ SPECIFICATION

```
void ReferenceCount::autoRelease()
```

DESCRIPTION

Decreases reference count of the object by one, but does not destroy the object, if the counter reaches zero.

NOTES

Since the object's reference count is initialized to one in the constructor, this function can be used to let dPVS "internally manage" the object. For example the user can create a `MeshModel` and assign it to some object. The object will take a reference to the model, thus increasing its reference count to two. By calling the auto-releasing function, the model's reference becomes one. When the object is destructed, the remaining reference is released, and the model is automatically destructed.

If the function is called multiple times, the subsequent calls do not decrease the reference count.

SEE ALSO

`ReferenceCount::release()`, page 217

C++ SPECIFICATION

```
const char* ReferenceCount::getName() const
```

DESCRIPTION

Returns name string associated with the instance

RETURNS

NULL-terminated string containing name of the instance

NOTES

If no name has been defined for the object, an empty string, i.e. "" is returned instead of NULL.

Note that name strings should mainly be used for debugging purposes.

SEE ALSO

ReferenceCount::setName(), page 218

C++ SPECIFICATION

```
int ReferenceCount::getReferenceCount() const
```

DESCRIPTION

Returns reference count of the object

RETURNS

Reference count of the object

C++ SPECIFICATION

```
void* ReferenceCount::getUserPointer() const
```

DESCRIPTION

Returns user pointer of the object

RETURNS

User pointer of the object

NOTES

If a user pointer has not been defined, the function returns NULL.

SEE ALSO

[ReferenceCount::setUserPointer\(\)](#), page 219

C++ SPECIFICATION

```
bool ReferenceCount::release() const
```

DESCRIPTION

Decreases reference count of the object by one. If the count reaches zero or becomes negative, the object's destructor is called.

RETURNS

True if the object got destructed, false otherwise

NOTES

It is valid to call this function for a NULL pointer - in such a case nothing is done and 'true' is returned. This corresponds with the convention used in C++ for deleting NULL pointers.

SEE ALSO

ReferenceCount::autoRelease(), page 213

C++ SPECIFICATION

```
void ReferenceCount::setName(const char* name)
```

DESCRIPTION

Defines a name string for the instance

PARAMETERS

name NULL-terminated string (may be NULL)

NOTES

The name strings are not used internally - they are only provided to facilitate debugging. A copy of the string is taken internally, so the original string can be discarded after this call. The copy of the string is destroyed when the reference counted object is destroyed.

The name string can be defined multiple times per object. If a NULL string is provided, the object will become unnamed. Initially all objects are unnamed.

SEE ALSO

ReferenceCount::getName(), page 214

C++ SPECIFICATION

```
void ReferenceCount::setUserPointer(void* p)
```

DESCRIPTION

Defines a user pointer for the object

PARAMETERS

p User-defined pointer (may be NULL)

NOTES

User pointers are used for mapping dPVS classes into the corresponding structure inside the user's application. See section 6.4.4 for further information.

dPVS never modifies this pointer or the data pointed by it. The user pointer initialized to NULL in the constructor of the object. The application user can define a new user pointer at any time.

On 64-bit architectures the pointer is 8 bytes long, i.e. it is long enough to point to any structure.

SEE ALSO

ReferenceCount::getUserPointer(), page 216

8.13 RegionOfInfluence

C++ SPECIFICATION

```
class RegionOfInfluence
```

DESCRIPTION

Class representing a *region of influence*

BASE CLASSES

ReferenceCount, Object

HEADER FILE

dpvsObject.hpp

NOTES

Regions Of Influence (or ROIs for short) are used to place light sources and similar objects into the scene. The camera's visibility resolving process will determine overlaps between ROIs and other objects.

The actual region is defined by assigning a test model to the ROI. In order to model a spot light, a mesh model of the corresponding shape can be used.

ROIs are tested during the visibility query in a similar fashion to other objects. This means that if a ROI is fully outside the view frustum or fully occluded, it will be rejected prior to the ROI vs. object overlap tests.

Directional lights and other unbounded Regions of Influence are best expressed by using the `Object::UNBOUNDED` property.

See section [6.10](#) for further information about the `RegionOfInfluence` class.

Because the class is derived from `ReferenceCount`, it does not have a public destructor. Use the function `ReferenceCount::release()` to release instances of the class.

PUBLIC INTERFACE

```
create()
```


C++ SPECIFICATION

```
static RegionOfInfluence* RegionOfInfluence::create(Model* testModel)
```

DESCRIPTION

Constructs a new RegionOfInfluence

PARAMETERS

testModel Pointer to a test model (non-NULL)

RETURNS

Pointer to the object or NULL on failure

NOTES

See documentation for `Object::create()` for information about how the ROI attributes are initialized in the constructor.

The function will fail if there is not enough memory in order to allocate the RegionOfInfluence object. It will also fail if the *testModel* parameter is NULL.

SEE ALSO

`Object::create()`, page 182

8.14 SphereModel

C++ SPECIFICATION

```
class SphereModel
```

DESCRIPTION

A model type describing a sphere with a center position and a radius

BASE CLASSES

ReferenceCount, Model

HEADER FILE

dpvsModel.hpp

NOTES

Sphere models are the simplest type of models. A `SphereModel` is a spherical shape defined by a translation vector in the object's local coordinate system and a positive radius value. Although many shapes are usually bounded poorly by spheres, this class is provided so that certain complicated systems - such as particle systems or point light sources - can be represented easily. An additional advantage is that sphere models consume very little memory. Note that `SphereModels` cannot be used as write models. Because the class is derived from `ReferenceCount`, it does not have a public destructor. Use the function `ReferenceCount::release()` to release instances of the class.

PUBLIC INTERFACE

`create()`

C++ SPECIFICATION

```
static SphereModel* SphereModel::create(const Vector3& center, float  
radius)  
static SphereModel* SphereModel::create(const Vector3* vertices, int  
numVertices)
```

DESCRIPTION

Constructs a new sphere model

PARAMETERS

<i>center</i>	Reference to object-space center position
<i>radius</i>	Reference to object-space radius (must be positive)
<i>vertices</i>	Array of object-space vertex locations (non-NULL)
<i>numVertices</i>	Number of vertices in the array (1 or more)

RETURNS

Pointer to the model or NULL on failure.

NOTES

A sphere model can be constructed either by providing the sphere itself, or an array of vertices from which a sphere can be computed.

In debug build an assertion is raised if *radius* is negative.

The function will fail if there is not enough memory in order to allocate the model. The function will also fail if any of the input parameters are invalid.

8.15 Vectors and Matrices

The dPVS API defines, but does not implement, several vector and matrix classes. It is likely that users want to use their own implementation for these classes. This can be achieved by first defining the macro `DPVS_OVERRIDE_VECTOR_TYPES` before including any dPVS headers and then using typedefs. The data layout for the vector/matrix classes can be found from the header file **dpvsDefs.hpp**. The matrix data organization, i.e. whether it is column or row major, can be selected at run-time when `Library::init()` is called. Functions dealing with projection matrices take additional parameters for indicating the matrix handedness.

C++ SPECIFICATION

```
class Vector3i
```

DESCRIPTION

Structure for holding three 32-bit signed integers

HEADER FILE

dpvsDefs.hpp

VALUES

INT32 i	First integer component
INT32 j	Second integer component
INT32 k	Third integer component

NOTES

This structure is used mainly to describe triangle vertex indices in some API functions.

C++ SPECIFICATION

```
class Vector2
```

DESCRIPTION

Single-precision two-component vector

HEADER FILE

```
dpvsDefs.hpp
```

VALUES

float x	First component
float y	Second component

C++ SPECIFICATION

```
class Vector2d
```

DESCRIPTION

Double-precision two-component vector

HEADER FILE

```
dpvsDefs.hpp
```

VALUES

<code>double x</code>	First component
<code>double y</code>	Second component

C++ SPECIFICATION

```
class Vector3
```

DESCRIPTION

Single-precision three-component vector

HEADER FILE

```
dpvsDefs.hpp
```

VALUES

float x	First component
float y	Second component
float z	Third component

C++ SPECIFICATION

```
class Vector3d
```

DESCRIPTION

Double-precision three-component vector

HEADER FILE

```
dpvsDefs.hpp
```

VALUES

<code>double x</code>	First component
<code>double y</code>	Second component
<code>double z</code>	Third component

C++ SPECIFICATION

```
class Vector4
```

DESCRIPTION

Single-precision four-component vector

HEADER FILE

```
dpvsDefs.hpp
```

VALUES

float x	First component
float y	Second component
float z	Third component
float w	Fourth component

C++ SPECIFICATION

```
class Vector4d
```

DESCRIPTION

Double-precision four-component vector

HEADER FILE

```
dpvsDefs.hpp
```

VALUES

<code>double x</code>	First component
<code>double y</code>	Second component
<code>double z</code>	Third component
<code>double w</code>	Fourth component

C++ SPECIFICATION

```
class Matrix4x4
```

DESCRIPTION

Single-precision 4x4 matrix

HEADER FILE

```
dpvsDefs.hpp
```

NOTES

dPVS uses 4x4 matrices for all matrix data, even if most functions can only accept 4x3 matrices, i.e. they do not allow a projection term.

Whether the matrices are interpreted to be column or row major is up to the user. The user should indicate this when calling `Library::init()`.

SEE ALSO

`Library::init()`, page 157

C++ SPECIFICATION

```
class Matrix4x4d
```

DESCRIPTION

Double-precision 4x4 matrix

HEADER FILE

dpvsDefs.hpp

NOTES

dPVS uses 4x4 matrices for all matrix data, even in most functions can only accept 4x3 matrices, i.e. they do not allow a projection term.

Whether the matrices are interpreted to be column or row major is up to the user. The user should indicate this when calling `Library::init()`.

SEE ALSO

`Library::init()`, page 157

8.16 VirtualPortal

C++ SPECIFICATION

```
class VirtualPortal
```

DESCRIPTION

Class describing an arbitrary transformation between portals

BASE CLASSES

ReferenceCount, Object, PhysicalPortal

HEADER FILE

`dpvsObject.hpp`

NOTES

Virtual portals express an arbitrary transformation between a source and a target cell. Special effects such as mirrors, teleports etc. can be created by using the `VirtualPortal` class.

See section [6.11](#) for an overview of physical and virtual portals.

Because the class is derived from `ReferenceCount`, it does not have a public destructor. Use the function `ReferenceCount::release()` to release instances of the class.

PUBLIC INTERFACE

```
create()  
getTargetPortal()  
getWarpMatrix()  
setTargetPortal()  
setWarpMatrix()
```

C++ SPECIFICATION

```
static VirtualPortal* VirtualPortal::create(Model* testModel,  
PhysicalPortal* targetPortal)
```

DESCRIPTION

Creates a new virtual portal

PARAMETERS

<i>testModel</i>	Pointer to test model (may not be NULL)
<i>targetPortal</i>	Pointer to physical portal

RETURNS

Pointer to the virtual portal or NULL on failure

NOTES

The function will fail if there is not enough memory in order to allocate the virtual portal or if the *testModel* parameter is NULL.

C++ SPECIFICATION

PhysicalPortal* VirtualPortal::getTargetPortal() const

DESCRIPTION

Returns pointer to target portal

RETURNS

Pointer to target portal

NOTES

The function returns NULL if no target portal has been defined.

SEE ALSO

VirtualPortal::setTargetPortal(), page 238

C++ SPECIFICATION

```
void VirtualPortal::getWarpMatrix(const Matrix4x4& mtx) const  
void VirtualPortal::getWarpMatrix(const Matrix4x4d& mtx) const
```

DESCRIPTION

Returns the transformation matrix used for portal-to-portal traversal.

PARAMETERS

mtx Reference to the warp matrix

NOTES

For more information about warp matrices, see section [6.11.5](#).

The matrix format used has been defined in the call to `Library::init()`. The matrix is initialized to identity in the object constructor.

SEE ALSO

`VirtualPortal::setWarpMatrix()`, page 239

C++ SPECIFICATION

```
void VirtualPortal::setTargetPortal(PhysicalPortal* target)
```

DESCRIPTION

Defines a target portal for the virtual portal

PARAMETERS

target Pointer to target (physical) portal

SEE ALSO

VirtualPortal::getTargetPortal(), page 236

C++ SPECIFICATION

```
void VirtualPortal::setWarpMatrix(const Matrix4x4& mtx)
void VirtualPortal::setWarpMatrix(const Matrix4x4d& mtx)
```

DESCRIPTION

Assigns a transformation matrix used for portal-to-portal traversal

PARAMETERS

<i>mtx</i>	Reference to a warp matrix
------------	----------------------------

NOTES

For more information about warp matrices, see section [6.11.5](#).

The matrix may not contain any projection (i.e. the fourth column/row must be 0, 0, 0, 1) The matrix format used has been defined in the call to `Library::init()`. The matrix is initialized to identity in the constructor.

SEE ALSO

`VirtualPortal::getWarpMatrix()`, page 237

Part III

Visibility Theory Guide

Chapter 9

Introduction

9.1 Exact Visibility

Visibility determination has been an integral part of computer graphics since the 1960s. In order to correctly display three-dimensional graphics on a flat display, two fundamental requirements arise. First, proportions and relative sizes of objects must behave as we observe them in the real world. This is solved by using a perspective projection. Secondly, the closest object at any given screen pixel should be drawn.¹ An analytical solution that is not tied to any fixed resolution is called *continuous exact visibility* whereas *point sampled exact visibility* resolves the visibility for a fixed-resolution screen. Insufficient resolution causes *aliasing*. For our purposes continuous and point sampled algorithms solve the same problem, and despite their fundamental difference, we consider both categories as *exact visibility algorithms*. Numerous algorithms for solving the problem have been proposed, and finally in 1992 Charles Grant presented a complete taxonomy of exact visibility algorithms. At present, practically all graphics hardware solve the exact visibility determination by using the Z-buffer algorithm [15].

9.2 Conservative Visibility

Having the exact visibility problem solved, the output image is guaranteed to be correct as long as all contributing geometry is processed. If we somehow know that a car is hidden by a building, there is clearly no point in processing the car. Conservative visibility algorithms try to find objects that do not contribute to the output image. Furthermore, there is a potential possibility for optimizing complex physics, AI and collision calculations, if there is no-one to check the results. Philosophers in ancient Greece were puzzled with the question of whether a falling tree makes a sound, if there is no-one listening. In virtual environments, it is both safe and wise to say that it does not have to. Indeed, it is beneficial to declare that nothing in the world needs to happen, or to exist, if it is not observed.

During the 1990s the research emphasis moved strongly towards conservative visibility algorithms and occlusion culling. Compared to exact visibility algorithms, they are significantly more complicated, and no single algorithm is expected to solve all types of applications satisfactorily.

¹If antialiasing is being performed, multiple objects can contribute to a single pixel.

9.3 Scope of This Thesis

Our goal is to create a platform-independent, general purpose visibility determination library, dPVS. The library must not require any pre-processing or static data structures. However, it should benefit from *a priori* visibility information, if such knowledge is available. The architecture must scale from marginal to huge depth complexity, from handful to millions of objects and from thousands to hundreds of millions of polygons.

We evaluate existing algorithms according to the requirements listed in the previous paragraph and review new contributions in detail. First, we consider exact visibility determination algorithms because the conservative algorithms inherit algorithmic properties from the exact ones. We briefly list some of the non-conservative research areas, since both conservative visibility determination and non-conservative methods are required for visualization of arbitrarily complex scenes.²

We do not go into any depth with the nature of visibility; an interested reader should refer to [31]. We do not consider area visibility and related issues of radiosity, soft shadows and penumbræ. Discussion of algorithms used to handle dynamic spatial databases is limited to references to the ones used in dPVS. This thesis concentrates mainly on visibility issues related to *direct image formation*; higher-order visibility, such as reflections, is considered only in the context of portal rendering.

9.4 Visibility Pipeline

The process of visibility determination can be divided into the following sub-categories.

1. *View Frustum Culling* removes objects that are outside a virtual camera's view volume. In other words it resolves whether an object can contribute to the final image when not obscured by other objects.
2. *Occlusion Culling* removes objects or parts of objects that are obstructed from view by other objects. This test involves more than one object, and is therefore a global problem and significantly harder than view frustum culling. The primary focus of this thesis is on occlusion culling.
3. *Back-face culling* removes parts of the object that are facing away from the camera and therefore do not contribute to the final image.
4. *Exact Visibility Determination* resolves a single or multiple contributing primitives for each output image pixel.
5. *Contribution culling* removes objects that would have only a minor impact on the final image. Whereas the other four sub-categories leave the output image intact, contribution culling introduces artifacts. The term *aggressive culling* is sometimes used to refer to such non-conservative methods.

²As the visible portions of the scene can be arbitrarily complex.

9.5 Target Applications of the Library

The library architecture presented in this thesis is designed to improve performance in several types of target applications. Unlike most of the research, practical applications do not generally have scenes with millions of polygons and a high depth complexity. Dominant portion of interactive 3D visualization happens in video games and in other VR applications. The library design is targeted to work well also with the modest scene complexities of video games. It is worth noticing that occlusion culling algorithms generally improve their performance as the scene complexity increases, and not many of them are of practical value with current video game complexities. The following target application types are considered:

- Architectural walkthroughs and indoor environments
- Urban environments
- Flight simulators
- Terrain visualization
- Fully dynamic scene visualization
- Applications with a pre-rendered background.

9.6 Plan of Development

Chapter 1 is this introduction, including the problem definition, the scope of this thesis, types of target applications, and the plan of development. Evaluation of previous work on both exact and conservative visibility determination is presented in chapter 2. New contributions are reviewed in detail in chapter 3. Chapter 4 describes the overall library architecture with discussion on design solutions made. Chapter 5 summarizes the results and conclusions. Finally, possible extensions to the library architecture and potentially useful pre-processing tools are listed in chapter 6.

9.7 New Algorithms and Concepts

- Temporally coherent occlusion-preserving silhouette extraction (section 11.8.1).
- Occlusion Write Postpone Queue (section 11.7).
- Validation Buffer (section 11.10.7).
- Construction-Time Contribution Culling (section 11.10.5).
- An incremental variation of Hierarchical Occlusion Maps (section 11.9).
- Visible Point Tracking (section 11.4).
- Hierarchical Depth Estimation Buffer (section 11.11).
- Adaptive Occluder Selection with Feedback (section 11.3.2).
- Portal PVS (section 11.14).

- A Taxonomy of Conservative Visibility Algorithms ([11.2](#)).
- Spatial database nodes as dynamic Virtual Occluders (section [11.13](#)).

Chapter 10

Previous Work

In this chapter we first state our primary design principles for algorithms. Then we evaluate exact visibility determination algorithms by following the taxonomy of basic visibility algorithms [50] shown in figure 38. Exact visibility algorithms are reviewed, since they can be seen as the basic building blocks of conservative visibility algorithms, and because conservative algorithms generally require an exact visibility algorithm as a back-end to visualize the scene correctly.

In 10.4 an overview and analysis of conservative visibility algorithms and their practical usability is presented.

Both conservative visibility determination and non-conservative methods are required for visualization of arbitrarily complex scenes. Non-conservative methods are briefly discussed in 10.5. They are seen as an important future extension to our framework, and should therefore be considered when making design decisions.

Data organization models for spatial databases are briefly discussed in 10.6.

Antialiasing is outside the primary scope of this thesis, but since visibility determination and antialiasing are tightly coupled, we refer to aliasing issues in a few places in the text.

10.1 Fundamental Concepts

Bounding Volumes

Instead of performing computations directly with the actual geometry, it is often beneficial to surround an object with something simpler and to perform the computation first using the simplified bounding volume. If the bounding volume is found to be hidden, all geometry inside it, however complicated, is hidden as well [116]. Usually this surrounding object would be a bounding sphere, an axis-aligned bounding box (AABB), an oriented bounding box (OBB), a k-dop¹ or the convex hull of a mesh.

¹Construction of a k-dop begins by selecting K directions that cover the entire direction-space as evenly as possible. 6-dop is a box. Vertices are projected onto the direction vectors and the maximal projection on each direction is chosen. These K maximal distances represent the k-dop.

Occluders and Occludees

An *Occluder* is an object that obstructs other objects from the viewpoint of the camera. Any visible object is a potential occluder. *Occlusion power* is a characteristic estimating how much geometry an object manages to obstruct from the viewpoint of the camera. An *occludee* is an object obstructed by one or more objects. An *occluder set* is a list of all objects that were chosen as occluders. The optimality of an occluder set can be evaluated using a *cost-benefit function*, which compares the time lost by using the set of objects as occluders and the processing time won by avoiding processing the occluded geometry. Typically the losses and gains have to be approximated. Finding the *optimal occluder set* is view-dependent and an NP-hard problem², therefore an approximation must be used.

Virtual Occluders

Law and Tan [72] made a superb observation that any hidden³ object or volume in the scene can act as an occluder. Occluding geometry can be replaced with considerably simpler virtual occluders.

A classic example is a dense forest where visibility is only 15 meters. If a virtual occluder could be placed at the distance of 15 meters from the camera, all the trees actually blocking the visibility could be ignored as occluders. Savings can be tremendous. Virtual occluders mainly reduce occluder setup time, and in some cases can provide pre-computed occluder fusion.

Schaufler *et al.* [96] discretize the entire scene and flood fill the occupied regions of space to create a discrete and simple presentation of combined occluders. Filling the interiors of non-manifold, non-closed surfaces is potentially an ambiguous problem, and certainly a challenging task to perform dynamically. This approach can be seen as a way of compactly expressing Potentially Visible Sets (see chapter 31).

The implementation of Koltun *et al.* [69] is limited to 2.5D and the authors present an algorithm for calculating the augmented umbrae from an area. Creating such umbrae in 3D is difficult, but would be the key for constructing temporally coherent virtual occluders. Such occluders would in fact solve many important issues. We believe that efficient construction of virtual occluders has become one of the most important research topics of the visibility research community.

Bernardini *et al.* [11] use octree faces as virtual occluders. They create the virtual occluder octree at a pre-processing phase. We adopt the principles of this approach and extend it to dynamic scenes in section 11.13.

Hierarchical bounding volumes are commonly used as virtual occludees. Therefore the introduction of virtual occluders provided a missing piece to a puzzle.

Occluder Selection

Selecting a good set of objects as occluders is a fundamental requirement for an efficient occlusion culling algorithm. Regardless of their importance, occluder selection algorithms have received only modest attention. The usual classification criteria are based on:

1. *Size*: Small objects are bad occluders.

²With N objects there are $O(N!)$ different occluder sets. Due to *occluder fusion*, the occlusion power of an object depends on possibly all other objects. Thus finding the optimal set is a combinatorial problem.

³Hidden object is an object that is occluded by other objects.

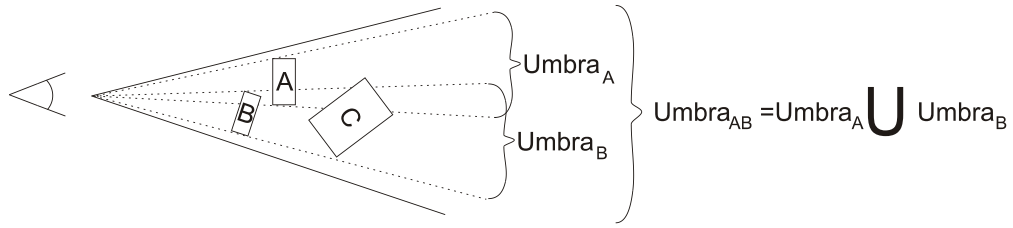


Figure 10.1 The principle of occluder fusion. Objects A and B can together occlude object C, even though neither of them can occlude C on their own.

2. *Complexity*: Complex objects are expensive to use as occluders.
3. *Redundancy*: Certain objects, such as a clock on a wall, do not contribute significantly to the overall occlusion. These objects are sometimes called *detail objects*.

The question of a good occluder set is *completely* context-dependent and none of the above criteria can provide actual information. A tennis ball can occlude the Statue of Liberty. When there is no feedback from the occlusion system, no context-dependent information can be obtained. We can go even further and to claim that the occlusion culling algorithm must be *incremental* to provide such information. See section 11.3 for a cost-benefit analysis of occlusion culling.

Occluder Fusion

Merging multiple objects into a single occluder always has greater or equal occlusion power than the sum of the separate occlusion powers. In many cases the difference is significant. In case of figure 10.1 neither occluder A nor B alone can obstruct object C, but together they can. To illustrate the idea, consider a forest. A single tree does not occlude much, but a hundred trees may be able to occlude large portions of the screen.

Shadows of point light sources and the hidden regions of a camera's view frustum are schematically identical. Thus the concept of shadow volumes, introduced by Crow [29], has an analogue in visibility determination, *occlusion volumes*. In fact, all algorithms applicable to shadow computation of point light sources can be used for visibility determination and vice versa.

Another aspect of occluder fusion are consecutive semi-transparent surfaces. Even if it is possible to see through each of the surfaces individually, stacking enough of these surfaces creates a virtually opaque surface⁴.

Potentially Visible Sets

The potentially visible set (PVS)⁵[4] is a conservative superset of the actually visible set (AVS) of primitives or objects. We call the initial PVS consisting of the entire scene database PVS_{scene} . View frustum culling bounds the PVS_{scene} to view-dependent $PVS_{frustum}$, where usually $\|PVS_{frustum}\| \ll \|PVS_{scene}\|$. If the scene has even modest depth complexity,

⁴Limited by the color resolution of the output device.

⁵A PVS contains *all* visible objects and some hidden ones.

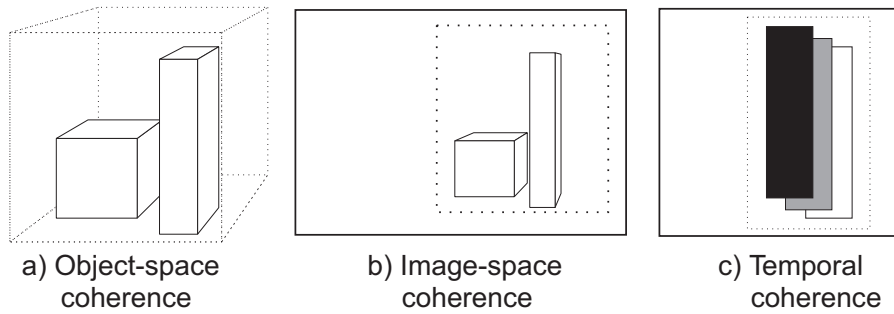


Figure 10.2 Object-space, image-space and temporal coherence

$\|PVS_{frustum}\| \gg \|AVS\|$ holds. The purpose of occlusion culling is to reduce $PVS_{frustum}$ to $PVS_{occlusion}$, that better matches AVS . Exact visibility algorithms are needed to resolve AVS from any given PVS . Conservative visibility algorithms minimize the input $\|PVS\|$ of the exact algorithm.⁶

10.2 Design Principles

Coherence

As recognized by Sutherland, Sproull and Schumacker some twenty-five years ago in their classic survey of visible-surface algorithms [106], exploiting coherence is the key for designing efficient visibility algorithms. A more recent study by Gröller and Purgathofer [55] lists more than a thirty different types of coherence present in computer graphics. Of these, our primary emphasis is on object-space, image-space and temporal coherence (figure 10.2).

Object-space coherence is based on some known relationships between objects or parts of an object. Adjacent regions of space are likely to be occupied by the same objects. Therefore a single computation may suffice to say something definitive about a group of objects or all primitives of an object. If the bounding volume of an object intersects only the left plane of the view frustum, its primitives need only be tested against the left plane. In a similar fashion, if a bounding volume is determined to be hidden, the actual object inside the volume does not have to be processed.

Image-space coherence is the view-dependent analogue of object-space coherence. Object-space coherence transforms into image-space coherence on the view plane under well-behaved projections. The primary difference is that a projection also *creates* coherence, since distant objects may form coherently behaving groups on the view plane. All operations taking place in 2D are considered to be exploiting image-space rather than object-space coherence.

Temporal coherence exists when there is a smooth change in the motion of either the objects or the camera. Exploiting temporal coherence can open possibilities for calculating results that are valid during a longer period of time, not just a single moment. We are using the word 'can' here, because temporal coherence is very hard to exploit properly and even though many algorithms claim to be temporally coherent, they usually benefit only marginally from the previous results. Literature is

⁶Some of the inequalities do not hold if higher-order visibility, i.e. reflections, is considered.

10.2 Design Principles

extremely ambiguous with the precise differences between temporal and *frame coherence*. Therefore we classify temporally coherent properties to the following categories:

1. An algorithm can decide that a calculated result is valid until a specific condition is broken. The condition can be a time period, a point inside a given volume or another preferably simple condition. In interactive worlds time alone is hardly ever a sufficient condition.
2. An algorithm can use results from the previous frames to speed up computations of the current frame. Due to cache architectures of modern computers, basically every algorithm benefits from the work done in the previous frames.
3. Algorithms that calculate results always from scratch.

Output-Sensitivity and Hierarchical Processing

In 1976 Clark [22] set an objective to design a visibility algorithm “in which the computation time increased linearly with the visible complexity of the scene”.⁷ A detailed model of New York could contain billions of objects, and yet only a small fraction would be visible from any given location. Selecting the visible objects without testing the hidden ones is clearly possible only, if the set of visible objects has been stored during a pre-process. In absence of pre-computed visibility information, the best one can do is to organize the objects into bounding volume hierarchies and use object-space coherence to apply the results downwards in the hierarchy. If Manhattan is hidden, there is no need to test the visibility of Carnegie Hall and definitely not a chair inside it. As first noted by Warnock [114], hierarchical processing can be thought of as a logarithmic search for primitives of interest. Maintaining output-sensitivity is a strict constraint. It should be noted that any operation accessing more objects than a linear function of visible objects violates the principle of output-sensitivity. Hierarchical processing has been found very efficient in acceleration of both object-space and image-space operations.

Lazy Evaluation

Nothing has to be processed before the result is needed. This principle has the following benefits:

1. Simplifies the logic outside the component. When not all control paths require all results, no performance penalty occurs.
2. Improves performance by skipping work.
3. Improves cache-coherence by executing consecutive operations on the same data.

Portability

The algorithms should not rely on specific hardware or rarely supported features.

⁷Clark limits the output-sensitivity to *linear* processing of visible complexity of the scene. As a result, more complex operations such as quicksort do therefore violate the original definition of output-sensitivity.

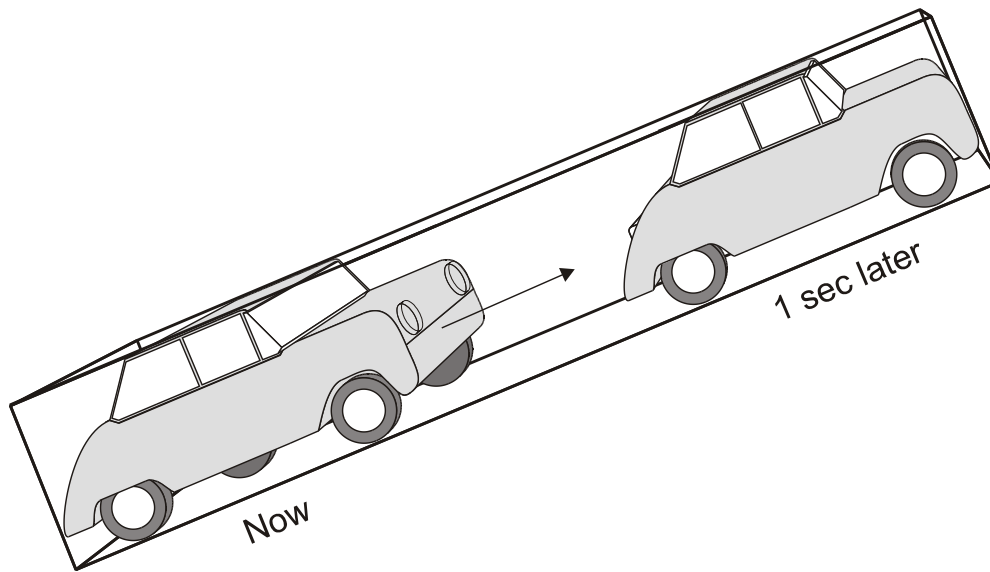


Figure 10.3 Temporal bounding volume of a car over the duration of one second

Generality

The algorithms should be applicable to arbitrary models. Limiting the applicability to convex models, models with large polygons, or architectural models are considered unacceptable.

Robustness

The algorithms must not be prone to floating-point errors. Computational geometry algorithms are often prone to degeneracies, discontinuities and other problems.

Applicability to Dynamic Environments

If we constrain the world to be fully static, everything becomes much simpler. First of all, the need for physics, dynamics and collision detection⁸ are all gone. Visibility determination and spatial data structures also become significantly easier. Unfortunately, such a constraint is only suitable for walkthroughs, and no decent virtual environment can be restricted to be static. However, the fact that most objects are indeed static is an important observation. A house does not move, but when the window gets broken, the house is a dynamic object for a short moment. Bearing the above in mind, we conclude that classifying something as static must be done dynamically, and the fact that many objects are static should be exploited. No two static objects can ever interact. Maintaining a dynamic hierarchical spatial index was thought impractical by several researchers during the past two decades, and finally shown feasible by Sudarsky in 1996 [104]. Indeed, the introduction of *temporal bounding volumes* (TBV) (figure 10.3) significantly reduces the amount of required database updates, and provides temporal coherence to bounding volume tests of dynamic

⁸Except for the camera itself.

10.2 Design Principles

objects. We consider algorithms relying solely on pre-processed static information unsuitable for practical use.

10.3 Exact Visibility Determination

In this section some of the basic visibility algorithms are reviewed and their most significant features are underlined. Especially suitability for parallel hardware implementation is considered.

10.3.1 Important Algorithmic Properties

Hardware Parallelization

It is crucial that an algorithm is suitable for *hardware implementation*, which in turn implies the requirement of *parallelization*. Parallelization can be divided into image-space parallelization (sub-division) and per-object parallelization.

Combinatorial Blowout

Many algorithms consider variable amounts of data for determining the visibility of a single element. Such an approach can lead to the problem commonly referred to as *combinatorial blowout*. Algorithms suffering of combinatorial blowout are usually not suitable for hardware parallelization due to the complex logic needed.

Cache-Coherence

An algorithm should preferably be *cache-coherent*. Since current computer hardware only has a small amount of fast cache memory, an object should optimally enter the cache only once per frame. This problem is in fact getting worse all the time, as silicon speeds grow faster than bus speeds. To illustrate this problem, consider a scene of 100 objects each consisting of 1000 triangles. Such a scene can be interactively rendered with current graphics hardware. Sorting these 100.000 triangles is theoretically an $O(n)$ operation⁹ that will fall out of the cache memory on almost all consumer-level computers. As a result, global sorting would indeed be a dominant cost in rendering.

Incremental

An algorithm is *incremental*¹⁰ if new primitives can be added to the current solution. In absence of this property, all primitives of the entire scene must be available at the same time. This is hardly ever practical. If the primitives are cached, and finally processed in a single batch, the principle of cache-coherence is violated and geometry $\leftrightarrow$ rasterization parallelism is lost.

10.3.2 Grant's Taxonomy

The first systematic study in exact visibility determination algorithms was published in 1973 and 1974 by Sutherland, Sproull and Schumacker¹¹ [106]. Since their study many new algorithms have

⁹With bucket sort, if cyclic overlaps are not handled properly. $O(n^2)$ worst case if cyclic overlaps are handled.

¹⁰Some authors use the term *progressive* instead.

¹¹SSS taxonomy.

10.3 Exact Visibility Determination

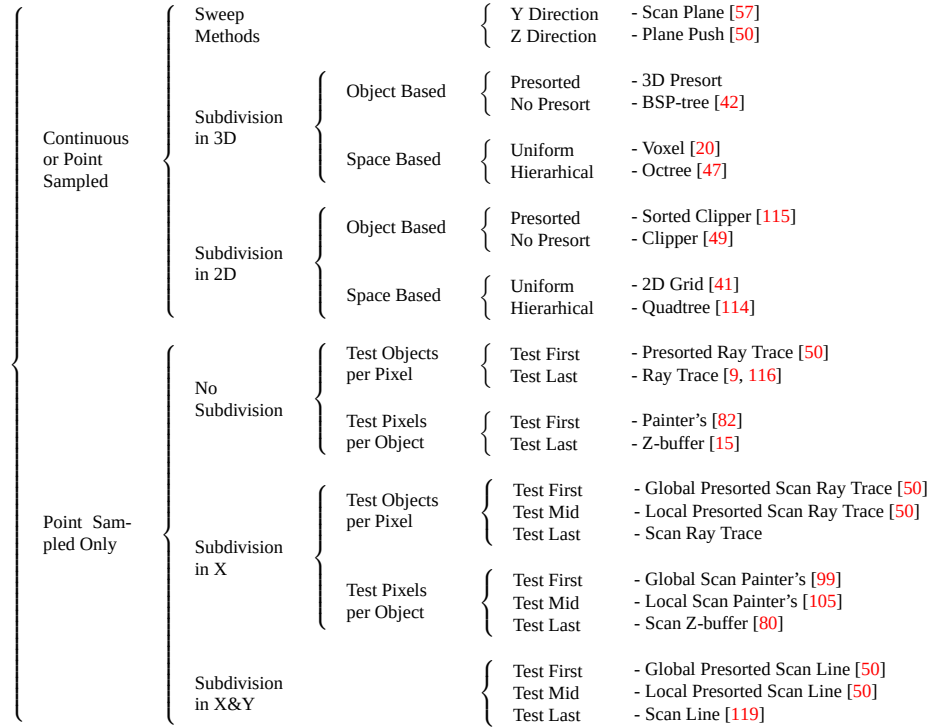


Figure 10.4 Complete taxonomy of exact visibility algorithms according to Grant[50]

been published and Grant presented his revised taxonomy in 1992[50]. This taxonomy did not consider conservative visibility algorithms, which have been the major area of research in the 1990s. Nor did it contain the fundamental new hierarchical algorithm HZ-buffer (section 10.4.2), which was introduced a few years later. Unfortunately it does not seem possible to add HZ-buffer into Grant's taxonomy.

10.3.3 Painter's Algorithm

An artist creating an oil painting first paints the background colors, then the distant objects and finally the foreground objects are painted on top of everything. Painter's algorithm[82] is motivated by this behavior. First the objects are sorted in order of decreasing depth, then drawn starting from the back.

In the first pass, primitives are sorted according to their largest depth values. Primitive P with the largest depth value is compared with other primitives for depth overlap. If no overlap occurs, the primitive is drawn. In the case of a depth overlap, additional tests are required to solve whether any of the primitives should be reordered. Reordering is unnecessary if any of the following tests are true.

1. The 2D screen-space bounding rectangles of the two primitives do not overlap.

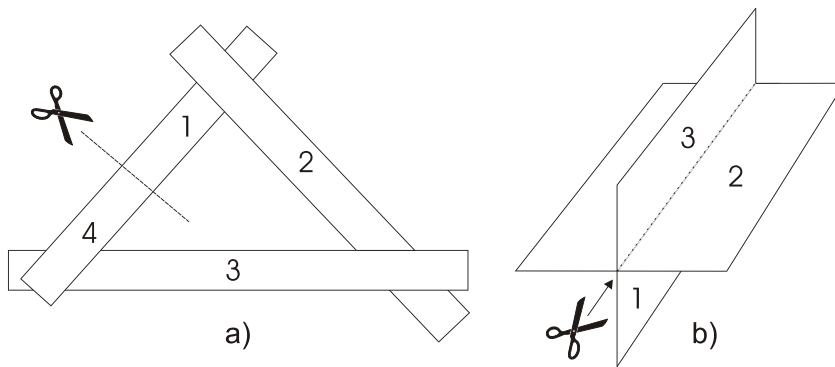


Figure 10.5 Classic problem cases of the Painter's algorithm. The numbers indicate the correct drawing order.

2. Primitive P is completely behind the overlapping primitive.
3. The screen projections of the two primitives do not overlap.

Reordering can get stuck in an infinite loop when two or more primitives alternately obscure each other (figure 10.5)¹². To avoid such situations, the primitives are flagged as they are reordered to a farther depth position. When attempting to reorder a flagged primitive, the primitive is split into two parts and the procedure continues.

Advantages

1. Primitives with transparencies are handled correctly.

Drawbacks

1. Combinatorial blowout due to sorting process.
2. All primitives must be considered at the same time, therefore the algorithm is not incremental.
3. Hardware parallelization is difficult and would be coarse-grained.
4. Every pixel in every primitive is processed, resulting in a lot of unnecessary work.

10.3.4 Z-buffer

In addition to the frame buffer, another equal-sized buffer holding depth values for each pixel is kept. First the depth buffer is initialized to the farthest possible value. For every pixel in every primitive, we test whether the pixel is located closer to the camera than the current depth buffer value. If the test fails, the current primitive pixel is not visible. Otherwise the depth buffer value is updated, and the pixel is drawn into the frame buffer (figure 10.6).

¹²Numbers on the primitives indicate the correct drawing order.

10.3 Exact Visibility Determination

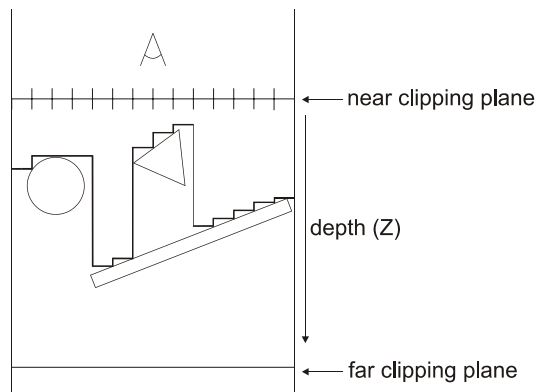


Figure 10.6 Z-buffer principle

Advantages

1. A single primitive can be considered at a time. Therefore Z-buffer is incremental and does not suffer from combinatorial blowout.
2. Different types of geometric primitives can be used together.
3. Parallelization is trivial via subdivision.
4. Rasterization is image-space coherent. The depth values can be computed incrementally.
5. Logic required for depth coordinate interpolation is the same as for all other surface parameters, and is therefore readily implemented into pixel hardware pipelines.
6. Can be generated from real-life imagery [1]¹³
7. The depth buffer can be stored along with the output image to allow further modifications.

Drawbacks

1. Every pixel in every primitive is processed, resulting in a lot of unnecessary work done in densely occluded scenes.¹⁴
2. Additional memory requirements per pixel¹⁵, although this drawback is constantly diminishing as memory becomes cheaper.
3. Transparent primitives are handled incorrectly.

Usually opaque primitives are processed first, and depth buffer updates are disabled before processing back-to-front sorted transparent primitives. To handle transparencies correctly, the sorting stage should be implemented using the Painter's algorithm. In practice this is hardly ever done, and some combinations of transparent primitives are indeed drawn incorrectly. These effects do not show

¹³Minolta VIVID 700 3D digitizer grabs a 200x200 depth buffer in addition to a 400x400 color buffer in 0.6 seconds.

¹⁴*Deferred shading* can be used to limit the unnecessary work to just the depth tests.

¹⁵Memory requirements can be bounded by dividing the screen to tiles but this removes some of the advantages.

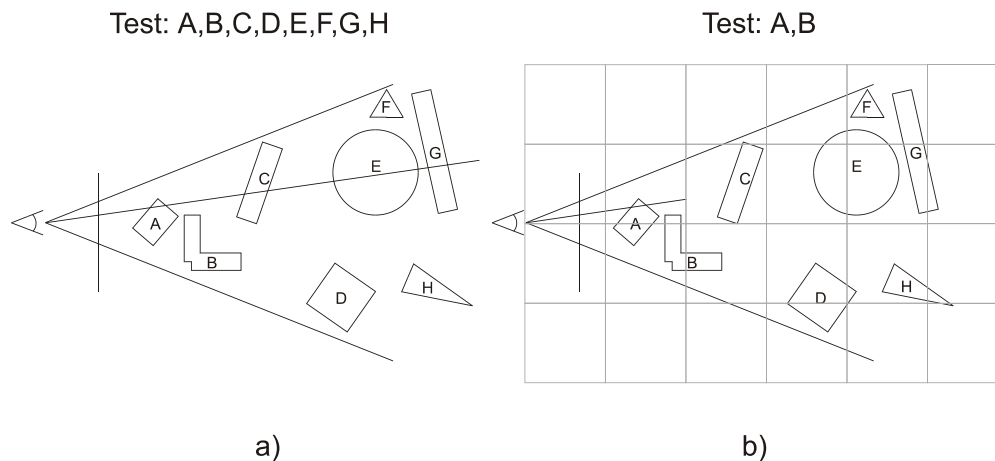


Figure 10.7 Raycast principle. A brute force approach (a) tests all objects for intersection with a ray. A more sophisticated approach (b) only seeks the first intersection and utilizes spatial subdivision to reduce the number of tests.

up very often, and can be almost fully avoided during the modeling phase by avoiding the known problem cases.

Mammen [76] was the first to propose a solution to make the Z-buffer handle transparencies correctly. Though theoretically a functional approach, his multi-pass algorithm is rarely used in practice.

*A-Buffer*¹⁶ [14, 39] handles transparencies correctly, and additionally provides substantial improvements to the image quality. The algorithm is a simplified version of Catmull's algorithm[16], which clips polygons to pixel boundaries. A-Buffer is currently used by Pixar in their rendering system called REYES¹⁷[25]. The basic idea is to store more than one contributing sample per output image pixel, and to finally combine all samples according to their coverage percentage, transparency and other surface attributes. The price to pay is a varying per-pixel memory usage. Variants with constant per-pixel memory usage have been proposed [66]. All algorithms in this category have a very high memory consumption, and are usually implemented by processing the output image in fixed-size tiles, which in turn violates the principle of incrementality.

An *item buffer* stores an ID for each output pixel. The ID is usually a 32-bit identifier indicating which object or primitive is visible at the output pixel. Potential uses of item buffers include the generation of PVSs, object/primitive picking and speeding up certain radiosity computations [24].

10.3.5 Ray Casting and Ray Tracing

Ray casting [9] casts a ray from the camera through the center of each output image pixel (figure 10.7). The output color is calculated from the primitive with the closest intersection. The brute-force version (a) would test every primitive against every ray, but we adopt here the somewhat more realistic approach (b) and expect an approximate front-to-back traversal, which is achieved usually with an octree [47], or some other object-space hierarchy[17]. More accurate results can be obtained by shooting multiple rays per pixel and filtering the resulting shading values. Whereas ray

¹⁶antialiased averaged area accumulation buffer.

¹⁷Renders Everything You Ever Saw

10.3 Exact Visibility Determination

casting stops when the first intersection is found, ray tracing [116] continues recursively by shooting reflected and refracted rays from the intersection points. The contributions of the recursive rays are finally accumulated to get a single color per pixel.

Advantages

1. Exploits object-space coherence by hierarchically culling hidden geometry. Works efficiently in densely occluded scenes.
2. Transparent primitives are handled correctly.
3. Highly parallelizable.
4. Handles complex primitive types such as spheres, metaballs and parametric surfaces.

Drawbacks

1. Since the depth of the closest intersection is not stored, new primitives cannot be added into an existing solution.¹⁸ Therefore ray casting is not incremental.
2. Intersection calculations are expensive.

There has been exhaustive research on data structures for speeding up ray tracing and trying to merge advantages of Z-buffer and ray casting.

ZZ-buffer by Salesin and Stolfi [90] approaches this by subdividing the output image into image-space tiles. A first pass assigns primitives into tiles, and a second pass ray casts the primitives assigned to the tiles. While being able to exploit image-space coherence, this approach loses the most significant advantage of ray casting: exploiting the available object-space coherence.

An alternative approach would be to use a tile-sized Z-buffer or A-Buffer. Whenever the ray intersects a primitive, the depth values of the primitive are applied to the Z-buffer values of every pixel it occupies¹⁹. The ray casting intersection depths are processed normally, but a single primitive has to be considered only once per tile since its depth values are already cached into the local Z-buffer. This results in exploitation of both image-space and object-space coherences.

10.3.6 Subdivision

Area subdivision by Warnock [114] is a divide-and-conquer algorithm, which recursively subdivides the output image until each rectangle is fully covered by a single primitive or no primitives at all. Unless terminated earlier, ultimately 1x1 pixel rectangles are tested.²⁰

Starting with a rectangle of the total output image area, we recursively perform tests to determine whether we should subdivide the area into quadrants. Each primitive has one of the following relations with the current rectangle (figure 10.8).

¹⁸This can be achieved by combining ray casting with Z-buffer and is currently used for example in the previewer of 3D Studio Max™.

¹⁹Inside the tile.

²⁰Assuming no antialiasing is performed.

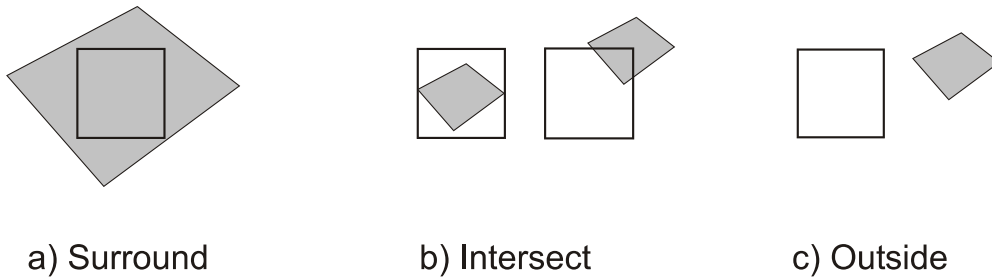


Figure 10.8 Warnock subdivision classification cases

1. The primitive completely encloses the rectangle (surrounding).
2. The primitive is partially or completely inside the rectangle (intersecting).
3. The primitive is completely outside the rectangle (outside).

The rectangular area does not have to be subdivided further if any of the following are true:

1. All primitives are outside the rectangle.
2. Only one primitive surrounds or intersects the rectangle.
3. A single surrounding primitive obscures all other primitives.

Advantages

1. Exploits image-space coherence by hierarchically culling hidden geometry.

Drawbacks

1. Scenes with a lot of small primitives suffer from substantial overhead due to subdivision. This corresponds to *bucket rendering* with very small buckets. The overheads involved in bucket rendering have been studied in [28, 21].

While Warnock's area subdivision algorithm has never been widely used in its original form, its principles have been utilized in several newer algorithms. Similar subdivision schemes have been used by Greene [52, 51] and Naylor [81].

Warnock's subdivision is always axis-aligned, and therefore may not be very efficient on diagonal primitive edges. To overcome this, Weiler and Atherton [115] presented a scheme where primitives are first sorted into front-to-back order, then the subdivision proceeds along the edges of primitives. This results in very awkward clipping problems and the algorithm has only theoretical and historical value.

10.3 Exact Visibility Determination

10.3.7 Scan-Line Algorithms

Scan-line algorithms [119, 99, 105, 80, 50] are not fundamentally different from previous algorithms, but simplify the problem by solving visibility in 1D instead of 2D. It could be stated that these are mainly data structures used to simplify the problem domain. Usage of scan-line algorithms usually reduces memory usage, but requires an additional sorting pass. Many of the scan-line algorithms can also be implemented using tiles.²¹

10.3.8 Beam Tracing

Beam tracing by Heckbert and Hanharan [58] is a hybrid technique of ray tracing and Weiler and Atherton's sorted clipper (see section 47).

Whereas ray casting casts a ray through the center of each output image pixel, beam tracing only casts a single viewport-sized beam. The beam is recursively clipped against the scene geometry and consequently several smaller beams result. While able to exploit both image-space and object-space coherence, beam tracing inherits the awkward clipping problems present in Weiler and Atherton's algorithm. Unfortunately, refraction is not a linear function and can therefore only be approximated by reflecting beams.

We shall refer to beam tracing later when discussing portals (section 10.4.8).

10.3.9 Frustum Casting

Frustum casting by Teller [109] is a hybrid technique of Warnock subdivision and ray casting.

The output image is subdivided in a fashion similar to Warnock subdivision, but instead of projecting geometry into image-space, the subdivided view is ray casted into the scene. From an analytical point of view the difference is significant. While Warnock subdivision needs to concern all primitives, frustum casting inherits the hierarchical culling property of ray casting and is therefore able to exploit object-space coherence in addition to image-space coherence inherited from subdivision. In scenes with a lot of small primitives, frustum casting converges to ray casting, and is therefore no longer able to exploit image-space coherence.

10.3.10 Immediate Mode Ray Casting

Immediate mode ray casting by Alex and Teller [5] is a hybrid technique of Z-buffer and ray casting.

Assume we have a surface of the form $F(x, y, z) = 0$ and a bounding mesh surrounding it. We first test the bounding mesh against the Z-buffer and tag all visible pixels. We then cast rays towards all visible pixels and calculate intersections with the actual surface. This method could be generalized to work with any parametric surfaces and modifications to existing hardware are quite small.

²¹Image-space coherence is often better exploited by processing tiles than scan-lines.

10.3.11 Conclusion

SSS taxonomy concluded that Z-buffer and ray tracing were impractical brute-force algorithms. The memory requirements of Z-buffer were considered outrageous.

Currently Z-buffer is the *de facto* standard for exact visibility determination due to its simplicity and easy hardware implementation.

Hybrid Algorithms

Hybrid algorithms inherit algorithmic properties from their base algorithms. There is some similarity to object-oriented programming and inheritance. However, the process of inheriting algorithmic properties is not as straightforward as class inheritance. The following three cases are possible when a hybrid algorithm H is created from algorithms A and B .

1. A property of A (or B) is present in H .
2. A property present in A is lost due to a fundamental change in behavior.
3. A and B contain conflicting properties and the effect may be either canceled out, blended or inherited from either A or B .

The resulting set of properties can only be deduced by understanding the base algorithms. We shall use this model when presenting and analyzing new visibility algorithms.

Future Directions

The lack of proper antialiasing is still a big problem in real-time rendering. Antialiasing and visibility algorithms are tightly connected [54, 50]. Additionally motion blur, depth of field and proper surface shaders are needed to create images with a convincing appearance. These additional features are possible by using the A-Buffer, which is currently being used in Pixar's Photorealistic RenderMan[112, 8], a software widely used by the movie industry. Hardware implementation of A-Buffer is still years away due to its inherent memory consumption.

The accumulation buffer [84] is the current solution for improved image quality. The accumulation buffer is an extra buffer where the frame buffer can be accumulated. The only operations required for antialiasing, motion blur and depth of field are addition and scaling with a constant.

Proper motion blur and depth of field require at least 64 samples per pixel, which puts enormous pressure on geometry processing, pixel fill rate and visibility algorithms. With 64 samples per frame the frame rate is effectively 64 times than what is needed without the special effects. Temporally coherent algorithms can provide immense boosts to performance.

During the past years graphics hardware performance has doubled annually. Extrapolation of this tendency indicates that hardware capable of full quality A-Buffer rendering of *current scenes* could be available around 2005.

The first steps towards real-time surface shaders are right now being taken by Microsoft with their DirectX8.0 specification. Peercy *et al.* [85] have made an interesting attempt to use OpenGL as

10.3 Exact Visibility Determination

a SIMD machine and have managed to create a low-level language that models RenderMan shading language using OpenGL as its primitive assembly language. It seems obvious that real-time graphics industry will follow the footsteps of the movie industry. Maybe in ten years we will *play* Dragonheart.

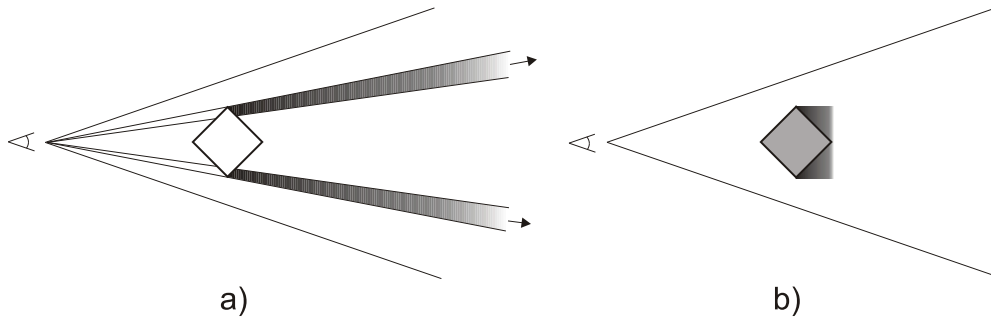


Figure 10.9 Sources of occlusion volume loss include image-space quantization (a) and depth quantization (b).

10.4 Conservative Visibility Determination

Exact visibility algorithms guarantee the correct output as long as all contributing geometry is processed. There might be a large amount of redundant geometry inside the view frustum. Various conservative algorithms have been proposed to avoid processing the hidden geometry. There is currently no standard way of estimating how conservative the algorithm is. Consequently we propose to use the following metric

$$\text{conservativity} = \frac{R}{V} \quad (10.1)$$

where R is the amount of visible objects reported by a conservative visibility algorithm and V is the amount of actually visible objects. V can be robustly calculated using an item buffer (see section 44). It can be easily seen from the equation that conservativity of 1.0 is the perfect result, and values under 1.0 indicate erroneous non-conservative results.

In this section we first consider conservative image-space algorithms in 10.4.1. Object-space algorithms are then reviewed in 10.4.5.

10.4.1 Image-Space Algorithms

Image-space algorithms are usually highly robust, since no computational geometry is involved. Degenerate primitives, non-manifold meshes, T-junctions, holes or missing primitives are all handled without special treatment. Parametric surfaces and everything that can be rendered and bounded with a volume can be used, at least in theory.

Lost Occlusion Volume

Occlusion culling is usually performed using a somewhat lower precision than the actual model geometry. Image-space algorithms may solve the visibility using a resolution smaller than the output resolution. This conservativity in image-space causes potentially significant loss in the *occlusion volume* [122]. An error in image-space extends to a sweep volume reaching the far plane of the view frustum. With a long view frustum an image-space error of a few pixels can correspond to

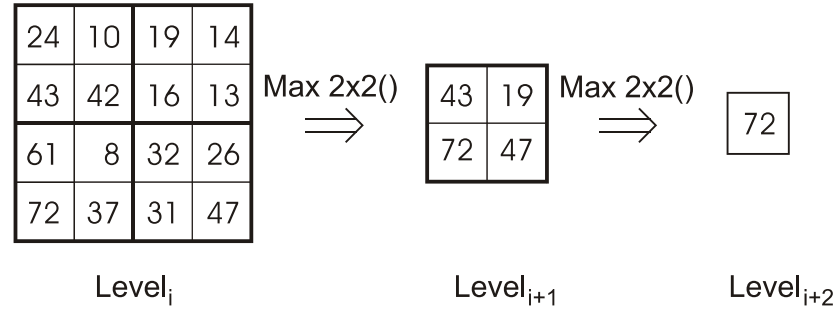


Figure 10.10 Hierarchical Z-buffer principle

a large volume. This is illustrated in figure 10.9 a). Similarly the depth values of an object may be approximated by using a single depth value instead of per-pixel depth values. This also causes loss in occlusion volume. However, being perpendicular to the viewing direction and bounded by the sides of view frustum, the resulting volume is usually smaller than with image-space error (see figure 10.9 b)). We call these losses *image-space occlusion volume loss* and *depth representation occlusion volume loss* respectively.

10.4.2 Hierarchical Z-buffer

Greene and Kass [53] propose extensions to existing Z-buffer hardware to avoid per pixel depth comparisons. Their first major idea is to organize the scene into an octree for approximate front-to-back traversal. For each octree node the question 'Would this node be visible if rendered?' is asked. If not, the entire node is skipped. If only hardware could answer this query, significant performance improvements would be possible. But current hardware does not and according to hardware vendors a fast feedback is against the one-way design principles of current hardware. This implies that the query would have to be implemented either in software or with a separate occlusion culling unit, which should in fact be possible right now. Georges [46] describes an implementation for Pixel-Planes 5 [44], a parallel graphics computer.

Another invention is the hierarchical Z-buffer itself, which is similar to the mipmap-pyramid²² [117] used in texture mapping, except that whereas a mipmap pyramid is constructed using the average operator, the Z-pyramid uses the *maximum* operator. Figure 10.10 demonstrates the contents of pyramid levels. All writes are performed to the lowest, full resolution pyramid level. A primitive can be quickly determined hidden if its bounding rectangle with the primitive's minimum depth value fails the visibility test. In many cases this can be done with a single comparison on the appropriate Z-pyramid level.

Unfortunately changes to current hardware are required and the efficient implementation of a constantly updating Z-pyramid is problematic in hardware. If the Z-pyramid updates only happened infrequently, the problem would be reduced. Intel has proposed a related concept of *Adaptive Hierarchical Visibility* [120] where Z-pyramid updates are only triggered after a certain amount of primitives have been processed. Their implementation is based on a tile architecture and the pyramid updates get simplified considerably. This causes depth representation occlusion volume loss as primitives introduced between two updates cannot occlude each other.

²²Mipmap-pyramid has the original image as the lowest pyramid level and each higher level is created from the lower one by averaging 2x2 blocks. Complete mipmap-pyramid of 256x256 texture consists of 9 levels (256x256, 128x128, ..., 1x1).

While the HZ-buffer is not available in consumer-level hardware, it still leaves the possibility of updating the Z-buffer with hardware, downloading it to the system memory and creating the required hierarchy in software. In general, a Z-buffer cannot be read back from the hardware, and in any case, that would be too slow with current bandwidths. Additionally Z-buffer formats are largely hardware-dependent and often unspecified. As a conclusion, we must say that hardware-software implementation is possible with certain hardware configurations, but not in a generic case.

Advantages

1. Provides occluder fusion.
2. Logarithmic search for visible primitives. Only visible or nearly visible primitives are processed.
3. Hierarchical Z-buffer is able to exploit both object-space and image-space coherence and also benefit from previous frames. This makes the HZ-buffer belong to temporal coherence class 2 (see section 10.2).
4. No restrictions on scene structure or primitive types.
5. Should hardware of required kind become available, there might not be need for further development of occlusion culling algorithms.²³

Drawbacks

1. Neither the depth query nor the Z-pyramid are implemented in current hardware.
2. Octree is hardly the best spatial structure, since most scenes are far from box shape and a waste of resources follows. BSP or kd-tree subdivisions adapt much better into many common scene types.

10.4.3 Hierarchical Polygon Tiling with Coverage Masks

In order to support high-quality antialiasing Greene [52] presented a hierarchical polygon tiling algorithm using coverage masks.

If strict front-to-back traversal is guaranteed, there is no need for depth tests. Greene achieved this, undeniably limiting, goal by organizing the scene into an octree of BSP-trees. The hierarchical culling of octree nodes, similar to the Hierarchical Z-buffer, can be used and each BSP-tree can be efficiently traversed in strict front-to-back order by employing the algorithm by Naylor [81].

If the projection area of an octree node is fully covered, the node is guaranteed to be hidden and all geometry inside it can be culled. Otherwise the primitives are hierarchically tiled into a mask pyramid. Masks used in the hierarchy are special tri-valued masks called *triage masks* (see Fig. 10.11). Pixel value V indicates that all pixels downwards in the hierarchy, starting from this one, are vacant. Value O indicates they are all occupied, whereas P indicates that only part of them are occupied. Hierarchical tiling only writes to fully covered pixels with value V. When tiling encounters value O, it terminates. Value P causes recursive subdivision to the next lower pyramid level. This

²³It is an interesting question whether hierarchical Z-buffer could become the *de facto* standard for conservative visibility determination as Z-buffer is for exact visibility determination.

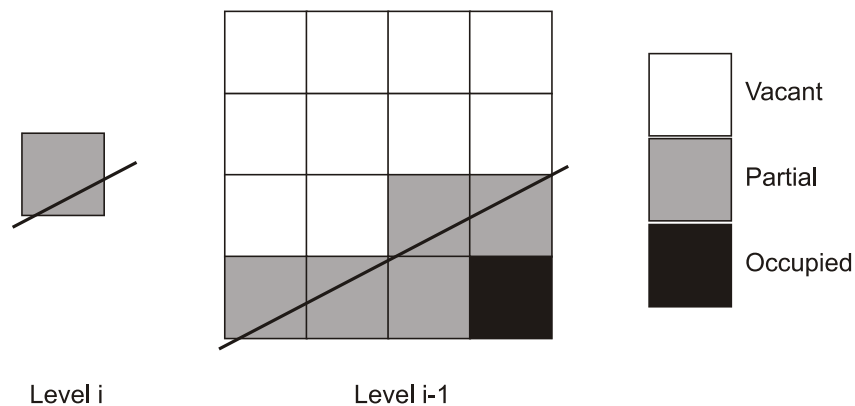


Figure 10.11 Triage masks

writing procedure is similar to Warnock subdivision and manages to avoid much of the tiling work with hierarchical rejection of parts of primitives.

The authors achieve antialiasing by setting the *second* level of the coverage pyramid to the output image resolution. The lowest level is therefore large enough to supply information for 4x4 super-sampling. Due to strict front-to-back traversal, it is possible to calculate the output color by simply accumulating the coverage-weighted samples. As a result, it is possible to get supersampled output quality in a single pass without the need for a 4x4 frame buffer.

Advantages

1. Elegantly bundles visibility determination and antialiasing to significantly reduce memory requirements of supersampling.
2. The algorithm is incremental. This property is inherited from the Z-buffer algorithm.

Drawbacks

1. Only works with fully static environments, since dynamic construction of an octree of BSP-trees is computationally very expensive.
2. Hierarchical tiling should be implemented into hardware to be competitive in scenes with anything less than millions of polygons.
3. BSP-trees of millions of polygons use a considerable amount of memory and in turn limit the usability with large models.

The first drawback is a totally unacceptable requirement and effectively rules out every interactive environment. The second and third are conflicting conditions and imply that while having millions of polygons, the BSP-trees must be kept small. This can only be achieved by making multiple instances of the same BSP-tree. Our conclusion is that while being a very elegant approach, 'Hierarchical Polygon Tiling with Coverage Masks' has mostly theoretical value.

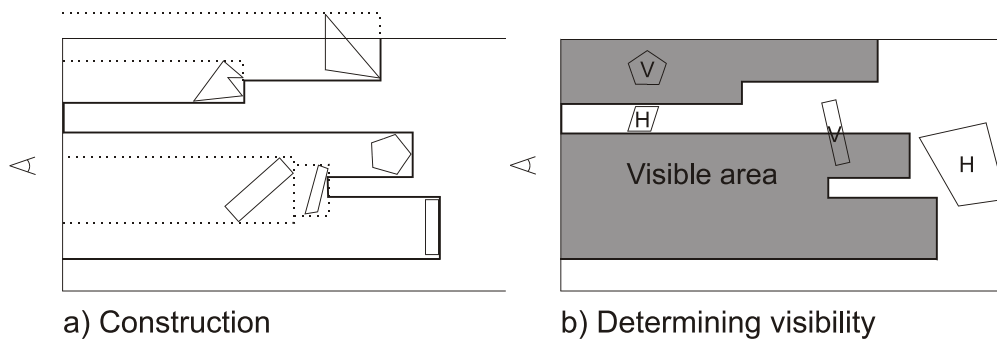


Figure 10.12 Depth Estimation Buffer principle

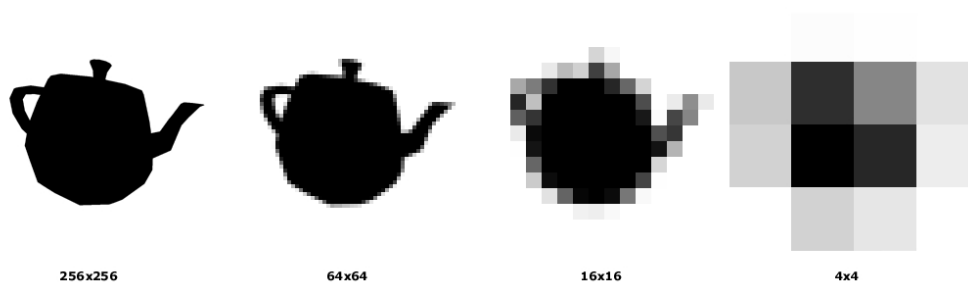


Figure 10.13 An example of Hierarchical Occlusion Maps

10.4.4 Hierarchical Occlusion Maps

Zhang *et al.* introduce Hierarchical Occlusion Maps (HOM) in [122, 125]. Their primary innovation is to split the visibility test into two sub-tests:

1. *Coverage test*: Is the primitive spatially covered by occluders?
2. *Depth test*: Is the primitive behind the occluders?

New data structures introduced are HOM (figure 10.13) for the coverage test and *depth estimation buffer* (DEB) (figure 10.12) for the depth test. After the visibility test has been split into the sub-tests, resolution of the more involved depth test can be decreased without introducing visible artifacts. Depth representation occlusion volume loss increases when the resolution of the depth test decreases.

Coverage

The Hierarchical Occlusion Map (HOM) is constructed by rendering the opacity values of all occluders into a monochromatic frame buffer. A mipmap pyramid of the frame buffer is built using mipmapping hardware if available. In absence of such hardware, the frame buffer is first downloaded to the system memory and the pyramid is constructed with software. Pixel values in the hierarchy indicate the average opacity percentage of the underlying pixels. Value 0.75 in the topmost layer

10.4 Conservative Visibility Determination

indicates that 75% of the highest resolution image pixels are set. Coverage tests of any given rectangle can now be carried out in a hierarchical fashion similar to Warnock subdivision (10.3.6) and *contribution culling* can be enabled by setting the opacity threshold to something less than 100%. It is worth pointing out that if contribution culling is either not required or is done at *construction time*, HOM can be presented with a single bit per pixel and memory requirements are reduced by 87.5%²⁴. This is the approach taken by our algorithm (see 11.9 for details).

HOM is conservative if “for each pixel in the output image, its corresponding pixels in HOM have the same or less opacity”. The exact condition for HOM being conservative is to store a floating point coverage percentage per pixel. At that point, HOM becomes conservative to the same extent as object-space algorithms [123]. Since authors use a limited base resolution (256x256), high quality antialiasing capabilities are required in the hardware.

Depth

DEB functionality has a strong resemblance with the Z-buffer. Whereas the Z-buffer is initialized to the farthest possible value, DEB is initialized to the *nearest* possible value. A DEB update occurs if the current value is *greater* than the value stored in DEB. It would be possible to implement DEB with current Z-buffer hardware by initializing the buffer value to near, and reversing the depth-test function. Each object is approximated with an axis-aligned rectangle and the farthest depth value of an object is chosen.

No-background Z-buffer [122] is an alternative depth representation scheme. It is constructed exactly like a traditional Z-buffer. During the depth tests *far* values must be interpreted as *near* values.

Occluder Selection

The occluder selection algorithm returns a list of objects to be used as occluders. These occluders are written both into HOM and DEB. All other objects inside the view frustum are tested first against HOM and then against DEB. If both of the tests succeed, the object is hidden.

The basic algorithm is not incremental, and receives no feedback from the occlusion system. In absence of context-dependent information, there is not much one can do. This is similar to a blind guy trying to hide behind objects; in order to succeed, user guidance is required. One can measure the distances to objects and to use the metrics listed in section 30 to aid in the selection. Our options are to select every object closer than X meters or to select N closest objects. Size and complexity can merely be used as thresholds, not as proper weights, since there is no cost-benefit function to balance. Only an incremental occluder selection algorithm can be guaranteed to break all lines of sight. Failing to do so can cause catastrophic situations when visualizing very large databases.

Advantages

1. Provides occluder fusion.
2. Depth estimation buffer is fully portable as it can be maintained with software.
3. The proposed implementation is able to handle alpha textures [15], which are often used to model impostors [75].

²⁴compared to 8 bits per pixel.

4. Can handle all degeneracies and different primitive types.
5. Is possible to implement with current hardware.

Drawbacks

1. No incremental versions of the algorithm were discussed.²⁵
2. Antialiasing is required to assure conservativity.
3. Occluder selection is insufficient.
4. DEB accuracy:
 - (a) Using bounding rectangle area to perform DEB writes results in unnecessary worsening of DEB. In [122] Zhang states that these effects are only local and therefore insignificant. While it is true that the effects are local, our empirical experiences indicate that the effects should not be considered insignificant.
 - (b) Using a single depth value for DEB writes results in an unacceptable amount of occlusion volume loss.
5. Contribution culling is prone to popping artifacts when an occluder has small holes and an object behind is either culled or not culled depending on discrete samples. Slow smooth camera motion causes discretized hole sizes to vary in pixels. In the worst case this results in totally random culling of slightly visible objects.
6. Reading back the frame buffer from the graphics hardware is very slow, except on high-end graphics computers such as Silicon Graphics Infinite Reality.²⁶

Ho and Wang [59] propose replacing the mipmap pyramid with *Summed-Area Tables* (SAT). Even though SAT allows significantly faster, constant time coverage queries, the memory requirements increase from $\frac{4}{3} * 8bpp \rightarrow 32bpp$. Remembering that $1bpp$ representation is sufficient, we observe that SAT uses 32 times as much memory. Authors also propose replacing the depth estimation buffer with a sparse span buffer. This does not have any benefits over the depth estimation buffer. We conclude that the proposed changes do not solve any of the real problems of HOM and only complicate the otherwise elegant algorithm.

Incremental Variants

Klosowski and Silva [67] present an algorithm for fixed-budget rendering. Their *Prioritized-Layered Projection* (PLP) algorithm is basically a database traversal, but can also be used for finding occluders for HOM. Zhang [122] discusses *multi-pass occlusion culling* and concludes that some properties of the current hardware are not favorable for it. Bormann [13] proposes slicing the frustum into several layers. He creates a Hierarchical Occlusion Map for each slice and uses only a single depth value in each slice.

Due to favorable features, we have selected a variant of HOM as our image-space solver. Our contributions are presented in section 11.9.

²⁵There is nothing inherent that prevents the algorithm from being incremental. However, when implemented using the current hardware, the update costs are prohibitively high.

²⁶The base resolution is generally smaller than the output resolution and therefore the bandwidth requirements are reduced.

10.4.5 Object-Space Algorithms

While image-space algorithms are characterized by robustness and applicability to a wide variety of problems, object-space algorithms usually have no such desirable features. What makes them important, is their theoretical possibility for utilizing temporal coherence, which is missing from every image-space algorithm.

10.4.6 Aspect Graph Variants

The scene can be divided into partitions of constant *aspect*. Any such partition, called Visibility Space Partition (VSP) [92], has the property of seeing a fixed set of visible primitives. Having such information provided, the visible primitives can be found by locating the current VSP. Moving from one VSP to another causes a *visual event*, which implies that the set of visible primitives has changed.

It was shown by Plantinga and Dyer [86] that a VSP with inconvex polyhedra and perspective projection yields the spatial complexity of $O(n^9)$, which is a totally ridiculous requirement. Even worse, some VSP boundaries are quadric surfaces. They proposed a complex structure called *asp* to represent the aspect graph, but apparently did not implement it. Durand *et al.* introduced the *3D Visibility Complex* [32], which obviously was not implemented either. Later they slightly simplified the concept and introduced the *Visibility Skeleton* [33, 34]. They report memory consumption of 400Mb for a 1500 polygon scene and because the complexity is not even close to linear, it is obvious that a scene with millions of polygons would consume memory beyond comprehension. From these results it seems safe to conclude that purely VSP-based approaches can only be used in certain special cases.

The aspect graph of a box consists of 26 VSPs²⁷. Since visibility events only occur when the VSP changes, the list of silhouette edges remains constant inside a single VSP [98]. Each VSP has a fixed set of features (faces, edges and vertices), one of which is the closest feature of an object. This information can be used to optimize box vs. plane intersection tests [60]. We first determine the direction of the normal of the plane in the object's space to obtain the current VSP. According to the current VSP we use the pre-computed closest and farthest vertex indices to test the intersection. As every object can be bounded with a box, this method is fully general.²⁸

Convex Aspect Graph Variants

Coorg and Teller [26] simplify the aspect graph complexity by limiting the occluders to convex polygons. While this certainly does simplify the problem, it also makes the algorithm quite useless in realistic scenes, where no large polygons are available. The algorithm might also not be suitable for complex or highly dynamic scenes due to the relatively high amount of visual events.

The above algorithm is temporally coherent and might be useful with large and convex *virtual occluders*.

In [27] Coorg and Teller generalize the algorithm to polyhedral occluders. They introduce a new

²⁷6 faces, 12 edges and 8 vertices

²⁸We use this method for optimizing our database voxel culling.

method for estimating the occlusion potential of a polygon:

$$-\frac{A(\vec{N} \cdot \vec{V})}{d^2} \quad (10.2)$$

Where A is the polygon area, $\vec{N}$ is the polygon normal, $\vec{V}$ denotes the viewing direction and d the distance from the viewpoint to the center of the polygon. How this relates to polyhedral occluders remains unspecified in the paper. One alternative is to estimate the size of the axis-aligned projection of the object. Regardless of the dynamic nature of equation 10.2, the algorithm selects potentially good occluders during a pre-process. If the occluders are selected at a pre-processing stage, one could consider the actual occlusion powers of the candidates, instead of selecting them based on such simple criteria.

The authors also propose an interesting idea of selecting a dedicated occluder for each detail object. How the detail objects are chosen is left as an exercise to the reader and might not be easy to perform automatically. As pointed out by Saona-Vázquez *et al.* [92], the correct amount of detail objects is an important question.

10.4.7 Shadow Volumes

Hudson and Manocha [63] first select all convex objects in a scene. For each one of them the solid angle²⁹ is measured. If the solid angle exceeds a fixed threshold, the object is accepted as an occluder. At navigation-time they first build shadow frusta [29] from some of the best occluders, and hierarchically cull axis-aligned bounding boxes of the scene hierarchy using the frusta.

The algorithm in its original form works only with convex occluders and can therefore only be used in special cases. While extending it support inconvex occluders is possible, it is unlikely that it will achieve the desired performance level.

Culling with convex shadow frusta is identical to hierarchical view frustum culling and could therefore be easily integrated into many existing occlusion systems.

10.4.8 Portals

Airey [4, 3] proposed subdividing the scene into *cells* connected by *portals*. This subdivision is particularly well-suited for architectural scenes. Most intuitively this is explained with cells corresponding to rooms, and portals corresponding to doors and windows, through which other rooms can be seen.

Concept

Usually we are taking considerable effort to prove that we cannot see through some part of the scene. Cells are implicitly assumed fully blocking, and are therefore a very powerful way of expressing *a priori* visibility information. The shapes of the cells are not restricted, although most practical purposes require convex cells. The places where this blocking property is untrue are places to insert

²⁹same as in [27].

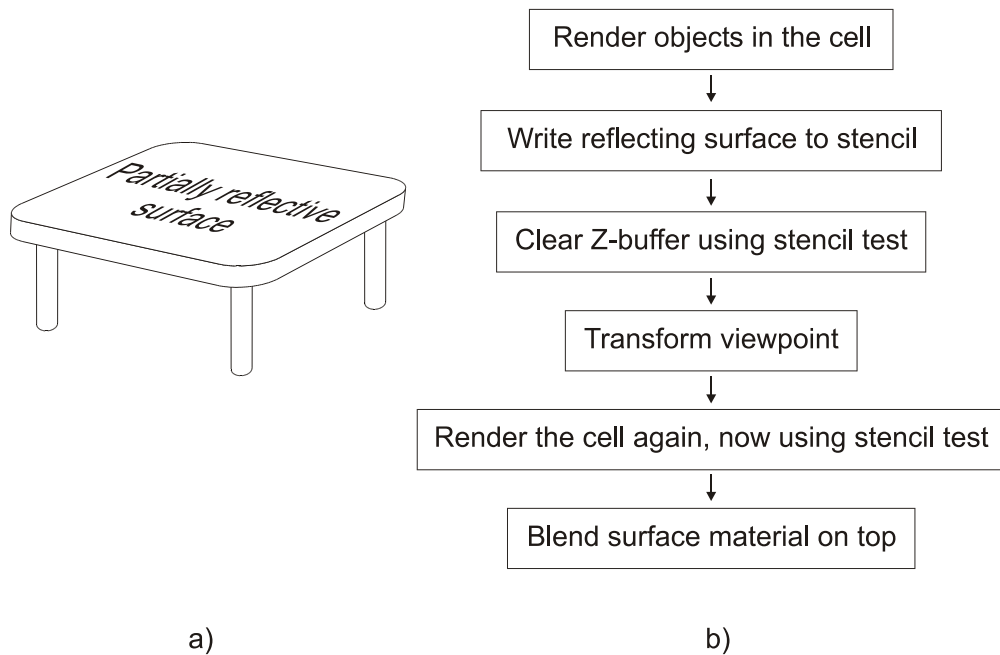


Figure 10.14 Portals can be used for creating partially reflective surfaces, such as the surface of the table. The process is outlined in *b*).

portals. Automatic portalization of generic scenes is a hard problem and no solution has been proposed. Teller [108] presented an algorithm for performing automatic portalization of axis-aligned scenes. Even though this is often the case with architectural scenes, no such assumptions should be made.

Static Version

In portalized scenes the visibility problem is reduced to resolving cell-to-cell visibilities. Both Airey and Teller [110] solved this by creating PVSs in a pre-processing step. To create PVSs Airey proposed sampling from several locations and using shadow frusta, neither of which can guarantee a correct result. Teller gave an analytical solution by formulating the problem into a linear programming problem.

Dynamic Version

Luebke and Georges [74] observe that the current view frustum can be clipped against the portal when traversing through it. If we clip the frustum against the axis-aligned projection of the portal instead of the actual geometry, we always get a frustum with four screen-space axis-aligned side planes. Thus the portal traversal and cell-to-cell visibility determination can be merged to view frustum culling and performed dynamically. The need for PVSs disappears and it is possible to open, close, move and even to create new portals at run-time. Furthermore, the authors observed that adding a transformation matrix to portals made it possible to create reflecting surfaces (figure 10.14) and non-physical continuities in space.

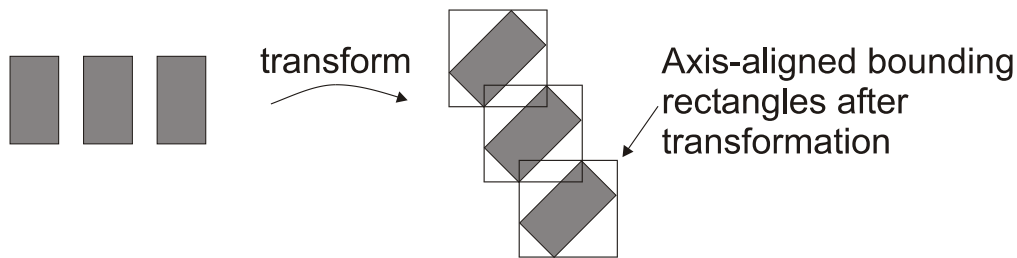


Figure 10.15 Portals are often approximated using axis-aligned bounding rectangles. The bounding rectangles may overlap after viewport transformation and therefore the rendered result is incorrect.

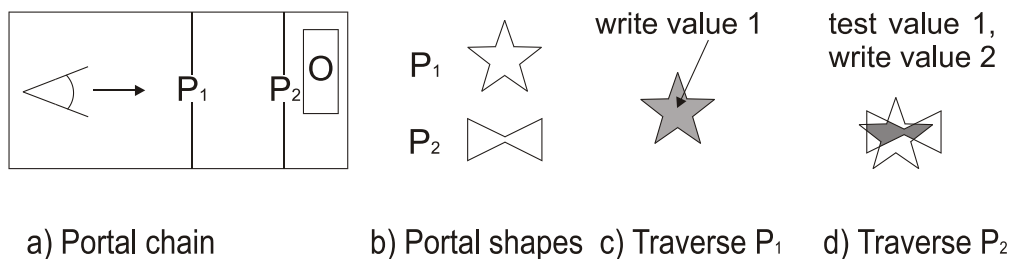


Figure 10.16 Stencil buffers can be used for solving arbitrary portal clipping. Object O is seen through two consecutive portals. Figures c) and d) illustrate the stencil buffer during the traversal and finally O is rendered using stencil test.

We call portals that do not define a transformation matrix *physical portals* and the ones that do *virtual portals*.

Portals can be seen as a way of compactly expressing a PVS. An interesting observation is that portal traversal can be seen as beam tracing (see section 10.3.8). Reflections and approximate refractions of beam tracing are directly applicable to portal traversal.

Overlap Problem

There is no literature giving solutions for certain common problems arising from using portals. Sometimes axis-aligned projections of two portals overlap on the screen. This causes problems when a partially transparent object is seen twice and parts of it are in fact rendered twice. Timestamping the objects is not a generic solution, since there can be any amount of portals between two cells. Merging these two portals into a single bigger portal may fix the problem, but can potentially trigger a chain reaction by overlapping a third portal. This is illustrated in figure 10.15. The only practical solution is to use a stencil buffer [84] to block writes outside the actual portal geometry as illustrated in 10.16. If a stencil buffer is not available, the alpha channel and a destination alpha test can be used instead.

10.4 Conservative Visibility Determination

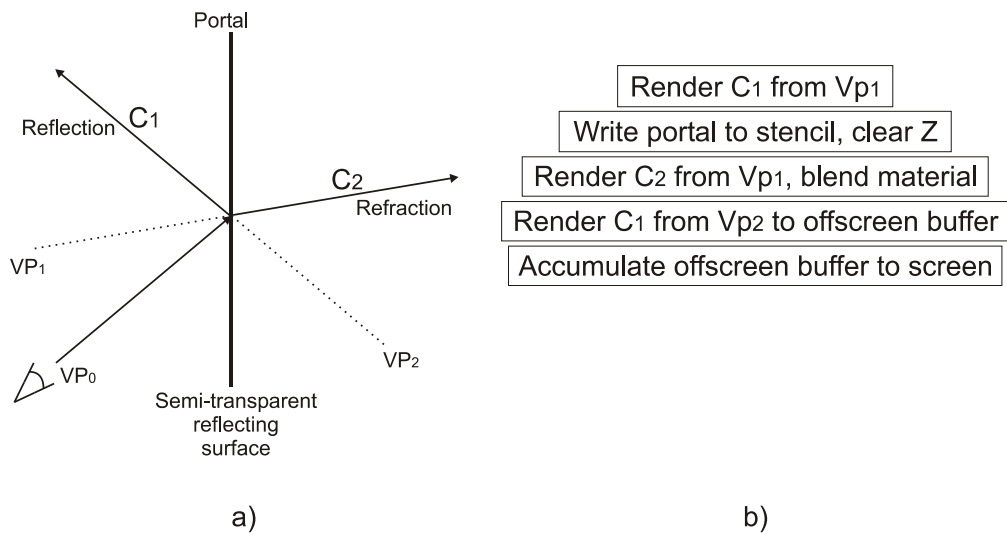


Figure 10.17 Creating simultaneous reflections and refractions using portals

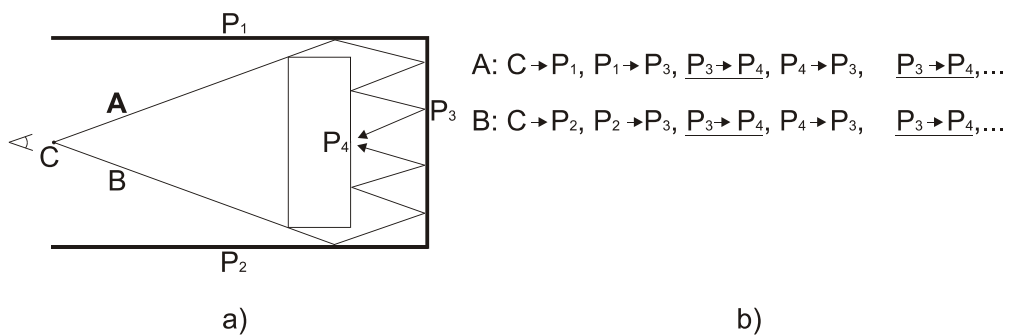


Figure 10.18 Limiting the recursion depth of portal traversal. Assuming the desired recursion depth is 1, the traversal is terminated when the transition $P_3 \rightarrow P_4$ occurs for the second time.

Special Effects

Surprisingly complex visibility relations, such as portable mirrors and reflecting cubes can be modeled with non-physical portals and stencil masks. By adding an off-screen buffer to the list of hardware requirements, both reflections and refractions³⁰ can be modeled simultaneously (figure 10.17). The need for an off-screen buffer results from both reflection and refraction requiring their own Z-buffering, while mapping into the same screen pixels.

Infinite Recursion Problem

When using non-physical portals, it is possible to create infinite loops by having mirrors that see each other in a circular fashion. Also, if free transformations are allowed for portals, loops inside

³⁰Refraction calculations are performed on per-portal base only, as in beam tracing.

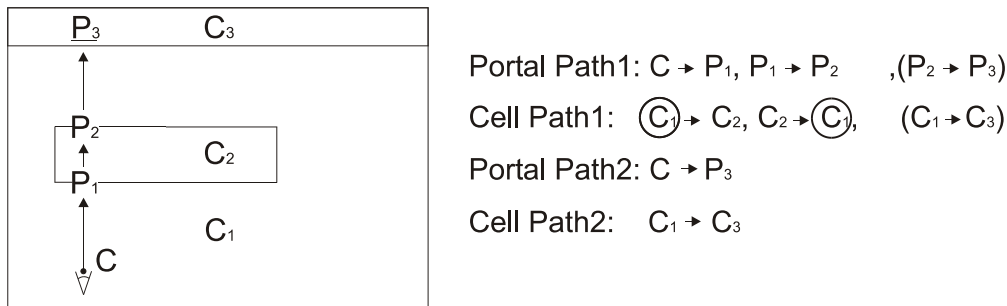


Figure 10.19 The traversal can process the same cell multiple times if nested cells are used. To prevent this, the traversal of “cell path 1” must be terminated since the path already contains C_1 .

the scene are possible. Consider a scene where a character looks into an alien device that transforms the view frustum 5 meters backwards. The character would see himself watching the device, and an infinite recursion follows.

The infinite recursion problem can be solved by writing a recursion solver, which sets the maximum number of times a portal can be traversed through a *per-portal sequence*. It is important to notice that simply setting the maximum number is insufficient. Recursion solving is illustrated in 10.18. Additionally portals must be back-face culled and two-sided portals evidently result in an infinite recursion³¹ in some cases. We made the observation that the portals do not have to be planar. The only condition is the possibility of back-face culling them. Indeed, a cube can be used as a portal.³²

Multiple Traversals Problem

Nested cells can result in situations where the outer cells are processed several times and therefore the objects inside the cell are also processed several times (see Fig. 10.19).

If the target cell has already been processed during the *portal sequence*, and all the portals were physical, the traversal should be terminated. If any of the portals are non-physical, we cannot guarantee that the transformations of the already processed objects are the same as previously. Thus the traversal cannot be terminated if non-physical portals are involved.

Floating Portals Problem

Special treatment is required if the portal is floating in the middle of the cell. The Z-buffer values may have been set closer than the objects in the destination cell and must therefore be cleared. The proper solution is to clear the Z-buffer if both the stencil test and depth test the for the portal pass.³³

³¹Which would be solved by the solver explained. Yet, it would result in processing the cells multiple times and possibly incorrect output.

³²This allows surrounding a floating cell, for example a space station, with a single cube-shaped portal.

³³This test needs to be done only for the area covered by the floating portal.

Warning

There are strong objections against using non-physical portals with free transformation. Physics systems potentially lose the concept of 'up' and the scales get twisted. How should a character passing through a magnifying portal be visualized? To avoid these and countless even trickier problems, the usage of free transformations should not be generally encouraged, except for pure "demo effects".

10.4.9 Hierarchical View Frustum Culling

View frustum culling is the first operation of a visibility pipeline. Because its input set is usually the whole scene database, the principle of output-sensitivity is of major importance. Naïve view frustum culling tests every object against the view frustum, thus breaking the output-sensitivity. View frustum culling can be done hierarchically [22]. Objects are first combined into bounding volume hierarchies. View frustum tests are then performed starting from the topmost level, and the results are propagated downwards. If a higher-level bounding volume fails the view frustum test, all the objects inside the bounding volume would fail as well, and can therefore be rejected without further tests. This is an important optimization for large scenes and for portalized scenes, where the view frustum is most of the time very narrow. Hierarchical view frustum culling can be seen as a logarithmic search of objects falling inside the view frustum.

View frustum culling can be made temporally coherent by using the algorithm presented in [102] or the one outlined in 11.14.

10.4.10 Hierarchical Back-face Culling

Removal of back-facing primitives can be done hierarchically [71, 124, 65]. This reduces the time complexity from a linear into a logarithmic search of visible primitives. Hierarchical back-face culling of triangles is not widely used in actual rendering, but the same techniques are usable for silhouette extraction.

Kumar and Manocha [70] present hierarchical *backpatch culling* for visibility culling of spline models. El-Sana *et al.* [36] introduce *skip strips* for hierarchical processing of triangle strips.

10.5 Non-Conservative Visibility Determination

Algorithms in this category produce artifacts by removing objects or primitives that should contribute to the output image. This means that the output image is not correct anymore, but in some cases this is considered acceptable. Most importantly, if the camera is moving rapidly, errors of a few pixels will not be noticeable. This principle is widely used in animation compression.

Both conservative visibility determination and non-conservative methods are required for visualization of arbitrarily complex scenes. The most common non-conservative methods include contribution culling (10.5.1), model simplification (10.5.2) and image-based rendering (10.5.3).

Non-conservative methods are not the main focus of this thesis and we mainly list some of the most relevant ideas with references. Nevertheless, we consider non-conservative methods as an important future extension to our library architecture (presented in chapter 4).

10.5.1 Contribution Culling

If an object occupies only a few pixels in the output image, we are tempted to cull it away. The easiest way to achieve this is to estimate the size of the object's bounding rectangle. This is called *screen-size culling* [78]. Zhang [122, 125] offered a more sophisticated method called *contribution culling*, which is able to cull objects according to the percentage of visibility³⁴. This method is however more prone to disturbing flickering when the camera moves smoothly. Setting the resolution of image-space algorithms lower than the output image resolution causes *subsampling* and potentially random rejection of slightly visible objects.

10.5.2 Multi-Resolution Presentation

Drawing a lot of detail into a small screen area produces aliasing. Aliasing of distant textures has been successfully solved using an image pyramid[107]-based technique called *mipmapping*[117], which both reduces aliasing and speeds up the rendering. Interpolation between mipmap levels provides additional improvements to the output quality.

It sounds plausible that a similar approach could be used with objects. The object equivalent of a mipmap pyramid is a list of objects with a progressively decreasing amount of detail. Morphing between the levels can be used to reduce visible artifacts. Hoppe [61] introduced the concept of a *Progressive Mesh* (PM)³⁵, which was later generalized by Popovic and Hoppe [88] to handle non-regular, non-manifold objects.

PM stores the entire process of simplification starting from the full resolution object. A presentation with N vertices can be constructed in linear time. Temporally coherent tracking can be utilized to reduce restoration time considerably. This algorithm is not well-suited for parallel processing and hardware implementation is challenging. PM is extremely well-suited for the progressive transmission of geometry over slow network connections, such as the Internet.

Vast research resources have been put into mesh simplification algorithms. Most of these ignore some or all surface attributes and are only able to handle connected, manifold geometry. This is hardly ever acceptable, and to be of practical use, the algorithm must handle non-manifold geometry

³⁴Authors use the term *Levels of Visibility*

³⁵Also referred to as continuous level-of-detail

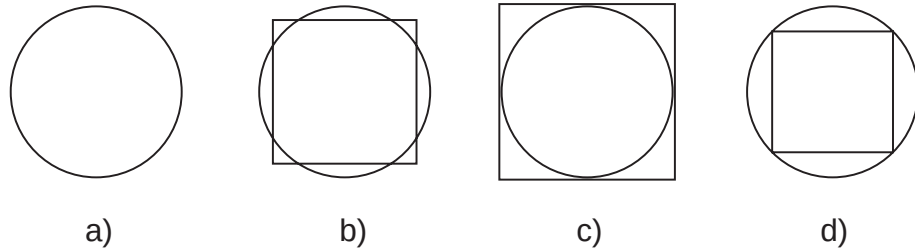


Figure 10.20 Occlusion-Preserving Simplification principle. For visualization purposes a distant sphere could be approximated using an area-preserving square (b). For occlusion culling purposes the approximation must fully contain the sphere when used for testing (c) and be fully inside the sphere when used for writing (d).

and all surface attributes. Proper handling of surface attributes was proposed by Garland and Heckbert [45]. Ericsson's and Manocha's GAPS[37] simplified the work of Garland and Heckbert by introducing *attribute clouds* for robust attribute handling, and also presented an algorithm capable of simplifying unconnected, non-manifold objects.

View-dependent simplification is based on the psycho-visual observation that the silhouette of an object is more important than its internals [68]. This is especially true for models being used in occlusion culling. Hoppe [62] proposed a temporally coherent view-dependent reconstruction algorithm for progressive meshes. Luebke and Ericsson [73] introduced Hierarchical Dynamic Simplification, which tracks view frustum changes and re-tessellates the polygonal environment accordingly. They were also able to exploit temporal coherence.

Occlusion-Preserving Simplification (OPS) Occlusion writes and tests could and should be performed using simplified geometry. The simplified geometry required for this purpose has the additional requirements of being entirely inside or outside the original model. Figure 10.20 illustrates the situation. A simplified test model must fully contain the original model (c) to assure conservative results. Similarly the simplified write model must be fully contained by the original model (d) in order to not cause false occlusion. While the simplified test model can be trivially constructed³⁶, constructing the simplified write models is a hard problem. Not a single general purpose algorithm solving the OPS for write models has been published. Cohen *et al.* introduced *Simplification Envelopes*[23] for maximum distance bounded simplification. The authors failed to note that setting either the inner or the outer envelope distance to zero indeed gives an occlusion-preserving result. This approach was later used by Zhang[122] to construct simplified geometry for HOM(section 10.4.4). Their simplification algorithm relied on connected, manifold geometry and is inherently hard to implement. Replacing this simplification algorithm with the more tolerant GAPS [37] improves the results, but the hard problem of extracting an envelope of unconnected, non-manifold geometry remains. Hoppe *et al.* [56] also describe an OPS algorithm for manifold objects.

³⁶Any bounding volume can be used.

10.5.3 Image-Based Rendering

The process of rendering a frame can be accelerated by reusing the image data from previous frames, instead of rendering the frame from its geometric description. The basic idea is to render one or more objects into a texture, which is then used to represent the objects as long as specific error metrics remain satisfied. *Image warping* is thoroughly covered in [118].

Impostors

Maciel and Shirley [75] assign several bitmaps into each face of every octree node during a pre-process. The authors use image processing techniques to estimate the accuracy of the generated bitmaps when seen from different viewpoints. The results are stored into a table, and at run-time the best-matching bitmap is chosen. The authors define an impostor to be an entity that is faster to draw than the true object, but retains the important visual characteristics of the true object. As all of the impostors are generated during a pre-process, the memory requirements are immense, and the scene is restricted to be fully static.

Schaufler [93] generates impostors dynamically per object. An impostor is always a transparent polygon with an opaque image of the object. The author argues that generating an impostor is hardly more costly than rendering the object into the frame buffer. Rendering into textures was not widely supported at that time, and the argument was at least partially untrue. However technology has advanced, and currently there is quite a bit of truth in this observation.

Aliaga [6] subdivides the scene uniformly into voxels as a pre-process. During navigation, every voxel further than a user-provided threshold is replaced by an impostor. Aliaga and Lastra [7] assign several textures to portals in a pre-processing phase and during navigation they only render the current room and appropriate portal impostors. Sillion *et al.* [101] present a specialized impostor placement algorithm for navigating in a city.

Talisman hardware architecture [111] internally renders objects into several layers and, whenever appropriate, performs affine transformations to layer data to generate subsequent frames. Not much has been said about the error metrics used to determine the needs for re-rendering the geometry. The host is responsible for assigning objects into the image layers and for maintaining a priority list of perceptible errors in the layers. The final output image is generated by a fast layer compositor, which collapses the layers from back-to-front with proper handling of transparencies.

Since the impostors are defined with a single polygon, the depth information is largely lost. This leads to visual artifacts when objects that intersect each other are not rendered correctly. Additionally, the parallax effect inside a single object cannot be simulated.

Layered Impostors

Nailboards [94] additionally store the per-pixel depth information of an object into impostor bitmaps. At the generation time, the depth values are read from the depth buffer, and when rendering the frame, the values are used to offset the depth values. With a correct transformation for the depth offset, nailboards can be mixed properly with geometric rendering. Nailboards cannot be used with current consumer-level hardware.

Layered impostors [95] replace an object with several bitmaps instead of a single one. These bitmaps represent different depths. Since the parallax effect can be approximated, layered impostors can be

10.5 Non-Conservative Visibility Determination

used much longer than single-layer impostors.

Spatial Impostors

Shade *et al.* [100] assign dynamically generated impostors to nodes of a spatial subdivision instead of assigning an impostor per object. Thus the impostors in the spatial hierarchy represent regions of space instead of objects. If no geometry intersects the node faces, the assigned impostors can be properly sorted, and occlusion is correctly solved. Schaufler [97] describes a similar approach.

3D Image Warping

To overcome the lack of parallax effect, out-of-plane displacement can be assigned to each impostor pixel. The idea is completely different from nailboards, where the depth was only used to offset the depth buffer. 3D warping equations [118] are unfortunately not invertible like conventional texture mapping. This means that it is impossible to analytically calculate the texture coordinates from the screen coordinates. As the mapping has to be done from texture coordinates to screen coordinates, holes are unavoidable and have to be filled by *ad hoc* methods. The WarpEngine [87] is a hardware architecture solely based on 3D image warping.

Oliveira *et al.* [83] have factored the 3D image warping equations into a 1D pre-warp followed by the conventional texture mapping. It should be possible to implement their *relief texture mapping* in hardware.

10.6 Data Organization and Traversal

Scene Graph vs Spatial Database

Traditionally applications use scene graphs to describe transformation hierarchies and optionally to limit the effects of lights, shaders etc. The scene graph does not contain proximity information, which makes it impossible to efficiently find objects located spatially close to each other. A *spatial database* (or *spatial index*) organizes the data spatially and is therefore suited for proximity queries and can generally provide approximate front-to-back traversal of the scene data. Any performance-optimized implementation of a spatial database is hierarchical.

Data Organization

Axis-aligned nodes of a hierarchical presentation are called *voxels* regardless of the particular data structure in use. Objects can usually belong to several voxels to avoid expensive clipping operations.

Bounding Volume Hierarchy

Rubin and Whitted [89] used bounding volume hierarchies to reduce intersection calculations in ray tracing. If the scene has a high amount of dynamic objects, the maintenance of the bounding volume hierarchy is a challenging task.

Regular Grids

Regular grids are perhaps the simplest spatial subdivision scheme. The drawback of regular grids is that dense and sparse areas of the scene are presented with identical subdivision. The consequence is that voxels in densely populated areas contain a high amount of objects while the ones in sparse areas are nearly empty. As a result, a regular grid is efficient only if the scene contents are uniformly distributed. For example, many terrains are like this.

Octree

Glassner [47] introduces the use of a hierarchical subdivision of a box called *octree*. Each voxel is divided into eight sub-voxels. Finer subdivision is performed in densely populated areas and consequently the amount of objects inside a voxel is nearly constant. The scene traversal is more expensive than with regular grids. The recursive subdivision procedure itself is independent of the data, and causes the octree to adapt rather slowly to any irregular scene structure. Since the octree is a subdivision of a box, it adapts rather poorly to flat scenes; for example, most urban scenes are relatively flat. Some volumetric visualization systems use octrees as the scene description itself [77].

Recursive Grids

Jevans and Wyvill [64] extend the idea of octrees by introducing a more generic subdivision procedure. Whereas octree subdivision always generates eight child voxels, the recursive grid can generate

10.6 Data Organization and Traversal

any amount of child voxels. Usually this amount is higher to reduce the depth of the hierarchy. The optimal number of sub-voxels created is approximately the same as the number of objects in the parent voxel [17]. If even subdivision is used, each axis would be subdivided $\sqrt[3]{N}$ times.

Hierarchical Uniform Grids

Cazals *et al.* [17][18] propose a new approach to spatial subdivision. They first cluster together the objects that are spatially close to each other and have similar sizes. A separate grid is used for each cluster and finally the grids are inserted into higher-level grids. Hierarchical Uniform Grids adapt to an irregular scene structure much quicker than octrees.

BSP

BSP tree [43] is fundamentally different from other spatial subdivision schemes. This is due primarily because it is not axis-aligned and the objects have to be splitted according to the partitioning planes. Splitting creates a considerable amount of *subpolygons* and makes BSP trees suitable mostly for static scenes. Timestamps can be used in many applications to avoid physical splitting of polygons.

Chen and Wang propose the *Feudal priority* algorithm [19] to reduce the amount of splits compared to BSP.

Axis-aligned BSP

If the objects are not clipped according to the planes, and plane orientation is limited to being axis-aligned, also dynamic scenes can be expressed easily. Axis-aligned BSP subdivision adapts to an irregular scene structure faster than octrees. We have selected axis-aligned BSP as our spatial database subdivision scheme.

Chapter 11

Contributions

In this chapter we introduce the new contributions and analyze them in detail. First, we state some of the basic concepts. A taxonomy of conservative visibility algorithms is given in 11.2. We review the contributions by following the flow of figure 11.1.

Occluder selection is reviewed in 11.3, and a new incremental algorithm with feedback is analyzed in detail in 11.3.2. Sections 11.4 and 11.7 introduce the Visible Point Tracking and Occlusion Write Postpone Queue algorithms respectively. Section 11.8 considers using silhouettes as rendering primitives. The Hierarchical Depth Estimation Buffer (HDEB) is introduced in 11.11. An incremental variant of Hierarchical Occlusion Maps is presented in 11.9. The use of dynamic virtual occluders is outlined in 11.13. Dynamic portal PVSs are briefly considered in 11.14. Finally, conclusions are presented in 11.15.

The camera model used in this chapter is the “standard” view frustum consisting of six planes unless stated otherwise.¹ For example, a portal chain can result in a view frustum with more (or less) than six planes. In such cases, the image-space algorithms use a scissor rectangle and the object-space algorithms use the frustum planes.

11.1 Building a Framework of Inter-Connected Algorithms

Bidirectional Visibility

In the real world, it is always true that if point **B** is directly visible from point **A** the opposite holds as well. Unfortunately in computer graphics it is possible to create objects that violate this fundamental property. One-sided primitives are commonly used to allow back-face culling. We reserve the right to assume that the above relationship always holds. By doing this, we acquire the right to reverse the visibility calculations whenever appropriate. To cope with violating objects we offer a possibility to manually disable the use of these objects as occluders. Furthermore, the objects that penetrate the viewport may result in situations where the interior of an opaque object is seen. Such cases can usually be considered a bug in the application code, but to avoid ambiguities we do not use such objects as occluders.

¹left, right, top, bottom, near and far.

Duality of Visibility

We believe that no single visibility algorithm will ever solve all application types satisfactorily. Therefore we propose to use a framework of inter-connected algorithms. Having multiple algorithms working together, a fine-grained subdivision of the problem domain is desired. The first, admittedly obvious, observation is that an object's visibility can be determined in two passes.

1. Prove that the object is visible.
2. Prove that the object is hidden.

This division may seem more than obvious, but it implicitly possesses an important property. An object is visible if *any* part of it is visible. On the contrary, the object is hidden only if *all* parts are hidden. The first test is an early-exit test and we thus first perform a series of tests trying to prove that the object is in fact visible. If all of these tests fail, we are almost sure that the object is hidden, and dare to spend substantially more time trying to prove this. The process is backed up by history data, so that we use results of the previous tests to make our initial guess.

Adaptive Hierarchical Testing

As proving either one of the above conditions is generally a difficult problem, we always use hierarchies of tests organized in order of increasing computational complexity. Whenever a simple test is able to answer the question, significant savings result by not executing the more involved tests. Whether a more involved test should be executed is context-dependent:

1. How much could be won by executing this test?
2. How much does the test itself cost?
3. Is it likely to succeed?

These are very important questions, and to be able to answer we must have some information on which to base the conclusion on. The full potential of an inter-connected framework can only be exploited by building feedback mechanisms to serve data for adaptation and learning. Due to our goal of building a general purpose library, we desperately need systems adapting to the problems at hand. To be able to answer the question "Is it wise to use this object as an occluder?" we should have some knowledge of the history. It is not difficult to construct scenes where the optimal amount of occluders ranges from zero to thousands. As user intervention or parameter tweaking is clearly out of the question, our framework must automatically adapt to the input data. However, rapid adaptation is a double-edged sword, because faster adaptation means having a shorter memory. This duality can be triggered by moving either swiftly or slowly in the same scene.

Resonance

A hierarchy of multiple adaptive algorithms can end up in a situation where algorithm $\mathcal{A}$ performs adjustments that cause $\mathcal{B}$ to adapt, which in turn causes $\mathcal{A}$ to adjust backwards. This is an unstable situation we refer to as *resonance*.

11.1 Building a Framework of Inter-Connected Algorithms

On the positive side, when $\mathcal{B}$ performs poorly, $\mathcal{A}$ will adapt to $\mathcal{B}$'s poor feedback and the effect is partially canceled out. This makes the adapting framework much more tolerant to problem cases than any single algorithm. Unfortunately, this is also true when there are bugs in some particular algorithm in the hierarchy. A sophisticated run-time debugging environment is required to locate such problems, otherwise a malfunctioning part might go unnoticed. This was indeed true with dPVS . Several bugs were extremely hard to find because other systems “fixed” them.

11.2 Taxonomy of Conservative Visibility Algorithms

The steps involved in conservative visibility determination algorithms are roughly as shown in figure 11.1. The pipeline begins when a frustum description is received and executes from top to down. The figure includes both static and dynamic “pieces” and for our purposes only the dynamic variants are acceptable. An algorithm is essentially static if any of the following is true:

1. Spatial database cannot support dynamic objects
2. Occluder selection is done at pre-process
3. Occlusion test is skipped or relies on static structures
4. Occlusion write is skipped or relies on pre-computed occluder fusion in the form of virtual occluders

The Spatial Database

The spatial database holds the entire scene and the format is usually one of the formats listed in section 10.6. The database can be either fully static or dynamically adapting. Most of the current implementations are limited to static scenes and are therefore not suitable for general purpose use. A notable exception is [104].

View Frustum Culling

Performance-optimized view frustum culling is always done hierarchically [22]. Dynamic Portals, Beam Tracing and Frustum Casting belong to the view frustum culling category. Optimized view frustum culling algorithms can be found in [10, 121]

Occluder Selection

Occluder selection can be either dynamic or static. Figure 11.1 includes a taxonomy of occluder selection. If *every object*, visible or hidden, is used as an occluder, the algorithm converges to an exact visibility algorithm. The simplest dynamic approach is to select all *potentially visible* objects as an occluder. Alternatively, only some of the closest and best² objects can be selected as occluders. Finally, the occluder selection process can rely on feedback information from the previous frames.

Algorithms relying on static occluder selection either do not select occluders, and thus store the list of potentially visible objects, or select good occluders from a pre-processed list, possibly with spatial information. These methods include aspect graph variants [86, 32, 33], original PVS implementations [4, 110] and other from-region PVSs. All these algorithms are limited to static environments and are highly memory-intensive. Some authors have proposed efficient compression methods [48, 113].

²According to feature classification.

11.2 Taxonomy of Conservative Visibility Algorithms

Spatial Database	Static	{ Construct Once	[53]
	Dynamic	{ Construct Adaptively	[104]
View Frustum Culling	Per Object	{	
	Hierarchical	{ [22]	
Occluder Selection	Static	{ Pre-Solved	[86]
		{ Select from Stored Set	[26, 74]
	Dynamic	{ Everything	[106, 15]
		{ Everything Visible	[53]
Occlusion Test	Image-Space	{ Depth Only	[15, 53]
		{ Coverage and Depth Separately	[125]
	Object-Space	{ Coverage Only	[52]
		{ All planes (shadow frusta)	[63]
Occluder Fusion	Static	{ Crossed planes (visibility events)	[26]
	Dynamic	{ Volumetric	[96]
		{ Projective	[35]
Occlusion Primitives	Test Primitive	{ Object-Space	[30]
		{ Image-Space	[53]
	Write Primitive	{ Objects	[53]
		{ Virtual Occludees	[22, 53]
Occluder Simplification	Static	{ Silhouettes	[53, 52]
	Dynamic	{ Objects	[72, 96]
		{ Virtual Occluders	[63, 2]
	Static	{ Silhouettes	[63, 2]
	Dynamic	{ Object-Space (view independent)	[56]
		{ Object-Space (view dependent)	
	Static	{ Image-Space	

Figure 11.1 Taxonomy of conservative visibility determination. Exploiting temporal coherence of camera transformation allows certain static relations to be constructed dynamically. Though theoretically possible, no such method has been published.

Occlusion Test

There are several fundamentally different ways for performing an occlusion test. Figure 11.1 includes a taxonomy of occlusion testing. The first question is whether the culling takes place in image-space or object-space. Image-space algorithms are limited to a fixed resolution and can be seen analogous to *point sampled* exact visibility algorithms, whereas object-space algorithms are analytical and thus *continuous*. Additionally, object-space algorithms can potentially exploit temporal coherence unlike any currently known image-space algorithm.

Image-Space occlusion test can be carried out in three different ways. Performance-optimized implementations are always hierarchical. Perhaps the most obvious approach is to perform depth tests using the output resolution [15, 53]. An alternative is to perform either the coverage or the depth test at the output resolution and to quantize the other one. Depth representation requires more memory than coverage. Therefore it is a better alternative to quantize the depth as in HOM [125] (see section 10.4.4). If strict front-to-back traversal is available, either coverage or depth alone is a sufficient representation. Since the depth value can be quantized into a single bit, these two tests become the same. Coverage-only representation was used in “Hierarchical Polygon Tiling with Coverage Masks” [52] (see section 10.4.3). The Validation Buffer introduced in section 11.10.7 also works in dynamic environments whenever possible, and uses other algorithms to handle the problematic cases.

Object-Space occlusion test can be carried out in two fundamentally different ways. The first alternative is to construct shadow frusta of the occluders and use those to cull the geometry [63]. As a result, all the occluding planes are tested every frame, and culling with shadow frusta is not temporally coherent. An alternative is to track visibility events when specific planes are crossed [26]. This is temporally coherent, since tests are only performed when visibility events occur.

Occluder Fusion

Fusing plurality of occluders into bigger entities can be done either as a pre-process or at run-time. In either case the two options are to fuse in object-space [96, 30] or in image-space [35, 53]. Of these four options, only the dynamic image-space solution has been demonstrated to be suitable for general purpose use.

Occlusion Primitives

Occlusion culling can be implemented using several different primitives in both occlusion writes and tests. Using simplified geometry is theoretically the same as using the object and hence is not listed separately.

The basic approach is to use the object itself as the occlusion write primitive [53, 125]. Alternatively, any hidden object can be used as an occluder [72]. See section 29 for a description of virtual occluders. Finally, an object’s silhouette can be used instead of the object itself. In object-space this approach has been taken by Hudson *et al.* [63] and in image-space by us (section 11.9).

Occlusion tests are usually performed at a somewhat lower precision than writes. The object itself can be used as a primitive [53, 52]. An alternative is to perform the test with *anything that encloses the object* [22, 53]. We use the term *virtual occludee* for such representations. Most commonly these are the nodes of the spatial hierarchy [53] or the bounding volumes of the objects [125]. In image-space algorithms the virtual occludee is often further conservatively simplified into an axis-aligned

11.2 Taxonomy of Conservative Visibility Algorithms

rectangle with a single depth value.

Occluder Simplification

The results of view independent occlusion preserving simplification (OPS) are valid only if the object is always observed from outside (or inside) [56]. Another option is to perform OPS dynamically according to the current view. We are not aware of such approaches.

Exact Visibility Determination

Any conservative algorithm must be backed up by an exact algorithm. Taxonomy of exact algorithms is show in figure 38. Almost every currently available graphics hardware uses the Z-buffer [15] (outlined in section 10.3.4).

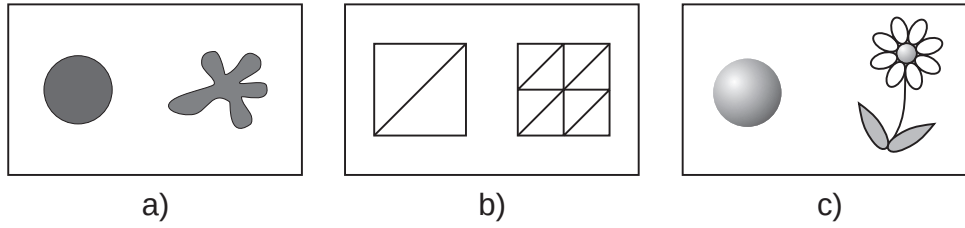


Figure 11.2 Compactness as a tessellation and shape descriptor. The figures on the left are compact whereas the ones on the right are not.

11.3 Occluder Selection

Instead of just selecting *simple* and *large* occluders or relying on *a priori* information, a more analytic approach is needed. The concept of being a good occluder is entirely context-dependent and has little to do with an object's projection size. Before proceeding with a new approach to occluder selection, we briefly explain some of the object characteristics we tried to use.

11.3.1 Potentially Good Occluders

Without context-dependent information we can calculate some descriptors that provide information whether an object *might* be a good occluder.

Visible Object

The most important rule is that a hidden object is generally a bad occluder.³ An exception to this are Virtual Occluders (see section 29).

Normalized Occlusion Property

Every object has a fixed occlusion property regardless of its transformation. When the scales are normalized, a coffee cup is generally a better occluder than a tree. *Compactness* is a transformation-independent shape descriptor widely used by the pattern recognition community [103]. Figure 11.2 illustrates the concept.⁴

$$\frac{edgelen\!g\!th^2}{area} \quad (11.1)$$

$$\frac{area^{3/2}}{volume} \quad (11.2)$$

³This information can be found in incremental algorithms.

⁴The shapes on the left are more compact than the ones on the right.

11.3 Occluder Selection

Equations 11.1 and 11.2 represent two and three dimensional compactness descriptors respectively. When used for triangle meshes, 2D compactness estimates the tessellation level. 3D compactness estimates how well the object occupies its bounding sphere. Therefore the global minimum is achieved with a highly tessellated sphere. Calculating an object's volume is a linear operation for closed manifold objects, but a hard and ambiguous problem for objects that are not closed.

Combining both 2D and 3D compactness formulas yields an estimate that tries to balance between low tessellation and a compact shape. According to our psycho-visual tests, the compactness measure reacts expectedly to holes and other problem areas, and sorts objects according to the normalized occlusion property.

Projection Size

From any viewpoint, the objects with a large projection area are likely to be better occluders than the smaller ones. The size of an object's screen projection can be quickly estimated from its bounding volume. Together with the Normalized Occlusion Property, it provides a reasonable estimate of the object's occlusion power. Unfortunately, this measure does not take into account the geometry actually being obstructed by the object. A building is a good occluder, but only if there is something behind it.

Static Context Dependency

In most cases, a small clock on a wall is a redundant occluder. Obtaining such information is hard unless provided by user. Static context-dependencies can be calculated automatically by selecting a large amount of viewpoints, and trying a large amount of occluder combinations to see which ones worked out the best [29]. Such extensive pre-processing is unacceptable for our purposes.

Dynamic Context Dependency

No proposed occluder selection algorithm can properly handle dynamic context-dependencies. Analyzing the results of different occluder sets can be used to tune the occluder selection process [122]. The accuracy obtainable with this approach is fairly limited.

11.3.2 Incremental Occluder Selection with Feedback

As pointed out, a potentially good occluder in the wrong place is a bad occluder. The problem of constructing the optimal occluder set is NP-hard⁵ and can only be approximated dynamically. Our goal is to estimate benefits of each selected occluder, and to use the results to predict the future context-dependent occlusion powers dynamically. This is appropriate, since the set of good occluders is usually temporally coherent. The spatial database provides approximate front-to-back traversal and the write queue (see 11.7) postpones the occluder writes up to the point the results are really needed. The resulting effect is mostly similar to the PLP-algorithm [67].

Let $\mathbb{F}$ denote the set of objects inside the view frustum, $\mathbb{O}$ the set of objects used as occluders, O_{base} the base cost of using occlusion culling and R_i , T_i and W_i the costs of rendering, performing an

⁵With N objects there are $O(N!)$ different occluder sets. Due to *occluder fusion* the occlusion power of an object depends on (possibly all) other objects. Thus finding the optimal set is a combinatorial problem.

occlusion test and using the i th object as an occluder respectively.

The cost of rendering a frame without occlusion culling:

$$\sum_{i \in \mathbb{F}} (R_i) \quad (11.3)$$

The worst-case cost of rendering a frame with occlusion culling:

$$\sum_{i \in \mathbb{F}} (R_i + T_i) + \sum_{j \in \mathbb{O}} (W_j) + O_{base} \quad (11.4)$$

Let $\mathbb{B}$ denote the set of objects culled away so that current object's contribution was necessary and $\mathbb{N}_i$ the number of objects participated in culling the i th object. Since the O_{base} is independent of the occluders selected, we can omit it from our cost-benefit evaluation. Then the total benefit of the current object is:

$$\underbrace{\sum_{i \in \mathbb{B}} \left(\frac{R_{\mathbb{B}_i} + T_{\mathbb{B}_i}}{\mathbb{N}_i} \right)}_{gain} - \underbrace{W_{object}}_{lose} \quad (11.5)$$

Constructing $\mathbb{B}$ requires the use of a full-resolution Z-buffer and an item buffer, which is an expensive operation and therefore impractical⁶. Let us define $\mathbb{C} \subseteq \mathbb{B}$ as the set of objects culled away, so that the current object contributes to the coverage buffer before the coverage test. Constructing this new set requires neither a Z-buffer nor an item buffer, but the evaluation still has to be performed with full resolution. Finally, we declare $\mathbb{D} \subseteq \mathbb{C}$ as the set, where the previous condition holds but the contribution area is a rectangle instead of a single pixel. It holds that $\lim_{area \rightarrow 1} \mathbb{D} = \mathbb{C}$. As an obvious consequence, we declare the size of this rectangle to be equal to the tiles used in our write queue. In general this would be 64x64 or 32x32 pixels. More accurate tiles have been observed to cause increasing overhead, while not providing significantly better results.

We believe that $\mathbb{D}$ is the closest possible approximation to $\mathbb{B}$, so that the time spent in building the set does not become a dominant cost. Construction of $\mathbb{D}$ proposes strict requirements for the occlusion sub-system:

1. Approximate front-to-back traversal.
2. Occlusion writes must be postponed until their contributions are needed.⁷
3. Incremental occlusion buffer.
4. Feedback from the coverage buffer, if the object contributed to the area in question.

⁶Except for testing the algorithm.

⁷This would be unnecessary if strict front-to-back traversal and exact depth writes were available.

11.3 Occluder Selection

The object test cost $T_{\mathbb{D}_i}$ in equation 11.5 is dominated by the rendering cost and cannot be reliably estimated due to hierarchical testing⁸. Omitting the term is analogous to slightly overestimating the rendering cost. So the final benefit for an object is:

$$\underbrace{\sum_{i \in \mathbb{D}} \frac{R_{\mathbb{D}_i}}{N_i}}_{\text{gain}} - \underbrace{W_{\text{object}}}_{\text{lose}} \quad (11.6)$$

Implementation

We can estimate W_{object} from the silhouette cache's hit ratio, silhouette extraction cost, screen projection area and silhouette complexity. N_i can be maintained incrementally inside write queue tiles by appending contributing occluders individually to a *benefit receiver list*. The coverage buffer must supply the information about whether the object contributed inside the tile area. $R_{\mathbb{D}_i}$ is a function of an object's geometrical and material complexity. Unfortunately this can only be approximated. However, occluder selection is rather insensitive to errors in calculation of either one of the costs. Both the *gain* and *lose* parts of equation 11.6 tend to get approximated incorrectly in the same direction, and what really matters is that the correct occluders get the benefit.

Whenever an occludee is found to be hidden, its rendering cost is divided among all tiles it touches. For each tile, we divide the assigned benefit with the number of occluders currently in the benefit receiver list and accumulate the value. When the entire frame has been processed, the benefit receiver lists of all tiles are collapsed and the benefits are assigned to the objects. The proposed two-pass approach is of linear complexity unlike the direct assignment, which grows quadratically with the amount of receivers.

Each processed object stores the benefit it received during the previous frame. Additionally, a fixed-size history of N previous benefits is maintained to stabilize the prediction. We call the balance between these two terms *responsivity*. Responsivity is used to indicate how quickly the predictor reacts to changes. Putting more weight to history results in a predictor that is less prone to accidental changes in occlusion power, but reacts slower to new potentially good occluders.

Outdated and Insufficient Information

We are able to get rather reliable information about the true occlusion power of the occluders used. Since data from previous frames is used to guide the selection process, problems arise from old information and from not getting information about objects that have not been used as occluders lately. In such cases, we must somehow try to use the object as an occluder to get updated information. The following cases arise:

1. The object has recent information and estimates are valid.
2. The object has been hidden for N frames, but has become visible during this frame, and its information is therefore outdated.
3. The object has performed poorly during the previous times it was used as an occluder, and has therefore a very low benefit estimate.

⁸Visible Point Tracking (section 11.4) diminishes the cost of occlusion testing considerably.

How should we react to objects becoming visible? Since there is no information about their potential occlusion power, or it is outdated, our best bet is to use them as occluders to get the information updated. However, if the object smoothly slides into the view frustum, it almost certainly receives very little or no benefit during the first frame it is visible. This is clearly unfair. Even if an object has been determined to be inadequate from place A, nothing guarantees that it could not perform extremely well from place B.

Including a random term into the selection procedure gives a possibility to distribute the chance of a new object getting selected to be an occluder over N frames. The expected time before selecting the object can be bounded when the repeated execution of the random function is considered to be a cumulative probability function. The *cost-benefit* function for occluder selection is:

$$\underbrace{\alpha\mathcal{P} + (1 - \alpha)\mathcal{A} + \frac{\mathcal{W}}{q}}_{\text{benefit}} - \underbrace{\mathcal{W} * \text{rnd}()}_{\text{cost}} \quad (11.7)$$

The first two terms denote the blend between the previous benefit ($\mathcal{P}$) and the N -frame average benefit ($\mathcal{A}$) using responsivity (α). The third term of the benefit part assures that objects of categories 2 and 3 (above) get a chance to prove their occlusion power during a bounded frame interval. The basic idea is to increase an object's possibility of getting selected along with the time passed since the object was previously visible. An estimate of the write cost ($\mathcal{W}$) is present to scale the values to match the cost side. Scaling value q is selected so that by executing the selection code N times, the cumulative probability of an object getting selected reaches 0.99. To enable this, the cost side of the equation includes $\text{rnd}()$, which returns a pseudo-random number in range $[0, 1[$. For example, if $q = 500$, the condition is fulfilled when $N = 30$. In other words, an object has a high probability to get a chance as an occluder during a period of 30 frames regardless of its observed occlusion power.

Let λ denote the number of frames elapsed since the object was last visible. The objects of category 3 usually have $\lambda = 1$. As the objects sliding into the view frustum tend to have limited occlusion power during the first few frames, and as the equation tends to select such objects straight away, we utilize a trick to assure they get another chance in a short while. When an object with $\lambda > 15$ is observed, we overwrite its average history

$$\mathcal{A} = \mathcal{W} \quad (11.8)$$

before executing equation 11.7 in order to give the object an increased chance to prove its abilities during the next few frames.

Global Knowledge

If several occluders are required to hide a single object, the algorithm proposed can fail to select all the required objects simultaneously, and to conclude that they are all bad occluders. We call the operation of using every visible object as an occluder a *flash*. Flashing increases the amount of occluders used every N frames by I percent. Typically we have observed 30 – 100% of visible objects being used as occluders (v). The overall cost of flashing is

$$I = \frac{100\%}{v\%} * \frac{1}{N} \quad (11.9)$$

11.3 Occluder Selection

which is 6.7% more occluders for typical values ($N = 30$ and $v = 50$). This is the cost of fixing the occluder selection information of every visible object and effectively prevents the convergence to a bad occluder set. Flashing increases the processing time of a single frame and the operation cannot be amortized over several frames. We have not been able to witness significant performance degradation in actual scenes during the flash. This is mainly due to lower amount of visible objects reported to the rendering subsystem during the flash. dPVS uses only a limited portion of the CPU time, and consequently the peaks are not that big. One of the reasons why we can use flashing is that we always select only the visible objects as occluders. Therefore the flash operation remains output-sensitive.

Another possibility is to use the flash only when there seems to be a need for it. When the increase in the amount of visible objects between two subsequent frames is more than a fixed threshold, say 30%, it is probably an indication of an abrupt change in the camera motion or the scene geometry. It may or may not be that the occluder selection algorithm has performed poorly. Regardless of the actual reason, we can guess that there was something wrong with the occluder selection, turn on the flash and re-resolve the visibility.⁹ The current implementation of uses both fixed frame (once every 32 frames) flashing and 'panic' flashing.

Discussion

The occluder selection algorithm proposed finds the occluders that actually hide other objects. It completely ignores previously used heuristics of using 'large' and 'simple' objects as occluders. Instead of such *a priori* classification, our algorithm dynamically learns to use the correct occluders, and we have observed the expected behavior with scenes ranging from a few objects to scenes where the fusion of thousands of occluders is required.

It is theoretically possible for our occluder selection algorithm to conclude that there are not enough good occluders and therefore infinite lines of sight can exist. Such a case can be a big problem when visualizing very large environments. Fortunately it can be detected by utilizing a heuristic that compares the amount of visible objects of the previous and the current frame. If the amount of visible objects increases by more than 40%, it is likely that infinite lines of sight exist and therefore we perform the query again with the flash enabled.

⁹Activating the flash for the *next* frame can cause the application to fetch unnecessary data to the memory.

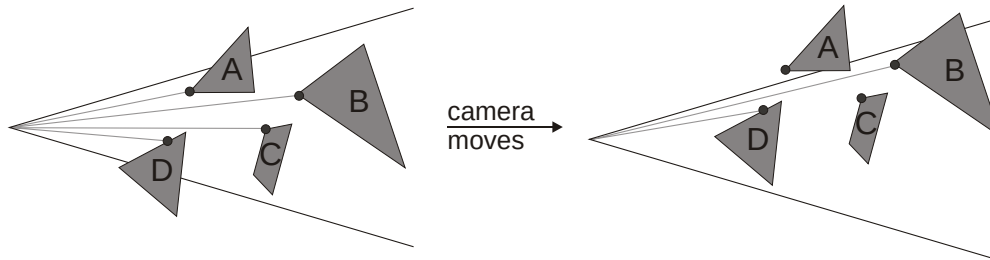


Figure 11.3 Principle of Visible Point Tracking. After the camera has moved, objects B and D can be proved visible with a single ray cast. Visible point candidates of objects A and C are hidden and therefore a more involved visibility test is required.

11.4 Visible Point Tracking

Any visible object can be theoretically proven visible with a single ray cast. If we always knew the correct point to cast the ray into, none of the visible objects would require further testing. Thus the visibility determination problem would be reduced into proving that the *nearly visible*¹⁰ objects are hidden. Being backed up by an image-space algorithm, we can replace the ray cast with a constant time image-space test. If the selected visible point candidate was visible, the object is visible. If the object gets determined visible by another more involved test, we should try to find a better visible point candidate. The problem is solved if the image-space algorithm used can supply coordinates of some actually visible point, otherwise we randomize a new location inside the object.

Figure 11.3 a) illustrates the concept of Visible Point Tracking. In 11.3 b) object A's visible point is no longer visible even though the object is. Therefore a new visible point should be calculated. Visible points expire quicker when the camera motion is rapid. Figure 11.4 shows an example of Visible Point Tracking in action. The camera is located at the origin of the rays. In this case only one of the objects fail the ray cast and therefore requires more involved visibility tests.

The same technique can be used to speed up view frustum culling. We have been able to find on average 80% of the visible objects using Visible Point Tracking. If the camera motion is subtle, the figure steadily approaches 100%. Additionally, we can expect that the objects passing the test really are hidden. Therefore we dare to use significantly more effort to prove this, and even if the tests fail, we get a better estimate for a new visible point candidate, and are possibly able to skip the more expensive tests next frame. While Visible Point Tracking provides temporal coherence into determining that the object is *visible*, it provides no solid information about objects being hidden.

¹⁰Nearly visible objects are the ones that are hidden, but need to be processed as their parent bounding volume (voxel) cannot be determined as hidden.

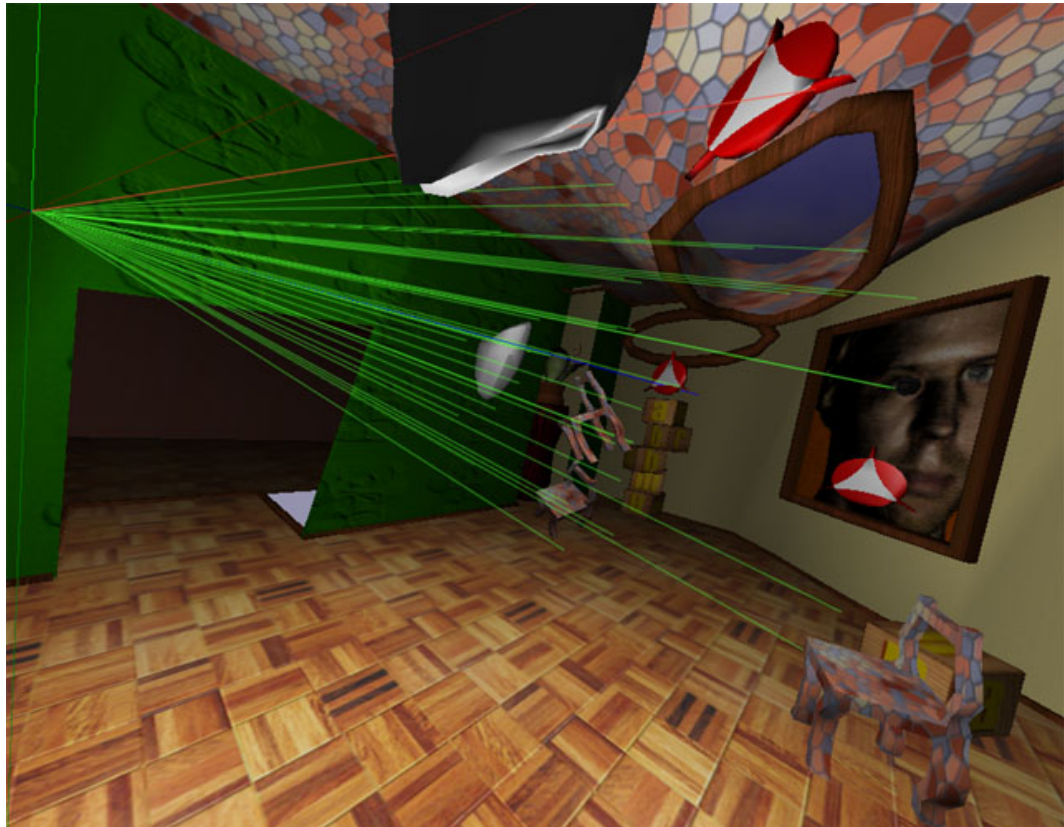


Figure 11.4 Example of Visible Point Tracking. The camera is located at the origin of the rays. In this case only the topmost ray (shown in red) fails to hit its target. That particular object is a *moving* logo and therefore its visible point candidates expire quicker than the ones used for the static objects.

OBJECT VISIBILITY TYPE	ACTION	
Visible	Prove by Visible Point Tracking	} temporal coherence
Hidden but nearly visible	Prove by Occlusion Buffering	
Well-hidden	Avoid testing by using hierarchical search	} spatial coherence

Figure 11.5 Object visibility types and corresponding actions

11.5 Object Visibility Types and Actions

An object's visibility type can be classified into the three categories shown in figure 11.5. The action taken is based on the visibility type. Temporal coherence is exploited by Visible Point Tracking and spatial coherence by occlusion buffering and the hierarchical spatial database search.

11.6 Occlusion Buffer and Plan of Development

Occlusion Buffer is a common term for image-space occlusion culling data structures. The most common occlusion buffer algorithms include hierarchical Z-buffer[53] and hierarchical occlusion maps (HOM)[122]. Non-hierarchical variants, such as 1bpp coverage buffers and span buffers exist, but are not used in speed-optimized implementations.

We present our incremental occlusion buffer in section 11.9.

Coverage Buffer is a part of the occlusion buffer that stores coverage information. In addition to coverage information, an occlusion buffer generally requires some representation of depth. The resolution of the depth representation can be chosen independently of the coverage buffer and is usually smaller in order to obtain lower memory consumption and faster processing speed. The most common representations include Z-buffer and depth estimation buffer (DEB).

We proceed by introducing an occlusion write postpone algorithm (11.7). Silhouettes as occluder primitives are considered in 11.8 and the results are used in section 11.9 for creating an incremental version of HOM. Hierarchical version of DEB is reviewed in section 11.11.

11.7 Occlusion Write Postpone Queue

With *incremental* occlusion buffer algorithms it is possible to delay writes to the occlusion buffer up to the point when their result is required. This is not important with very high depth complexity scenes, but a useful optimization with scenes of modest depth complexity. Practically all real-time graphics falls into the latter category. By using an Occlusion Write Postpone Queue the following useful optimizations are introduced:

1. The depth estimation buffer contains better (i.e. less conservative) values during the occlusion tests as the non-contributing occluders have not yet “trashed” the depth estimation buffer values.
2. The write queue working together with the incremental occlusion buffer can be used to calculate context-dependent occlusion powers. Section 11.3.2 discusses this in detail.
3. The depth complexity of the occlusion buffer is reduced as far-away occluders that cannot occlude anything are not rasterized
4. When combined with portal rendering, the write queue removes a large amount of unnecessary writes. In the best case, only the objects that overlap subsequent visible portals are rendered.
5. Improves CPU cache-coherence, because consecutive writes and reads are subjected to the same part of the screen, and thus to the same memory area.

Since occluder rasterization may not be a by-product of the actual scene rasterization¹¹, it is necessary to rasterize only useful occluders to minimize the rasterization costs. The properties listed above all help to reduce rasterization costs *and* to improve the quality of the occlusion and depth estimation buffers.

Our current implementation subdivides the screen into tiles (axis-aligned rectangles) and the write queue is locally *flushed*, if a visibility test passes and there are pending writes with closer depth values than the visibility test depth value. As a result, the contents of the occlusion buffer are not updated, unless there is a possibility to hide the object.

Flush Tabus

If we expect an object to be visible¹², we do not allow the object to flush the write queue. This rule decreases the amount of write queue flushes immensely. As we want every adapting algorithm to contain a small amount of random behavior, the objects that are expected to be visible have a 6% chance of flushing the write queue. Objects that are expected to be hidden can always flush the write queue.

¹¹For instance, in HZ-buffer[53] the occluder rasterization is a by-product of the rendering algorithm, in HOM[122] it is not.

¹²Based on history data.

11.8 Silhouettes as Occluder Primitives

The silhouette of an object represents exact coverage information. This is easily proven by observing that drawing the object with a single color in fact creates the object's silhouette. Therefore it is not necessary to consider the actual object geometry when performing coverage writes¹³. HOM (10.4.4) renders the actual geometry to produce an image of silhouettes. This is unnecessary and we take an alternative approach and omit the phase of rendering the objects. Instead we extract the silhouettes in object-space, and directly use them to create the coverage buffer. This has the substantial advantage of reducing the edge complexity from $O(n)$ to approximately $O(\sqrt{n})$ in many typical models [91].

In order to efficiently use silhouettes as occluder primitives, we must be able to extract, clip and render them as quickly as possible. We proceed by first optimizing silhouette extraction, then rendering and finally clipping. Algorithm for converting alpha textures into silhouettes is given in 11.8.4.

11.8.1 Temporally Coherent Silhouette Extraction

Brute-Force Extraction

Brute-force silhouette extraction is a linear algorithm and would be a dominating cost in our occlusion buffer algorithm (section 11.9). Occlusion-preserving simplification algorithms are still inadequate to be used in a commercial library, and usually the actual geometry has to be considered when rendering the occluders.

Output-Sensitive Extraction

Most output-sensitive silhouette extraction algorithms only work under orthographic projection and are thus not useful for our purposes. Sander *et al.* [91] describe an algorithm that also works with perspective projection. According to their results, their algorithm is faster than the brute-force algorithm, if the object has more than a 1000 edges.

Occlusion-Preserving Temporally Coherent Extraction

Theorem 1. *Silhouette of an opaque mesh $\mathcal{M}$ extracted from location $\mathcal{L}_0$ is a valid occluder when observed from $\mathbf{R}^3$ if a) $\mathcal{M}$ is two-sided or if b) $\mathcal{M}$ is a closed surface and only observed from the outside.*

Proof. Back-face culling $\mathcal{M}$ from location $\mathcal{L}_0$ produces a list of N opaque triangles T_i . Outlines of T_i are a piecewise representation of the silhouette of $\mathcal{M}$. Each one of the triangle outlines accurately bounds a piece of the actual surface, and is therefore a valid occluder when observed from $\mathbf{R}^3$. The above is true only if all of the triangles T_i are opaque from both sides.

If two triangle outlines share an edge, the edge can be removed to merge the triangles into a single outline, which is guaranteed to be a valid occluder when observed from $\mathbf{R}^3$. This relation holds when all shared edges are recursively removed. The operation of removing the shared edges is silhouette extraction. Therefore the relation holds for a silhouette.

¹³This is not of course true for the depth tests.

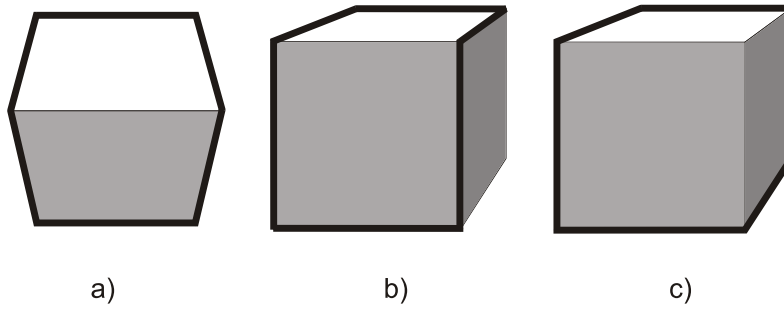


Figure 11.6 Re-transforming a previously extracted silhouette (*a*) to a new orientation (*b*) produces a conservatively wrong silhouette compared to (*c*).

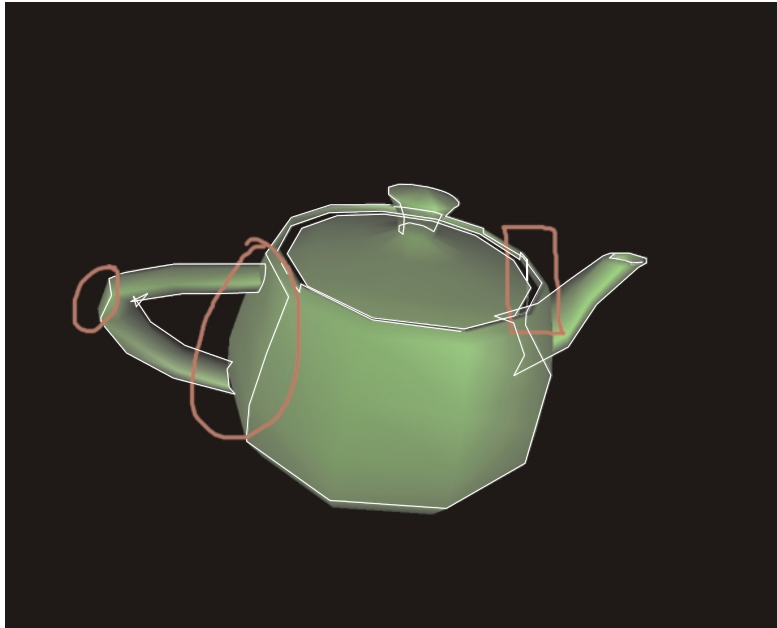


Figure 11.7 An example of a re-transformed silhouette. Two marked areas on the left are conservatively wrong, but the one on the right can produce false occlusion due to issues explained in the text.

11.8 Silhouettes as Occluder Primitives

The triangles T_i do not have to be opaque from both sides if only one side of each triangle can be observed. This is only true if the mesh is closed and observed from the outside. $\square$

Any silhouette of an object is always a valid occluder. As long as the specified angular distortion constraint remains satisfied, the same silhouette can be re-transformed instead of extracting a new one. The re-transformed silhouette is an *occlusion-preserving impostor*. Figure 11.6 a) illustrates the original silhouette b) the re-transformed conservative silhouette and c) the correct silhouette.

We have implemented a silhouette cache, which stores object silhouettes and either finds the best existing match or creates a new silhouette depending on object and camera transformations. The re-transformed silhouette in figure 11.7 is extremely conservative. Most notably this can be seen in places marked with rounded symbols. The occurrence marked with a square symbol is the only true defect of re-transformed silhouettes. The teapot model violates the principle of bidirectional visibility and the re-transformed silhouette can therefore cause false occlusion. If that is not acceptable, such models should not be used as occluders or the models should be fixed.

We have witnessed silhouette cache hit ratios ranging from 50% to 100% depending on camera and object motion. On average we are able to skip approximately 95-100% of the silhouette extractions¹⁴. These figures could be further enhanced by allowing more distortion, which would in turn decrease the occlusion power of an object. Where the break-even lies is scene- and platform-dependent and we are currently adjusting the limits at run-time using simple feedback heuristics. To prevent all sudden moves from invalidating the caches, we allow the objects to initially use worse silhouettes and tighten the constraint according to time the silhouette has been used.¹⁵

The silhouette cache is independent of the actual extraction algorithm and effectively multiplies the performance. For relatively simple meshes with less than ~2000 triangles we opt for using brute-force extraction to save memory. We have not implemented an output-sensitive extraction algorithm but currently the most appealing approach seems to be the *search tree* by Sander *et al.* [91]. It remains to be seen whether the output-sensitive extraction is actually needed. The options for avoiding it are to split the model into several parts, use simplified geometry, or to disable the occlusion writes of a complex model.

The first versions of dPVS used the approach described above, but the latest releases favored another method, explained in the next section. The primary reason for switching to another approach was lost occlusion volume in case of splitted input meshes, most prominently with terrain scenes.

Distance-based Temporally Coherent Extraction

The algorithm works solely using *edges* of meshes, so it does not need triangle data at run-time. As a pre-process, we find all edges of a triangle mesh that are *not* shared by coplanar surfaces, i.e. all edges that can potentially contribute to a silhouette. For each such edge we store indices to the vertices forming the edge and indices to the plane equations of the 1 or 2 connected surfaces. If there are more than two surfaces connected to an edge we duplicate the edge.

The actual silhouette extraction works in two parts: *caching* “incremental” silhouettes from certain viewpoints and *extracting* silhouettes from the incremental representations.

Creating an incremental representation is straightforward. First, we transform the camera position into the model’s space, `invCamPos`. Then we compute the distance from `invCamPos` to each of the

¹⁴Silhouettes are cached on a per-model basis, thus many objects sharing a model can share a cached silhouette.

¹⁵We have also limited the total size of the silhouette cache and perform aggressive cleaning of the cache.

planes of the model¹⁶. After this we traverse all edges of the model and

1. Classify edges into two groups, as described below
2. Compute smaller absolute distance to all the planes connected to the edge (*visual event distance*)

The edges are classified into two groups, the “in” group and the “out” group, based on whether the edge is a silhouette edge as viewed from `invCamPos` or a non-silhouette edge respectively. The smaller absolute distance indicates the minimum distance from `invCamPos` that a *visual event* can happen, i.e. the status of the silhouette edge may change from silhouette to non-silhouette or vice versa.

Then we sort all “in” and “out” edges separately based on the visual event distance. After this we select N closest “out” edges¹⁷. We allocate memory for all of the “in” edges and N “out” edges and store pointers to the edges and the visual event distance values¹⁸. We define `radius` as the largest “out” edge distance in the selection and store it along with the `invCamPos`. The data structure contains thus a silhouette as viewed from `invCamPos` and a set of edges along with “visual event distances” up to a certain distance (`radius`).

The silhouette extraction part is trivial and very fast. We transform the camera position into the model’s space and find *any* cached representation that satisfies the equation $\| \text{invCamPos} - \text{cached} \rightarrow \text{invCamPos} \| \leq \text{cached} \rightarrow \text{radius}$. If there are multiple cached entries that satisfy the equation, we choose the one with the smallest distance between the camera positions.

We define `dist` as the distance between the current camera position and the cached structure’s camera position. Then all we have to do is to traverse the edges in the cached structure. For “in” edges we compare `dist` to the visual event distance. If the current distance is smaller, we automatically select the edge¹⁹. If it’s greater, we perform two dot products²⁰ for classifying the edge as silhouette or non-silhouette.

For the sorted “out” edges we have to perform the plane equation tests, two tests per edge. However, we need only to traverse this list until we reach a visual event distance greater or equal to `dist`. We additionally use a simple vertex cache mechanism for removing duplicate vertices.

The end result is that if we have a cached silhouette, the absolute worst case work per extraction happens when `dist` equals `radius` and is bounded by $3 * \text{numSilhouetteEdges} * 2$ dot products. The best-case scenario, when `dist` is zero, requires just copying the “in” edges.

We’ve replaced the silhouette extraction algorithm of dPVS with this and have got very nice results. On an x86 box, we spend an average of ~200 cycles per *output edge* for the entire caching and extraction process. That’s 5M output edges per second on a 1GHz box. The algorithm scales well when there’s not that much camera coherence, is capable of sharing silhouettes when models are heavily instanced (or there are mirrors in the scene) and doesn’t consume much memory.²¹

Although we use the algorithm for extracting occluder silhouettes, it should work very nicely for shadow frustum extraction or perhaps for some cartoon rendering stuff.

¹⁶As the plane equations are normalized as a pre-process, this work is a dot product per plane.

¹⁷Currently we use $N = \text{number of “in” edges} * 2$.

¹⁸A total of 8 bytes per cached edge.

¹⁹As the status of the edge cannot change.

²⁰Camera position vs. plane equations connected to the edge.

²¹The total cache size has been limited to 256kB

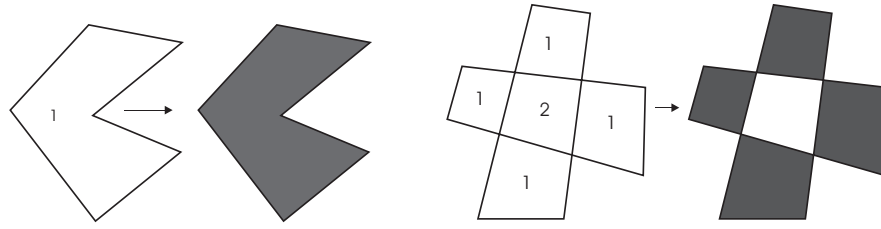


Figure 11.8 XOR filler handles correctly a single overlap (a), but fails when the overlap count is two or more (b). The numbers indicate the overlap count.

11.8.2 Silhouette Rasterization

A single bit per pixel is a sufficient representation of coverage. To alleviate the need for graphics hardware and downloading the frame buffer we create a specialized 1 bpp silhouette renderer, which needs a very high fill rate to compete against hardware rasterization.

Scan-line Filler

The usual approach [40]²² for silhouette filling is to assign all active edges into scan-line buckets. Value +1 is assigned to left edges and -1 to right edges. For each scan-line, the entries are sorted from left to right and *depth* is initialized zero. For each entry in the scan-line, $depth = depth + entry.value$ and if $depth > 0$, a span is drawn from the previous to the current position-1. Though very simple to implement, this algorithm is not very efficient in practice. The need for sorting and the high amount of conditional processing limit the performance.

XOR Filler

Commodore Amiga 500[®] presented all graphics with bit planes²³. Up to 5 bit planes could be displayed simultaneously and the final color of a pixel was indexed from a 32-entry palette. Amiga 500 also had a co-processor called *blitter*, that performed simple combinations of logical operations between two sources. Simple filled 3D objects were rendered by first drawing the outlines to the bit planes in XOR mode. Then the blitter was configured to perform operation 11.10 for each scan-line²⁴.

$$scanline_i = scanline_{i-1} \text{ XOR } scanline_i \quad (11.10)$$

This approach only works for silhouettes without overlaps (Figure 11.8), but has the very desirable property of being totally unconditional and highly parallel.

²²Active edge table.

²³bit plane = 1 bit per pixel buffer

²⁴In fact the blitter did not perform the operation vertically but horizontally.

***N*-bit Parallel Filler**

We merge two previous algorithms by employing the algorithmic outline of the XOR filler and extending the logical operation to handle multiple overlaps. The basic idea is to store the per-pixel overlap counts into N bitplanes. The bitplane₀ stores the lowest bits, bitplane₁ the next bits and so on. The logical operations needed are identical to a hardware ALU accumulator. We first accumulate the left edges with value $+1$ and then the right edges with value -1 . As our N -bit numbers are unsigned, adding -1 is equal to adding $2^N - 1$. The filler output bit is set if any of the bitplanes have a set bit. This operation reduces to a logical OR among all bitplanes.

Technically this never guarantees the correct output, since a silhouette can potentially have an unlimited number of overlaps. In practice, the amount seldom reaches $2^3 = 8$, which is the reason we are currently using $N = 3$. One key observation is that even if the capacity of the accumulator is exceeded ($N > 8$) the output is only conservatively wrong. This can be verified by observing that $M * 8 \Rightarrow 0$, which is incorrect, but no input sequence can result in $0 \Rightarrow !0$. Thus we ignore the problematic cases.

Each pixel on the scan-line is processed independently of each other and we only need logical operations. Therefore we can fit the bitplane data of 64 pixels into N 64-bit registers and are able perform the accumulation in parallel. On Intel Pentium III[®]-based machines a 3-bit 64 pixel parallel accumulator executes in approximately 16 cycles, which gives the peak fill rate of $64/16 = 4$ pixels per cycle. Running at 700MHz this sums to 2.8 billion pixels per second. Even if such figures cannot be demonstrated in real-life scenarios, these numbers indicate that the filler is unlikely to become a bottleneck of the system.

11.8.3 Silhouettes of Clipping Objects

Our silhouettes are a simple collection of edges. The mesh facing is always clockwise and therefore a single comparison of y -coordinates suffices to indicate whether an edge is a left or a right edge. An arbitrary silhouette can have any amount of distinct closed regions and we do not require a specific ordering of silhouette edges. The silhouettes are defined with (x, y, z) -coordinates and are generally not planar before perspective projection.

Spatial Clipping

Since our silhouettes do not have connectivity information, the clipping procedure only needs to consider a single edge at a time. Clipping to top, bottom and right planes of the view frustum are simple rejections. If the edge is completely outside any of these planes, it can be rejected. The edges partially outside are clipped, and the parts outside the planes are rejected.

On the left border the rejection is not possible. To assure correct filler functionality none of the edges outside the left plane must be discarded. Instead the edge should be clipped to the left plane and the clipped part of the edge should be clamped to the leftmost column. This is illustrated in figure 11.9 where a) shows the consequences of discarding an edge and b) shows the clamping and subsequent correct filler output.

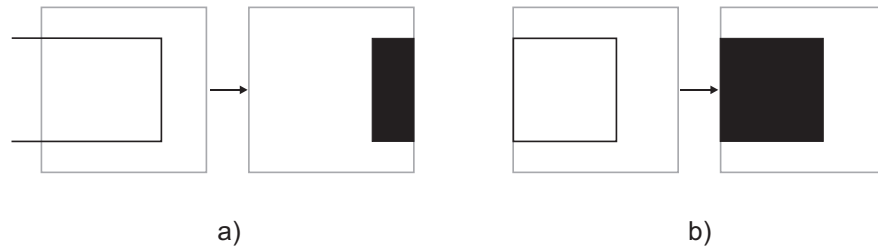


Figure 11.9 A silhouette clipping to the left plane. Clipping produces an incorrect result (a) whereas clamping produces the visually correct filler output (b).

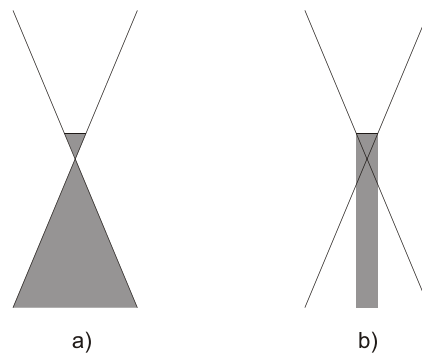


Figure 11.10 The gray area indicates the behind-the-viewport region of space from where a perspective projection (a) and a parallel projection (b) can project points into the viewport. Such a projection causes incorrect visual output and cannot thus be allowed. As can be seen from the figure, parallel projection has a significantly smaller gray region and consequently a larger valid backprojection area.

Silhouette Splatting

Very often the best occluders are the ones that intersect the front clipping plane. By intersecting the front plane we do not mean objects that penetrate the viewport, but the ones that intersect the infinite plane outside the viewport. For example, most walls and floors in an architectural scene intersect the front plane.

An obvious approach is to clip the object to the front plane, and then to extract the clipped silhouette. This suffers from the fundamental flaw of not being able to utilize temporal coherence.²⁵

The key observation is that the front clipping operation does not need to produce correct output, since it is only performed outside the screen. If an edge, or a part of an edge, is behind the front plane, we can *clamp* it to the front plane by using a parallel projection. A point can only get parallel-projected inside the viewport if it lies inside the gray area of figure 11.10 b). We call this gray area *back projection shaft*. An object intersecting the back projection shaft is considered an invalid occluder. Since anything *outside* the back projection shaft gets projected *outside* the viewport, the observed

²⁵Silhouette cache in our case.

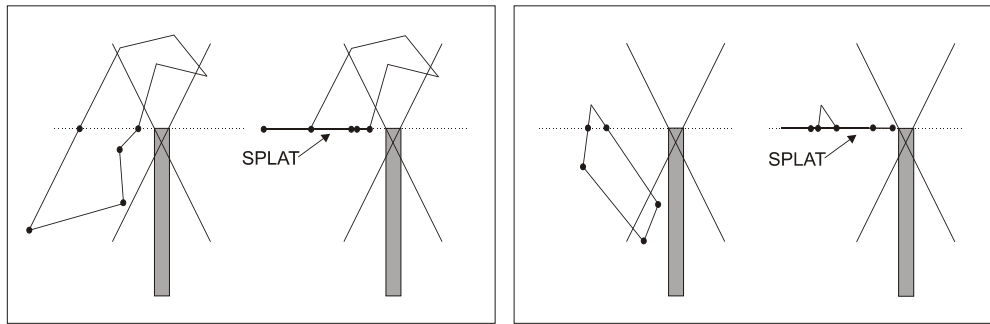


Figure 11.11 Silhouette splatting. If an edge of a front clipping silhouette is partially outside the front plane, the intersection point is calculated and the outside part is parallel projected to the front plane. Note that the silhouette on the right intersects invalid backprojection region of perspective projection but is a valid occluder if parallel projection is used (see figure 11.10).

filler output is always correct *inside* the viewport.

What actually needs to be done is a parallel projection of the *front clipping parts of edges of the silhouette* to the front plane. If the edge is partially outside the front plane, the intersection point is calculated and the outside part is parallel projected to the front plane. We call this operation *silhouette splatting*. The operation is illustrated in figure 11.11.

For comparison, if perspective projection is used, the points inside the gray area in figure 11.10 a) can get projected inside the the viewport. Normally the view plane distance is very small²⁶ and the *back projection shaft* of b) converges to a line. Therefore the invalid gray area in b) is much smaller than the one in a) and thus more front clipping occluders can be accepted.

The final rules for using cached silhouettes are:

1. If the *object* does not intersect the front plane, it is a valid occluder.
2. If the *object* does not intersect the back projection shaft, it is a valid occluder.
3. The front clipping silhouettes of valid occluders must be splatted.

Discussion

Not having to reject front clipping occluders more than doubled the performance of some of our test applications. During the development of dPVS this has been performance-wise the single most important issue. The silhouette splatting algorithm currently used can still reject some of the good occluders, but we consider the ability to use temporally coherent silhouette extraction more valuable than the rare unnecessary rejections. In fact, such rejection can only occur when an object intersects the back projection shaft without penetrating the viewport.

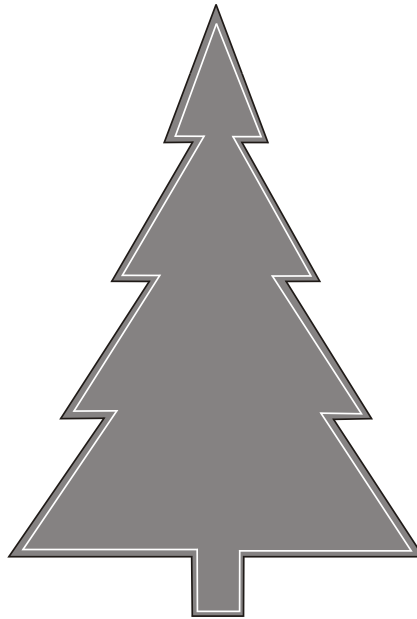


Figure 11.12 Converting an alpha texture into a silhouette. Any picture can be converted into a silhouette according to the alpha channel. The silhouette can be simplified as long as every edge is contained by the opaque area.

11.8.4 Silhouettes of Alpha Textures

Rendering the silhouettes directly loses the possibility of using primitives with alpha textures as occluders. This may or may not be an important issue depending on the application type. If alpha textures contribute significantly to occlusion, the texture can be converted into a simplified silhouette according to the alpha channel of the texture²⁷. Figure 11.12 illustrates the general idea. The job is easy to perform manually, but in the case of extensive use of alpha textures as occluders, an automatic extraction is preferable.

The general outline of a brute-force algorithm goes as follows:

1. Separate the opaque and transparent areas with flood fill.
2. Construct closed boundary silhouettes of all opaque areas.
3. Store vertices in a priority queue, sorted by *opaque area loss*.
4. Simplify boundary by removing one vertex at a time.
5. Allow the removal only if both new edges are fully within the opaque area.

The algorithm outlined is suitable for pre-processing purposes. Whereas the occlusion-preserving simplification of 3D meshes is a challenging problem, in 2D it is mostly trivial.

²⁶Optimally it would be zero, but is usually set to > 0 in order to get floating-point math working and to avoid certain singularities.

²⁷A texture mapped on a curved surface can create a very complex silhouette and the approach presented here is mainly suitable for planar surfaces.

Consider the case of a checkerboard texture where the black texels are opaque and the white ones are transparent. Such a texture map can be tiled arbitrarily many times over a single polygon. Thus, the exact alpha mesh of the polygon can be arbitrarily complex. Therefore meshing according to alpha channel should not attempt to be exact; occlusion losses should be accepted in areas where the edge complexity would become too high.

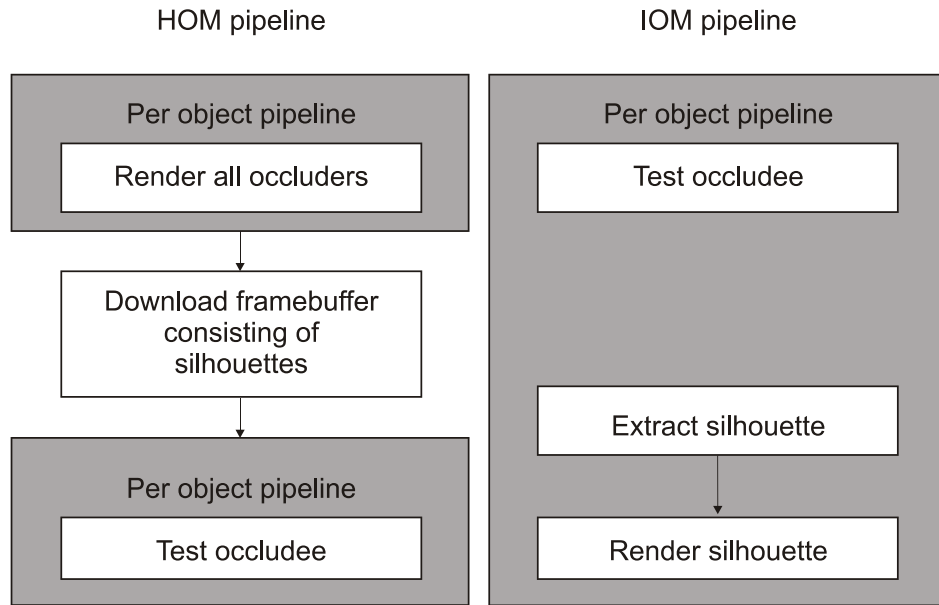


Figure 11.13 Comparison of HOM and IOM pipelines shows that IOM pipeline only includes per-object operations whereas HOM pipeline has a pipeline wreck in the middle.

11.9 Incremental Occlusion Maps (IOM)

11.9.1 HOM and IOM Pipelines

HOM (section 10.4.4) and IOM pipelines are shown in 11.13. The fundamental difference is that HOM first renders all occluders and then downloads the frame buffer consisting of silhouettes of the occluders. Downloading the frame buffer is not a per-object operation. This pipeline interrupt is important, since it disables the possibility of getting valuable feedback of the usefulness of the occluders.

IOM pipeline only consists of per object operations and makes it possible to incrementally add more occluders when needed. This in turn allows us to get estimates of the occlusion powers of the individual occluders. Occluder selection algorithm that uses this information is explained in section 11.3.

11.9.2 Occluder Fusion

Initially the *coverage buffer* is empty and the DEB is initialized to nearest depth value. We store a single floating-point depth value for each 8x8 *block* in the coverage buffer. This corresponds to a memory consumption of 0.5 bits per pixel.

For each occluder we fill the silhouette to a *cache* and then perform occluder fusion by ORing the cache to the coverage buffer. If the block in the coverage buffer changes during the occluder fusion,

we update the corresponding depth value.²⁸ As a result, the depth updates are limited to the blocks that satisfy both of the following conditions:

1. The silhouette at least partially covers the block.
2. The silhouette actually contributed to the corresponding coverage buffer block.

Comparison with HOM

HOM updates the depth values of all blocks that are at least partially covered by the *axis-aligned rectangle*, regardless of the actual contribution to the coverage buffer. As a result, our depth estimation buffer contains less conservative depth values. Since we can collect the coordinates of changed coverage buffer blocks into a list, we can optionally derive more accurate depth values only for the required blocks.

²⁸Depth update only occurs if the current depth value is *bigger* than the value stored for the block.

11.10 Implementing Incremental Occlusion Maps

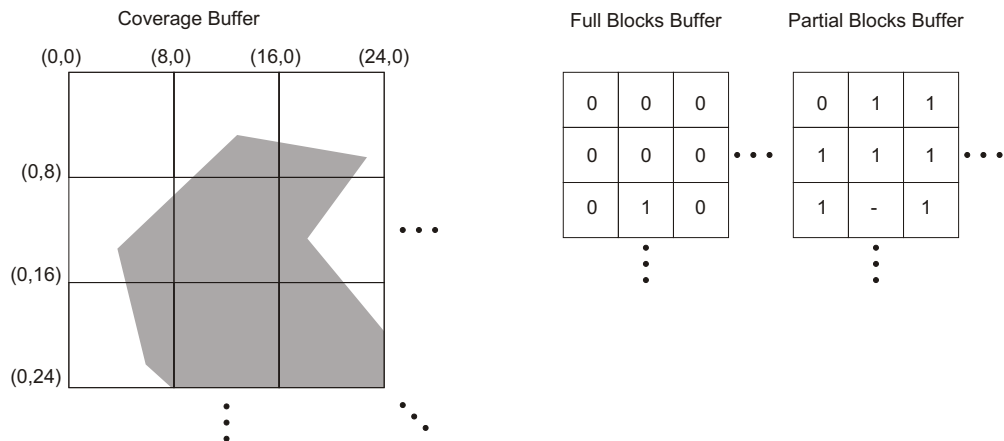


Figure 11.14 The coverage information of IOM is stored in three buffers. The Coverage Buffer contains the actual coverage information. The Full Blocks Buffer has a set bit if the corresponding 8x8 region in the Coverage Buffer is full and the Partial Blocks Buffer has a set bit if the corresponding 8x8 region is partially full. If the Full Blocks Buffer has a set bit, the Partial Blocks Buffer value is redundant.

11.10 Implementing Incremental Occlusion Maps

This section outlines an efficient implementation of incremental occlusion maps. The implementation uses a custom three-level hierarchy of the coverage buffer. We call these levels *bit*, *block* and *tile* resolution buffers respectively. Bit resolution equals output resolution, block resolution is $\frac{1}{8 \times 8}$ th and tile resolution is $\frac{1}{32 \times 64}$ th of the output resolution.

Each tile has a corresponding tile in the write queue (section 11.7) and stores information whether the corresponding coverage buffer area is completely covered. When the coverage buffer is cleared, the *Pending Clear Flags* are set to the tiles instead of actually executing the clear operation.

Whereas HOM uses a flat depth estimation buffer, IOM uses a hierarchical presentation in order to have faster depth queries for objects with a large screen coverage.

We begin by introducing the different buffers and proceed with tiled silhouette rasterization and hierarchical construction. Occlusion queries are explained in section 11.10.4. An alternative contribution culling technique, called *construction-time contribution culling*, is introduced in section 11.10.5. Incorporation with a stencil buffer and a useful optimization algorithm called *validation buffer* are explained in 11.10.6 and 11.10.7 respectively. Finally, conclusions on IOM are given in 11.10.8.

11.10.1 Buffers

Contents of the buffers to be explained next are illustrated in figure 11.14.

Coverage Buffer is a bit resolution 1 bpp coverage buffer, where each output pixel has a corresponding “coverage bit”. The bit is set if the pixel is covered.

Full Blocks Buffer is a block resolution buffer. A bit is set if all of the corresponding 8x8 pixels in

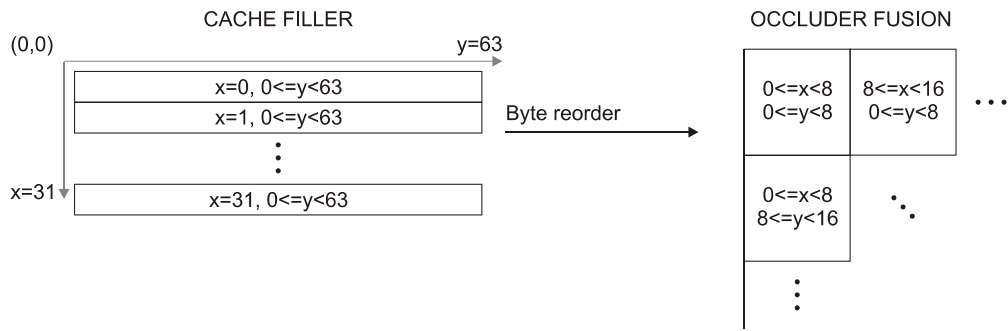


Figure 11.15 “Byte reorder” operation arranges filler output so that the data of each 8x8 blocks is consecutive in the memory. After the byte reorder operation each 8x8 block can be very efficiently manipulated and on native 64-bit architectures it fits into a single register.

the Coverage Buffer are set.

Partial Blocks Buffer is a block resolution buffer. A bit is set if any of the corresponding 8x8 pixels in the Coverage Buffer are set.

Hierarchical Depth Estimation Buffer is a block resolution buffer at the lowest hierarchy level. The higher levels are updated only when needed (see section 11.11).

11.10.2 Tiled Silhouette Rasterization

We have chosen to subdivide the output surface into 32x64 pixel tiles and to perform all filling operations inside a tile-sized 3-bit plane cache. We run the parallel filler in the cache from left to right. For vertical lines inside the cache we use the term *column*. Performing the filling operation using a fixed-size cache is straightforward and each tile only requires the last column of the previous tile. We call this last column the *fvalue*.

After the tile is filled into the cache, we perform a *byte reordering* operation to get 8x8 pixel masks into 64-bit registers (figure 11.15). The coverage buffer is updated by performing a logical OR operation with the masks. This results in occluder fusion.

Tiles that are 64 pixels high result in a lot of unnecessary rasterization. However, modern CPUs execute 64-bit operations generally faster than 8-bit operations. On the other hand, the byte reordering and the merging to coverage buffer are costly operations and should be avoided if possible. Thus we need to track *dirty rectangles* for each tile. Vertical bounds are easily obtained by ORing all of the columns together inside the filler. As the *outlines* are drawn into the cache before executing the filler, we can track the horizontal bounds there (figure 11.16).

We know the horizontal bounds before executing the filler and can therefore limit the execution to the dirty region. In a large number of cases, the size of the dirty region is only a fraction of the full tile. A favorable special case is a tile without any outlines crossing it, as large objects usually cover several tiles completely. All we have to do to fill such tiles is to set the current *fvalue* to each column of the cache. If the *fvalue* has all bits set, the tile is fully contained by the silhouette and the cache filler and the byte reordering operations can be skipped.

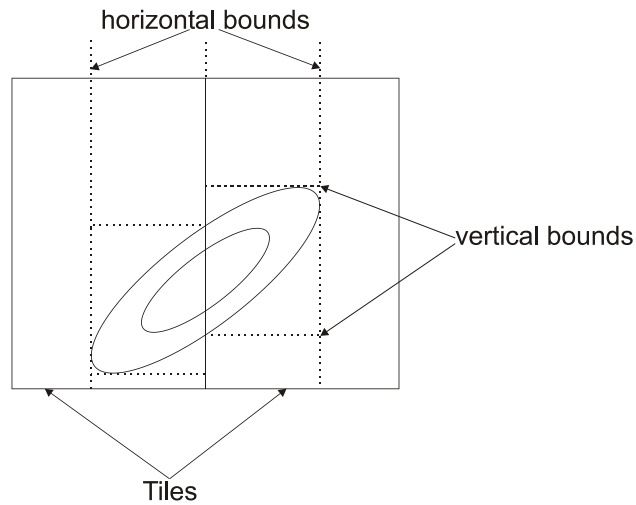


Figure 11.16 Vertical and horizontal bounds inside the tiles of the IOM filler. The horizontal bounds are used to limit the filler work and both of the bounds limit the byte reorder work (see figure 11.15).

11.10.3 Hierarchical Construction

Having introduced the buffer hierarchy, we can construct a faster version of the occluder fusion algorithm that both uses and updates the hierarchical buffers:

1. Skip the tile, if marked as full.
2. If the tile has a pending clear, clear the corresponding coverage buffer area.
3. Execute parallel filler to the cache.
4. For each dirty block inside the cache:
 - (a) Skip the block, if corresponding bit is set in Full Blocks Buffer.
 - (b) Merge cache block with Coverage Buffer block if no pending clear was set.
 - (c) Skip the block, if Coverage Buffer did not change.
 - (d) Append block coordinates to dirty blocks list.
 - (e) If the block became full, set bit to Full Blocks Buffer.
 - (f) If the block is partially full, set bit to Partial Blocks Buffer.
 - (g) After all the blocks have been processed, update the HDEB for all blocks listed in the dirty blocks list. Set the modified region to the HDEB to allow lazy updating of the hierarchy.

The first step deserves further investigation. If the processing of a tile is skipped without regarding the outlines inside the tile, the `fvalue` is incorrect when starting to process the next tile. To alleviate the problem, when a tile becomes full, the next non-full tile on the right is searched and set as a *reflection target*. If no such tile is found, NULL is set to indicate that the outlines can be fully ignored. The outlines crossing a full tile are set to the leftmost column of the reflection target instead

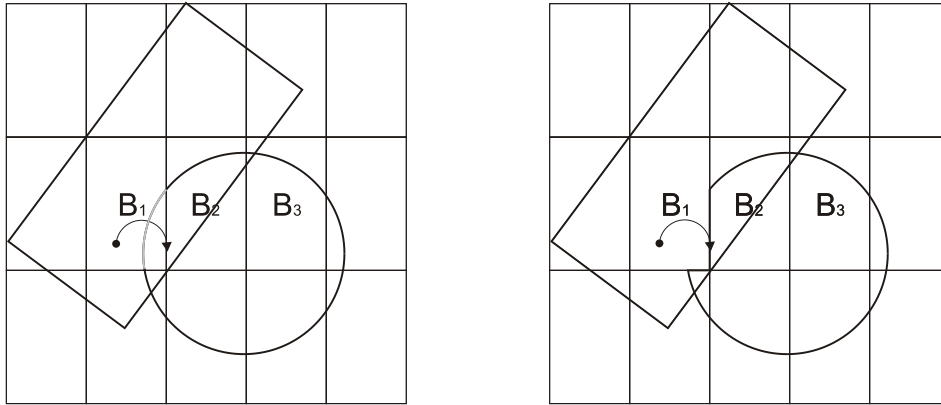


Figure 11.17 The box on the left has already been rasterized and therefore the bucket B_1 is full. A full bucket starts rejecting writes by setting its “reflection target” to the bucket on the right, in this case B_2 . When the circle is about to get rasterized, the part marked with lighter gray gets “reflected” to the leftmost pixel column of B_2 , as indicated in the figure on the right. After the modified circle has been rasterized, B_2 becomes full and its reflection target is set to B_3 . Furthermore, the reflection target of B_1 has to be changed to B_3 as B_2 no longer accepts writes.

of the cache. Figure 11.17 illustrates the process. On the left, the B_1 tile is full and has its reflection target set to B_2 . The parts of the circle that overlap B_1 are assigned to the leftmost column of the B_2 tile. Consequently no outlines are assigned to B_1 tile and therefore the tile can be skipped while rasterizing the circle. After the circle has been rasterized, B_2 becomes full and reflection targets of both B_1 and B_2 are set to point into B_3 . Had B_3 been outside the screen, both reflection targets would have been assigned NULL instead.

The HDEB is only updated in places where the silhouette alters the contents of the coverage buffer. Therefore the depth estimation buffer has less conservative values than the original HOM approach, which used axis-aligned rectangles to determine the DEB area to update.

When the depth values are written into the HDEB, they are either approximated to be the object’s z_{Far} or the conservative Z value obtained by casting a frustum through the object’s oriented bounding box. For planar objects, such as walls, this will produce exact depth values.

The algorithm presented skips work at every level. Figure 11.18 shows the actual contents of the Full Blocks Buffer (gray) and the Coverage Buffer (white). It is clear from the picture that our algorithm avoids touching the coverage buffer whenever possible. As a result, the fill rate increases when the screen becomes more covered. First the blocks, then the tiles and ultimately the entire coverage buffer get full, and start rejecting writes.

11.10.4 Occlusion Queries

We support three types of queries:



Figure 11.18 Contents of the Full Blocks Buffer and the Coverage Buffer marked as white and gray respectively. Cache coherence is improved since the higher resolution Coverage Buffer must only be accessed if partial 8x8 coverage is encountered.

Point Query

A point query tests the visibility of a single screen-space point. A depth test is performed prior to the coverage test. The coverage is first tested from the Full Blocks Buffer. Unless full, the Partial Blocks Buffer is tested to make sure there is a chance to succeed, if a full resolution query is used. Last, the partially full Coverage Buffer block is tested for accurate coverage information.

Rectangle Query

A rectangle query tests visibility of an axis-aligned rectangle. The coverage test is first performed using the block (8x8) resolution. If only partial coverage was encountered, a list of partially filled blocks is built and their accurate coverage is tested from the Coverage Buffer. This behavior is identical to the triage masks used by Greene in [52]. Finally, a depth test is carried out using HDEB.

Each bit of the Full Blocks Buffer encodes the visibility status of 8x8 pixels. A single test of 32/64 bits can be carried out in approximately 2 cycles²⁹, which results in a peak test rate of 1024/2048 pixels per cycle³⁰ and gives 0.7/1.4 TeraPixels at 700MHz. This clearly indicates that coverage testing will hardly be a bottleneck of the overall system.

Silhouette Query

A silhouette query tests the visibility of a *convex* silhouette. Each block that is even partially covered by the silhouette is tested for both coverage and depth. As the silhouettes are convex, we can use the XOR filler (see section 85) for additional fill rate.

All occlusion queries first execute a point test, then a rectangle test, and finally a silhouette test. The test silhouette for an object is extracted from its OBB. Spatial database nodes are tested one face at a time, and each node has 1-3 visible faces.³¹ Since the faces are always planar, we employ a more accurate depth test for database nodes. If the normal depth test that uses the closest depth value of the face fails, we evaluate a more accurate depth value for each 8x8 block using the plane equation of the face.

11.10.5 Construction-Time Contribution Culling

Traditionally contribution culling has been done in the occlusion *test* (section 10.5.1). If contribution culling is done at construction time, there is no need for transparency values and a single bit per pixel representation is sufficient for testing.

We normally set a bit in the Full Blocks Buffer when all 64 (8x8) pixels are set. We might just as well set the bit when a certain amount of pixels are set. For example, setting the bit into the Full Blocks Buffer when 60 of the 64 corresponding pixels are set, results in potential rejection of objects with less than 6% visible pixels.

²⁹On current x86 processors

³⁰For a 32-bit CPU: $32bits * 64pixels / 2cycles = 1024pixels/cycle$.

³¹If the camera is located inside a voxel, we always consider the voxel visible.

Error Bounds

With HOM (section 10.4.4) the *levels of visibility* are calculated hierarchically and the values in the higher levels provide no information about how the vacant pixels are distributed. In the worst case all of the vacant pixels can be consecutive. If contribution culling is done on an 8x8 block basis, the visible error is tightly bounded.

11.10.6 Stencil Buffer

In order to support complex portal sequences, the hardware must have a stencil buffer. Similarly the occlusion buffer must support stencil buffers to allow occlusion writes behind portals that use stencil masks.

A single bit presentation is sufficient for portal traversal purposes. We can implement the stencil buffer very efficiently by storing it in the same format as the cache. Initially the stencil buffer is set to fully covered, and subsequent stencil masks are applied to the stencil buffer by using a logical AND operation. The stencil buffer is applied to the silhouette being rendered by performing a logical AND operation *before* merging the cache contents into the coverage buffer.

The stencil test is performed for 64 pixels at a time and executes virtually for free. Rendering the stencil masks is the same operation as the silhouette rendering and an almost identical code is executed.

11.10.7 Validation Buffer

Our database traversal can guarantee that objects that are in front of a database node are processed before the node itself. In other words, when a node is processed it is known that nothing that is processed later can be closer to the camera than the node.³² This opens a new possibility.

Whenever a pixel of a node is found to be hidden, a bit can be set to the *validation buffer*. A set bit in the validation buffer indicates that *no depth comparison is required* for the pixel. The Validation Buffer can be made hierarchical in the same way as the Coverage Buffer (see 11.10.1).

Whenever a coverage test is being made, the Validation Buffer is used to mask parts of the primitive that are hidden regardless of the depth. As the implementation is hierarchical, the rejections become very fast when the *occlusion frontier* is reached. Ultimately the entire validation buffer becomes full, and all subsequent occlusion queries succeed without actual tests.

The Validation Buffer essentially converts a HOM into a coverage buffer of “Hierarchical Polygon Tiling with Coverage Masks” [52], but unlike Greene’s algorithm, it can also be used without strict front-to-back traversal.

The Validation Buffer has the following important properties.

1. Reduces depth tests.
2. Reduces HDEB subsampling loss.³³

³²In the node’s screen projection

³³If a lower-resolution depth estimation buffer is used, occluders that are rasterized into the occlusion buffer “leak” their depth values to nearby pixels. After a pixel becomes validated, it cannot be affected by such leaks.

3. Can be maintained with very little extra cost.

11.10.8 Conclusions on IOM

Advantages

1. Provides incremental occluder fusion with feedback and therefore enables sophisticated occluder selection.
2. No hardware rendering is required and therefore the need to download the frame buffer is alleviated. Performance of our software implementation is comparable to current high-end hardware rasterization.
3. Uses full output resolution and is therefore truly conservative, unlike HOM. Subsampling can be performed internally to reduce rasterization costs at very high resolutions.
4. More accurate updates of the depth estimation buffer compared to HOM.

Drawbacks

1. Cannot directly handle alpha textures as occluders.
2. A lot more complicated than HOM. Our current implementation of the IOM pipeline is about 10.000 lines of C++ code.

Additionally static coverage masks can contribute to occlusion. This is useful when the application has a static head-up display (HUD) or other kind of on-screen display that can cause occlusion.

11.11 Hierarchical Depth Estimation Buffer

Depth Estimation Buffer (DEB)[122], reviewed in section 51, is particularly suitable depth representation for software implementation. The data structure of the *hierarchical depth estimation buffer* (HDEB) is very similar to the Hierarchical Z-buffer (section 10.4.2). The lowest level of the HDEB pyramid is identical to DEB. For practical reasons we limit the horizontal and vertical resolutions of higher pyramid levels to be powers of two. The buffer does not need to be square. The highest-precision level, however, can be of arbitrary resolution. Thus memory is only wasted in the much smaller hierarchical layer structures.

Construction

Lazy evaluation is used to postpone and to combine update operations. This postponing/combining makes the HDEB updates feasible. A similar idea was proposed by Xie and Shantz [120] for a hardware implementation of HZ-buffer. They construct the HZ-buffer from the real Z-buffer several times per frame. The main difference is that we never get worse results than by using a true Z-buffer. In addition to the depth values (32-bit floats) a single-bit *modification mask* is maintained per pixel. Whenever the contents of the real Z-buffer are modified, the modification masks of the screen-space bounding rectangle of the object are set. Additionally a single-bit mask is maintained for each horizontal scan-line of each layer. This bit contains the logical OR of all modification bits in the scan-line. Thus if the *scan-line modification bit* is not set, none of the pixels in the scan-line have been modified.

Whenever a query is made, depth buffer levels are updated based on the modification masks. Levels are updated bottom $\rightarrow$ up and modification masks are propagated upwards by performing a logical OR of the 2x2 modification bits when changes happen. Lookup tables are used to perform the bit-swizzling required in this process. Only the modified pixels are updated, and only true modifications are propagated upwards. The HDEB depth values are never cleared. Instead the modification bits of the bottom level are all set at the beginning of each frame. Thus the HDEB contains a lot of the time “false depth information”.³⁴ This does not matter, since the depth values will become correct when an actual query is made. Conditional logic in updates can be removed by using the conditional move and maximum instructions available in newer processors.

Queries

A depth query resolves whether the current object is entirely behind the depth values stored into the HDEB. The queries are either point or axis-aligned rectangle queries. Point queries, that in our occlusion culling system mainly originate from Visible Point Tracking and silhouette queries (IOM), are performed directly on the lowest pyramid level. Rectangle queries are performed in a fashion similar to the HZ-buffer.³⁵ Even large objects can thus be tested extremely quickly. For each query we determine the correct *starting level* based on $\log_2$ of the smaller spatial dimension of the query rectangle. Only levels up to the query start level need to be updated. For small objects (small query rectangles) we perform the queries using the original depth buffer. This means that the HZ-buffer does not need to be updated for small objects, which is an important saving. Also, it is faster to sample the depth buffer directly than to use the more complicated traversal when there is only a few pixels to test. Currently we use an area of 32 pixels as the break-even point.

³⁴See section 14.3 for discussion about shadow acceleration. In that case the invalidations are only needed for moving visible objects.

³⁵logarithmic search.

Results

The HDEB works very well in practice. The time required to perform the updates is insignificant in all of our test applications, typically being 1-2% of dPVS 's CPU consumption³⁶. In worst-case situations, a time usage of 4% has been measured. In many test scenes the updates are performed only ~1-4 times per frame on average. In worst-case scenarios over 80 updates/frame have been measured. However such updates are very localized, and we need to update only a very small portion of the HDEB at a time. The queries are very efficient: time consumption is typically < 1% of overall dPVS CPU usage. Major reasons for efficiency are that relatively low resolution HDEBs are used and that queries are made only for objects that have passed the coverage test. Also, the depth buffer is updated only for pixels that actually contributed to the depth buffer.

³⁶CPU consumption of the visibility query, not rendering time.

11.12 Object-space Techniques

Object-space techniques can be exploited in order to better utilize temporal with occlusion buffer algorithms. In the next two sections we briefly review two optimization techniques that can be used to increase the performance of occlusion buffer algorithms.

11.13 Virtual Occluders

The research on this subject continues and dPVS does not currently use virtual occluders (VO). Virtual occluders are introduced and their short history is outlined in section 29.

Our virtual occluders will be the *nodes of the spatial database* as in [11]. Thus we can recursively combine virtual occluders into bigger entities. Since we track how long objects and nodes have been hidden, we have a rather good initial guess for virtual occluder candidates. We also know if an object is *currently* static and consequently accept only static occluders to hide virtual occluders.

Whenever a virtual occluder is created, we store the set of occluders used to hide it. If any of the objects used to hide a virtual object becomes dynamic, this *agreement* expires and the VO is destructed. Our agreements are similar to those in [26]³⁷ except that we create and delete the agreements dynamically.

When the database traversal encounters a virtual occluder, that VO is rasterized into the occlusion buffer. VOs that cannot be merged in object-space are thus merged using image-space processing. The occluder selection algorithm used in dPVS can automatically adjust its selection process when VOs are introduced. The main performance improvement that can be achieved from using VOs is the diminished occluder rasterization cost, as the real geometry is not necessarily needed.

VOs fit extremely well into the framework, and we are currently trying to find a generic algorithm for proving that a virtual occluder is hidden from a region. Our goal is to be able to prove the existence of one new virtual occluder per frame.

³⁷Used to assign personal occluders for detail objects.

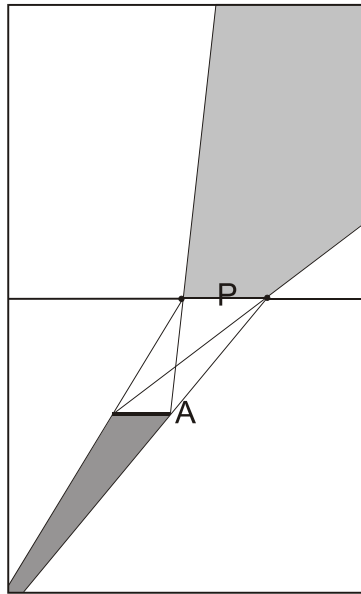


Figure 11.19 The Potentially Visible Set marked in light gray is conservatively valid for all viewpoints in the dark gray area. The portal is marked with the letter *P* and the area light source polygon with the letter *A*.

11.14 Portal PVS

All view frustum culling algorithms can exploit temporal coherence by constructing and caching Potentially Visible Sets of objects using temporal bounding volumes (TBVs) of view frusta over a period of time. Such an algorithm periodically constructs new PVSs and performs exact view frustum culling only for objects in the PVS. The PVS can be constructed and stored hierarchically, if the scene is organized in a spatial database.

Because of the similarity between portal culling and view frustum culling, the same method can be used for constructing *portal PVSs*. The Potentially Visible Set is created by finding all objects that intersect the penumbra formed by an area light source and a convex portal. See figure 11.19 for an example. The light gray area is the PVS penumbra formed by the portal *P* and the area light source *A*. The dark gray area is the corresponding *view cell* of the PVS.

The area light source is a convex polygon, having the same shape as the portal and located perpendicular to the portal. The size and exact position of the polygon may vary.³⁸ The *view cell* for the PVS is the open-ended frustum formed by connecting each edge of the area light source with the corresponding portal edge, and then *capping* the other end by the area light source polygon.

The penumbra frustum is created by constructing planes between each edge in the area light source and the furthest vertex³⁹ in the portal, and then capping by the portal polygon.⁴⁰ Finding the objects intersecting the penumbra is trivial, as the intersection test is an ordinary view frustum culling test.

A new PVS is constructed whenever the old one becomes invalid, i.e. when the camera moves

³⁸This choice of shape and orientation avoids generating quadratic surfaces for the view cell boundaries.

³⁹in the negative half-space of the plane

⁴⁰This overestimates the size of the PVS frustum, but avoids constructing quadratic surfaces and keeps the frustum convex.

11.14 Portal PVS

outside the associated view cell. The size and actual positioning of the area light source can be selected based on the velocity of the camera and by deciding an *expected validity period*. The algorithm can utilize additional coherence by caching several PVSs with view cells of different sizes. A larger PVS can then be used as a starting point for constructing a tighter one.

The portal PVS algorithm can be used to store Potentially Visible Sets of arbitrarily long portal sequences. Because the view cell is represented in the portal's space, the portal PVS can be also used for storing PVSs of virtual portals, such as mirrors.

Although dynamic objects cannot be stored into the portal PVS, they can be easily used by storing visible voxels of the spatial database into the PVS - the dynamic objects inside these voxels must then be tested each frame. By storing volumetric information rather than objects in the PVS, the PVS stays valid regardless of object motion.

The portal PVS is not particularly effective when the camera comes very close to the portal, as the view frustum becomes then extremely wide, and the PVS converges into the full set of objects in the target cell. When such a case is detected, the portal PVS should be temporarily disabled.⁴¹

Note that back-face culling can be used when creating the PVS. If a polygon is back-facing when viewed from all of the vertices of the area light source, it can be removed from the PVS.

⁴¹ Alternatively, if volumetric information is stored rather than object lists, a pointer to the root node can be stored. This way the algorithm requires no special treatment of the bad cases.

Feature	Virtual Occluder	OcclusionBuffer	Portal
Resolution-Independent	+	—	+
Can handle transparencies	—	some can	—
Can use dynamic occluders	+	+	—
Static occluder fusion	+	+	+
Dynamic occluder fusion	—	+	—
Temporally coherent	+	—	section 11.14
Source area culling	+	—	—
Destination area culling	subdivision	subdivision	cell
Needs pre-processing	—	—	+
Contribution culling	—	+	—
Depth estimation loss	small (view indep)	high (view dep)	small
Memory consumption	agreements	view size	small
Occluder selection	complex dynamic	dynamic	static
Object processing order	arbitrary	front-to-back	portal order
Occludees processed	once per frame	once per frame	once per portal

Figure 11.20 Comparison of algorithmic properties of Virtual Occluders, Portals and Occlusion Buffer.

11.15 Conclusion

No single algorithm provides all desirable features, and a combination of some sort seems appealing. Combination of the occlusion buffer and virtual occluders forms a nice algorithm handling everything but transparencies correctly (Fig. 11.20). Neither of these algorithms require pre-processing, although both can benefit from it. Portals require pre-processing, but are extremely powerful in architectural scenes, and can additionally be used to create a wide variety of special effects. Portals are essentially a view frustum culling algorithm, and can thus be easily incorporated into a visibility determination framework.

Portal culling and occlusion culling are complementing algorithms, since portal culling can be used to reduce occlusion culling work, and occlusion culling can be used to cull portals.

Chapter 12

dPVS Architecture

In this chapter we list the algorithms used inside the library, outline the dPVS architecture and introduce the dPVS Visualizer run-time debugging tool. Accuracy and robustness issues are considered in section 12.4.

The information presented in this chapter is intended to help the reader to understand the library architecture - not how to *use* the library. How the library can be integrated into applications is discussed in the “Programmer’s Guide”.

12.1 Algorithms Used in dPVS

Our implementation is approximately 70.000 lines of C++ code. The primary reason for the amount of code is the number of algorithms used. We have also tried to make the framework as extendable as possible in order to integrate impostors and other future extensions (see chapter 6) into the architecture with modest effort. The following algorithms are currently used inside the library:

- Hierarchical View Frustum Culling (section 10.4.9)
- Visible Point Tracking (section 11.4)
- Incremental Occlusion Maps (section 11.9)
- Hierarchical Depth Estimation Buffer (section 11.11)
- Occlusion Write Postpone Queue (section 11.7)
- Silhouette Cache (section 11.8.1)
- Incremental Occluder Selection with Feedback (section 11.3.2)
- Portals (section 10.4.8)
- Dynamic Spatial Subdivision and Traversal (section 12.6)
- Temporal Bounding Volumes (section 34)
- Validation Buffer (section 11.10.7)
- Construction-Time Contribution Culling (section 11.10.5)
- Dynamic Virtual Occluders (section 11.13)

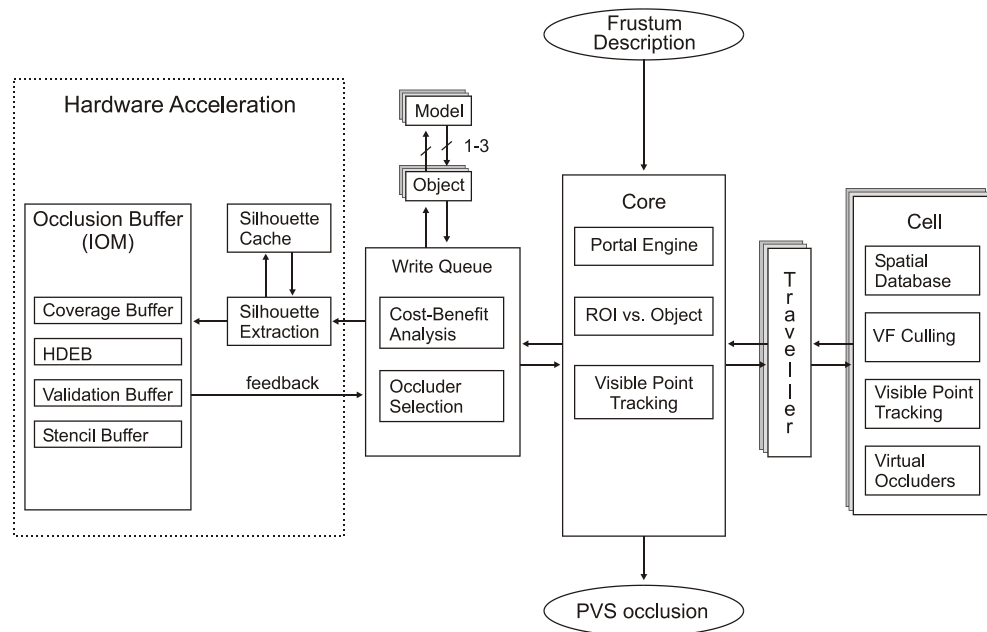


Figure 12.1 Block diagram of dPVS

12.2 A Block Diagram of dPVS

Figure 12.1 shows the block diagram of dPVS . There are a few issues in the figure that need further explanation.

12.2.1 Concepts

Visible Point Tracking

Visible Point Tracking (VPT), see section 11.4, is performed in two different places (core and cell). This is because VPT is used for both view frustum culling in the spatial database and for occlusion culling in the core.

Traveller

Traveller is an interface class between the core and a cell. The current view frustum description is set to the traveller before the traversal of the cell begins. All information exchange between the core and a cell is handled through a traveller object.

ROI vs. Object

Region of influence (ROI) is a term used for a “thing” that affects other objects somehow. The most obvious usage are light sources. Thus “ROI vs. Object” stands for resolving which lights affect which objects. dPVS defines ROIs as special objects that are handled as normal objects all the way through the visibility pipeline. Before informing the user about visible objects, the intersections between ROIs and other objects are resolved. Since the visibility pipeline of normal objects is used, ROIs can be eliminated by VF and occlusion culling. As a result, dPVS only reports ROIs that are visible or almost visible. dPVS does not concern different light types or lighting models in any way. Since the ROIs are objects, the test model can be anything from a sphere to a mesh. For example, point light sources can be modeled by setting a sphere as the test model and spot lights by submitting a cone-shaped mesh.

The basic algorithm used for object versus ROI intersection testing is sweep and prune. The optimal sorting axis is found by incrementally calculating the standard deviation of ROI center-point locations in three dimensions and normalizing the result to get the direction vector. The choice of sorting axis has been found very important, and the maximum variance algorithm being used is extremely good in practice. To further enhance the performance of our sweep and prune implementation, bucketing is performed in the remaining two dimensions to eliminate redundant overlap tests.¹ The number of buckets is chosen based on the number of objects/ROIs.² The total number of object/ROI overlap tests converges to a linear function when objects become “well-separated”.

The communication to the user is handled through the *commander interface*³. A “ROI active” command is executed whenever a ROI becomes active. The ROI reported stays active until a “ROI inactive” command is executed. OpenGL has a similar light manipulation mechanism.

Theoretically the region of influence can be, for example, a sound emitter. The implementation is generic, and we are expecting to encounter surprising uses for regions of influence in the near future.

Hardware Acceleration

Should occlusion culling become available in consumer-level hardware, the part marked with a dashed line could be accelerated. As a hardware implementation will almost certainly not use silhouettes but triangle meshes as rendering primitives, the silhouette extraction would not be needed.

An important question is the feedback information from the image-space occlusion system⁴. Our occluder selection algorithm relies solely on this feedback, and if not available, some other occluder selection algorithm should be used. As the required feedback information is only a single bit per screen-space tile, and a boolean value whether the object is visible, it should be possible to implement the hardware so that the required information could be obtained. From our point of view, this is very important, since other occluder selection schemes have shown drastically worse performance figures in our experiments.

Another important consideration is the *latency*. If it takes a significant amount of processor cycles to get the feedback information from the hardware, we would need to adjust the granularity of the visibility queries to be somewhat higher. That is an awkward thing to do and basically the time consumption of the query itself and the feedback latency has to be less than when implemented with

¹Our implementation can also be seen as performing the sweep and prune simultaneously in all three dimension by using bucket sorting.

²We use $N^{2/3}$ buckets, where N is the number of visible objects and ROIs.

³see “Programmer’s Guide” for explanation of the Commander class

⁴See 11.3.2 for details of our occluder selection algorithm and the feedback information required.

12.2 A Block Diagram of dPVS

software. HP Visualize-fx workstations have occlusion culling in the hardware, but they can only issue 1000-6000 queries per second, which is considerably less than what we are able to handle with our software. As a conclusion we must say that the latency of a hardware implementation must be low.

12.3 Visibility Pipeline

The visibility determination pipeline is divided roughly into two categories: view frustum culling and occlusion culling. View frustum culling is managed by the spatial database and a new view frustum is created every time we pass through a portal. The resulting view frustum is the intersection of the portal chain. Basically the same pipeline is executed for objects and for the nodes of the spatial database.

For each object and each node of the spatial database:

1. If VPT is inside the view frustum → test occlusion.
2. If AABB intersects the view frustum → test occlusion.
3. If convex hull intersects the view frustum → test occlusion.

For each object and each node inside the view frustum:

1. If VPT is visible → visible.
2. If axis-aligned rectangle is hidden → hidden.
3. If silhouette of OBB is hidden → hidden.
4. Flush the write queue, if expected to be useful. Re-execute tests 2 & 3.

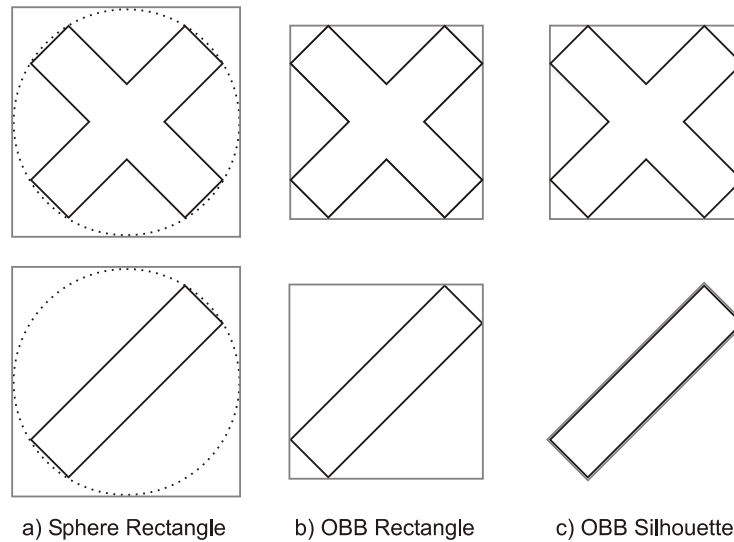


Figure 12.2 Different coverage estimation schemes. An axis-aligned rectangle constructed from a bounding sphere approximates most objects rather poorly. An axis-aligned rectangle constructed from an OBB is a suitable presentation for the upper object but very poor for the lower one. A silhouette constructed from OBB is a more expensive test primitive, but provides good coverage estimations for a wide variety of objects.

12.4 Accuracy and Robustness Considerations

In this section we first explain several accuracy issues of dPVS. After that we list known robustness issues and problem scenarios.

12.4.1 Accuracy

Granularity

dPVS performs culling at per-object precision. Polygon-precision rejection is clearly out of the question, primarily because it would be too slow and altering the model description causes bandwidth problems.⁵ Consequently dPVS cannot occlude parts of a model and therefore spatially large models should be splitted into several sub-models.

Coverage

Stored coverage information is pixel-exact by default. Optionally sampling density can be increased or decreased by setting the image-space scaling to something else than 1.0. An object's screen coverage is first estimated with an axis-aligned rectangle and then with a silhouette of the OBB. Figure 12.2 illustrates occlusion test accuracies obtained by using a) rectangle of a bounding sphere, b) rectangle of an OBB and c) silhouette of an OBB. From the figure it becomes prominent that

⁵If even a single triangle is removed, the model has to be re-uploaded to the graphics hardware.

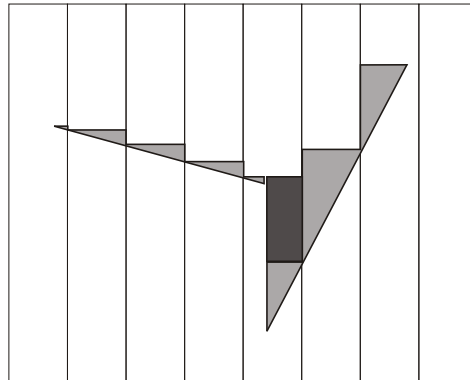


Figure 12.3 Effects caused by a low depth buffer resolution. Occlusion volume losses are marked with light gray and the area marked with dark gray is caused by several primitives getting mapped into the same 8x8 block and the furthest value getting selected.

the upper shape is relatively well presented with all three approximations, but the lower shape is captured well only by the OBB silhouette. As shapes such as the lower one are not rare, we are first using a rectangle, and only if the test fails, do we use the OBB silhouette.

Depth

dPVS stores a single depth value for each 8x8 block of the image-space resolution. Such low resolution depth values are useful only if the *greatest* depth of the corresponding 64 pixels is stored. This causes depth representation occlusion volume loss. Figure 12.3 illustrates occlusion volume losses (light grey) caused by modest and huge depth slopes. The area marked with darker gray is caused by several values mapping into the same 8x8 block and the greatest value getting selected. These effect have been found rather insignificant in actual scenes both by Zhang [122] and by us in our empirical experiments. The primary reason for this is the accurate coverage information.

Primarily due to performance reasons, the depth values used both in occlusion tests and occlusion writes are usually conservative. Figure 12.4 illustrates depth values obtained from a) the nearest and the farthest points of an OBB and b) interpolated depth values of an OBB. As can be seen from the figure, in a) the depth values fail to separate objects A and B, and therefore the object B is determined visible. On the contrary, in b) the more accurate depth values succeed in separating the objects and therefore B is determined hidden. The current version of dPVS uses approach b) for objects and for spatial database nodes.

12.4.2 Robustness

All image-space algorithms are numerically stable as they use integer math. Silhouette extraction is considered stable in the sense that degeneracies may produce non-optimal, but still closed silhouettes. We use OBBs internally and the OBB construction code itself is prone to certain degeneracies and cannot therefore be considered robust. If the OBB creation fails, we use AABB instead. Portal traversal is prone to mathematical inaccuracies when the portal chain is very long. This effect cannot fully be eliminated, but we expect no “feasible” cases to show visible artifacts. Should the problem become visible, double precision matrices will be used internally. Infinitesimal numerical values are

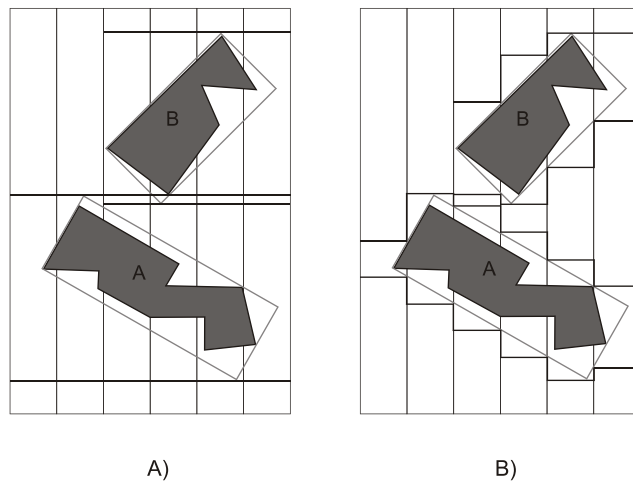


Figure 12.4 Different depth estimation schemes. If the depth values are obtained from the nearest and farthest points of the OBBs (*a*), object A fails to hide object B. If the depth values are obtained from interpolated depth values of the OBBs (*b*), object B is hidden.

explicitly flushed to zero in order to avoid underflows and inaccurate results.

12.5 Data Organization and Traversal

In this section we briefly list the primary concepts that are related to data organization and traversal.

Traversal

Objects and *cameras* are assigned into *cells*. The traversal starts from the cell into which the camera is assigned. The database is traversed hierarchically, and whenever a portal is found visible, a new view frustum is created and the target cell is processed. The traversal proceeds in depth-first order so that the current cell is always processed fully before continuing to the next cell.

Cells

Cells are object containers. A typical application only defines a single cell that contains all objects. Each cell has its own local coordinate system and a spatial database. Certain types of applications⁶ divide the scene into several cells that are connected through *portals*.

Portals

We perform portal traversal computations in 3D⁷, in the coordinate system of the cell. Image-space tests are supported in the form of scissoring [84]. We support both physical⁸ and virtual⁹ portals. Portals are treated the same way as other objects and therefore our occluder selection algorithm automatically takes portals into account.

Objects

Objects are nodes of the scene graph that are organized into the spatial databases.

Models

An object can have separate occlusion test and an occlusion write *models*. The test model is used when the object is tested for occlusion. In many cases a simple bounding volume is a sufficient representation. The write model is used when the object is being used as an occluder. Typically the write model is either the original model or simplified¹⁰ geometry. The same model can be assigned to any number of objects.

⁶Mostly architectural scenes

⁷Luebke and Georges [74] performed portal intersection calculations in screen-space.

⁸Continuous in space.

⁹Free transformation between two portals.

¹⁰Simplified with Occlusion-Preserving Simplification.

Visibility Parent

An object can have a *visibility parent*. If the visibility parent is determined hidden, dPVS can cull the child as well without performing more expensive occlusion tests. *On-demand loading* can be implemented using unnecessarily large *bounding volumes* that are visibility-parented to actual objects. Visibility parents can be seen as a generalized method for expressing bounding volume hierarchies.

Bounding Volumes

Whenever dPVS needs a bounding volume, it creates it internally. The interface does not define the concept of a bounding volume, but the supported model types include, for example, a sphere. Certain complex entities such as particle systems should be inserted to dPVS as bounding volumes.

12.6 Spatial Database

12.6.1 Construction

A separate spatial database is created for each cell. The database is an axis-aligned BSP tree. Thus each non-leaf cell has either 1 or 2 children¹¹. Objects are originally inserted into the database based on their AABBs¹². All database updates are performed during visibility query traversals. This means that hidden portions of the database are not updated. Objects are always inserted into the database root node and “pushed down” during the traversal. If an object is pushed down into a node that is found hidden, it will not be pushed further down until the node becomes visible. Nodes containing more than N objects (N = 12 currently) are split into two child nodes. The splitting plane is always axis-aligned, but can be in any position inside the node. The algorithm used for finding the splitting plane location is presented in[79]. The algorithm attempts to minimize the cost function

$$L.surfaceArea * L.numObjects + R.surfaceArea * R.numObjects \quad (12.1)$$

Where *L* and *R* stand for the left and right child respectively. This algorithm works very well in practice and also attempts to minimize object splits. If more than 30 of the optimal subdivision, the voxel is not split. Objects are never split, but can belong to several nodes instead. A mailbox mechanism is used during the traversal to ensure that objects are accessed only once per traversal.

When an object moves, it is not deleted and re-inserted. Instead, the update is performed locally by moving the object upwards in the tree until it fits completely inside a node. The object is then potentially later pushed down by a traversal. In many cases, an object does not exit the node where it is, and thus no updates need to be performed. Objects are pushed downwards until either a leaf node is reached or the volume of the node is smaller than the volume of the object. Leaf nodes containing no objects are destroyed.

A *tabu counter* is used for node updates. This distributes the database construction work over multiple traversals and avoids sudden hits. For example, the time to create a 10,000 object database was reduced from a 3 second start-up time to 0.3 secs for each of the first ten frames.

12.6.2 Visibility Query

The query traverses the database in an approximate front-to-back order. View frustum tests are applied to all nodes. A hierarchical frustum mask (see [10]) is propagated downwards to limit the VF tests only to the planes that clip the parent node. The AABB test used is an optimized version of Greene’s / Hoff’s AABB[60] test using N and P points¹³. Non-leaf nodes use their full AABB, leaf nodes use *tight bounds*, i.e. the total AABB of the objects inside it. This can be a considerably smaller volume than the node itself.

An occlusion test is performed for all potentially hidden nodes. All nodes containing objects are potentially hidden. Also nodes that had only hidden children during the previous query are potentially hidden. This is the same as Bittner’s “node skipping” [12] algorithm and reduces tests by approximately ~50%. Bittner’s probabilistic node skipping [12] was tried, but not found useful. Occlusion

¹¹left/right children.

¹²That can be temporal bounding volumes for moving objects. Later, if an object is determined to be static, a more accurate triangle-level intersection test is made to place the object into the correct voxels.

¹³ $d.x * ||p.x|| + d.y * ||p.y|| + d.z * ||p.z|| + m.x * p.x + m.y * p.y + m.z * p.z + p.w \leq 0$; where p = plane equation of the clip plane, m = center of the bounding box and d = half-diagonal of the bounding-box (all components positive)

12.6 Spatial Database

tests are performed using silhouettes of the visible faces of the node. A single face is tested at a time. If a node is found visible, all objects in it are VF tested and occlusion tested. The VF test uses the frustum mask of the node. Objects inside a node are locally sorted (quicksort) to front → back order to improve occlusion tests. Visible point tracking (page 296) is used in node occlusion tests, object VF tests and object occlusion tests. This reduces the amount of work considerably and is the most important optimization of the database. Object VF testing uses initially bounding spheres, then visible point tracking, then a more exact convex hull vs. view frustum test¹⁴. Spending additional time in VF testing to get rid of objects that are “just outside the view frustum” is extremely important. In our implementation VF testing takes only ~10% of the time used by the occlusion culling, so reducing the number of objects to be tested and subsequently applied as occluders is crucial.

12.6.3 Optimizations

Custom pool allocators are used throughout the database system to ensure data coherence and fast memory allocations/frees. Most of the optimization work is memory-related: packing data, accessing cache lines coherently, avoiding unnecessary writes into memory. The actual traversal and testing takes very little time and the system is clearly bounded by the memory bandwidth. Originally other database formats were tried, but the axis-aligned BSP trees were found to be by far the most flexible mechanism for dynamic worlds. The lazy evaluation and postponing strategies used ensure that database updates are output-sensitive.

¹⁴Vertices of the convex hull are stored in a random order to provide early exits.

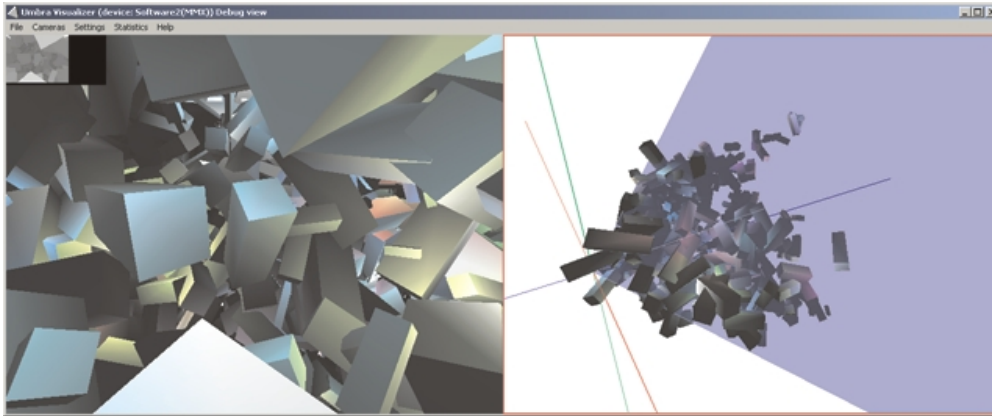


Figure 12.5 A scene with 32000 random boxes and 1000 point light sources rendered with occlusion culling.

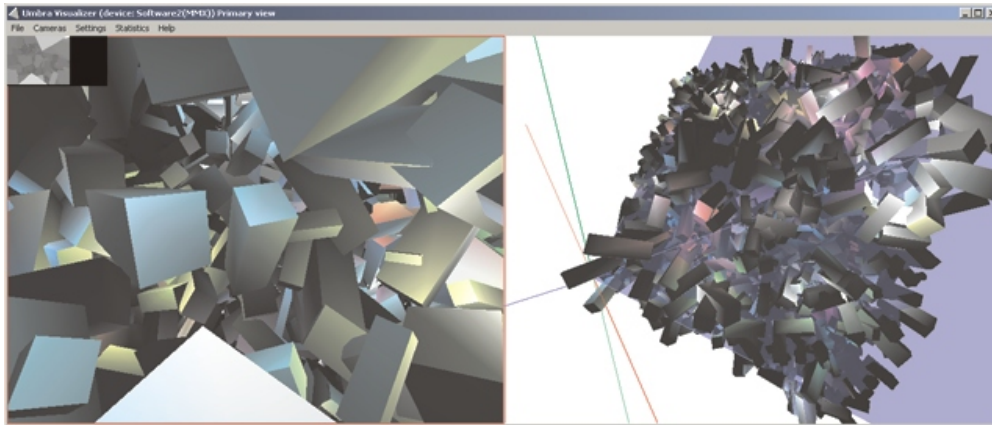


Figure 12.6 A scene with 32000 random boxes and 1000 point light sources rendered without occlusion culling (view frustum culling is enabled).

12.7 dPVS Visualizer

dPVS Visualizer is a run-time debugging and performance analysis tool. Debugging features include a wide variety of statistics (12.7.2) and visual debugging information (12.7.1). Figures 12.5, 12.6, 12.7 and 12.8 are screenshots taken using the dPVS Visualizer.

12.7.1 Visual Debugging Information

The visual debugging information available includes the following:

Secondary View (figure 12.5 can be used to observe the culling process from any location in the scene. This is highly useful for observing what gets processed and drawn.

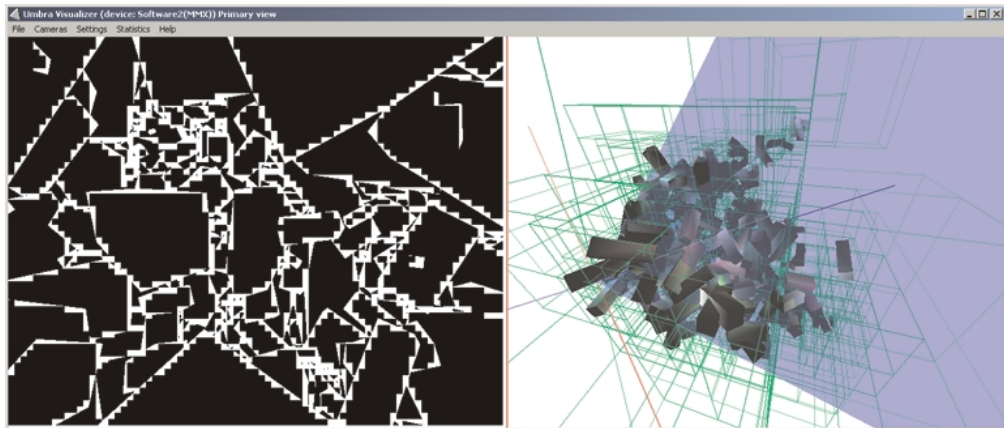


Figure 12.7 Occlusion buffer and voxels for the scene in figure 12.5

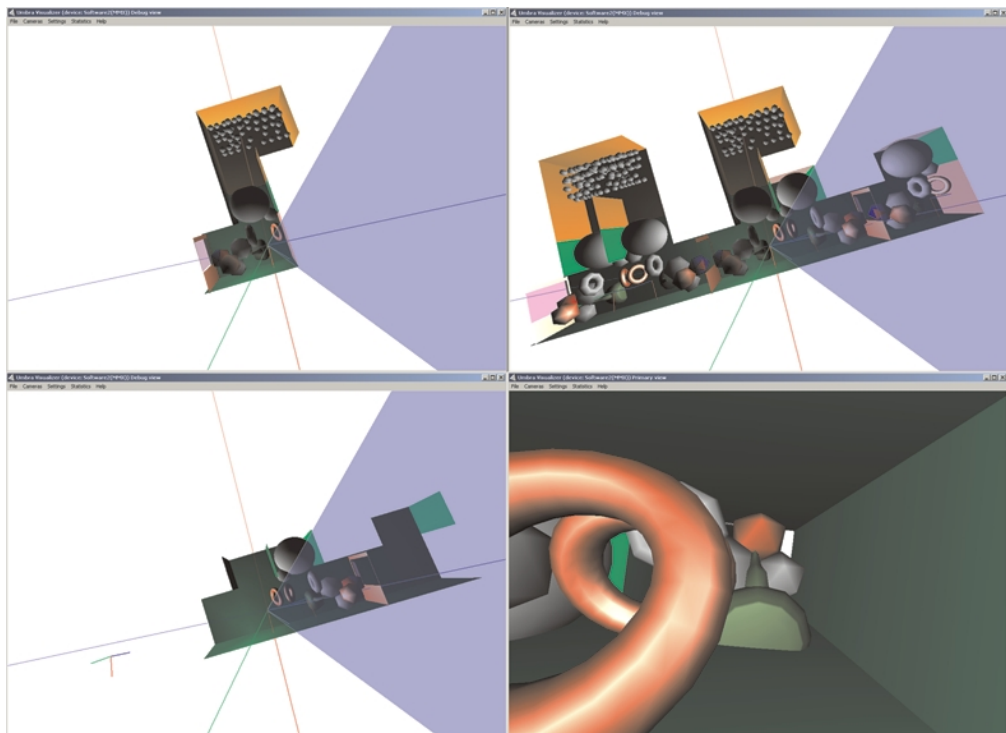


Figure 12.8 A scene with circular mirrors. The upper left figure is the scene without active mirrors. The upper right figure shows the result of enabling mirrors both on the right and on the left (the recursion depth is limited). When both view frustum culling and occlusion culling are enabled, most of the objects get culled away and the result is shown in the lower left figure. The lower right figure shows the first-person view of the final scene with two circular mirrors. Notice that only the first torus is a real object, all others are reflections.

Camera Slots: Up to 10 configurations can be stored/loaded in order to ease run-time debugging.

Crash Recovery: dPVS Visualizer can store all parameters to a disk every frame in order to trap severe bugs that cause a crash. The crash situation can thus be quickly located and consequently debugged.

Hierarchical View Frustum Culling can be disabled to simulate a basic rendering engine.

Occlusion Frustum Culling can be disabled to test the performance of the scene with and without occlusion culling.

Traversed Database Voxels can be visualized (figure 12.7) to get precise information about the traversal depth. If it seems that a traversed voxel should be occluded, the reason is an unacceptable amount of occlusion volume loss. Using this debug information we observed that depth testing the voxels with a single depth value is not accurate enough, and reports a considerable amount of voxels as visible, even though a more accurate depth test indicates that they are occluded.

Occluder Silhouettes can be visualized. A different color is used for the contributing and non-contributing silhouettes. A large amount of non-contributing silhouettes indicates that the coverage tests should have been made using a higher precision. If the same non-contributing objects are continuously being selected as occluders, there might be a bug in the occluder selection algorithm, as such objects should receive no benefits.

Occlusion Test Rectangles can be visualized to observe how conservative the axis-aligned rectangles actually are.

Occlusion Buffer can be shown. The options available are the full-resolution Coverage Buffer (figure 12.7), the Full Blocks Buffer and the Depth Estimation Buffer (figure 12.5).

Virtual Portals can be disabled in order to disable all mirrors and other special effects.

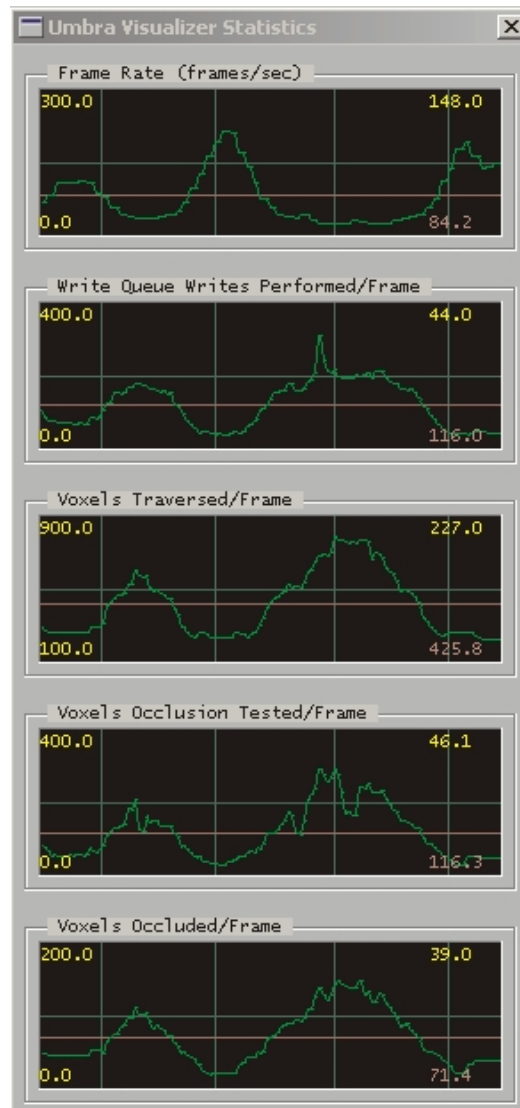


Figure 12.9 Example of the statistics output from dPVS Visualizer

12.7.2 Statistics

dPVS maintains approximately 100 different run-time statistics. dPVS Visualizer can show any combination of the statistics available in the fashion presented in figure 12.9. Additionally derivatives such as ratios between two statistics are available. More expensive statistics such as conservativity (section 10.4) require additional processing, and are optionally calculated by the dPVS Visualizer, not inside the library.

Consult the API Reference for a complete list of the currently available statistics.

12.8 Portability

dPVS does not use any rendering hardware and is not dependent on a specific platform or an operating system. Consult the “Programmer’s Guide”-part of this book for a detailed description of portability issues.

Our implementation is written in C++, but the interface can be easily wrapped to support a number of languages, including C, Java and Python.

Occlusion Buffer and Hierarchical Depth Estimation Buffer are the only algorithms prone to little/big endian issues. The issue is solved with a `SELECT_ENDIAN( little_expr , big_expr )` macro that executes the correct expression based on a global setting.

Some of the data structures use 64-bit elements. We have implemented a `QWORD` class that emulates a subset of 64-bit arithmetics on 32-bit processors. The only operations we need are logical operations and the pre-increment (`++`) operator. These operations can be emulated very efficiently without conditional code.

Chapter 13

Results and Conclusions

13.1 Test Results

The results presented in most of the occlusion culling literature have very little practical value. As there are *no* standard test scenes, “Authors report over 90% culling ratios” - style results are not very useful.

Conservativity (see section 10.4) is a useful measurement for estimating how conservative the algorithm is. Additionally some kind of measurement is needed for estimating how “difficult” the scene is.

Due to the above reasons we have no intention to release specific numbers. Additionally the end result, dPVS, is a commercial library, and stating anything like “culls 75% of the hidden geometry in typical scenes” would not be very wise. Every scene is different and there is no such thing as a typical scene. We proceed by examining a few scenarios and analyzing their results.

A Large Scene with Simple Models

In order to simulate an urban scene, we have run tests with a scene that consists of 30.000 - 100.000 randomly oriented and scaled boxes and 0 - 1000 local light sources (see figures 12.5 and 12.6). The optimal occluder count varies from zero to hundreds of objects depending on the camera transformation.

When starting from the center of the scene with 30.000 objects and 1000 lights (figure 12.6), we had to wait approximately 5 minutes to see the first frame, if occlusion culling was turned off. The first frame took approximately 0.2 seconds to render when occlusion culling was enabled. No pre-processing stage was involved. Figure 12.9 shows the frame rate when occlusion culling was enabled and actual rendering was disabled. Enabling the rendering slows down the frame rate approximately from 30 → 20 fps. Increasing the object count to 100.000 while keeping the same object density has no effect on the frame rate, if occlusion culling is used. If the density of the scene is decreased, and consequently we see much further, the frame rate decreases expectedly. According to our tests, the frame rate follows the size of the output set, not the size of the input. Average conservativity value is slightly below 2.0.

A Large Scene with Complex Models

Replacing the boxes of the previous scene with teapots (a few thousand triangles each) and other equally complex objects pushes the total polygon count to over 100 million triangles. Without stating exact numbers, we can say that we have observed dPVS's CPU time consumption growing slower than rendering engine's¹. When rendering is disabled, dPVS is able to solve the visibility several frames per second while retaining the average conservativity value of 2.0. Rendering slows the frame rate down considerably, but that is pretty much out of our hands, since the conservativity value is rather good. There is simply too many polygons visible on the screen, and simplification or impostors would be needed for interactive navigation. We have not tried to turn off occlusion culling, as the result would not be seen anytime soon.

A Large Scene with Very Complex Models

We are currently using temporally coherent silhouettes, but have not implemented an output-sensitive extraction algorithm. Very complex models would suffer from substantial extraction penalty, and the guideline is to split complex models into sub-models and to use *visibility parents*² in this sub-model hierarchy. We have not performed tests with very complex models.

A Simple Scene with Nothing to Occlude

dPVS is constantly adapting to its input data, but does occasionally try to use objects as occluders even when it knows there is hardly a chance to succeed. The performance penalty in simple scenes with nothing to occlude has been measured to be a few percents of the CPU time.

Portal Sequences

We have experimented with physical and virtual portals. Figure 12.8 shows our test scene that contains two cells connected by a physical portal, and two mirrors on opposite walls³. If virtual portals are disabled in the Visualizer, no reflections are generated. As the mirrors see each other in a circular fashion, infinite recursion follows, unless addressed properly. dPVS uses both maximum recursion depth and importance decay terms to terminate recursions. Objects can cull the reflections in the mirrors. Portals are also occlusion culled. The light sources are correctly reflected by the mirrors and properly culled. Stencil models were assigned to some or all of the portals to test functionality of the stencil buffer. The rendering speed was increased by occlusion culling whenever there was something to occlude, otherwise the frame rate was practically intact.

¹OpenGL and NVIDIA GeForce256

²If an object's visibility parent is hidden, dPVS can cull the child without additional occlusion tests.

³Application code does not need to consider mirrors, it is purely an internal issue.

13.2 Conclusions

We have presented a framework for conservative visibility determination in dynamic scenes. Our framework is able to find visible geometry by logarithmic search and is therefore output-sensitive. The framework was designed so that no platform- or hardware-specific features were used, and therefore we are able to use the library on basically any platform. We have been able to show significant acceleration of rendering of both relatively simple and very complex scenes.

To alleviate the need for pre-processing, we developed several new algorithms. Our occluder selection algorithm is adaptive, and estimates the expected occlusion powers of objects at run-time. The spatial subdivision scheme used adapts dynamically to changes in the scene structure. It internally classifies objects to be either static or dynamic and selects optimal execution paths accordingly.

Our variation of Hierarchical Occlusion Maps is incremental and does not require hardware acceleration. The framework has feedback paths between every algorithm, and most of the algorithms used perform dynamic adaptation to the input data. Even though the adaptation processes are generally very simple, the framework “learns” to process each scene in a slightly different way.

Future extensions, such as image-based rendering, have been taken into account when developing the framework.

Arbitrarily complex scenes pose difficult challenges to real-time visualization. In order to render such scenes at interactive frame rates, both conservative visibility determination algorithms and non-conservative methods, such as impostors and model simplification, are required. Although output-sensitivity can be reached in the visualization process, the challenging problems of avoiding physics, collision detection, AI calculations etc. in the hidden regions of the scene remain.

Chapter 14

Future Work

14.1 Algorithms

dPVS has a very solid image-space solver, but does not utilize temporal coherence maximally. Every known object-space algorithm is too limited for general purpose use. The concept of virtual occluders is very promising, and should an efficient algorithm to construct them be published, it would certainly be added to dPVS. The current implementation is ready to use virtual occluders as soon as an algorithm for proving region-to-virtual occluder visibility is found (see section [11.13](#)).

14.2 Pre-Processing Tools

Occlusion-Preserving Simplification

Not a single general purpose algorithm has been proposed for Occlusion-Preserving Simplification (OPS). Should such an algorithm be published, the occlusion write models used in dPVS could be replaced with simplified versions.

Automatic Portal Generation

Being able to automatically create cell-portal subdivision of scenes would extend the usability of portals. Currently only custom modeling software have cells and portals as built-in primitives. Being able to automatically create such structures from arbitrary models would result in substantial savings of time and money in the games industry. A special-case solution for axis-aligned scenes has been proposed by Teller [[108](#)].

Alpha-Meshing

dPVS cannot handle alpha maps and automatic alpha-meshing code would simplify their use.

14.3 Accelerating Shadow Generation

Shadow generation is a problem tightly connected to visibility determination. dPVS can be used to accelerate the shadow generation by tracking the geometry seen by the light sources. Since most light sources are static, unlike cameras, a specialized version that only updates the changed portions of the coverage buffer could be used. Furthermore, stencil shadows work with object silhouettes. Since we have already optimized the silhouette extraction, the same algorithms could be utilized to accelerate the generation of shadow volumes.

14.4 On-Demand Loading

Modern consoles have a very high polygon drawing capability compared to the amount of memory available¹. This imbalance forces the scenes to be unreasonably small, unless the data can be loaded from the DVD drive when needed. dPVS tracks the potentially visible set of objects at any given time, so it could be used to predict which objects are likely to be needed in the near future. Similarly, it would be able to predict which objects are unlikely to be needed for a while, and to give hints to swap these out in order to free resources.

In a similar fashion, the clients of massive online games could only request the data actually needed from the server. Such an optimization would result in remarkable savings in the bandwidth required [38].

14.5 Image-Based Rendering

dPVS could be extended to create and handle bitmapped impostors. A large percentage of the code and data structures needed has already been implemented. The current interface was designed with impostors in mind, so they will fit into the framework very well. The first step is most likely generic reflections and refractions that require rendering into an off-screen surface. The actual rendering will be done outside the library, just as the rendering of the primary view. dPVS would be used to handle distortion calculations, redraw commands and texture management commands. Distortion calculations are in fact very similar to the current silhouette cache distortion calculations. dPVS could then transparently replace distant scene geometry with bitmapped impostors for increased rendering speed. Our impostors will be mapped to the voxel faces of the spatial database, thus combining spatially close objects and avoiding depth overlap problems. Should an impostor be completely opaque, the voxel face can be automatically determined to be a virtual occluder.

14.6 Interaction Pre-Processor

dPVS already solves ROI vs. object interactions and the logic could be easily extended to handle object-object interactions. Internally, we know whether an object is *currently static* and can thus eliminate the interaction tests between static objects. To maintain output-sensitivity, interactions must only be processed for the visible portion of the scene. It is a challenging task to design physics and collision systems so that physics in the hidden parts of the scene can be postponed until needed.

¹Especially Sony PlayStation2®

14.6 Interaction Pre-Processor

On the other hand, if a character becomes visible after a long while, the exact animation frame and position may not be important.

Part IV

Appendices

Appendix A

Checklists

This chapter contains several checklists the reader may want to go through. They also serve as a useful reminder during troubleshooting.

Quick Start

Start-up

1. optionally create a `Library::Services` object and override its member functions
2. initialize the library by calling `Library::init()`. At this point you will also decide which matrix format you want to use.
3. create test and write models either procedurally or by importing them from a scene description or a model database
4. create cells and portals connecting the cells
5. create objects, place them into the cells and assign models to them. Split large objects/models into smaller ones, especially if the whole scene is just one big object in the input format.
6. create at least one camera and configure it properly
7. create a derived class from `Commander` and override its `Commander::command()` function

Each Frame

1. release objects that need to be destructed
2. if new objects need to be created, create and configure them. This can happen due to on-demand loading.
3. update the transformations of all moving objects, ROIs and cameras
4. perform a visibility query by using `Camera::resolveVisibility()`
5. optionally make immediate mode visibility queries

Occasionally

1. optionally remove hidden objects from the database by using `Library::suggestGarbageCollect()`
2. optionally claim back memory used by temporary allocations by calling `Library::minimizeMemoryUsage()`

Cleanup

1. release all dPVS -related resources. Some of this work may happen automatically if you have used the `ReferenceCount::autoRelease()` mechanism
2. call `Library::exit()`. If there are any dPVS -related memory leaks, these will be reported in the debug build.
3. destruct your error handler

Checklist for Reference Counting

1. the reference counts of all objects are initialized to 1. This means that in order for an object to be destructed, someone will have to call `ReferenceCount::release()` at some point
2. the `ReferenceCount::autoRelease()` will decrease the count by one. However, the object will never be released if no-one releases the reference. So make sure that you use auto-releasing only in conjunction with systems that take references, such as `Cells` holding references to `Objects`.
3. the debug build of the library will perform assertions on unreleased objects when `Library::exit()` is called

Checklist for Cameras

Creating the Camera

1. construct a camera by calling `Camera::create()`
2. assign the camera into a cell by calling `Camera::setCell()`
3. assign a transformation matrix to the camera by `Camera::setCameraToCellMatrix()`
4. define a view frustum by calling `Camera::setFrustum()`
5. define the screen resolution and culling methods by `Camera::setParameters()`
6. optionally define a pre-computed depth buffer by `Camera::setStaticZBuffer()`

-
7. optionally define a pre-computed coverage mask by
`Camera::setStaticCoverageMask()`
 8. optionally enable screen-size culling by
`Camera::setObjectMinimumCoverage()`
 9. if you want to debug with names, assign a name for the camera by `Camera::setName()` (section 6.4.3)

Updating the Camera

1. if the size of the display has changed, call `Camera::setParameters()`. Note that if static coverage masks or depth buffers are used, they must be redefined as well. Also, if a scissor rectangle is used, it must be re-specified.
2. redefine the cameraToCell matrix if the camera has been moved or rotated
3. redefine the view frustum, if it has been changed

Checklist for Models

1. construct a model of desired type by calling `XXXModel::create()`
2. if the model is planar or nearly planar, or the model is to be used as a test model for a portal, enable back-face culling by setting the `BACKFACE_CULLABLE` flag
3. if you want to have the model automatically destructed once it is no longer referenced by any objects, call `ReferenceCount::autoRelease()`
4. if you want to debug with names, assign a name for the model by `Model::setName()` (section 6.4.3)

Checklist for Objects

1. construct an object of desired type by calling `Object::create()`. Pass a pointer to its test model in the constructor call
2. assign the object to some cell by calling `Object::setCell()`
3. assign the object $\rightarrow$ cell transformation matrix with
`Object::setObjectToCellMatrix()`
4. to give dPVS a better idea about the rendering cost of the object, call `Object::setCost()` (section 6.9.8)
5. if you intend to use the object as an occluder, assign a write model to it by using
`Object::setWriteModel()`
6. if you want to link the object back to some object class of your own, assign a user pointer by
`Object::setUserPointer()` (section 6.4.4)

7. if you know that the object will only be visible if some other (enclosing) object is visible, assign that object as the current one's *visibility parent* by calling `Object::setVisibilityParent()` (section 6.9.6)
8. if you want to debug objects with names, assign a name for the object by `Object::setName()` (section 6.4.3)
9. if you want the object to be automatically destroyed when its cell is destroyed, call `ReferenceCount::autoRelease()`

Checklist for Creating a PhysicalPortal

1. construct a `PhysicalPortal` by calling `PhysicalPortal::create()`. Pass a pointer to its test model and its target cell as the constructor parameters. Make sure that the test model has back-face culling enabled (section 6.8.5)
2. assign the portal to a cell by calling `Object::setCell()`
3. assign the portal $\rightarrow$ cell transformation matrix with `Object::setObjectToCellMatrix()`
4. remember to pass a positive recursion depth (1 or greater) to `Camera::resolveVisibility()`
5. if you want to link the portal back to some object class of your own, assign a user pointer by `Object::setUserPointer()`
6. if you know that the portal will only be visible if some other (enclosing) portal is visible, assign that portal as the current one's *visibility parent* by calling `Object::setVisibilityParent()`
7. if you want to debug portals with names, assign a name for the object by `Object::setName()`
8. if you want the portal to be automatically destroyed when its cell is destroyed, call `ReferenceCount::autoRelease()`
9. if the portal requires a stencil mask, set it by calling `PhysicalPortal::setStencilModel()`. The second parameter tells dPVS whether the portal is floating in the middle of a cell.
10. if you want assistance to LOD level selection, set portal's importance decay by calling `PhysicalPortal::setImportanceDecay()` (section 6.11.8)
11. if you want to be informed when a visibility query traverses through a portal, set `Object::INFORM_PORTAL_ENTER` and `Object::INFORM_PORTAL_EXIT`

Checklist for Creating a VirtualPortal

1. construct a `VirtualPortal` by calling `VirtualPortal::create()`. Pass a pointer to its test model and target portal as the constructor parameters
2. assign the portal to a cell by calling `Object::setCell()`

-
3. assign the portal → cell transformation matrix with `Object::setObjectToCellMatrix()`
 4. set the portal's warp matrix by calling `VirtualPortal::setWarpMatrix()` (section 6.11.5)
 5. remember to pass a positive (1 or greater) recursion depth to `Camera::resolveVisibility()`
 6. if you want to link the portal back to some object class of your own, assign a user pointer by `Object::setUserPointer()`
 7. if you know that the portal will only be visible if some other (enclosing) portal is visible, assign that portal as the current one's *visibility parent* by calling `Object::setVisibilityParent()`
 8. if you want to debug portals with names, assign a name for the object by `Object::setName()`
 9. if you want the portal to be automatically destroyed when its cell is destroyed, call `ReferenceCount::autoRelease()`
 10. if the portal requires a stencil mask, set it by calling `PhysicalPortal::setStencilModel()`. The second parameter tells dPVS whether the portal is floating in the middle of a cell.
 11. if you want assistance to LOD level selection, set portal's importance decay by calling `PhysicalPortal::setImportanceDecay()`
 12. if you want to be informed when a visibility query traverses through a portal, set `Object::INFORM_PORTAL_ENTER` and `Object::INFORM_PORTAL_EXIT`

Appendix B

dPVS Visualizer

dPVS Visualizer is an analysis and debugging tool for dPVS . Visualizer is a programming framework that uses dPVS for object database management and OpenGL for rendering. The windowing and menu handling uses GLUT. Visualizer is written in fully portable C++ and uses only libraries available on most modern platforms.

Plugged into the basic framework is a number of demos. These demos are used to showcase different features of dPVS . All of the demos share the same camera controls and other debug features provided by the Visualizer framework. The source code of dPVS Visualizer is distributed with the libraries. Adding a new demo requires only a couple of hundred lines of new code.

Views and Cameras

Visualizer supports multiple simultaneous views of the scene. The number of active views is selected using the *Grid Layout* menu. All camera and most menu controls affect only the currently selected (active) view. This view is marked with yellow borders. The active view can be selected either by pressing the TAB key or by moving the mouse cursor over the window and clicking any mouse button.

For each view, a different camera can be assigned. Visualizer has two kinds of cameras: *dPVS cameras* and *debug cameras*. dPVS cameras are cameras that have been placed into the scene. For each dPVS camera, there is a corresponding debug camera. The debug camera can view the output of the dPVS camera from a different location. Either kinds of cameras can be assigned to a specific view by activating the view and then using the *Assign Camera* menu.

If multiple views are used, Visualizer updates the active view every frame. All other views are updates less frequently. If you want to update all views every frame, you should disable the *Global/Fast Primary View Update* option.

The dimensions of the entire window can be changed the same way as other windowed applications are, or by using the keyboard shortcuts F1-F6.

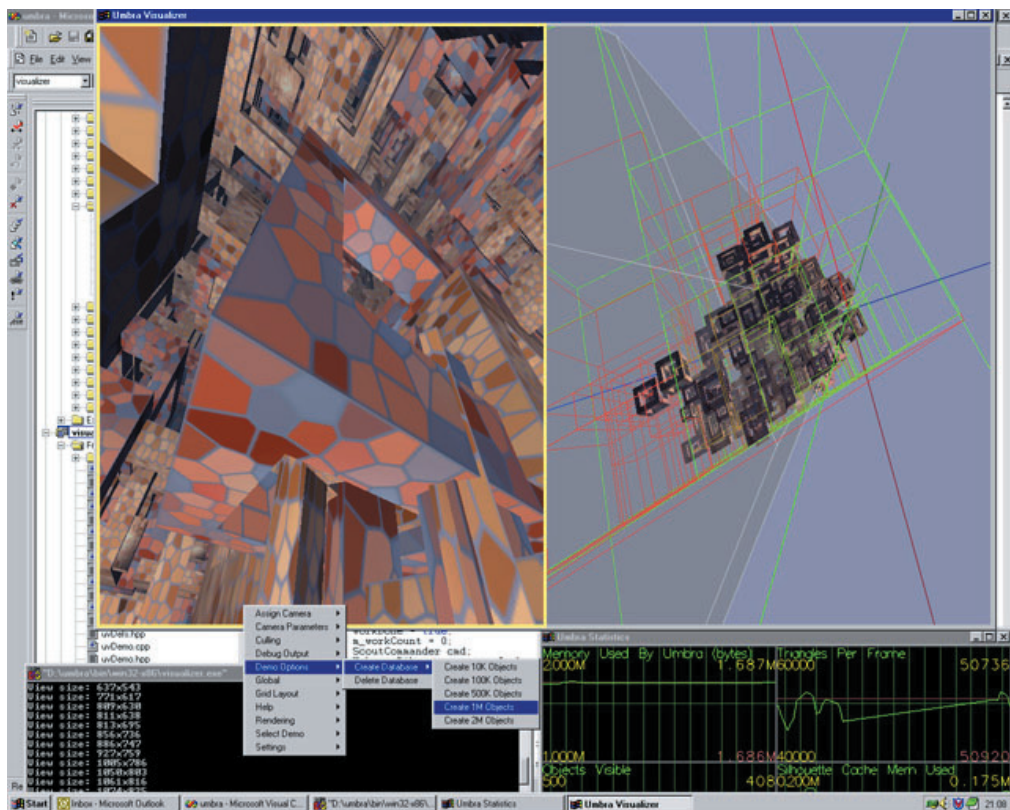


Figure B.1 The picture shows dPVS Visualizer in action. The left view is the real camera and the right view is the corresponding debug camera. The text output window can be seen in the bottom left corner and the statistics window in the bottom right corner.

Camera Controls

The camera can be controlled with both the mouse and the keyboard. The camera can be rotated either by using the arrow keys or by holding the left mouse button and moving the mouse. The camera can be moved forward or backward by the keys W and S. Strafing, i.e. horizontal or vertical motion, is achieved by pressing CTRL and the arrow keys or the center mouse button¹ and moving the mouse. The camera can be rolled with the keys Q and E.

Other camera-related parameters, such as the speed of the camera or the field of view angles, are controlled with the *gray plus* and *gray minus* keys. These keys modify the currently selected parameter. The parameter to be modified is selected from the menu *Camera Parameters*. The *gray multiply* and *gray divide* keys have the same effect, except that the parameter is modified more quickly.

The camera control can be transferred to any object in the scene by *picking* the object. An object can be picked by moving the mouse cursor on top of it and pressing the P key. The picked object is drawn with a flashing wireframe on top of it. The control can be transferred back to the camera by pressing P again. Note that objects that have demo-specific controllers, i.e. the ones that automatically move around the scene, cannot be picked.

Demos

Visualizer consists of a number of demos that are used for testing and showing different capabilities of dPVS . The current demo can be selected by using the *Select Demo* menu. Some demos have local options of their own; these can be accessed by the *Demo Options* menu.

Statistics

The statistics window can be opened or closed by selecting *Global/Statistics Window*. The contents of the statistics window can be changed by clicking the right mouse button when the statistics window has been selected. This will open up a pop-up menu that displays all available statistics. See figure [B.2](#) for an explanation of the statistics graph.

Settings

The current settings can be stored into a file. There are nine available save slots. When Visualizer starts, it loads its settings from the first slot. When the application exits, the settings are always stored into a special slot called 'slot 0'. Settings can be loaded from different slots with keys 0-9 and stored into different slots by pressing ALT and 1-9. The same functionality can also be found from the *Settings* menu.

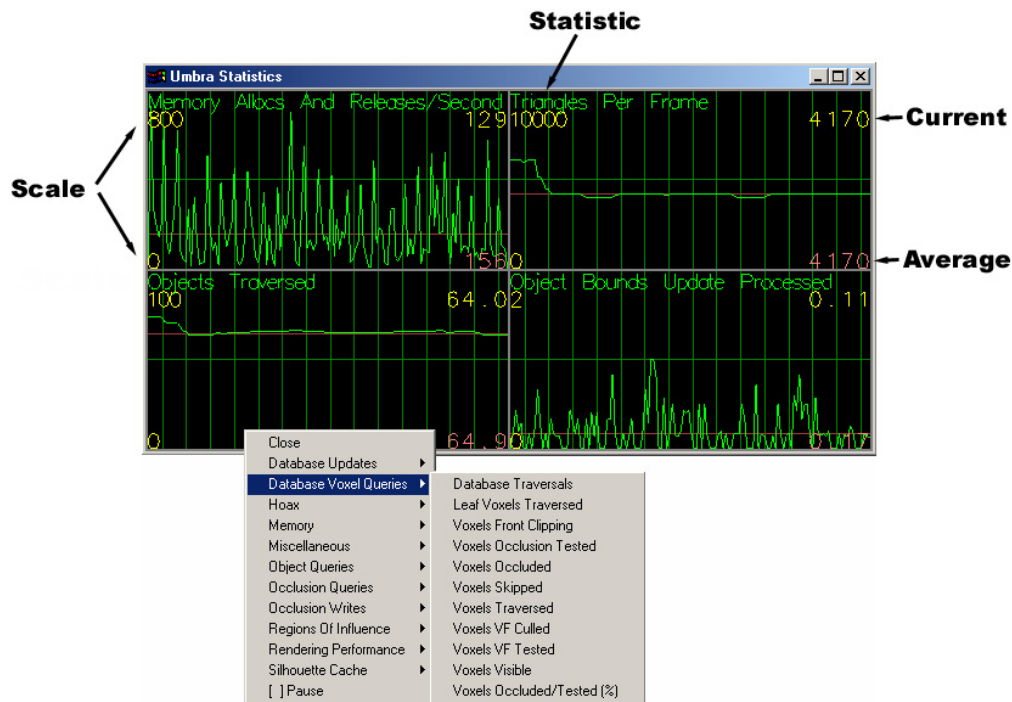


Figure B.2 Statistics menu

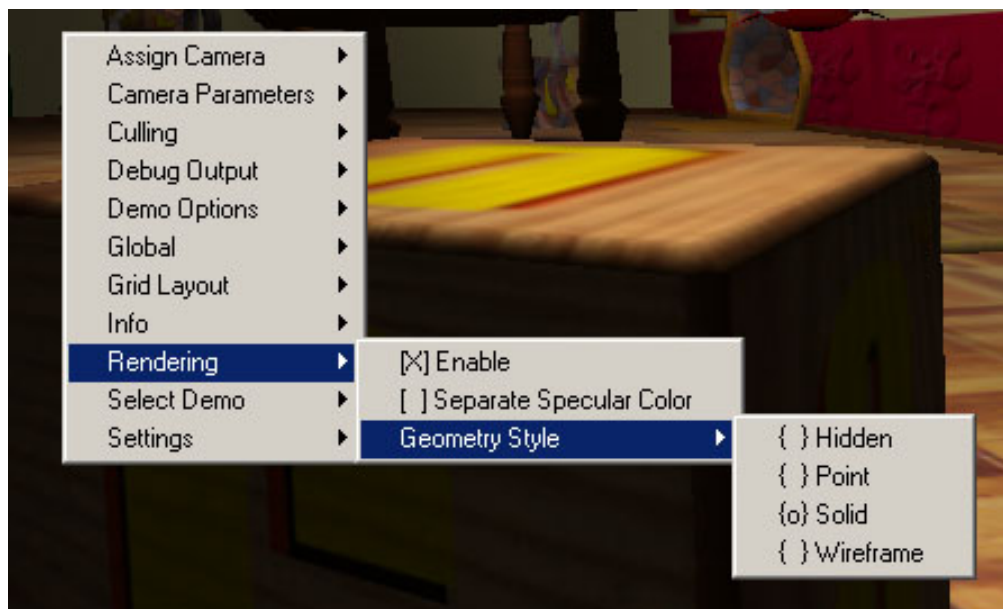


Figure B.3 Menu System of dPVS Visualizer

Assign Camera	Menu for selecting which camera is assigned to the current window
Camera Parameters	Menu for selecting which camera parameters are controlled by the gray +/- keys
Culling	Menu for selecting between different culling modes
Debug Output	Menu for controlling debug output
Demo Options	Menu for demo-specific options
Global	Menu for controlling global parameters
Grid Layout	Menu for configuring current window grid layout
Info	Menu for querying information about the system and current demo
Rendering	Menu for controlling rendering options
Select Demo	Menu for selecting the current demo
Settings	Menu for loading and saving current settings

Figure B.4 Top Level Menus

Strafe	Makes mouse button + drag to control strafing
Angular Velocity	Controls angular velocity of the camera/object
Clip Plane (Far)	Moves the far clipping plane
Clip Plane (Near)	Moves the front clipping plane
FOV (Both)	Scales both FOV angles (perspective projection only)
FOV (Horizontal)	Scales horizontal FOV angle (perspective projection only)
FOV (Vertical)	Scales vertical FOV angle (perspective projection only)
Ortho Projection Size	Scales projection size (ortho projection only)
Ortho Projection Width	Scales projection width (ortho projection only)
Ortho Projection Height	Scales projection height (ortho projection only)
Screen Cull Size	Scales screen-space culling size (default = 1x1)
Screen Cull Opacity	Scales screen-space culling opacity (default = 1.0)
Velocity	Controls velocity of the camera/object
Projection	Sub-menu for selecting between perspective and orthographic projection
Background	Sub-menu for selecting between different background colors

Figure B.5 Menu: Camera Parameters

Menus

All options are available through a single pop-up menu. The menu can be opened by clicking the right button of the mouse. On platforms, such as Apple Macintosh, that do not have a right mouse button, the menu appears in the list of pull-down menus for the application.

The menus are always specific to the currently selected window. The contents of the menu may also vary based on whether the camera assigned to the current window is a real camera or a debug camera.

¹Alternatively the left and right buttons can be pressed simultaneously to achieve the same effect.

Perform Occlusion Culling	Enable this to perform occlusion culling
Perform View Frustum Culling	Enable this to perform view frustum culling
Scissoring	Enable this to test scissoring
Static Coverage Mask	Enable this to test a static coverage mask
Sub-Sampling	Sub-menu for selecting between sub-sampling modes (1x1,2x2,3x3,4x4)
Traverse Virtual Portals	Enable traversal through virtual portals (mirrors etc.)

Figure B.6 Menu: Culling

Occlusion Buffer	Sub-menu for selecting which occlusion buffer is visualized
Disable Visual Debug Output	Disables all visual debug output
Show OpenGL Stencil Buffer	Visualize OpenGL stencil buffer contents (slow)
Show Object Bounds	Visualize object motion bounds
Show Portal Rectangles	Visualize portal scissor rectangles
Show Rectangles	Visualize object test rectangles
Show Silhouettes	Visualize occluder write silhouettes
Show Virtual Cameras	Visualize virtual cameras
Show Voxels	Visualize spatial database layout
Show VPT	Visualize visible point tracking
Visualize Overdraw	Visualize overdraw per pixel (slow)
Print Debug Info	Dump information about current camera/object to output window

Figure B.7 Menu: Debug Output

Disable	Do not display any occlusion buffers (default)
Coverage Buffer	Show high-precision coverage buffer
Depth Buffer	Show contents of the depth buffer
Full Blocks Buffer	Show contents of the full blocks buffer
Stencil Buffer	Show contents of the stencil buffer of dPVS

Figure B.8 Menu: Debug Output/Occlusion Buffer

Advanced Keyboard Settings	Changes keyboard layout
Disable All Visual Output	Disables all visual output (but still runs dPVS for each camera)
Fast Primary View Update	Update currently selected window faster than the other windows
Statistics Window	Open/close the statistics window
Verbose Output	Enable to allow verbose debug messages
Vertical Sync	Enable to force synchronization to vertical retrace
Assign Controls To Camera	Assign mouse/keyboard controls to camera
Assign Controls To Object	Assign mouse/keyboard controls to currently selected object
Disable Selected Object	Remove (temporarily) selected object from database
Dump Debug Info To File	Dumps debug information into a file
Enable All Disabled Objects	Returns all disabled objects back into the database
Minimize Memory Usage	Forces dPVS to minimize its memory footprint
Reset Settings To Factory Defaults	Resets all settings to the default state

Figure B.9 Menu: Global

Display Credits	Shows credits
Display Demo Info	Displays help text associated with the current demo
Display Key Shortcuts	Displays all keyboard shortcuts
Display Renderer Info	Displays information about OGL driver and hardware used
Display System Info	Displays information about the CPU
Display Version Info	Displays information about the version of dPVS and Visualizer used

Figure B.10 Menu: Info

Enable	Enable rendering to current window
Separate Specular Color	Enable use of the separate specular color OGL extension (if hardware supports)
Geometry Style	Select style of geometry (hidden, point, solid, wireframe)

Figure B.11 Menu: Rendering

Load Settings	Loads settings from selected slot
Save Settings	Stores current settings to selected slot
Save Settings Every Frame	Stores settings every frame (used for debugging)

Figure B.12 Menu: Settings

Arrow keys	Rotate camera or selected object
A/D	Strafe camera or selected object
Arrow keys (+ctrl)	Strafe camera or selected object
Space	Speed up rotation and motion by 10x
W/S	Move camera or selected object forward/backward
Q/E	Roll camera or selected object camera
Tab (w/shift)	Select next (previous) camera
Numpad +/-	Modify currently selected camera parameter
Numpad */	Modify currently selected camera parameter (10x speed)
0-9	Load settings for specified slot
0-9 (w/alt)	Save settings to specified slot
P	Pick/unpick object at mouse cursor
F	Print selected camera/object information
R	Disable selected object
E	Enable all disabled objects
H	Display keyboard shortcuts
V	Display version information
F1-F6	Set window size (320x240,512x384,640x480,800x600)
esc	Quit Visualizer

Figure B.13 Keyboard Shortcuts

Appendix C

Example Implementation of the Commander class

```
class MyCommander : public DPVS::Commander
{
public:
    MyCommander (void);
    virtual ~MyCommander (void);
    virtual void command (Command);
private:
    MyCommander (const MyCommander&);
    MyCommander& operator= (const MyCommander&);
};

// nothing to do here
MyCommander::MyCommander (void) {}

// nothing to do here
MyCommander::~MyCommander (void) {}

void MyCommander::command (Command c)
{
    switch (c)
    {
    case QUERY_BEGIN:
        // This marks the beginning of the visibility query, no
        // action needs to be taken. However, if we want, we can
        // setup the rendering states here
        break;

    case QUERY_END:
        // This marks the end of the visibility query. Again,
        // no action needs to be taken. Here we could restore
        // the rendering states to default, if we wanted.
        break;

    case PORTAL_ENTER:
    {
        // This function is called before the query passes
        // through a portal. No action needs to be taken.
        // If we want an access to the portal object, we can
        // use the getInstance() call.
    }
    }
}
```

```
    const Instance* instance = getInstance();
    // here do whatever we want
}
break;

case PORTAL_EXIT:
{
    // This function is called before the query passes back
    // through a portal (there's always a PORTAL_EXIT
    // message for each PORTAL_ENTER).

    const Instance* instance = getInstance();
    // here do whatever we want
}
break;

case VIEW_PARAMETERS_CHANGED:
{
    // View parameters have changed. This is signaled at
    // the beginning of the query and each time the query
    // passes through a portal.

    // We want to change the projection matrix of the
    // rendering engine. This can be done either by
    // querying the matrix from the viewer, querying the
    // frustum structure of the frustum planes.

    const Viewer* viewer = getViewer();
    assert (viewer);

    // example of using the projection matrix
    Matrix4x4 matrix;
    viewer->getProjectionMatrix(matrix, Viewer::LEFT_HANDED);
    // here pass projection matrix to the rendering engine

    // example of using the frustum
    Frustum frustum;
    viewer->getFrustum(frustum);
    // here pass frustum to the rendering engine

    // If we're using portals or scissoring, the scissor
    // may have changed

    int left,right,top,bottom;
    viewer->getScissor(left,top,right,bottom);
    // here pass new scissor to the rendering engine

    // If we're using virtual portals or a flipped frustum,
    // the 2D culling mode of the rendering engine may need
    // to be altered.

    if (viewer->isMirrored())
    {
        // change back-face culling mode
    } else
    {
        // set back-face culling mode to normal
    }

    // If we're using floating portals, an additional
    // clip plane may have been introduced.

    if (viewer->getFrustumPlaneCount() > 6)
    {
```

```

        Vector4 plane;
        viewer->getFrustumPlane(6, plane);
        // set this plane as the rendering engine's
        // user clip plane
    }
}
break;

case INSTANCE_VISIBLE:
{
    // An object is determined to be visible. Time to
    // either store the object into a list or to render
    // it immediately.

    const Instance* instance = getInstance();
    assert (instance);

    // Get pointer to the object

    Object* object = instance->getObject();
    assert (object);

    // Here perform rendering of the object or store the
    // pointer somewhere. If we need access to the
    // corresponding class on the application side, we can
    // use the 'user pointer' (assuming it was assigned to
    // the object earlier).

    void* userPointer = object->getUserPointer();

    // (MyObject*)(userPointer)->doThisAndThat();
}
break;

case INSTANCE_IMMEDIATE_REPORT:
{
    // An object is signaled as visible (during the query).
    // Here we may want to change the write model of the
    // object (in case we're using a LOD scheme or
    // something).

    const Instance* instance = getInstance();
    assert (instance);

    Object* object = instance->getObject();
    assert (object);

    // object->setWriteModel(foo);
}
break;

case REMOVAL_SUGGESTED:
{
    // This event can only happen if we have called
    // Library::suggestGarbageCollect().

    const Instance* instance = getInstance();
    assert (instance);

    Object* object = instance->getObject();
    assert (object);

    // maybe delete the object here?
    // object->release();
}
}

```

```
}
break;

case REGION_OF_INFLUENCE_ACTIVE:
{
    // A light source becomes active. Enable it in
    // your rendering engine.

    const Instance* instance = getInstance();
    assert (instance);

    Object* object = instance->getObject();
    assert (object);

    // 'object' points now to the RegionOfInfluence.
    // get user point and turn the light on..

}
break;

case REGION_OF_INFLUENCE_INACTIVE:
{
    // A previously activated light source has become
    // inactive. Disable it in your rendering engine.

    const Instance* instance = getInstance();
    assert (instance);

    Object* object = instance->getObject();
    assert (object);

    // 'object' points now to the RegionOfInfluence.
    // get user point and turn the light off.
}
break;

case STENCIL_VALUES_CHANGED:
{
    // Stencil test/write values have changed. This
    // event can only happen when using floating
    // portals.

    int test,write;

    getStencilValues(test,write);

    // here apply the new test / write values to the
    // rendering engine.
}
break;

case STENCIL_MASK:

    // Called whenever there are changes to be made to the
    // stencil buffer. The commands are either INCREMENT or
    // DECREMENT commands. In absence of stencil buffer,
    // a destination alpha test can be used for performing
    // this action.

    // Recommended action:
    // -----

    // Step 1:
    // - disable all color channels.
    // (glColorMask(GL_FALSE, GL_FALSE, GL_FALSE, GL_FALSE))
```

```

// - getStencilValues(). INC = true, if write>test
// - enable stencil test (glEnable(GL_STENCIL_TEST))
// - set stencil func (glStencilFunc(GL_EQUAL, test, 0xff))
// - if(INC) glStencilOp(GL_KEEP, GL_KEEP, GL_INCR);
// else glStencilOp(GL_KEEP, GL DECR, GL DECR);
// - optionally force polygon rendering mode to filled
// - render instance
//
// Step 2 (only INC):
// - set depth value to far plane
// - disable depth test
// - set stencil function
// - disable stencil operations
// - enable all color channels

// glDepthRange(1,1);
// glDepthFunc(GL_ALWAYS);
// glStencilFunc(GL_EQUAL, write, 0xff);
// glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
// glColorMask(GL_TRUE, GL_TRUE, GL_TRUE, GL_TRUE);
//
// Step 3:
// - enable all color channels
// - restore depth func
// - restore depth range
// - set stencil func
// - disable stencil operations
// - optionally restore polygon fill mode (wireframe)
//
// glColorMask(GL_TRUE, GL_TRUE, GL_TRUE, GL_TRUE);
// glDepthFunc(GL_LEQUAL);
// glDepthRange(0,1);
// glStencilFunc(GL_EQUAL, write, 0xff);
// glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);

// Considerations:
// -----

// - additional clip plane has been enabled/disabled in
// VIEW_PARAMETERS_CHANGED
// - clears color buffer in INC mode
// - leaves OpenGL color/depth states intact
// - adjusts OpenGL stencil state for object rendering
//
// Valid methods:
// -----
// - getInstance(), do not use getClipMask() here
// - getStencilValues()

break;

case TEXT_MESSAGE:
{
    // [debug information] dPVS has a text message to the
    // user.
    const char* s = getFormattedMessage();
    if (s)
        printf ("dPVS text message: %s",s);
}
break;

case DRAW_LINE_2D:
{
    // [debug information] There is a 2D line to be drawn.

```

```
Vector2 start,end;
Vector4 color;

Library::LineType type = getLine2D(start,end,color);

// draw the line here or buffer it for later use.
// The 'type' value indicates what debug information
// the line represents.
}
break;

case DRAW_LINE_3D:
{
    // [debug information] There is a 3D line to be drawn.

    Vector3 start,end;
    Vector4 color;

    Library::LineType type = getLine3D(start,end,color);

    // draw the line here or buffer it for later use.
    // The 'type' value indicates what debug information
    // the line represents.
}
break;

case DRAW_BUFFER:
{
    // [debug information]. There is a buffer (bitmap
    // image) to be displayed.

    const unsigned char* data;
    int width,height;

    Library::BufferType type = getBuffer(data,width,height);

    // We can either display the buffer here or take a copy
    // of it (width*height bytes) for later use. The 'type'
    // value indicates what the contents of the buffer are.
}
break;

default:
    // In debug build, perform an assertion
    assert (!"Unrecognized command enumeration");
    break;
}
}
```

Appendix D

Troubleshooting

Below are listed some common problems the user can encounter and how to resolve the problems.

- many errors are not trapped in the release build; use the debug build when hunting bugs
- see the checklists for different classes to see if you have missed something important
- if either the near or far clipping plane are defined to have negative values, invalid projections will occur. See `Camera::setFrustum()` for further details.
- if model vertex data contains invalid floating-point values, a number of systems can fail. This is trapped in the debug build, but not in the release build.
- make sure that the triangle vertex indices are specified with the correct winding; otherwise wrong triangles will be culled
- if models used by portals are not defined as `BACKFACE_CULLABLE`, infinite recursion can occur. Such recursions are trapped and handled internally, but will produce artifacts (section 6.8.5)
- the interface always uses 4x4 matrices even if only 4x3 matrices should be submitted. Please ensure that the last row/column does not contain a projection (i.e. it is 0,0,0,1). This is trapped in the debug build
- matrices with negative scaling terms can produce extremely surprising results and should be avoided. Non-invertible matrices, i.e. ones with zero scaling terms, may not be used at all
- in the debug build, memory leaks are detected when `Library::exit()` is called. An assertion will be raised if any leaks are found. See documentation for the `ReferenceCount` class to better understand how the reference-counting mechanism works
- if you have models with both fine detail and vertices that are placed extremely far apart, the resolution of single-precision floats may not be enough. Try to split such a model into several parts and to use the double-precision transformation matrices for locating the parts correctly
- objects must belong to a cell in order to be reported as visible
- if objects are incorrectly back-face culled and you are using virtual portals, make sure you have checked the mirroring status. See `Commander::Viewer::isMirrored()` for further details.

- if portals are not visible, make sure that the recursion count parameter in `Camera::resolveVisibility()` is at least 1.
- in order to use objects as occluders, a write model must be assigned to them.
- if your vertex lighting is flickering, the edge lengths of your models are too long. You can fix this by either subdividing the polygons with long edges ¹, or by assigning *safe distances* to your ROIs (section 6.10).
- small artifacts can occur if the internal rasterization pixel offsets of dPVS do not match the ones used in your renderer (section 6.12.3)
- getting a hardware floating-point exception when running the debug build is an indicator of bad mesh data, invalid frustum settings or object matrices containing invalid floats
- note that the initial clearing of the depth buffer is not signaled by the Commander
- when using the DLL version of dPVS, the dynamic link library must be found somewhere in the DLL search path

¹When tessellating polygons, use *edge tessellation* instead of *surface tessellation*.

Appendix E

FAQ

This section contains Frequently Asked Questions about the dPVS library and answers to the questions. Most of this information can be found elsewhere in this documentation as well.

Q: Can I use dPVS with OpenGL / DirectX / my own 3D engine?

A: Yes. dPVS does not expect anything from the rendering engine used. Some special features however are available only if certain hardware features exist. For example, usage of stencil models requires stencil buffer capabilities from the rendering engine.

Q: Does dPVS require special rendering hardware?

A: No. The library does not *render* anything, thus it does not need any rendering hardware or rasterization libraries. Of course, in order to visualize something, the user needs a rendering engine of some kind.

Q: Is there a version of dPVS available on platform XYZ?

A: Yes. dPVS is written in fully portable C++ and compiles as-is on a number of compilers and platforms. It does not rely on any external libraries. New and poorly-supported features of the C++ language have not been used. See appendix [G](#) for further information about minimum platform requirements.

Q: Can I add new objects to the system after initialization? Can I delete objects?

A: Yes. The spatial databases used in dPVS are all dynamic and support dynamic insertion and removal of objects.

Q: Does dPVS support LODs/multi-resolution geometry/image-based rendering?

A: All of the above-mentioned can be used in combination with dPVS, and the algorithms used internally take into account the effect of LODs, for example. The library however does not contain any support for the generation of LODs/MRG/IBR objects. See the dPVS website for information about the upcoming dPVS Impostor library.

Q: What matrix formats and coordinate systems can I use?

A: The interface allows specifying the matrix format used. dPVS is coordinate-system and orientation-independent. Projection terms cannot be used in transformation matrices.

Q: Are there any numeric limits (such as number of objects, object complexity etc.) ?

A: Very few. Integer data used inside the system is 32-bit and has natural limits. For example, having more than 2 billion references to a single object will cause problems. The image-space systems support resolutions up to 16384x16384.

Q: Do I need any other input than the mesh data and objects' orientation matrices, i.e. do I need to create convex hulls for objects, or calculate bounding volumes?

A: No. Whenever dPVS needs such information, it calculates and caches it internally.

Q: Can a pre-calculated depth buffer (such as in the game Alone in the Dark) be used with dPVS ?

A: Yes, dPVS can combine information from a pre-calculated depth buffer into its occlusion calculations. See documentation for `Camera::setStaticZBuffer()`.

Q: Can a 1-bit per pixel bitmap be used to mask out portions of the image?

A: Yes. See documentation for `Camera::setStaticCoverageMask()`.

Q: Our game engine uses portals. Can these be integrated into dPVS ? Are there any reasons for using other culling techniques than portals?

A: Yes, dPVS supports arbitrary-shaped portals. The other occlusion techniques provide faster VF culling and occlusion due to dynamic objects. Portals are highly efficient in architectural scenes and should be used if available in your modeling package. Additionally, portals are useful for special effects. dPVS is able to combine all these culling methods: for example, a moving creature can hide a portal and thus terminate the portal traversal.

Q: Can I use dPVS for calculating real-time shadows?

A: Not directly. However, dPVS can be used as a per-frame shadow pre-processor, i.e. to find all the objects that may contribute to the shadow volume / shadow depth buffer. This can optimize the shadow computations heavily. See section [6.12.12](#) for further details.

Q: Can I use it in a landscape environment / Quake-style dungeon environment / space game / flight simulator / urban scene?

A: Yes. dPVS uses a mixture of algorithms and it adjusts to the type of the scene. This allows easy mixing of different types of environments.

Q: Can user-defined object hierarchies be used with the library?

A: Yes and no. dPVS has the concept of a *visibility parent* relation. If object *A* is object *B*'s visibility parent and object *A* is hidden, then object *B* can be automatically discarded without further tests. dPVS does not support object transformation hierarchies, as these should be represented in the user's scene graph.

Q: Can morphing/skinning objects be used as occluders/occludees?

A: They can be used as occludees. There are no theoretical obstacles for using them as occludes, but usually that is not worth doing.

Q: Can dPVS handle inconvex objects or objects with holes? How about objects with interpenetrating polygons?

A: Yes, it can handle objects with arbitrary topology. However, note that a wall with a small hole can be a surprisingly bad occluder.

Q: Can alpha channels be used in textures?

A: Yes and no. The materials or pixel shaders of occluders may not contain transparencies. dPVS does not know anything about the materials/shaders/textures used in the actual rendering process. Only completely opaque polygons should thus be specified to dPVS as occluders. See section 6.8.8 for further details.

Q: Can I use multiple cameras simultaneously? Does this affect the temporal coherence somehow?

A: Yes. The various caches have all been designed to handle an unlimited number of cameras. Additionally, some of the algorithms can share information between cameras, so that part of the work done by the first camera can be used by the second.

Q: Can dPVS resolve visibility on a per-polygon level? Does it do back-face culling?

A: No. Individual polygon culling is a trivial task beyond the scope of this library and should be handled by the rendering engine. dPVS resolves visibility at object level. Defining the object granularity is entirely up to the user - be it 10, 100 or 1000 polygons - when the model data is specified to dPVS. On newer rendering hardware it is not worthwhile to perform software culling at small granularities.

Q: Can dPVS produce artifacts due to false occlusion?

A: Normally no, but there is one exception. False occlusion is possible if an occluder is a single-sided mesh with holes on the surface. Occlusion write of such objects should be disabled. Note that artifacts can occur if the user explicitly enables some of the non-conservative culling methods provided in the library.

Q: Why can't I allocate objects/models from the stack?

A: All dPVS objects are reference-counted and their destructors are protected. There are several good reasons for this approach. Use the `ReferenceCount::release()` method for destructing objects.

Q: How do I know which version of dPVS I'm using?

A: By calling `Library::getInfoString(Library::VERSION)` you can query the version number of the current build. By querying the `BUILD_TIME` you can get the date and time the library was built.

Q: Are there cases when occlusion culling should be disabled?

A: No, except for debugging or experimental purposes. When objects do not occlude each other well, dPVS automatically adjusts its occlusion culling to have as little overhead as possible. It is

important to keep the occlusion culling turned on, so that when it becomes useful again, the library will be prepared for it.

Q: Why does dPVS produce different sets of visible objects even if no objects have moved and the camera is stationary?

A: dPVS constantly attempts to optimize its selection of occluders and rules for proving that objects are hidden. This process is heuristic in nature, and is subject to mutation rules. Therefore the results of the visibility query can vary somewhat, although the results of all queries are conservative.

Q: Where can I find more information about visibility algorithms?

A: Chapter 10 contains an overview about the visibility determination problem and the algorithms used to solve it. The end of this book has a large bibliography section with references to recent publications on the subject. Additionally, the dPVS web site has a page with links to online publications and home pages of visibility researchers.

Appendix F

Glossary

This section contains short definitions and explanations for terms and acronyms used in this manual.

AABB	Axis-Aligned Bounding Box
Accumulation Buffer	Additional high-resolution buffer for combining color buffer images. Can be used for antialiasing or effects such as motion blur or depth of field
Aggressive Culling	Culling of hidden objects as well as some visible objects (compare to conservative culling)
AI	Artificial Intelligence
API	Application Programming Interface
Area Visibility	Determination of visibility from a volume in space rather than a single point
Axis-Aligned	Something that is aligned with the three primary axis (X,Y,Z)
Back-Face Culling	Culling of polygons or objects that are completely facing away from the camera
Bounding Box	A box that fully contains an object
Bounding Sphere	A sphere that fully contains an object
Bounding Volume	Any volume that fully contains an object
Bounding Volume Hierarchy	A set of bounding volumes organized hierarchically and used to improve various tests
BSP	Binary Space Partitioning
BSP Tree	A tree structure where each node partitions a set of objects or polygons into two halves
BVH	Bounding Volume Hierarchy

Cell	A part of the world connected to other cells by portals
Conservative Culling	Culling of (most of the) objects that do not contribute to the output image
Contribution Culling	Culling of objects that have little significance to the output image
Convex Hull	Vertices on the convex boundary of the object
Culling	Removal of objects that do not contribute to the output image
CW	Clockwise
CCW	Counter-clockwise
Depth Buffer	A data structure used for pixel-level hidden-surface determination
DHS	Definitely Hidden Set
DLL	Dynamic Link Library
DOF	Direction Of Flight (or View Plane Normal)
Floating Portal	A virtual portal that is not on a cell boundary (such as a hand mirror)
FPS	Frames Per Second
Frame Coherence	Coherence between subsequent frames
Frame Rate	Number of frames rendered per second
Free Clipping Plane	Any clipping plane beyond the initial six planes forming the view frustum
HOM	Hierarchical Occlusion Map
Hz	Hertz (unit of frequency). Measures number of times something occurs per second
HZB	Hierarchical Z-Buffer
HWTL	HardWare Transform and Lighting
IBR	Image-Based Rendering
Image-Based Rendering	Creating the output image from previously rendered images
LOD	Level Of Detail
Motion Prediction	Heuristic prediction of future locations and orientations of objects
OBB	Oriented Bounding Box, i.e. a bounding box where the axes are not aligned to the three primary axes
Occludee	An object obstructed from the view by other objects
Occluder	An object used for obstructing geometry from the view

Occluder Fusion	A property of certain occlusion culling algorithms of merging the umbrae of multiple occluders
Occlusion Culling	Culling of objects that are hidden by other objects
Octree	A spatial data structure where each node has eight child nodes
OPS	Occlusion-Preserving Simplification
Penumbra	The partial shadow region of an area light source
Physical Portal	A real-life portal where no warping of light rays is performed
Point Visibility	Visibility from a single point in space
Portal	A connection (either physical or virtual) between two cells
Potentially Visible Set	List of all visible and some hidden objects
PVS	Potentially Visible Set
Recycler	A memory-management structure for reusing allocated memory used to reduce memory allocations/frees
Region Of Influence	The volumetric region in space where an object (such as a light source) can influence other objects
ROI	Region Of Influence
Screen-Size Culling	Removal of objects that have an axis-aligned screen projection smaller than some specified size
Spatial Database	A data structure where objects are organized based on their spatial extents
Spatial Index	Spatial Database
Stencil Buffer	A buffer used for enabling or disabling frame buffer writes on a per-pixel level
Tabu Counter	A counter used to prevent some action for a period of time
TBV	Temporal Bounding Volume
Temporal Bounding Volume	A bounding volume for an object that is valid for a certain period of time
Temporal Coherence	Coherence in the objects' positions and orientations over long periods of time
Test Model	A model used for determining the visibility of an object
Umbra	Portions of the scene that are fully hidden from a light source
Vertex Indices	Indices to a vertex position array. Used for describing triangle mesh data. Each triangle is specified using three indices
VF	View Frustum
View Frustum	Portion of the scene visible from a camera
View Frustum Culling	Culling of objects that fall completely outside the view frustum

Virtual Occluder	A hidden region of space that can be used as an occluder
Virtual Portal	Non-physical link between two portals
Warp Matrix	A matrix used for bending light rays when they travel through a virtual portal
Winding	The order in which a polygon's vertices are defined. Used to indicate which side is considered the front-side of the polygon
Write Model	A model that participates in the occlusion culling process as an occluder
Z-Buffer	A depth buffer where the raster-space Z-values of pixels are used as depth values

Appendix G

Technical Information

Platforms supported

- IBM PC and compatibles (Pentium or newer) running on Win32
- Microsoft Xbox
- Apple Macintosh (PowerPC) running on MacOS 8.6 or newer
- all other platforms available on request

Minimum Platform Requirements

dPVS can be ported to any platform with the following minimum requirements:

- 32 or 64 bit integer instructions
- hardware single-precision floating-point unit
- in order to compile the examples, OpenGL 1.1 and GLUT 3.x are needed

Compilers Supported

The dPVS library is written in fully portable C++, where many of the newer C++ constructs have been avoided. The code handles endianness issues properly and scales to 64-bit architectures.

- Microsoft Visual C++ 5/6
- Intel C++ W1.2 or newer
- GCC 2.95.2 or newer
- Borland C++ Builder 4.0

- Metrowerks CodeWarrior C/C++ version 2.2 or newer
- library files for other compilers available on request
- C, Java and Python wrappers available

Minimum Compiler Requirements

dPVS can be ported to any modern C++ compiler satisfying the following minimum requirements:

- support for namespaces
- support for basic templates. No member function templates needed.
- no support for exception handling needed
- no support for RTTI needed
- neither STD nor STL needed

API

- all classes are inside a common namespace - dPVS does not conflict with other libraries
- public interface consists of 10 header files, about a dozen classes, and about one hundred public member functions
- no global functions, variables or enumerations
- all API functions are fully documented
- consistent naming conventions for enumerations, classes and member functions
- most of the complicated concepts are internal and not exposed in the API
- no implementation details exposed in the API

Debugging

- separate debug build containing assertions for NULL pointers, invalid data and other user errors
- symbolic debugger info available in debug build
- most internal components perform self-tests in debug build
- system tested with a number of test applications and error-detection tools such as BoundsChecker
- entire source code compiles with zero warnings at maximum warning levels on all supported compilers
- dPVS Visualizer tool with external correctness tests of the system

Profiling

- API contains functions for querying a large amount of internal statistics
- dPVS Visualizer tool with statistics graphs

Resources Used

- no pre-processing of data required
- library object file is ~150 kB
- no disk space needed directly as the library does not perform any file I/O
- memory consumption is linear to input data
- small consumption of CPU resources

Input Data

- models used as occluders are defined as triangle/vertex arrays
- models used as occludees are defined either as triangle/vertex arrays or bounding volumes
- objects can be instantiated, i.e. the same model can be shared by multiple object instances

Multi-Threading Issues

- only a single thread can access any components of the library at a time
- compiled with multi-thread version of the standard C library
- does not use non-thread safe C library functions such as `rand()`

x86

- detects and utilizes MMX instructions if available; speeds up occlusion buffering
- detects and utilizes SIMD FP instruction sets if available; speeds up floating-point processing

Appendix H

Versions: Changes and Porting

This appendix contains relevant porting issues and a list of changes with their primary causes.

Porting

To version 2.3

- Replace calls to dPVS constructors with `create()`. For example `Object::create()`. Note that inheritance of dPVS objects has been explicitly banned. If you happened to use inheritance, the calls have to be redirected either with an inline wrapper (preferred) or directly through a pointer.
- Convert doubles in dPVS API calls into floats.
- Delete all references to `Model::WRITABLE` as this is not supported anymore.
 - Never set a write model that is not supposed to be used as a write model.
- *The users of `Camera::RESEND` flag:*
A camera must be prepared with `Camera::PREPARE_RESEND` before using `Camera::RESEND` facility. Consult the API Reference for details.
- *The users of `SelectorModel`, `DynamicModel`:*
These classes were removed. A new flag `Object::REPORT_IMMEDIATELY` can be used for obtaining an immediate report of the object's visibility (`Commander::INSTANCE_IMMEDIATE_REPORT`), *before* the object is used in any way. The write model of the object can be changed during this callback. As a result, level-of-detail selection is now applied without delay. The creation of dynamic write models (morphing) is (and was) unlikely to increase frame rate and should be generally discouraged. However, the API allows it.

List of Changes

Below are listed the most relevant API and internal changes along with their primary causes.

Versions 1.9 - 2.2

First public release. No major changes in the API. Various internal modifications and improvements.

Version 2.3

Constructors were replaced with static `create()` functions

This was a difficult but mandatory decision to make, since we wanted to have the system running also on fixed-memory systems (such as Sony PlayStation2 and Microsoft Xbox). The change to the API itself was minor, but the side-effect of disabling inheritance potentially causes some porting work. Should that happen, we sincerely apologize for the inconvenience.

Reasons:

1. Constructors must be able to fail due to out-of-memory error (without exception handling).
2. Enables more efficient memory management as the allocation sizes remain fixed. This directly correlates with execution speed, especially with large data sets.
3. Inheritance might have caused naming conflicts in the future, should we have added new functions.
4. Cleans the API by removing the unnecessary `m_imp`-pointers and `getImplementation()`-functions.
5. **NEGATIVE:** Disables inheritance and causes changes to application code.

Converted all doubles into floats

Reasons:

1. Certain machines, such as PlayStation2 execute double-precision floats using an emulator, thus incurring tremendous penalties in execution speed
2. Single-precision is a sufficient precision for all of our API functions
3. Implications to application code are small

Changes in flags

Change: `Model : WRITABLE` was removed.

Reason: The flag was useless and the removal simplified the API and the internal logic.

Change: `Model : SOLID` was added.

Reason: Allows a number of optimizations. Optional flag.

Change: `Object : REPORT_IMMEDIATELY` was added.

Reason: Allows changing the write model without a delay.

Change: `Commander::INSTANCE_IMMEDIATE_REPORT` was added.

Reason: `Pairs Object::REPORT_IMMEDIATELY`.

Change: `Camera::PREPARE_RESEND` was added.

Reason: The `Camera::RESEND` - facility is only used in certain editors and debugging tools. Therefore the normal applications should not pay the price of caching all commands for potential use of `RESEND` flag later.

Moved names from `Object` to `ReferenceCount`

Reason: Names take 0 bytes of memory when not used. Names are a useful debug functionality and thus setting names to models, cameras, etc. can be beneficial in the debug build. Previously only `Objects` had names, but now `getName()` and `setName()` have been moved to the `ReferenceCount` class, thus becoming accessible from several classes.

Bounds can be queried from `Models` and `Objects`

Reason: `getAABB()`, `getOBB()` and `getSphere()` are useful for an application programmer. `dPVS` stores the bounding volume information internally in any case.

`SelectorModel` and `DynamicModel` classes were removed

Reasons:

1. These classes were highly (or totally) unused and the same effect can now be accomplished in a more flexible manner using the newly introduced `Object::REPORT_IMMEDIATELY` flag.
2. The old approach had a one frame latency, which was eliminated.
3. Simplified the API.

Added an alternative constructor to `SphereModel`

Reason: Usability.

Pixel center can be defined for a camera

Reason: Mandatory for some platforms. The default is correct for OpenGL and DirectX.

Added `OBBModel` class to the API

Reasons: The only simple bounding volume supported by the previous versions was a sphere. As a sphere bounds many objects rather poorly, an oriented box was added. Axis-aligned boxes are a special case of OBBs.

Support for orthographic projection

Reasons: The Camera class now supports also orthographic projection. This was added because orthographic projection is useful in many editor tools and when viewing CAD models.

Utility header now provides a scene splitter

Reason: When importing unstructured data, such as CAD models, a manual partitioning into objects would be cumbersome. The splitter functionality provided can generate meaningful objects out of unstructured data.

Changes to the Commander API

Reason: The constructor of the Commander class is now protected. The `Commander::command()` function is now changed to pure virtual. This modification does not alter existing code, but reflects better the meaning of the Commander class.

User pointers are available for all classes

Reason: The user pointer mechanism previously used only for the Object class is now available for all classes inherited from ReferenceCount. This may help mapping the user's classes with the dPVS equivalents.

Internal: Approximate silhouettes no longer used

Reasons:

1. Splitted input, for instance terrain, occasionally suffered immense penalties due to accidental "see-through" scenarios. With exact silhouettes this simply cannot happen
2. Managed to design a very fast and memory efficient silhouette extraction algorithm

Version 2.10

Occluder selection behaves slightly differently

Reason: Scenes containing a lot of bad occluders suffered an unreasonable performance penalty. This was due to the fact that cost-benefit evaluation has to "try things out". Mathematical models were updated in a number of ways and this generally decreases the work spent on trying out bad occluders.

We tested the occluder selection using all of our test scenes and confirmed everything worked equally well or better than before. However, changes in mathematical models of multi-variable environments can cause certain cases to behave worse than expected. Should your scene be one of those, please contact us with a general description (number of objects, occluders used etc.) of the problematic

scene along with the description of type of the problem, if possible. Basically the type can be either "selecting too many occluders", "selecting too few" or "not reacting quickly enough".

Support for unbounded objects

Reason: Objects having an infinite size, such as directional light sources were difficult to express in dPVS . A new object flag UNBOUNDED has been added to easily define such objects.

Appendix I

Contact Information

General Information, Licensing Inquiries and Developer Relations

mr. Jouni Mannonen
Tel. +358-9-686 63860
Fax. +358-9-685 2030
Email: jouni.mannonen@surrender3d.com

Technical Questions, Bug Reports and Feature Requests

Email: rw-support@cs1.com
dPVS WWW-Site: <http://renderware.com/dpvs>

Hybrid Offices

Telephone, main line: +358-9-686 6380
Telefax: +358-9-685 2030

Hybrid Holding
Eteläinen Makasiinikatu 4
00130 Helsinki
Finland

For a guaranteed speedy delivery to our offices, be it contract documents, cheques or hardware, we have had consistently positive results with Federal Express. There has sometimes been unexpected delays or costs associated with other types of air cargo. Our normal office hours are from 9 a.m. to 6 p.m. (GMT +2 hours). We may be present at other times as well, but that cannot be guaranteed. Incoming questions are usually answered within 24 hours during weekdays.

Part V

Index

Index

- A-buffer, 257
- accuracy, 333
- adaptive hierarchical visibility, 264
- adaptive testing, 284
- alpha meshing, 309
- antialiasing, 261
- aspect graph, 270

- back-face culling, 276
- beam tracing, 260
- bidirectional visibility, 283
- binary files, 23
- bounding volume, 247
 - temporal, 252
- bounding volume hierarchy, 281
- BSP, 282
 - axis-aligned, 282
- bugs, 25

- Camera, 76
 - create(), 78
 - getCameraToCellMatrix(), 79
 - getCameraToWorldMatrix(), 80
 - getCell(), 81
 - getFrustum(), 82
 - getHeight(), 83
 - getObjectMinimumCoverage(), 84
 - getPixelCenter(), 85
 - getProperties(), 86
 - getScissor(), 87
 - getWidth(), 88
 - isPointVisible(), 89
 - isRectangleVisible(), 90
 - Property, 77
 - resolveVisibility(), 91
 - setCameraToCellMatrix(), 92
 - setCell(), 93
 - setFrustum(), 94
 - getObjectMinimumCoverage(), 95
 - setParameters(), 96
 - setPixelCenter(), 97
 - setScissor(), 98
 - setStaticCoverageMask(), 99
 - setStaticZBuffer(), 100
- cameras, 59, 358
- Cell, 101
 - create(), 103
 - getCellToWorldMatrix(), 104
 - getWorldToCellMatrix(), 105
 - Property, 102
 - set(), 106
 - setCellToWorldMatrix(), 107
 - test(), 108
- cell-space, 30
- cells, 39
- changes, 391
- checklists, 357
- coherence, 250
 - cache, 253
 - image-space, 250
 - object-space, 250
 - temporal, 250
- combinatorial blowout, 253
- Commander, 109
 - Command, 110
 - command(), 113
 - Commander(), 111
 - example implementation, 371
 - getBuffer(), 115
 - getInstance(), 116
 - getLine2D(), 117
 - getLine3D(), 118
 - getStencilValues(), 119
 - getTextMessage(), 120
 - getViewer(), 121
- commander, 37
- Commander::Instance, 122
 - Clip, 123
 - getClipMask(), 125
 - getImportance(), 126
 - getObject(), 127
 - getObjectToCameraMatrix(), 128
 - getProjectionSize(), 129
 - Projection, 124
- Commander::Viewer, 130
 - getCameraToWorldMatrix(), 132

- getFrustum(), 133
- getFrustumPlane(), 134
- getFrustumPlaneCount(), 135
- getProjectionMatrix(), 136
- getScissor(), 137
- Handedness, 131
- isMirrored(), 138
- conservativity, 263
- contribution culling, 277
 - construction time, 318
- conventions, 28
- debug build, 24
- depth estimation buffer, 268
 - hierarchical, 321
- depth of field, 261
- dPVS , 327
 - block diagram, 329
 - Visualizer, 24, 340–344
- dPVS Visualizer, 363
- endian, 345
- eye-space, 30
- feudal priority, 282
- frame rate, 62
- Frustum, 139
 - Type, 140
- frustum casting, 260
- hardware acceleration, 330
- header files, 23
- hierarchical occlusion maps, 267
 - incremental, 311
- hierarchical uniform grid, 282
- image-based rendering, 67, 278
- image-space algorithms, 263
- immediate mode ray casting, 260
- importance decay, 57
- impostors, 278
- incremental algorithms, 253
- item buffer, 257
- lazy evaluation, 251
- Library, 141
 - BufferType, 142
 - clearFlags(), 152
 - exit(), 153
 - FlagType, 143
 - getFlags(), 154
 - getInfoString(), 155
 - getStatistic(), 156
 - InfoString, 144
 - init(), 157
 - LineType, 145
 - MatrixFormat, 146
 - minimizeMemoryUsage(), 158
 - resetStatistics(), 159
 - setFlags(), 160
 - Statistic, 147
 - suggestGarbageCollect(), 161
 - textCommand(), 162
- Library::Services, 163
 - allocateMemory(), 164
 - error(), 165
 - getTime(), 166
 - releaseMemory(), 167
- light sources, 19, 330
- matrices, 32
- Matrix4x4, 232
- Matrix4x4d, 233
- memory usage, 68
- MeshModel, 168
 - create(), 169
- Microsoft
 - DirectX, 261
- mipmap pyramid, 264
- Model, 170
 - getAABB(), 172
 - getOBB(), 173
 - getSphere(), 174
 - Property, 171
 - set(), 175
 - test(), 176
- models, 41
 - back-face culling, 43
 - bad, 45
 - granularity, 41
 - simplification, 42
 - transparent, 45
- motion blur, 261
- multi-threading, 25, 68
- normalized occlusion property, 290
- OBBModel, 177
 - create(), 178
- Object, 179
 - create(), 182
 - getAABB(), 183
 - getCell(), 184
 - getOBB(), 185
 - getObjectToCellMatrix(), 186
 - getSphere(), 187

- getTestModel(), 188
- getVisibilityParent(), 189
- getWriteModel(), 190
- Property, 180
- set(), 191
- setCell(), 192
- setCost(), 193
- setObjectToCellMatrix(), 194
- setTestModel(), 195
- setVisibilityParent(), 196
- setWriteModel(), 197
- test(), 198
- objects, 47, 359
 - costs, 49
 - disabling, 49
 - hierarchies, 50
 - infinite, 51
 - unbounded, 51
- ObjectSplitter, 199
 - split(), 200
- occludee, 248
- occluder, 248
 - virtual, 248, 323
- occluder fusion, 249, 311
- occluder selection, 248, 268, 290
 - classification, 290
 - incremental, 291–295
 - user assistance, 43
- occluder set, 248
- occlusion power, 248
- occlusion volumes, 249, 263
- occlusion write queue, 300
- octree, 264, 281
- on-demand loading, 65
- OpenGL, 261
- optimizations, 70
- output-sensitivity, 251
- painter’s algorithm, 254
- parallelization, 253
- particle systems, 50
- PhysicalPortal, 201
 - create(), 202
 - getImportanceDecay(), 203
 - getStencilModel(), 205
 - getTargetCell(), 204
 - setImportanceDecay(), 206
 - setStencilModel(), 207
 - setTargetCell(), 208
- Pixar, 257
- pixel center, 59
- portability, 345
- portals, 54, 271–275, 360
 - mirrors, 56
- potentially visible sets, 66, 249
 - dynamic, 324
- progressive mesh, 277
- raster-space, 30
- ray casting, 257
- ray tracing, 257
- recursive grid, 281
- reference counting, 33
- ReferenceCount, 209
 - addReference(), 212
 - autoRelease(), 213
 - getName(), 214
 - getReferenceCount(), 215
 - getUserPointer(), 216
 - ReferenceCount(), 210
 - release(), 217
 - setName(), 218
 - setUserPointer(), 219
- RegionOfInfluence, 220
 - create(), 221
- regions of influence, 52, 330
- regular grid, 281
- RenderMan, 261
- resonance, 284
- REYES, 257
- robustness, 333
- scan-line algorithms, 260
- scene graph, 281
- scout cameras, 61
- screen-size culling, 60
- shadow volumes, 249, 271
- shadows, 62
- silhouette extraction
 - hierarchical, 276
 - output-sensitive, 301
 - temporally coherent, 301
- silhouette rasterization, 305
- silhouette splatting, 307
- simplification, 277
 - models, 277
 - occlusion-preserving, 278
 - textures, 277
 - view-dependent, 278
- solid angle, 270
- sorted clipper, 259
- spatial database, 281, 338
- SphereModel, 222
 - create(), 223

- static coverage masks, 60
- statistics, 35
- stencil buffer, 319
- sub-sampling, 60
- summed-area table, 269
- surface shaders, 261
- taxonomy
 - conservative visibility, 286–289
 - Grant’s, 253
 - SSS, 261
- terrains, 51
- test models, 47
- triage mask, 265
- troubleshooting, 377
- validation buffer, 319
- Vector2, 226
- Vector2d, 227
- Vector3, 228
- Vector3d, 229
- Vector3i, 225
- Vector4, 230
- Vector4d, 231
- vectors, 32
- view frustum culling
 - hierarchical, 275
 - shadow frusta, 271
- VirtualPortal, 234
 - create(), 235
 - getTargetPortal(), 236
 - getWarpMatrix(), 237
 - setTargetPortal(), 238
 - setWarpMatrix(), 239
- visibility determination
 - conservative, 243, 263
 - exact, 243, 253
 - non-conservative, 277
- visibility parents, 49
- visibility queries, 61
 - traversal order, 61
- visibility types, 298
- visible point tracking, 296
- Warnock subdivision, 258
- write models, 49
- x86, 25
- Z-buffer, 255
 - depth estimation, 268
 - hierarchical, 264
 - Mammen’s algorithm, 257
 - pre-defined, 60
- Z-pyramid, 264
- ZZ-buffer, 258

Part VI

Bibliography

Bibliography

- [1] <http://www.minolta3d.com>.
- [2] Timo Aila. Surrender umbra: A visibility determination framework for dynamic environments. Master's thesis, Helsinki University of Technology, October 2000.
- [3] John Airey. *Increasing Update Rates in the Building Walkthrough System with Automatic Model-Space Subdivision and Potentially Visible Set Calculations*. Technical report 90-027, Department of Computer Science, University of North Carolina at Chapel Hill, July 1990.
- [4] John Airey, John Rohlf, and Fredrick Brooks. Towards image realism with interactive update rates in complex virtual building environments. *Computer Graphics (1990 Symposium on Interactive 3D Graphics)*, 24(2):141–150, March 1990.
- [5] John Alex and Seth Teller. Immediate-mode ray casting. Technical Report 784, MIT LCS, 1999.
- [6] Daniel Aliaga. Visualization of complex models using dynamic texture-based simplification. In *IEEE Visualization '96*, October 1996.
- [7] Daniel Aliaga and Anselmo Lastra. Architectural walkthroughs using portal textures. In *IEEE Visualization '97*, October 1996.
- [8] Anthony A. Apodaca and Larry Gritz. *Advanced RenderMan - Creating CGI for Motion Pictures*. Morgan Kaufmann, 2000.
- [9] Arthur Appel. Some techniques for shading machine renderings of solids. In *AFIPS Conference Proceedings*, volume 32, pages 37–45, 1968.
- [10] Ulf Assarsson and Tomas Möller. Optimized view frustum culling algorithms for bounding boxes. Technical report, Department of Computer Engineering, Chalmers University of Technology, Sweden, 2000. Accepted for publication in *Journals of Graphics Tools*.
- [11] Fausto Bernardini, Jihad El-Sana, and James Klosowski. Directional discretized occluders for accelerated occlusion culling. In *Proceedings of Eurographics '2000*, 2000.
- [12] Jiri Bittner. Global visibility computations. Master's thesis, Department of Computers, Czech Technical University, 1997.
- [13] Karsten Bormann. An adaptive occlusion culling algorithm for use in large ves. In *Proceedings of the IEEE Virtual Reality 2000 Conference*, 2000.
- [14] Loren Carpenter. The a-buffer, an antialiased hidden surface method. In *Computer Graphics (SIGGRAPH '84 Poceedings)*, volume 18, pages 103–108, July 1984.

- [15] Edwin Catmull. *A Subdivision Algorithm for Computer Display of Curved Surfaces*. PhD thesis, University of Utah, 1974.
- [16] Edwin Catmull. A hidden-surface algorithm with anti-aliasing. *Computer Graphics (Proceedings of SIGGRAPH 78)*, 12(3):6–11, August 1978.
- [17] Frédéric Cazals, George Drettakis, and Claude Puech. Filtering, clustering and hierarchy construction: a new solution for ray-tracing complex scenes. *Computer Graphics Forum*, 14(3):371–382, August 1995. ISSN 1067-7055.
- [18] Frédéric Cazals and Claude Puech. Bucket-like space partitioning data structures with applications to ray-tracing. In *Proceedings of the 13th ACM Symposium on Computational Geometry*, pages 242–248, New York, June 1997. ACM Press.
- [19] Han-Ming Chen and Wen-Teng Wang. The feudal priority algorithm on hidden-surface removal. *Proceedings of SIGGRAPH 96*, pages 55–64, August 1996.
- [20] Lih-Shyang Chen, Gabor Herman, Anthony Reynolds, and Jayaram Udupa. Surface shading in cuberille environments. *IEEE Computer Graphics and Applications*, 5(15), December 1985.
- [21] Milton Chen, Gordon Stoll, Homan Igehy, Kekoa Proudfoot, and Pat Hanrahan. Simple models of the impact of overlap in bucket rendering. In *Proceedings of the 1998 EUROGRAPHICS/SIGGRAPH workshop on Graphics hardware*, pages 105–112, August 1998.
- [22] James Clark. Hierarchical geometric model for visible surface algorithms. *Communications of ACM*, 19(10):547–554, October 1976. Reprinted in [?].
- [23] Jonathan Cohen, Amitabh Varshney, Dinesh Manocha, Greg Turk, Hans Weber, Pankaj Agarwal, Frederick Brooks, and William Wright. Simplification envelopes. In *SIGGRAPH '96 Proc.*, pages 119–128, Aug. 1996.
- [24] Michael Cohen and John Wallace. *Radiosity and Realistic Image Synthesis*. Morgan Kaufmann Publishers, 1993.
- [25] Robert L. Cook, Loren Carpenter, and Edwin Catmull. The reyes image rendering architecture. *Computer Graphics (Proceedings of SIGGRAPH 87)*, pages 95–102, July 1987.
- [26] Satyan Coorg and Seth Teller. Temporally coherent conservative visibility. In *Proceedings of the Twelfth Annual Symposium On Computational Geometry*, pages 78–87. ACM Press, May 1996.
- [27] Satyan Coorg and Seth Teller. Real-time occlusion culling for models with large occluders. *1997 Symposium on Interactive 3D Graphics*, pages 83–90, April 1997. ISBN 0-89791-884-3.
- [28] Michael Cox and Narendra Bhandari. Architectural implications of hardware-accelerated bucket rendering on the pc. In *Proceedings of the 1997 SIGGRAPH/Eurographics workshop on Graphics hardware*, pages 25–34, August 1997.
- [29] Franklin Crow. Shadow algorithms for computer graphics. *Computer Graphics (Proceedings of SIGGRAPH 77)*, 11(2):242–248, July 1977.
- [30] Pavan Desikan, T.M.Murali, and Pankaj Agarwal. Occlusion culling using exact shadow computation and ray shooting queries. Technical report, Duke University, 1999.
- [31] Frédo Durand. *3D Visibility: Analytical Study and Applications*. PhD thesis, Université Grenoble I - Joseph Fourier Sciences et Géographie, July 1999.

BIBLIOGRAPHY

- [32] Frédo Durand, George Drettakis, and Claude Puech. The 3d visibility complex: A new approach to the problem of accurate visibility. In *Eurographics Rendering Workshop 1996*, pages 246–256. Eurographics, June 1996.
- [33] Frédo Durand, George Drettakis, and Claude Puech. 3d visibility made visibly simple an introduction to the visibility skeleton. In *Proceedings of the thirteenth annual symposium on Computational geometry*, pages 89–100, 1997.
- [34] Frédo Durand, George Drettakis, and Claude Puech. The visibility skeleton: A powerful and efficient multi-purpose global visibility tool. *Proceedings of SIGGRAPH 97*, pages 89–100, August 1997.
- [35] Frédo Durand, George Drettakis, Joëlle Thollot, and Claude Puech. Conservative visibility preprocessing using extended projections. In *Proceedings of SIGGRAPH 2000*, Computer Graphics Proceedings, Annual Conference Series, pages 239–248, July 2000.
- [36] Jihad El-Sana, Elvir Azanli, and Amitabh Varshney. Skip strips: Maintaining triangle strips for view-dependent rendering. *IEEE Visualization '99*, pages 131–138, October 1999.
- [37] Carl Ericsson and Dinesh Manocha. Gaps: General and automatic polygonal simplification. Technical Report 98-033, UNC Chapel Hill Computer Science, 1998. Appeared in Proc. of ACM Symposium on Interactive 3D Graphics, 1999, Atlanta.
- [38] Chris Faisstnauer, Dieter Schmalstieg, and Werner Purgathofer. Scheduling for very large virtual environments and networked games using visibility and priorities. Technical Report TR-186-2-00-09, Institute of Computer Graphics, Vienna University of Technology, A-1040 Karlsplatz 13/186/2, April 2000.
- [39] Eugene Fiume, Alan Fournier, and L. Rudolph. A parallel scan conversion algorithm with anti-aliasing for a general purpose ultracomputer. *Computer Graphics (Proceedings of SIGGRAPH 83)*, 17(3):141–150, July 1983.
- [40] Foley, van Dam, Feiner, and Hughes. *Computer Graphics: Principles and Practice*. Addison-Wesley, second edition, 1990.
- [41] W. R. Franklin. A linear time exact hidden surface algorithm. *Computer Graphics (Proceedings of SIGGRAPH 80)*, 14(3):117–123, July 1980.
- [42] Henry Fuchs, Zvi Kedem, and Bruce Naylor. Predetermining visibility priority in 3-d scenes (preliminary report). *Computer Graphics*, 13(2), August 1979.
- [43] Henry Fuchs, Zvi Kedem, and Bruce Naylor. On visible surface generation by a priori tree structures. *Computer Graphics*, 14(3), July 1980.
- [44] Henry Fuchs, John Poulton, John Eyles, Trey Greer, Jack Goldfeather, David Ellsworth, Steve Molnar, Greg Turk, Brice Tebbs, and Laura Israel. Pixel-planes 5: a heterogeneous multiprocessor graphics system using processor-enhanced memories. In *International Conference on Computer Graphics and Interactive Techniques Conference Proceedings on Computer Graphics*, pages 79–88, Boston, MA, USA, July 1989.
- [45] Michael Garland and Paul Heckbert. Surface simplification using quadric error metrics. *Proceedings of SIGGRAPH 97*, pages 209–216, August 1997.
- [46] Chris Georges. Obscuration culling on parallel graphics architectures. Technical Report 95–017, Department of Computer Science, UNC-Chapel Hill, 1995.
- [47] Andrew Glassner. Space subdivision for fast ray tracing. *IEEE Computer Graphics and Applications*, 1984.

- [48] Craig Gotsman, Oded Sudarsky, and Jeffrey Fayman. Optimized occlusion culling using five-dimensional subdivision. *Computers & Graphics*, 23(5):645–654, October 1999. ISSN 0097-8493.
- [49] Charles Grant. Integrated analytic spatial and temporal anti-aliasing for polyhedra in 4-space. *Computer Graphics*, 19(3), July 1985.
- [50] Charles Grant. *Visibility Algorithms in Image Synthesis*. PhD thesis, University of California, 1992.
- [51] Ned Greene. *Hierarchical Rendering of Complex Environments*. PhD thesis, University of California at Santa Cruz, June 1995.
- [52] Ned Greene. Hierarchical polygon tiling with coverage masks. *Proceedings of SIGGRAPH 96*, pages 65–74, August 1996.
- [53] Ned Greene and Michael Kass. Hierarchical z-buffer visibility. *Proceedings of SIGGRAPH 93*, pages 231–240, 1993.
- [54] Ned Greene and Michael Kass. Error-bounded antialiased rendering of complex environments. *Proceedings of SIGGRAPH 94*, pages 59–66, July 1994.
- [55] Eduard Gröller and Werner Purgathofer. Coherence in computer graphics. Technical Report TR-186-2-95-04, Institute of Computer Graphics, Vienna University of Technology, A-1040 Karlsplatz 13/186/2, March 1995.
- [56] Xianfeng Gu, Steven Gortler, Hugues Hoppe, Leonard McMillan, B. Brown, and A. Stone. Silhouette mapping. Technical Report TR-1-99, Department of Computer Science, Harvard University, March 1999.
- [57] G. Hamlin, Jr., and C. W. Gear. Raster-scan hidden surface algorithms. *Computer Graphics (Proceedings of SIGGRAPH 77)*, 11(2):206–213, July 1977.
- [58] Paul Heckbert and Pat Hanrahan. Beam tracing polygonal objects. *Computer Graphics (Proceedings of SIGGRAPH 84)*, 18(3):119–127, July 1984.
- [59] Poon Chun Ho and Wenping Wang. Occlusion culling using minimum occluder set and opacity map. In *Proceedings of the 1999 International Conference on Information Visualization*, 1999.
- [60] Kenny Hoff. A faster overlap test for a plane and a bounding box. Technical report, University of North Carolina, 1996. <http://www.cs.unc.edu/~hoff/research/vfculler/boxplane.html>.
- [61] Hugues Hoppe. Progressive meshes. *Proceedings of SIGGRAPH 96*, pages 99–108, August 1996.
- [62] Hugues Hoppe. View-dependent refinement of progressive meshes. *Proceedings of SIGGRAPH 97*, pages 189–198, August 1997.
- [63] Thomas Hudson, Dinesh Manocha, Jonathan Cohen, Ming Lin, Kenneth Hoff, and Hansong Zhang. Accelerated occlusion culling using shadow frusta. In *Proceedings of ACM Symposium on Computational Geometry*, pages 1–10, 1997.
- [64] David Jevans and Brian Wyvill. Adaptive voxel subdivision for ray tracing. In *Proceedings of Graphics Interface '89*, pages 164–172, June 1989.
- [65] Andreas Johanssen and Michael Carter. Clustered backface culling. *Journal of Graphics Tools*, 3(1):1–14, 1998.

BIBLIOGRAPHY

- [66] Norman Jouppi and Chun-Fa Chang. Z3: an economical hardware technique for high-quality antialiasing and transparency. *1999 SIGGRAPH / Eurographics Workshop on Graphics Hardware*, pages 85–93, August 1999.
- [67] James Klosowski and Cláudio Silva. Rendering on a budget: A framework for time-critical rendering. *Proceedings of the IEEE Computer Society Visualization '99*, October 1999.
- [68] Jan Koenderink. What does the occluding contour tell us about solid shape. *Perception* 13, pages 321–330, 1984.
- [69] Vladlen Koltun, Yiorgos Chrystanthou, and Daniel Cohen-Or. Virtual occluders: a from-region visibility techniques. In *Eurographics Rendering Workshop*, 2000.
- [70] Subodh Kumar and Dinesh Manocha. Hierarchical visibility culling for spline models. In *Graphics Interface*, pages 142–150, May 1996.
- [71] Subodh Kumar, Dinesh Manocha, William Garrett, and Ming Lin. Hierarchical back-face computation. In Xavier Pueyo and Peter Schröder, editors, *Eurographics Rendering Workshop 1996*, pages 235–244. Eurographics, Springer Wein, June 1996.
- [72] Fei-Ah Law and Tiow-Seng Tan. Preprocessing occlusion for real-time selective refinement. In *Proceedings of the 1999 symposium on Interactive 3D graphics*, pages 47–53, 1999.
- [73] David Luebke and Carl Erikson. View-dependent simplification of arbitrary polygonal environments. *Proceedings of SIGGRAPH 97*, pages 199–208, August 1997.
- [74] David Luebke and Chris Georges. Portals and mirrors: Simple, fast evaluation of potentially visible sets. In *Proceedings 1995 Symposium on Interactive Computer Graphics*, pages 105–106, April 1995.
- [75] Paulo Maciel and Peter Shirley. Visual navigation of large environments using textured clusters. In P. Hanharan and J. Winget, editors, *1995 Symposium on Interactive 3D Graphics*, pages 95–102. ACM SIGGRAPH, April 1995.
- [76] Abraham Mammen. Transparency and antialiasing algorithms implemented with the virtual pixel maps technique. *CG & A*, 9(4):43–55, July 1989.
- [77] Donald Meagher. *The Octree Encoding Method for Efficient Solid Modeling*. PhD thesis, Electrical Engineering Department, Rensselaer Polytechnic Institute, New York, August 1982.
- [78] Jim Miller. Directmodel 1.0 specification - revision c. Available online: http://www.op.dlr.de/FF-DR/dr_fs/staff/trautenberg/DirectModelspec.pdf, October 1997.
- [79] Gordon Müller, , and Dieter Fellner. Hybrid scene structuring with application to ray tracing. In *Proceedings of International Conference on Visual Computing (ICVC'99)*, pages 19–26, February 1999.
- [80] A.J. Myers. An efficient visible surface program. Technical Report DCR 74-00768A01, National Science Foundation, 1975.
- [81] Bruce Naylor. Partitioning tree image representation and generation from 3d geometric models. In *Proceedings of Graphics Interface '92*, pages 201–212, May 1992.
- [82] M. Newell, R. Newell, and T. Sancha. A solution to the hidden surface problem. In *Proceedings of the ACM 1972 National Conference*, pages 443–450, 1972. Reprinted in [?].

- [83] Manuel Oliveira, Gary Bishop, and David McAllister. Relief texture mapping. In *Proceedings of SIGGRAPH 2000*, Computer Graphics Proceedings, Annual Conference Series, July 2000.
- [84] Dave Schreiner OpenGL Architecture Review Board. *OpenGL Reference Manual: The Official Reference Document to OpenGL, Version 1.2*. Addison-Wesley, December 1999.
- [85] Mark Peercy, Marc Olano, John Airey, and Jeff Ungar. Interactive multi-pass programmable shading. In *Proceedings of SIGGRAPH 2000*, Computer Graphics Proceedings, Annual Conference Series, July 2000.
- [86] Harry Plantinga and Charles Dyer. Visibility, occlusion and the aspect graph. *International Journal of Computer Vision*, 5(2):137–160, 1990.
- [87] Voicu Popescu, John Eyles, Anselmo Lastra, Josh Steinhurst, Nick England, and Lars Nyland. The warpengine: An architecture for the post-polygonal age. In *Proceedings of SIGGRAPH 2000*, Computer Graphics Proceedings, Annual Conference Series, July 2000.
- [88] Jovan Popovic and Hugues Hoppe. Progressive simplicial complexes. *Proceedings of SIGGRAPH 97*, pages 217–224, August 1997.
- [89] Steven Rubin and Turner Whitted. A 3-dimensional representation for fast rendering of complex scenes. *Computer Graphics (Proceedings of SIGGRAPH 80)*, 14(3):110–116, July 1980.
- [90] David Salesin and J. Stolfi. The zz-buffer: A simple and efficient rendering algorithm with reliable antialiasing. In *Proceedings of the PIXIM '89 Conference*, pages 451–466. Hermes Editions, Paris, September 1989.
- [91] Pedro Sander, Xianfeng Gu, Steven Gortler, Hugues Hoppe, and John Snyder. Silhouette clipping. In *Proceedings of SIGGRAPH 2000*, Computer Graphics Proceedings, Annual Conference Series, July 2000.
- [92] Carlos Saona-Vázquez, Isabel Navazo, and Pere Brunet. Data structures and algorithms for navigation in highly polygon-populated scenes. Technical Report Technical Report LSI-98-58R, Universitat Politècnica de Catalunya, 1998.
- [93] Gernot Schaufler. Dynamically generated impostors. In D. W. Fellner, editor, *Modeling Virtual Worlds - Distributed Graphics*, pages 129–136. MVD'95 Workshop, November 1995.
- [94] Gernot Schaufler. Nailboards: A rendering primitive for image caching in dynamic scenes. In Julie Dorsey and Philipp Slusallek, editors, *Rendering Techniques '97*, Eurographics, pages 151–162. Springer-Verlag Wien New York, 1997.
- [95] Gernot Schaufler. Per-object image warping with layered impostors. In George Drettakis and Nelson Max, editors, *Rendering Techniques '98*, Eurographics, pages 145–156. Springer-Verlag Wien New York, 1998.
- [96] Gernot Schaufler, Julie Dorsey, Xavier Decoret, and Francois Sillion. Conservative volumetric visibility with occluder fusion. In *Proceedings of SIGGRAPH 2000*, Computer Graphics Proceedings, Annual Conference Series, pages 229–238, July 2000.
- [97] Gernot Schaufler and Wolfgang Stürzlinger. A three dimensional image cache for virtual reality. *Computer Graphics Forum*, 15(3):227–236, August 1996. ISSN 1067-7055.
- [98] Dieter Schmalstieg and Robert Tobler. Real-time bounding box area computation. Technical Report TR-186-2-99-05, Vienna University of Technology, 1999. <http://www.cg.tuwien.ac.at/research/TR/>.

BIBLIOGRAPHY

- [99] Robert Schumacker, B. Brand, M. Gilliland, and W. Sharp. Study for applying computer generated images to visual simulation. Technical Report AFHRL-TR-69-14, U.S. Air Force Human resources Laboratory, September 1969.
- [100] Jonathan Shade, Dani Lischinski, David Salesin, Tony DeRose, and John Snyder. Hierarchical image caching for accelerated walkthroughs of complex environments. *Proceedings of SIGGRAPH 96*, pages 75–82, August 1996.
- [101] François Sillion, George Drettakis, and Benoit Bodelet. Efficient impostor manipulation for real-time visualization of urban scenery. In *Computer Graphics Forum*, pages 207–218, 1997.
- [102] Mel Slater and Yiorgos Chrysanthou. View volume culling using a probabilistic caching scheme. In *Proceedings of the ACM symposium on Virtual reality software and technology*, pages 71–77, 1997.
- [103] Milan Sonka, Vaclav Hlavac, and Roger Boyle. *Image Processing, Analysis and Machine Vision*. International Thomson Computer Press, 1996.
- [104] Oded Sudarsky and Craig Gotsman. Output-sensitive visibility algorithms for dynamic scenes with applications to virtual reality. *Computer Graphics Forum*, 15(3):249–258, August 1996. ISSN 1067-7055.
- [105] Ivan Sutherland, Robert Sproull, and Robert Schumacker. Sorting and hidden-surface problem. In *Proceedings for the AFIPS National Conference*, volume 42, 1973.
- [106] Ivan Sutherland, Robert Sproull, and Robert Schumacker. A characterization of ten hidden surface algorithms. *ACM Computing Surveys*, 6, 1974.
- [107] S. L. Tanimoto and Thoe Pavlidis. A hierarchical data structure for picture processing. *Computer Graphics and Image Processing*, 4(2):104–119, 1975.
- [108] Seth Teller. *Visibility Computations in Densely Occluded Polyhedral Environments*. PhD thesis, CS Division, UC Berkeley, October 1992. Tech. Report UCB/CSD-92-708.
- [109] Seth Teller. Frustum casting for progressive, interactive rendering. Technical Report 740, MIT LCS, January 1998.
- [110] Seth Teller and Carlo Séquin. Visibility preprocessing for interactive walkthroughs. *Computer Graphics (Proceedings of SIGGRAPH 91)*, 25(4):61–69, July 1991.
- [111] Jay Torborg and James Kajiya. Talisman: Commodity realtime 3D graphics for the PC. *Computer Graphics*, 30(Annual Conference Series):353–363, 1996.
- [112] Steve Upstill. *The Renderman Companion*. Addison Wesley, Reading, MA, 1990.
- [113] Michiel van de Panne and A. James Stewart. Effective compression techniques for precomputed visibility. In *Eurographics Workshop on Rendering*, pages 305–316, June 1999.
- [114] John Warnock. A hidden surface algorithm for computer generated half-tone pictures. Technical Report RADC-TR-69-249, Dept. of CS U of Utah, June 1969. Reprinted in [40].
- [115] Kevin Weiler and Peter Atherton. Hidden surface removal using polygon area sorting. *Computer Graphics (Proceedings of SIGGRAPH 77)*, 11(2):214–222, July 1977.
- [116] Turner Whitted. An improved illumination model for shaded display. *Communications of the ACM*, 23(6), June 1980.
- [117] Lance Williams. Pyramidal parametrics. *Computer Graphics (Proceedings of SIGGRAPH 83)*, 17(3):1–11, 1983. Reprinted in [?].

- [118] George Wolberg. *Digital Image Warping*. IEEE Computer Society Press, 1990.
- [119] C. Wylie, G.W. Romney, D.C. Evans, and A.C. Erdahl. Halftone perspective drawings by computer. In *Proceedings of AFIPS Fall Joint Computer Conference*, 1967.
- [120] Feng Xie and Michael Shantz. Adaptive hierarchical visibility in a tiled architecture. In *Proceedings 1999 Eurographics/SIGGRAPH workshop on Graphics hardware*, pages 75–84, 1999.
- [121] Hansong Zhang. Fast incremental transformation of bounding boxes, with generalizations to regular grids like terrain and volume data. Technical report, University of North Carolina, September 1997.
- [122] Hansong Zhang. *Effective Occlusion Culling for the Interactive Display of Arbitrary Models*. PhD thesis, Department of Computer Science, University of North Carolina at Chapel Hill, 1998.
- [123] Hansong Zhang. Personal communication, 2000.
- [124] Hansong Zhang and Kenneth E. Hoff III. Fast backface culling using normal masks. *1997 Symposium on Interactive 3D Graphics*, pages 103–106, April 1997. ISBN 0-89791-884-3.
- [125] Hansong Zhang, Dinesh Manocha, Thomas Hudson, and Kenneth E. Hoff III. Visibility culling using hierarchical occlusion maps. *Proceedings of SIGGRAPH 97*, pages 77–88, August 1997.